University of Warsaw
Faculty of Mathematics, Informatics and Mechanics

Piotr Styczyński
Student no. 386038

An integrated and practical database solution for storage and efficient processing of large genomic datasets

Master's thesis
in COMPUTER SCIENCE

Warsaw, 2026-07-28

Abstract

Genomic workflows commonly connect specialized programs through files. This arrangement separates sequences from relational metadata, repeats parsing and indexing work, and prevents the database planner from representing the cost of alignment. This thesis presents BioSeqDB, a PostgreSQL extension that stores compact biological sequences and runs a minimap2-derived seeding, chaining, and extension pipeline during database scans.

BioSeqDB provides alphabet-generic packed sequence views, scalar and single instruction, multiple data (SIMD) minimizer extraction, and Vulkan implementations for selected sketching and chaining operations. Bounded CPU dynamic-programming kernels and GPU alignment routes perform final extension. Optional SneakySnake and GateKeeper filters reject unsuitable candidate regions before that stage. The database layer adds persistent minimizer indexes, bounded ingestion, external-provider relation definitions, coordinated caching, and generated PostgreSQL boundary declarations.

Alignment is not evaluated as an independent scalar function for every row. Planner support introduces custom paths whose execution nodes batch sequences, execute the explicitly selected implementation when the input and hardware pass admission, and place alignment results into planned expressions. The same operation can therefore participate in joins, filtering, metadata projection, and persistent storage. The thesis describes how the mapper's data and control flow become a database execution primitive while retaining explicit resource bounds and support for heterogeneous processors.

Keywords

sequence alignment, bioinformatics, minimap2, PostgreSQL, databases, genomic data management, SIMD, GPU computing

Thesis domain (Socrates-Erasmus subject area codes)

11.3 Informatics and Computer Science

Subject classification

Applied computing, life and medical sciences, bioinformatics

Tytuł pracy w języku polskim

Zintegrowane i praktyczne rozwiązanie bazodanowe do przechowywania i wydajnego przetwarzania dużych zbiorów danych genomicznych

Contents

Introduction . . . 7
Timeline . . . 10

1. Sequence representations and lazy evaluation . . . 11
1.1. Designing sequence abstraction . . . 11
1.2. Packed symbols and a common C++ interface . . . 15
1.3. Slices keep logical coordinates . . . 17
1.4. Composing sequences without joining their payloads . . . 19
1.5. Reading only what the algorithm requires . . . 20

2. PostgreSQL API and custom scans . . . 25
2.1. Designing SQL API for an aligner . . . 25
2.2. Integrating Postgres . . . 28
2.2.1. Designing SQL API . . . 28
2.2.2. Plan lifecycle . . . 29
2.2.3. Different types of scans . . . 31
2.3. Dispatch buffer as glue between Postgres scan and alignment pipeline . . . 32
2.3.1. Returning results back to Postgres . . . 32
2.3.2. Supporting joins . . . 34
2.3.3. Dispatch buffer internals . . . 34
2.3.4. MVCC support . . . 38
2.3.5. Filter placement . . . 38
2.4. Joins support . . . 39
2.5. Caching results . . . 42
2.5.1. Reusing fragments of queries . . . 43

3. Physical sequence storage in PostgreSQL . . . 45
3.1. Store sequences together with their metadata . . . 45
3.2. Columnar storage . . . 46
3.2.1. Columnar layout . . . 47
3.2.2. Smart reads . . . 50
3.2.3. Compression . . . 51
3.3. Raw byte storage . . . 52
3.4. Locator implementation . . . 54
3.4.1. Two levels of sharding . . . 57
3.5. MVCC and deletion for sequence fragments . . . 58
3.5.1. BioseqDB all cache layers . . . 59

4. Persistent sequence indexes . . . 63
4.1. Keep the lookup work between queries . . . 63
4.2. What existing sequence indexes store . . . 63
4.2.1. Indexing: minimap2 . . . 63
4.2.2. Indexing: mmseqs2 . . . 64
4.3. BioseqDB indexing abstractions . . . 64
4.4. BioseqDB supported indexing . . . 68
4.4.1. MPF follows source intervals . . . 69
4.4.2. MPS follows hash ranges . . . 70
4.4.3. A smaller index answers symbol counts . . . 70
4.5. Build a large index with bounded work buffers . . . 71
4.6. Read counts before large position lists . . . 73
4.7. A saved index needs a source version . . . 74
4.7.1. Publish complete generations . . . 74
4.7.2. From stored index rows to a CPU reader file . . . 75
4.8. Choose reuse without changing the search meaning . . . 83
4.8.1. Keep useful bytes while their owners remain valid . . . 85
4.9. What the database integration adds . . . 86

5. BioSeqDB architecture . . . 87
5.1. Connecting sequence processing to the database . . . 87
5.1.1. Reuse at the alphabet boundary . . . 88
5.2. Contracts describe the operations a stage needs . . . 88
5.3. Ownership completes the interface . . . 89
5.4. Runtime choices and compiled implementations . . . 90
5.4.1. Turn SQL settings into a request . . . 91
5.4.2. Separate algorithm selection from hardware selection . . . 91
5.4.3. Require compatible indexes and execution routes . . . 92
5.5. Schedule independent work within a backend . . . 93
5.5.1. PostgreSQL processes and oneTBB threads . . . 93
5.5.2. Choose the unit of parallel work . . . 94
5.5.3. Admit memory before increasing overlap . . . 96
5.5.4. Completion, cancellation, and progress . . . 97
5.6. Load sequence files through a reusable parser . . . 98
5.6.1. Separate byte sources from record framing . . . 98
5.6.2. Deliver chunks while a record remains open . . . 98
5.6.3. Route header fields into SQL columns . . . 99
5.7. Remote sequences and local persistence . . . 100
5.8. Generation and validation . . . 102
5.8.1. Deriving the PostgreSQL boundary . . . 102
5.8.2. Controlling specialization cost . . . 103
5.8.3. Benchmark specifications and validation . . . 104
5.9. Authorship, development and AI assistance . . . 104
5.9.1. Where AI tools were used . . . 105
5.10. Validation limits of the complete system . . . 105

6. Benchmarking and performance evaluation 107
6.1. Benchmarking and performance evaluation 107
6.1.1. End-to-end results and their interpretation 109
6.1.2. Recorded machines and completed coverage 113
6.1.3. Comparison with other aligners 143

7. Conclusions 149

References 151

Introduction

Sequencing instruments turn biological material into collections of strings. The useful interpretation of those strings rarely exists in isolation. A read has an instrument and sample provenance. A reference interval belongs to an assembly and annotation release. An alignment has a score, a strand, a position, an edit path, and a relationship to experimental metadata. Conventional workflows keep the strings in FASTA and FASTQ sequence files, Sequence Alignment/Map (SAM) and Binary Alignment/Map (BAM) alignment files, or mapper-specific index files while keeping much of the surrounding information in tables or ad-hoc scripts. Each tool sees only the portion of the workflow represented by its command-line arguments.

Relational database systems solve a different part of the problem. PostgreSQL provides durable storage, transactions, typed values, indexing, joins, extensible operators, and a cost-based planner. It does not, by itself, provide the representations or execution kernels required for billions of packed symbols and computationally expensive sequence searches. A naive user-defined function is not sufficient. Evaluating one alignment function independently for every row prevents batching, hides accelerator opportunities, and gives the planner little information about a predicate that can dominate the query.

BioSeqDB investigates the boundary between these two systems. It is implemented as a PostgreSQL extension whose non-database core is written in C++26. Biological symbols are represented by generic compact ranges rather than by a deoxyribonucleic acid (DNA)-only byte string. The alignment path is derived from minimap2: representative k-mers are extracted as minimizers, matching minimizers become anchors, compatible anchors are chained, and CPU or GPU alignment routes extend the selected regions. The port is not a line-for-line translation. The implementation introduces template-selected alphabets and layouts, Highway SIMD kernels, task-based parallel execution, Vulkan compute paths, persistent PostgreSQL-backed indexes, and prefilters for candidate regions. These changes must preserve the semantics that make minimap2 useful while operating on data that may be lazy, fragmented, cached, or obtained from an external provider.

The database boundary is equally important. An SQL expression such as `mmsearch(a, b)` has a conventional function definition so that PostgreSQL can type and plan it. The ordinary function body is not an alignment fallback: it fails closed if no BioSeqDB custom executor owns the expression. Its planner-support function assigns the unowned expression a high startup cost, while structural planner hooks can construct an owning custom path with explicit nonnegative costs. The resulting custom scan batches input, invokes the alignment pipeline, and makes its results available to planned expressions through injected PostgreSQL nodes. This is what permits alignment to participate in filters, projections, and joins while retaining the execution structure required by central processing unit (CPU) vector units and graphics processing units (GPUs).

The central research question is therefore not whether an existing aligner can be called from SQL. It is whether the aligner's data and control flow can become a database execution

primitive without losing algorithmic fidelity, resource bounds, or the ability to exploit modern processors. The investigation is organized around four supporting questions:

RQ1. Which parts of seeding, chaining, and extension can be expressed over a common compact sequence abstraction without eager conversion?

RQ2. Which stages admit SIMD or GPU implementations, and which input and resource constraints govern their selection?

RQ3. How can PostgreSQL plan and execute a set-valued, batch-oriented search while presenting an ordinary typed SQL interface?

RQ4. How should sequences, indexes, ingestion, external data, caches, and generated boundary code fit around the kernels so that the resulting system remains inspectable and reliable?

This thesis answers these questions through implementation and analysis of algorithmic and execution contracts. Historical designs appear only when they explain a constraint or a rejected optimization. They are not presented as simultaneous production architectures. Comparisons with minimap2, SIMD-Minimizers, the G3SA GPU seed-chain-extend system, Block Aligner, SneakySnake, GateKeeper, and Citus Columnar distinguish inherited mechanisms from BioSeqDB-specific changes.

The main contributions developed in the following chapters are:

C1. An alphabet-generic sequence representation that supports packed, fragmented, lazily composed, and PostgreSQL-backed data through a common range contract.

C2. A staged minimap2-derived mapper with scalar, SIMD, and Vulkan components and explicit runtime dispatch.

C3. Optional GPU-capable SneakySnake and GateKeeper candidate filters integrated without changing the meaning of an accepted alignment.

C4. Persistent sequence and minimizer-index storage derived from a columnar PostgreSQL access method, with bounded loading, caching, and selective cold-index reads.

C5. A planner and custom-scan interface that connects SQL expressions and joins to batch-oriented alignment.

C6. Structural generation and validation of SQL installation definitions and PostgreSQL function-manager boundary declarations from annotated C++ sources.

The document first establishes the biological and algorithmic context, then states the shortcomings that motivate an integrated system. The algorithm chapters follow data through minimizer extraction and chaining. They then describe the expensive downstream alignment stage before stepping back to the prefilter that guards that stage. This expository inversion makes the filter's safety and cost contract concrete without changing the runtime order, which remains chaining, prefiltering, and alignment. It then moves outward through sequence abstractions, PostgreSQL planning, physical storage, persistent indexes, ingestion, external providers, and generated SQL boundary code.

- Chapter 1 develops compact sequence ranges and bit-level views. It then presents fragmented and lazy sequences with PostgreSQL-backed adapters and alphabet dispatch.

- Chapter 2 follows an mmsearch expression through planning and batched execution. It explains joins and per-row result substitution.
- Chapter 3 explains the columnar-derived storage layout and fragment metadata. It also covers caching and request coordination.
- Chapter 4 describes index construction and storage with compatibility checks and selective cold loading. It also explains updates and differences from minimap2 indexes.
- Chapter 5 connects algorithm selection and oneTBB scheduling to a reusable processing core. It also covers sequence ingestion; remote access; code generation; validation and authorship.
- Chapter 6 measuring overall performance of BioseqDB
- Chapter 7 answers the research questions and summarizes the final system. It identifies the principal limitations and possible extensions.

Timeline

This thesis required several years of exploration and experiments. I began without a formal background in computational biology, so learning the field was part of the research. I proposed the database idea and prepared the initial design for our internal team. I directed the 2022 prototype, whose implementation was written by colleagues.[1] The MMseqs2 discussions did not lead to a collaboration. The Python prototypes that led to the implementation studied here began in December 2023.



Selected milestones from my project history. Spacing is not proportional to elapsed time. Linked entries point to the related discussion and prototype repositories.

[1] The current BioSeqDB is a separate project, designed and written by me. It contains no code or intellectual property from the early BWA-MEM integration experiments.

[2] I was a co-founder (shareholder) until 15 July 2022 (KRS update on 3 Jun 2022) in Covid Genomics Sp. z o.o. (registered in Poland in Jun 2020, KRS 0000846289) which then changed its legal name to CGen Sp. z o.o. (on 3 Jun 2022) and then changed its branding name to Deepflare (on Jul 2022). I was also a Chief Engineering Officer from 1 Apr 2020 till 31 Aug 2022.

Chapter 1

Sequence representations and lazy evaluation

1.1. Designing sequence abstraction

Suppose user stores large genome sequence in one row, but only wants to align specific small part of it (let's say we know location of one protein-coding human gene and are interested only in finding occurences of that gene). It would be very wasteful if I didn't provide built-in functionality to load only that small region. This region could be stored on disk, so we only have it's locator poiting to a specific chunk in the table.

Physical storage

"Locator" refers to a small structure holding sequence metadata and pointer to its actal location in the storage system. More on is explained in physcial storage chapter 3.1. For now, I assume it's only an opaque pointer to sequence and use term "hydration" describing the process of converting it into a sequence itself or its fragment (as continous byte array in RAM).

On the other hand, we can load sequences from fasta file, where parser can be writing to RAM, so our sequence could refer to location in memory, where all sequences are pooled into one memory region to avoid constant allocation/reallocation. In yet another case, in tests we would like to use standard library containers to prepare and examine sequences (potentially even somebody could directly call some internal BioseqDB functions as standalone alignment library). This created a design challenge: same sequence can arrive as relationship locator, region in memory pool in RAM, can be provided as a whole as standrad container or as only a fragment (if user wants to operate only on specific slice or if it does not fit in RAM or RAM is limited, so we cannot fit entire sequence directly there), while these fragments should also be shareable, so parallel alignment to the same sequence doesn't have to load them multiple times.

There are several existing libraries for interfacing with genomic sequences. SeqAn[1] is a C++ library that provides modern interface for composing efficient sequence-analysis software from generic C++ components. Its library includes sequence representations, indexes and configurable alignment algorithms, with SIMD alignment, but it has it's own limitations.

[1] Reinert et al., "The SeqAn C++ template library for efficient sequence analysis: A resource for programmers"; SeqAn Project, SeqAn Manual.

However SeqAn SIMD algorithms are mostly limited to auto-vectoriztion provided by the compiler (which doesn't have to be portable in most cases) and offers very narrow set of features for custom pipelines. I would be hard to directly reuse it for implementation of mentioned feautres. Using SeqAn would require deep integration, that's why I decided to address the design challenges by designing own C++ system of templates with different specializations driven by C++ concepts (inspired by nice-looking HPC MPI code Uhl, Schimek, and Brommer, Concepts in Practice: C++ MPI Bindings for the HPC Ecosystem. From a Standardizable Core to a Composable Interface).

SeqAn compatibility

Because the architecture of BioseqDB pipeline is very custom, I decided to only use SeqAn's alphabet layer[a] and utilities exposed by its public API. This means that libraries using SeqAn code as base, could be slightly adjusted to be able to work with BioseqDB pipelines.

[a] SeqAn Project, Alphabets in SeqAn.

Table 1.1 lists every specialization of all BioseqDB sequence representations, to showcase how they are used in different scenarios. SeqAn offers abstractions over lazy sequences as well SeqAn Project, SeqAn Tutorial: Sequences In-Depth, but they're not as sophisticated as in my case. The lazy interface is really important for BioseqDB to improve it's performance utilising late-materialization strategies similar to those described in literature[3]

Table 1.1: Iterfaces for sequence representations in BioSeqDB.

Template	Specialization, and when it is selected	Description and usage context
`compact_sequence_range<R>` This is the core type representing sequence that resides completely in memory. Template allows only four available specializations depending on what `R` is and each of them have entirely separate handling logic.	`R` is a container of alphabet values, such as `std::vector<dna4>`	This is only adapter for existing sequence containers to provide compatibility with SeqAn and standard library containers. There is no alphabet packing and the type borrows `const R&` Adapter just provides common sequence interface. Adapting one costs $O(1)$ as we proxy all operations to the underlying container
	`R` is a packing configuration `sequence_custom_alphabet<A, U,b>`	Bitpacked vector that stores sequences according to alphabet rules passed as type paramter. Owns storage. Can convert to and from other bitpacked or not-bitpacked sequences, but if constructed from non-bitpacked sequence, then constructor costs $O(N)$
	`R` is an SeqAn alphabet type, as in `compact_sequence_range<dna4>`	The same as `sequence_custom_alphabet<A,U,b>`, but has sensible defaults for used underlying alphabet's storage unit width etc. Commonly used in tests. Code interfacing directly with BioseqDB C++ API should use this specialization.

[3] Chernishev et al., "A Comprehensive Study of Late Materialization Strategies for a Disk-Based Column-Store."

Template	Specialization, and when it is selected	Description and usage context
	`R` is `compact_sequence_stdc_instance<...>`	Holds only reference to the underlying object. Adapter to convert `compact_sequence_stdc_instance<...>` to `compact_sequence_range<R>`. `compact_sequence_stdc_instance<...>` is a type that comes from `compact_sequence_stdc<...>` - described below Used internally
`compact_sequence_stdc<...>` Represents standard layout C type cppreference.com contributors, C++ named requirements: StandardLayoutType that is used to type-erase `compact_sequence_range<R>` types i.e allow user to have one type for storing possibly multiple different sequences types. Metadata like alphabet type and storage unit width are stored inside the struct and specialized template dispatch lambda is called to convert it back to the `compact_sequence_range<R>` type. This is heavily used by Postgres code to store sequences as C structures.		Owns everything and stores fixed header that records struct (self) size in bytes (compatible with Postgres varlena), alphabet code, unit width, padding, and symbol count Used heavy by Postgres code as we cannot store template instances directly and need to hold them in one object that converts type parameters of templates to runtime values (with correct runtime dispatch). If you want to store sequences of different alphabets in one single container, you should use this.
	`compact_sequence_stdc_instance<Stdc,A,W>`, internal type obtained when dispatching the type at runtime	Carries nothing, this is primarily runtime tag Shouldn't be used directly, one should receive `compact_sequence_range<compact_sequence_stdc_instance<...>>` when using dispatch methods of the stdc template
`lazy_sequence<R, Scanner>` Represents sequence that can be loaded in fragments by some storage provider represented by type parameter `Scanner`	`Scanner` has static constexpr method `is_fragmented()` that returns false (this means that Scanner always holds entire sequence in memory and there's only one fragment). This is used as compatibility layer so you can directly create lazy sequence from compact sequence range instance. I have `resident_sequence_scanner<U>` for trivial implementation of non-fragmented scanners so you can write `lazy_sequence<R,resident_sequence_scanner<R>>` and don't have to implement scanner type on your own.	Holds single non-owning pointer to `compact_sequence_range<R>` Used as adapter to convert compact sequence range to lazy sequence. All primary pipeline operations should accept it as parameter

Template	Specialization, and when it is selected	Description and usage context
	`Scanner` is fragmented (`is_fragmented()` returns true)	Ows scanner, internal small cache for sequence fragments that are neither copyable or movable Allows iteration over data source of sequences that doesn't fit into memory at once. This is primary specialisation for loading sequences from PostgreSQL relations. The Postgres side implements large scanner type that tells the lazy sequence template how to fetch blocks of sequences
`bit_casted_view<Base, Out>` Only legal for few types of alphabets and storage widths.	Special non-owning type that is used to provide zero-cost changes to pitpacking. It allows to cast bitpacked sequences to different storage widths i.e cast sequence encoded on 32-bit int to be read as sequence encoded on 64-bit or 16-bit or 8-bit units. There are special requirements for alphabet to be able to do that (discussed below).	Holds only reference to underlying compact sequence range This is adapter type that provides zero-cost conversion between different specialisations of compact sequence range. Used for example by SIMD algorithms to have specific vector lane widths
`lazy_sequence_view<A>` Type-erased, non-owning facade over every representation listed above. It is templated on the alphabet only, so exactly one type name crosses module and Postgres boundaries. The object itself is two pointers: opaque backend state and a static `backend_ops` table that holds `bp_size`, four orientation-specific `read_at` slots (read forward or backward, write forward or backward), `try_resident_packed`, `copy_packed_sequence` and `worker_shareable`. Specialization here means which callback table is installed.	`owned_sequence_adapter<R, Scanner>`, created by `make_lazy_sequence_view` for a concrete `lazy_sequence<R,Scanner>`	Complete table over an owning lazy sequence. This is the table behind every lazy sequence that pipeline code receives, where Postgres provides pointers to actual functions.
	`compact_range_adapter<R>`, created by `make_lazy_sequence_view` for `compact_sequence_range<R>`	Used as convenient adapter to convert compact sequence range to lazy sequence view without worrying about scanner type and ownership. Useful for testing.
	`facade_adapter<A>`, created by `make_pg_sequence_call_owner` on the Postgres side	This struct holds Postgres Scanner state - it is either a detoasted flat value or a piece tree root together with its read session, because sequences need to be represented as trees of slice operations

Template	Specialization, and when it is selected	Description and usage context
	`resident_adapter<A>`, created by `admitted_packed_sequence::view()`	Postgres Scanner for sequence only holds descriptor for already materialized sequence
`retained_lazy_sequence_view<A>` Has only one form. This type pairs view mentioned above with opaque lifetime pointer. It's used in Postgres code for management of sequence lifecycle during DB operations	Constructed from a view, an opaque lifetime pointer and a `noexcept` release callback, so there is nothing to specialize	Used wherever a view has to outlive the expression that produced it. The Postgres read session allocates the owner in its own memory context and registers it there, so the retain is dropped even when Postgres unwinds with `longjmp` and skips C++ destructors. This means that this type is specific version of `std::unique_ptr` that allows us to provide custom lifecycle management during scans.
`admitted_packed_sequence<A>` Supporting type to propagate sequence along with memory permit (memory permits are used in BioseqDB to manage large memory regions for sequences)	Created during materialization by `materialize_packed_sequence`. In BioseqDB there's a little sequence access planner that is responsible for deciding if we can materialize it all at once and directly or it needs to be streamed. `plan_sequence_access` picks this type when a contiguous resident region and we cannot use existing materialization (for example we access specific nucleotide that doesn't align well with storage unit boundary, but the GPU algorithm requires continous region of sequence, so we need to copy it)	Owns the permit, the aligned buffer and its descriptor together, so accounted bytes are released with the object. Its `view()` returns an ordinary view over `resident_adapter<A>`, which is how SIMD and GPU kernels obtain contiguous packed input from a sequence that is tree-backed or fragmented

1.2. Packed symbols and a common C++ interface

The template-driven design allows usage of any arbitrary alphabet for sequences, though for now I only support DNA4 and DNA5.

Sequence is encoded using bitpacking - the symbols are converted into integers values with specified fixed number of bits per symbol, then several symbols are packed into one single storage unit. Storage unit can be integer of any physical size. This is generalisation of bitpacking strategies commonly used in other software like packed-seq Groot Koerkamp and Martayan, packed-seq: Bitpacked sequence trait and implementation Rust library developed by author of SIMD minimizers paper. packed-seq Rust library offers non-alphabet-generic packing Groot Koerkamp and Martayan, packed_seq crate documentation. My design was more inspired by SeqAn approach where packing container is available, but very limited in features SeqAn Project, SeqAn Manual.

SeqAn alphabet assigns each symbol an integer called its rank. For DNA4, the four ranks are $A = 0$, $C = 1$, $G = 2$, and $T = 3$. A rank is an encoding of a symbol. Each `compact_sequence_range<R>` type needs to know certain properties of the encoding that are held

(using CRTP) in `compact_sequence_range<R>::traits` type. In my storage symbols never spill to other storage units i.e to retrieve some symbol you always need to access at most one storage unit. The symbols in the storage unit are always packed in "little-endian" order i.e the first symbol is in the lowest bits of the storage unit and the last symbol is in the highest bits. The properties in traits type are either types or constexpr static expressions/functions:

- `alphabet_type` - type of the alphabet
- `storage_unit` - type of the underlying storage unit (usually `uint8_t`, `uint16_t`, `uint32_t`, `uint64_t`)
- `alphabet_size` - number of symbols in the alphabet
- `unit_width` - width in bits of the storage unit: $\text{sizeof(storage_unit)} \times 8$
- `bits_per_char` - number of bits per symbol: $\lfloor \log_2(\text{alphabet_size}) \rfloor$
- `letters_per_unit` - number of symbols that can be packed into one storage unit: $\left\lfloor \frac{\text{unit_width}}{\text{bits_per_char}} \right\rfloor$
- `unit_used_bits` - number of bits used in the storage unit (for alphabet sizes that are not a power of two, some bits must stay unassigned): $\text{letters_per_unit} \times \text{bits_per_char}$
- `unit_leftover_bits` - number of bits leftover in the storage unit: $\text{unit_width} - \text{unit_used_bits}$
- `char_mask` - mask for the last symbol in the storage unit
- `rev_char_mask` - mask for the first symbol in the storage unit
- `used_storage_mask` - mask for the used bits in the storage unit
- Conversion functions for converting between storage unit and symbol: `letter_to_symbol`, `letter_to_char`, `letter_complement`
- Utilities for generating k-mer masks: `kmer_mask`

The figure below 1.1 shows an example of how I bitpack DNA4 sequence in storage units.

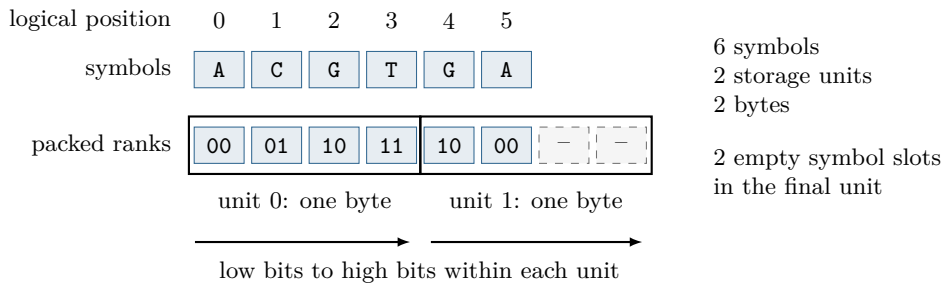


Figure 1.1: Packing `ACGTGA` with four DNA4 symbols per byte. The dashed slots are padding, which occurs at the end of the sequence as rarely sequence size is multiple of `compact_sequence_range<R>::traits::letters_per_unit` so they are not part of the sequence.

Template instance explosion

In BioseqDB I tried to allow user to specify any storage width that is power of two (8, 16, 32, 64) and alphabet (currently supported is DNA4 and DNA5). Because both of parameters are constexpr template parameters, this leads in some parts to creation of hundreds separate versions of pipeline. This slows downs compilation times and led to creation of absurdly large BMIs resulting in multi-gigabyte Clang .pcm files. Resulting binary still is optimised down to more convenient size, however I put lots of burden onto the compiler and linker. The goal of user-supplied storage widths was to observe how they affect performance and if they affect it at all. Currently in some parts of the pipeline I still cast the stored types to uint32_t, this happens because some algorithms like SIMD chaining require larger widths to store indices and scores. Extracting symbols directly from storage unit is a complex process involving lots of bitwise operations, so at this time most algorithms (SIMD/GPU) cannot operate directly on packed sequences as this would require lots of additional engineering effort. I added conversion helpers like one presented in the listing below 1.1 that convert requested ranges of sequence into non-bitpacked sequence that is easier to work with. This operation is done on demand and stored in separate memory pool region, so pipeline doesn't have to convert sequence from bitpacked to non-bitpacked format multiple times. Directly unpacking sequence inside pipeline at the latest point in time is potential opportunity for future improvements.

The following example uses actual code snippet from BioseqDB - in this case the caller owns `output`, which has at least $last - first$ bytes, and passes bounds satisfying $0 \le first \le last \le N$.

Listing 1.1: A consumer requests only the ranks it needs.

```
template<supports_compact_sequence_range R>
void read_fragment(const compact_sequence_range<R>& sequence,
                   std::size_t first, std::size_t last,
                   std::span<std::uint8_t> output) {
    // One rank per output byte, in forward order.
    sequence.template to_encoded_into<false>(output, first, last);
}
```

1.3. Slices keep logical coordinates

Postgres support array subscription operations natively, so I decided to support slicing as well in first version of BioseqDB.

A slice selects positions $[a, b)$: it includes a and excludes b. BioSeqDB uses zero-based positions, so $S[0]$ is the first symbol of the slice.

0-based indexing

This contradicts the fact that Postgres slices are 1-based, but it wasn't intentional decision. I had limited experience with Postgres array functionality and this realisation came only after most of the support was already implemented. What is more interesting that SeqKit - tool for operating on FASTA/Q files use 1-based indexing Shen, Sipos, and Zhao, SeqKit usage documentation: the subseq subcommand. This incompatibility might be something to reconsider in the future.

I provided utility functions for common operations like reversing and reverse-complementing slices as well, example available in listing 1.2.

Listing 1.2: Zero-based slices and sequence transforms in SQL.

```
WITH v(s) AS (
  VALUES ('ACGTAACCGTTA'::bioseq.seq('dna4', 32))      -- ACGTAACCGTTA
)
SELECT s[2:10]::text AS middle,                        -- GTAACCGT
       ((s[2:10])[1:5])::text AS nested,               -- TAAC
       s[-4:]::text AS suffix,                         -- GTTA
       bioseq.reverse(s[2:10])::text AS rev,           -- TGCCAATG
       bioseq.reverse_complement(s[2:10])::text AS rc -- ACGGTTAC
FROM v;
```

The bounds follow a small set of intuitive rules simiar to design of slices in Python Python Software Foundation, Python 3 Documentation: Common Sequence Operations:

- A missing lower bound means zero. A missing upper bound means the sequence length.
- A negative bound counts from the end (example: for length 12, -4 means 8) and bounds outside the sequence are clamped to its ends.
- An upper bound below the lower bound gives an empty slice, so `s[4:2]` has length zero.
- User can extract any element (symbol) from slice, which has separate SQL type `LETTER`. An out-of-range element, such as `s[12]` in this example, gives SQL `NULL`. An explicit null slice bound also gives `NULL` i.e $s[a:\text{NULL}]$ is equivalent to $s[\text{NULL}:a]$ and is also equivalent to $s[\text{NULL}:\text{NULL}]$ (all of them result in `NULL`)

The slicing can be implemented by old holding slice coordinates, without actually loading any sequences, by very simple coordinates arithmetic visible at figure below 1.2 For a slice view $V = S[a,b)$, view position j refers to $S[a+j]$. A second slice therefore combines its offsets directly:

$$V[c,d) = S[a+c, a+d).$$

The example's $S[2,10)[1,5)$ is $S[3,7)$, which reads `TAAC`. Reversal is represented by reversing start and end coordinates: For $W = \text{reverse}(S[a,b))$ we have:

$$W[j] = S[b-1-j], \qquad W[c,d) = \text{reverse}(S[b-d, b-c)).$$

Complementing operation leaves the coordinates unchanged, but alters binary flag representing if the slice is complemented or not.

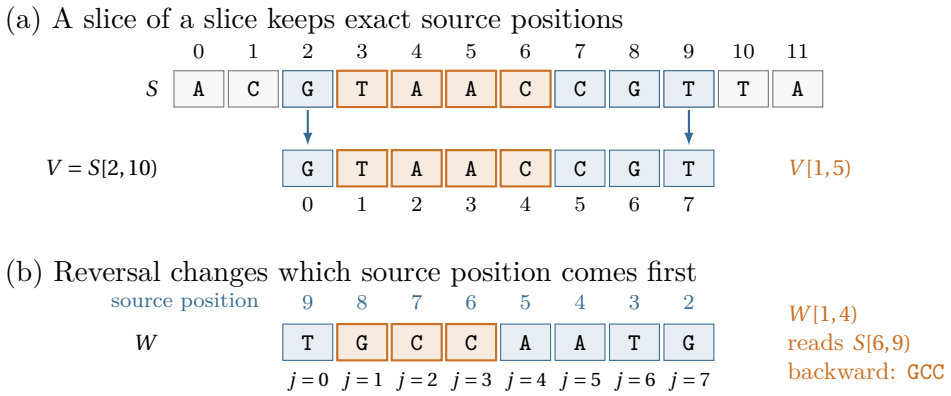


Figure 1.2: Blue cells form the first slice and orange cells show a second slice

1.4. Composing sequences without joining their payloads

The slice model I showed is very primitive. If we add ability to concatenate sequences we can represent any slice as a tree of pieces that represent operations on original sequence. In BioseqDB this tree refers to Postgres operators and datums.

The slice tree is constructed from the following elements:

- A source leaf describes interval in the source sequence.
- A literal leaf holds literal value referencing Postgres datum with additional metadata.
- A branch node that represents concatenation of two children. It holds total length, piece count (total number of leafs in subtree) and tree height. We collect total length to estimate how long the memory region in the pool should be to accomodate the materialized sequence.
- An edge that connects a node to its child. It holds reverse and complement flags, which affect the order of traversal and the interpretation of the leafs.

Figure 1.3 illustrates concatenation of three pieces including two source leaves and one literal leaf:

$$Z = S[2,6) \parallel \mathrm{reverse}(S[8,12)) \parallel \texttt{AC},$$

The result is `GTAAATTGAC`, with length $4+4+2=10$.

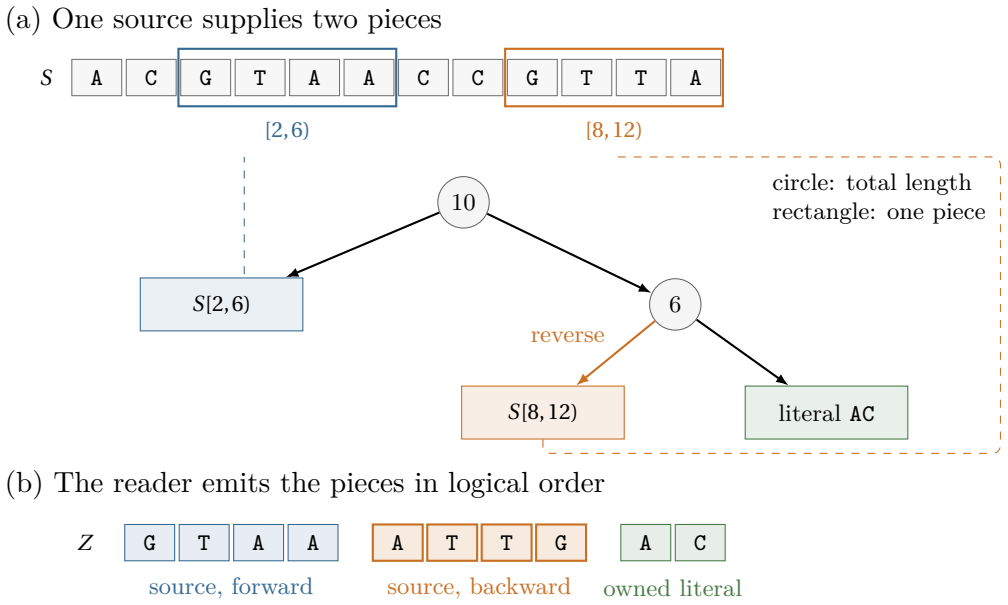


Figure 1.3: A piece tree describes how BioseqDB process slices of a sequence.

Each of the operations can easily be implemented by traversing the tree: concatenation joins roots and restores tree balance, slicing splits at the two bounds and leaves the middle part, reversing just as complementation flipps flags on the edges.

Reading actual slice content in BioseqDB is implemented using `lazy_sequence_view<A>` of table 1.1. It's built for the duration of a single call and the tree operations read only metadata from sequence locators (such as lengths). This means that we don't need to fetch large relations with sequence content and slicing is very performant. In the pipeline specific stages (like SIMD or GPU kernels) can request contiguous region of sequence and BioseqDB will materialize it into its own packed buffer first.

I implemented concatenation support for SQL text literals as well. This means that we can concatenate sequences with text directly without casting them, but this has it's own limitations:

- Slicing and reversal can operate on text without no known alphabet because they need only positions and order.
- Joining a known-alphabet sequence with a text literal validates the literal against that alphabet, so we require to go through entire literal (which shouldn't be a problem as we don't expect very long literals as this should be done via loading FASTA first into the table)
- Reverse-complement requires known alphabet with a complement rule for every symbol in that alphabet.
- Two different alphabets are not converted automatically to one another and this will result in an error.

1.5. Reading only what the algorithm requires

The storage can be provided in different forms as explained previously in the first section 1.1 In BioseqDB main implementation of Postgres-based storage is done via `lazy_sequence_`

`view<A>` and `retained_lazy_sequence_view<A>` mentioned in table 1.1. These types holds pointers to functions that implement the storage and represent the following operations:

- Report sizes of sequences
- Copy selected intervals into caller-owned memory, which in this case can be GPU-owned memory in GPU-accelerated pipeline stages
- Reading successive chunks
- Checking if the compatible interval was already materialized and can be borrowed (caching)

Internally I distinguish several access policies which are specified by the pipeline stage, they are the following choices:

1. Stream. If the algorithm accepts a stream, read the selected interval in bounded chunks and save it to reusable scratch memory (large memory pool)
2. Borrow. If the algorithm needs packed data, first check if's already in cache and safe to borrow.
3. Pack. In case if borrowing is not possible, algorithm needs to request new memory permit (this is routed through custom pool allocator). Caller is then responsible for this memory chunk.

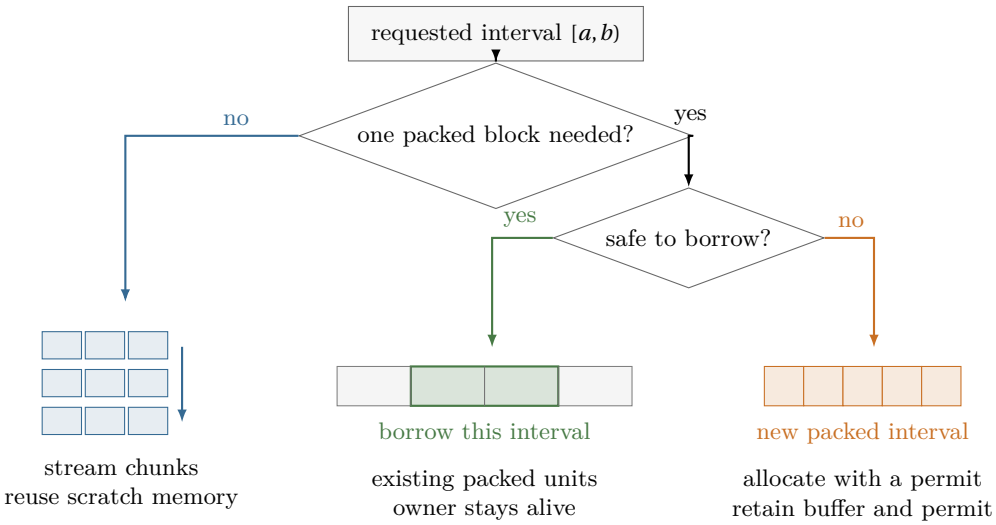


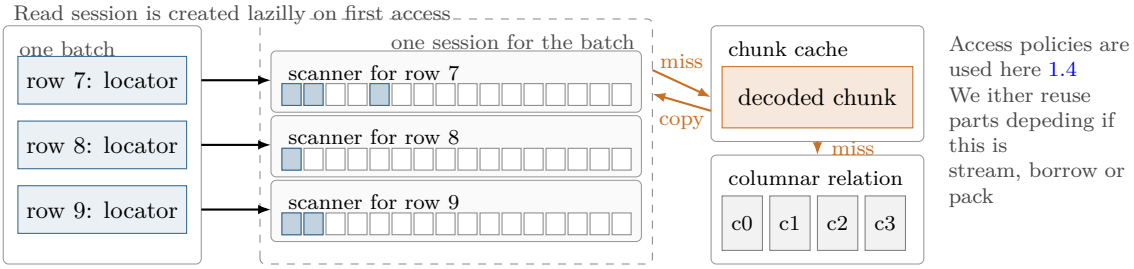
Figure 1.4: Simplified view of different access policies

Borrowing

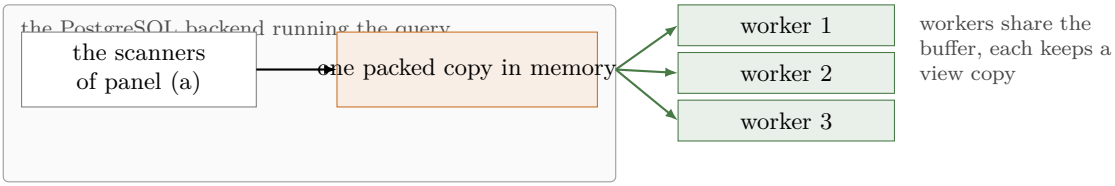
Borrowing can fail if the region for example starts in the middle of storage unit in packed sequence, then we cannot reuse that memory chunk as it's misaligned

At this point I need to describe lifecycles of various entities to illustrate how reading is performed. Figure 1.5 shows how what is happening under the hood during the scan:

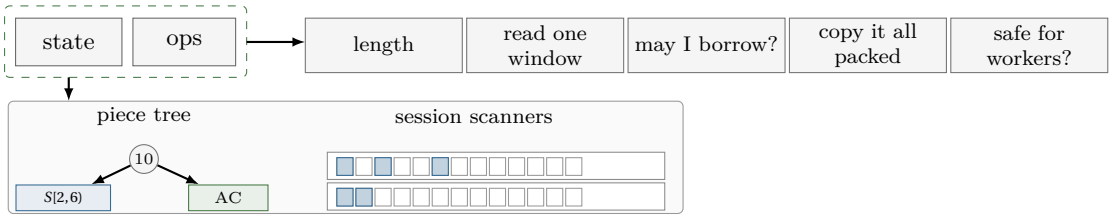
(a) One batch of rows gets reads session with multiple scanner. Each scanner has it's own slots for caching.



(b) Reading is single-threaded with distribution to multiple thread workers



(c) View copy sent to workers holds two base pointers (and release callback)



(d) Timeline of the scan

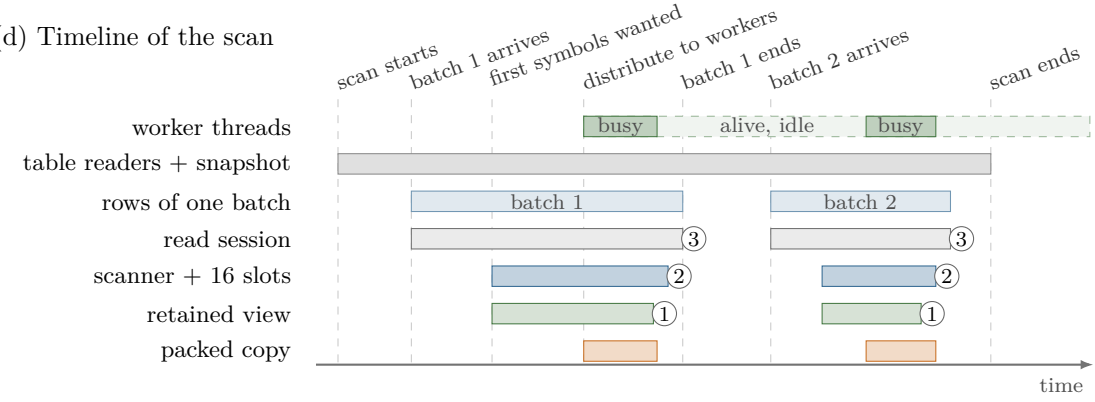


Figure 1.5: Illustration of structures used and operations performed on sequences during scan lifecycle

- (a) Each row of a batch gets its own scanner with 16 fragment slots for caching. A miss looks in the decoded chunk cache (this is lower-layer cache for sequences chunks which may store multiple unrelated sequences and is discussed in later chapters), then in the columnar relation, and copies one fragment into a slot.

- (b) The incoming queries are grouped in batches. Each batch has it's own session, which is destroyed when batch processing is finished. The session is different entity to the chunk cache so relation isn't reopen when batch is finished and we keep large sequence chunks still in memory. At the start of the batch the backend first builds one packed copy in memory and distributes that copy to workers from the pool. It's important that in this architecture we only have one backend thread which is responsible for reading relation. This means that the actual pipeline is run by single thread that submits various work to worker pool. I needed to read the relation on a single thread to avoid race conditions on PostgreSQL's process-local state which is not thread-safe - PostgreSQL starts (forks) a new server process for each client connection, and even parallel SQL starts extra background worker processes rather than extra threads inside the same backend.[4] The same read could still be done in parallel by giving each reader its own backend: one way would be to fork a background worker with a database connection (which in this context is problematic as forking with pool running will result in segfaults) or open a second client session through libpq (libpq is thread-safe when each thread uses its own connection).[5] I did not take those paths for fragment reads during a query as it would complicate the implementation and this is something to be done in the future.

- (c) A view is built from two pointers - `state` is a piece tree together with the session that holds one scanner per source row and `ops` contains length, two flags and two function pointers. Providing function pointer serves as abstraction layer to divide non-Postgres code with actual Postgres implementation. Using templated types could be possible here, but they still would need type erasure. A retained view additionally holds a release callback (C-style destructor)

- (d) At every batch end, in this order: (1) release retained views, (2) finish each scanner so its slot copies go, (3) drop the session's memory. The worker pool and the snapshot naturally outlive the batch as discussed here. It might happen that we encounter an error and failing query skips C++ destructors, so steps 1–3 also run from a memory context callback (retained view release callback)

Panel (d) of Figure 1.5 has a gap between the end of batch 1 and the start of batch 2. In that gap the backend collects the next tuples from the relation (metadata and sequence locators, not entire sequences), applies the query's `WHERE` clauses, and places surviving tuples in the dispatch buffer. When enough tuples are collected, it submits the next batch. That collection still uses one relation-reading thread per PostgreSQL process. There's however support for `bioseq.scheduler.postgres_parallel.enabled` for any query that do not use alignment. The reasoning behind that is that only the thread pool wasn't designed around being multi-process safe and current statistics collection doesn't support it. The scans that do not use alignment are marked parallel-safe. This means that example shown on lsiting below 1.3 can be executed by postgres parallel scan.

[4] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Architectural Fundamentals; PostgreSQL Global Development Group, PostgreSQL 17 Documentation: How Parallel Query Works.

[5] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Background Worker Processes; PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Behavior in Threaded Programs.

Scans in details are covered in the next chapter 2.1 in detail.

Listing 1.3: Simple table read

```
SELECT id, bioseq.to_text(s[0:8]) FROM t;
```

Parallel scan

This is potential optimisation in the future. Statistics collection can be easily made multi-process safe by implementing reduce function, while thread pool design needs to be revisited and reimplemented.

Chapter 2

PostgreSQL API and custom scans

2.1. Designing SQL API for an aligner

In the previous chapter I presented how the sequence abstrations work and how they interact with the workers, we also had opportunity to look on how batching is done on figure 1.5. This chapter focuses on the main problem - how I incorporated this infrastructure into Postgres.

First, I want to discsuss how we can design SQL API for the aligner. Currently BioseqDB can be used in the following way:

Listing 2.1: Search once and return the fields of each hit.

```
CREATE TABLE targets (
  id bigint PRIMARY KEY,
  seq bioseq.seq('dna5', 64)
) USING bioseq;

SELECT t.id,
       bioseq.dp_score(t.seq) AS score,
       bioseq.target_start(t.seq) AS start,
       bioseq.cigar(t.seq) AS cigar
FROM targets AS t
WHERE t.id > 20
  AND mmsearch(t.seq, $1::bioseq.seq('dna5', 64));
```

The first `mmsearch` argument is the target sequence and the second is the query, - here query is supplied by `$1` (text parameter in prepared statement). The `score` alias means the dynamic-programming score of the current hit, CIGAR string describes the alignment and we also request position of the match in the sequence coming from target table.

One insteresting observation is that naturally each alignment can produce multiple hits, so this operation can return multiple rows for a single input row. We intuitively expect that there will be two kinds of selected expressions: Some of them will not depend on alignment (in this case it's primary key), so they will be the same for all these hits, while some of them like score and cigar will be different. Figure 2.1 illustrates that alignment should be set-returning function.

The first idea for implementation of such search was the simplest one - implementing alignment as callable Postgres function, which is very easy to do in C++. The function accepts query as raw SQL string:

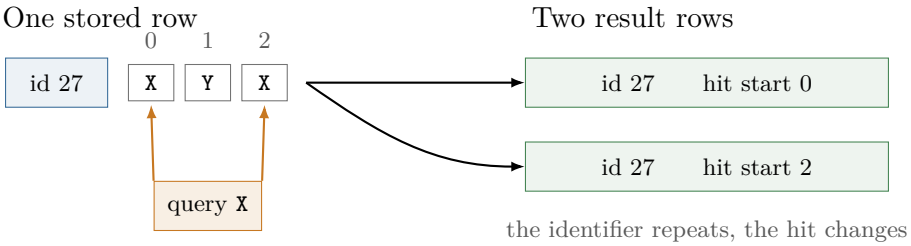


Figure 2.1: Searching `XYX` for `X` gives two occurrences, so the function needs to "clone" metadata of tuple (id key in this case) and for each generated tuple fill-in the alignment hit position.

Listing 2.2: An early interface sketch passes SQL as text.

```
SELECT * FROM search_sql(
  'targets',
  'SELECT id, seq FROM reads WHERE id > 20'
);
```

This however introduces serveral problems:

- Which is the column we want to align? We can make a choice to hardcode column name to be `seq` at all times, but this limits flexibility heavily. Other choice is to allow passing the aligned column name as a parameter, which resolves this problem.
- How do optimiser know about inner query? As the inner query can be complex the optimiser doesn't know about it. We can call libpq inside the function, which can optimise the query separately, but then we should change it depending on the context: you can imagine very trivial case that the outer SELECT statement only selects primary key for some reason, so we don't have to do any alignment. In that case we would expect the function to do this optimisation, but this means that its implementation needs to do the same work as Postgres optimizer.

Listing 2.3: An outer condition cannot automatically filter a hidden scan.

```
-- Hypothetical SQL-string interface
SELECT h.*
FROM search_sql('targets', 'SELECT id, seq FROM reads') AS h
WHERE h.target_id = 27;
-- The function must itself restrict the hidden target scan.
```

The listing illustrates point stated by me: the function sees the target table through the string `'targets'`, but it's used in a context, where we will do only one alignment. If the function doesn't interpret its context, then it will perform alignment on entire table. The solution could be to pass this reponsibility to the user, but this generates huge inconvenience - for more complex queries, user would need to transfer SQL statements to and from the raw text pased to search function and manually be responsible for optimisation. In that case this should mean that when we forgot to do that the different is enormous - the statement will align two tables instead of one (could be orders of magnitude slower).

SQL already has a direct way to express one-to-many output. A set-returning function in a `LATERAL` join could return a record for every hit.[1]. The `LATERAL` clause lets that call refer to the current target row.[2]

Listing 2.4: A different API could return explicit hit records.

```
-- Alternative API sketch, not an existing BioSeqDB function
SELECT t.id, h.score, h.start, h.cigar
FROM targets AS t
CROSS JOIN LATERAL align_hits(t.seq, $1) AS h
WHERE t.id > 20;
```

This means that optimiser can work with queries, but user needs to do `CROSS JOIN LATERAL` for each alignment, which adds lots of boilerplate code. Additionally we loose ability to conditional evaluation in this case. Sometimes alignment may not require CIGAR which is more expensive operation as it requires running base alignment and the `SELECT` clause may only specify score. This prevents us from evaluating alignments differently based on what fields are needed.

To more easily indentify what parts of the alignment are needed (because otherwise we would need to track where the record type is used and how), I decided to separate returned record type into calls to functions. The function takes sequence as argument and returns some information about alignment. This concept is presented below.

Listing 2.5: Hidden state does not establish an evaluation order.

```
-- Hypothetical functions with shared hidden state
SELECT id, last_score(seq)
FROM targets
WHERE save_search(seq, $1)
  AND last_score(seq) > 100;
```

Now the fields are easy to track as we can directly look for a call to a function in the AST expression tree. However if the extension implemented only Postgres function this example couldn't work. We would need to pass some internal state between function calls - `save_search` would need to save some pointer to lazy execution object invisible to the user so it can be later called by `last_score`. This way we compute only what we need. Declaring these functions `VOLATILE` limits some planner transformations, but does not guarantee execution order between them[3] This means that it's not possible to reliably pass this hidden lazy object between functions.

We can see that example above is much cleaner than `LATERAL` join and easier to understand, but to implement it we need to tell Postgres how we want to execute this query - in which order expressions are evaluated and create our own tuples inside. Historically before PostgreSQL 9.5 only way this could be done was via foreign-data wrapper interface, which treated remote table as opaque object. Later Postgres version introduced custom scans and table access methods. There's overlap in all these interfaces, but the main advantage of TAMs and custom scans was that the logic wasn't only tied to one specific table and there was deeper and more cleaner integration with the executor.

[1] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Set Returning Functions.
[2] PostgreSQL 17, `LATERAL` Subqueries.
[3] PostgreSQL 17, Function Volatility Categories.

1. PostgreSQL 9.1 added foreign tables and the foreign-data wrapper (FDW) API, a partial implementation of SQL/MED.[4] PostgreSQL 9.3 added `postgres_fdw`, the in-tree wrapper that queries another PostgreSQL server.[5]

2. PostgreSQL 9.5 added custom path and scan methods, so an extension can insert its own executor node into a plan.[6]

3. PostgreSQL 12 added table access methods (TAM) through `CREATE ACCESS METHOD …TYPE TABLE`. Heap remains the default TAM.[7]

Table 2.1 summarises the split.

Table 2.1: PostgreSQL interfaces that can supply rows to a query.

	FDW	Custom scan	TAM
First in core	9.1 API, 9.3 `postgres_fdw`	9.5	12
Attaches to	a foreign table	a plan node	a local table
Stores bytes here	no	no	yes
Typical work	fetch remote rows	replace a scan or join	layout, insert, vacuum

The BioseqDB implements the following components:

- Custom scans - allow to implement syntax suggested here with selective evaluation of the pipeline, efficient batching integrated with optimizer, so I rely on optimisations from Postgres as well. Theoretically custom scan can replace scans to any table, but I don not offer support for tables other than BioseqDB tables (created using `CREATE TABLE USING bioseq` TAM)

- Table access method - stores data in columnar format, so I can load large chunks of metadata and sequence in single row fetch

2.2. Integrating Postgres

2.2.1. Designing SQL API

A full text search interface (`pg_search`) from ParadeDB was an important motivation for the SQL API design of BioseqDB. ParadeDB is an extension written in Rust that uses custom text storage engine under the hood. I deeply inspected its source code to understand how we can implement our own syntax in BioseqDB. ParadeDB resolves similar problem as FTS can also return multiple hits for a single query.

ParadeDB score expression
score_from_relation 30

To return custom results instead of relying on Postgres evaluating the function (We don't want to let Postgres call actual `bioseq.score()` function, but rether provide it ourselves. ParadeDB has similar syntax and the same problem) ParadeDB replaces score expressions

[4] PostgreSQL Global Development Group, PostgreSQL 9.1 Release Notes; PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Foreign Data.

[5] PostgreSQL Global Development Group, PostgreSQL 9.3 Release Notes; PostgreSQL Global Development Group, PostgreSQL 17 Documentation: `postgres_fdw`.

[6] PostgreSQL Global Development Group, PostgreSQL 9.5 Release Notes; PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Writing a Custom Scan Provider.

[7] PostgreSQL Global Development Group, PostgreSQL 12 Release Notes; PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Table Access Method Interface Definition.

with typed placeholders. These placefolders are filled from the current search result before projection. ParadeDB placeholder replacement inject_placeholders 493 Similar design was used in BioseqDB - described in Section 2.3.

Listing 2.6: ParadeDB pg_search examples

```
-- Some examples for pg_search
SELECT id, title, pdb.score(id) AS score
FROM documents
WHERE body ||| 'postgresql search'
ORDER BY score DESC, id;

SELECT id, pdb.snippet(body) AS excerpt
FROM documents
WHERE body &&& 'postgresql indexing';
```

Listing 2.7 shows how similar ParadeDB's problem is to my initial API designs.[8][9]

2.2.2. Plan lifecycle

To implement custom scan, we must understand different objects in Postgres:

1. Path: Describes possible inputs, estimated work, output row count, and ordering. The planner compares several paths and keeps the cheapest one[10].[11] The Genetic Query Optimizer is used only when the number of joins exceeds `geqo_threshold`. A single-table `BioseqScan` is ordinary cost comparison, not GEQO. The path is later converted into a plan. I register my own paths so PostgreSQL knows how to access the table.

2. Plan: In our case `CustomScan` contains child plans, expression trees, and descriptions of the search.

3. State: When plan is executed, postgres creates its state via my code. `CustomScanState` holds the input cursor, alignment buffers for sequences and metadata, and slots for generated hit. Postgres will call scan repeatedly to get new rows, so I use the state to hold any information that needs to be preserved between calls.

All types of Postgres API described in table earlier 2.1 mainly rely on callbacks (in Postgres called hooks). Extension fills the structure with function pointers and let postgres know which call is handled by which function. In this case PostgreSQL calls `PlanCustomPath` when it converts a selected custom path into a plan and later calls `CreateCustomScanState` to create the executor state[12] Figure 2.2 shows when PostgreSQL calls those hooks, when `BioseqScan` runs, and when the columnar table access method returns a tuple.

`ProcessUtility_hook` is used for utility statements such as `CREATE`, `DROP`, `COPY`, and `CREATE INDEX`. This is useful to forbid user from doing certain operations (Currently I only use it to log statements).

The TAM stores the user table in columnar format: A long sequence is not kept in that table row, only a locator is stored there. The sequence itself is stored in companion fragment

[8] Pigsty, pg_search: full text search for PostgreSQL using BM25.
[9] Neon, The pg_search extension.
[10] PostgreSQL Global Development Group, PostgreSQL Documentation: Planner/Optimizer.
[11] Conway, Inside the PostgreSQL Query Optimizer.
[12] PostgreSQL 17, Creating Custom Scan Paths and Creating Custom Scan Plans.

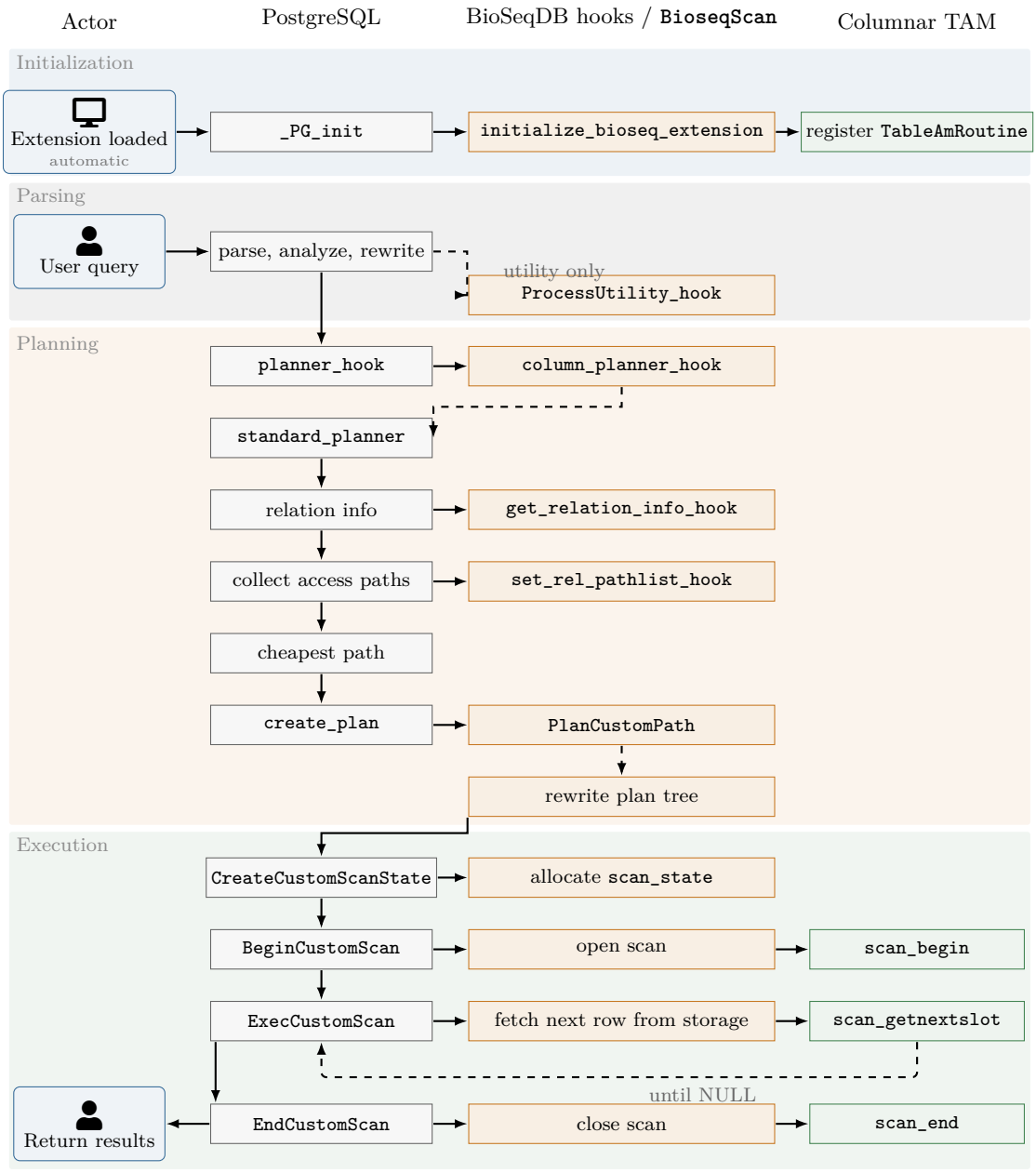


Figure 2.2: Lifecycle of `BioseqScan` and the columnar table access method. Bands mark exeuction states: initialization, parsing, planning, and execution. Boxes have various colours: Grey boxes are PostgreSQL, orange boxes are BioSeqDB hooks or the custom scan and green boxes are TAM callbacks. Arrows follow the order in which Postgres calls specific hooks. `column_planner_hook` calls `standard_planner`, then rewrites the finished plan, so we don't explicitly control GQO, but can affect it using costs functions, registering paths and even after plan is choosen, we have complete constrol over it (we are free to rewrite its parts). The scan is calling TAM to fetch rows from storage and returns them to Postgres until it returns NULL (no more tuples to fetch)

sub-tables, and all of them use the bioseq TAM as well. More on storage format is discussed in Chapter 3.1. I called the process of fetching fragment from locator "hydration" and TAM is responsible for it.

BioseqDB have control over the plan using `column_planner_hook`. This hook is called when PostgreSQL needs to perform planning and SQL query is already parsed. First BioseqDB hook validates `mmsearch` options, then it calls `standard_planner`. `get_relation_info_hook` and `set_rel_pathlist_hook` hooks, where we set options like disabling parallel scans or index-only scans and register custom access paths for cust scan, run inside that call. Then GQO is run and finds the best path that needs to be converted into a plan, so the `PlanCustomPath` is run. When `standard_planner` returns we are still in the `column_planner_hook`, so I'm free to rewrite plan tree freely. The rewrite mutor performs the following changes:

- Enable vectorized execution for simple aggregates Using modernized Citus/Hydra columnar vectorized optimisations I allow simple aggregates on metadta columns to be vectorized (processed in batches) for example: `COUNT(*)` or `SUM(foo_bar)`. If current node is Postgres `Agg` it will be replaced by custom aggregator `VectorAggNode`. If this plan has `VectorAggNode` as parent, then I return batches of rows instead of single tuples. Rewrite adds falgs to mark that the plan needs to change its output mode. Additionally Postgres `Agg`

- Wrap an index scans on columnar tables PostgreSQL can still choose a metadata index on a `USING bioseq` table and use external `IndexScan`. If that happens this index can read some tuples that shouldn't be red (columnar TAM allows removing rows. If that happens row is marked in bitmask, but index scan can go around it and still read the row, because it's physically there). To prevent that the `IndexScan` is replaced by custom `BioseqIndexScan` that hounours bitmask.

As entities like `bioseq.score(id)` are still ordinary functions, I discourage Postgres by using prohibitive execution costs[13]. The function implementation itself mostly just emits an error, because it shouldn't be ever called. Instead `ExecCustomScan` will using special Postgres expression nodes i.e `Const*` node to tell Postgres that the scan itself calculated the function's value and it shouldn't evaluate it by itself.

2.2.3. Different types of scans

When I talk about a custom scan I focus on `BioseqScan`, which is the primary way to read a `USING bioseq` table, but there are other types of custom scans used for different purposes in BioseqDB, some of them already mentioned earlier. All scans types are the following ones:

- Dedicated join support: `BioseqBatchJoin` is a custom scan with two children: one child is the target plan, the other is the query plan. Normally Postgres would use nested loop join that would read one target and one query and perform alignment for each pair, however we want to avoid this behaviour as target index can be loaded once (running `BioseqScan` for each pair would cause index for one table be loaded, then unloaded and loaded again for each query). This scan, dedicated for joins, prepares that target once (open the stored minimizer reader, or build index manually if not available) and runs the same logic as `BioseqScan` for each query in the batch. Dispatch buffer has separate API for joins, but they mostly effectively do same processing as for `BioseqScan` (collect queries in batches etc.)

[13] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Function Optimization Information.

- Slices across operations: `BioseqSliceProject` wraps a child plan when the query asks for a slice or other piece-tree operation, for example `SELECT seq[100:200]` or `mmsearch(t.seq[a:b], q.seq)`. It evaluates those expressions into expanded piece-tree Datums (Section 1.4) and sends them to Postgres as read-only expanded objects in a virtual slot. This means that the piece-tree is stored in the tuple itself and can be passed around other scans, joins etc. without translating it into a string yet: later `BioseqScan` or `BioseqBatchJoin` can align that slice lazily. When user requests a slice as in `SELECT seq[100:200]` and it doesn't go through other operation nodes, then it will be "hydrated" from piece-tree into actual byte sequence.

- Support for indexing: `BioseqIndexScan` is the wrapper around `IndexScan` described above, that allows to correctly read deleted columnar rows (support for bitmasks).

- Sorting by sequence: `BioseqBatchSortScan` is only for sorting by the sequence itself, for example `ORDER BY seq` or `ORDER BY seq DESC`. The planner will choose it only when there is exactly one `ORDER BY` key and that key has type `bioseq.seq`, and only when `bioseq.planner.sequence_sort_scan.enabled` is on (`ORDER BY seq, pk` and `ORDER BY seq::text` are still ordinary PostgreSQL sorts) This is experimental feature to allow faster sorting by sequence without the need to flatten entire sequence first. The custom scan buffers whole child rows, compares the sequence column as lazy views (not as printed text), and emits those same rows in the resulting permutation.

2.3. Dispatch buffer as glue between Postgres scan and alignment pipeline

I still need to explain how exactly `CustomBioseqScan` works with worker-architecture shown in Figure 1.5. The main abstraction here is the dispatch buffer which is reposible for transferring data between Postgres and workers.

Figure 2.3 shows how Panel (a) of Figure 1.5 looks in the context of Postgres scan. `BioseqScan` asks for the next row from TAM. This row is placed into dispatch buffer (locator only) and dispatch buffer holds all the structures presented before 1.5 - scanners. The dispatch buffer code then handles all view copies to the workers.

The collect loop is the gap between batch 1 and batch 2 in panel (d) of Figure 1.5. `BioseqScan` may call the user-table TAM many times before it returns to fill in the dispatch buffer. The backend thread that owns the scanners performs fragment reads, while workers only see the view copy described in panel (b).

2.3.1. Returning results back to Postgres

Now I not only need to build the tuple but somehow explain to Postgres that certain values correspond to requested values of function calls like `bioseq.dp_score(t.seq)`. PostgreSQL evaluates output expressions through an expression tree. Before returning its first alignment hit, the executor walks this tree and replaces the function calls with `Const` nodes. It writes the hit's values and null flags into these nodes and rebuilds the output-expression evaluator so that it reads the changed fields. This makes Postgres know that we already computed these values. The idea was created by analyzing how ParadeDB provides its search results

ParadeDB current-result score
exec_custom_scan 1784

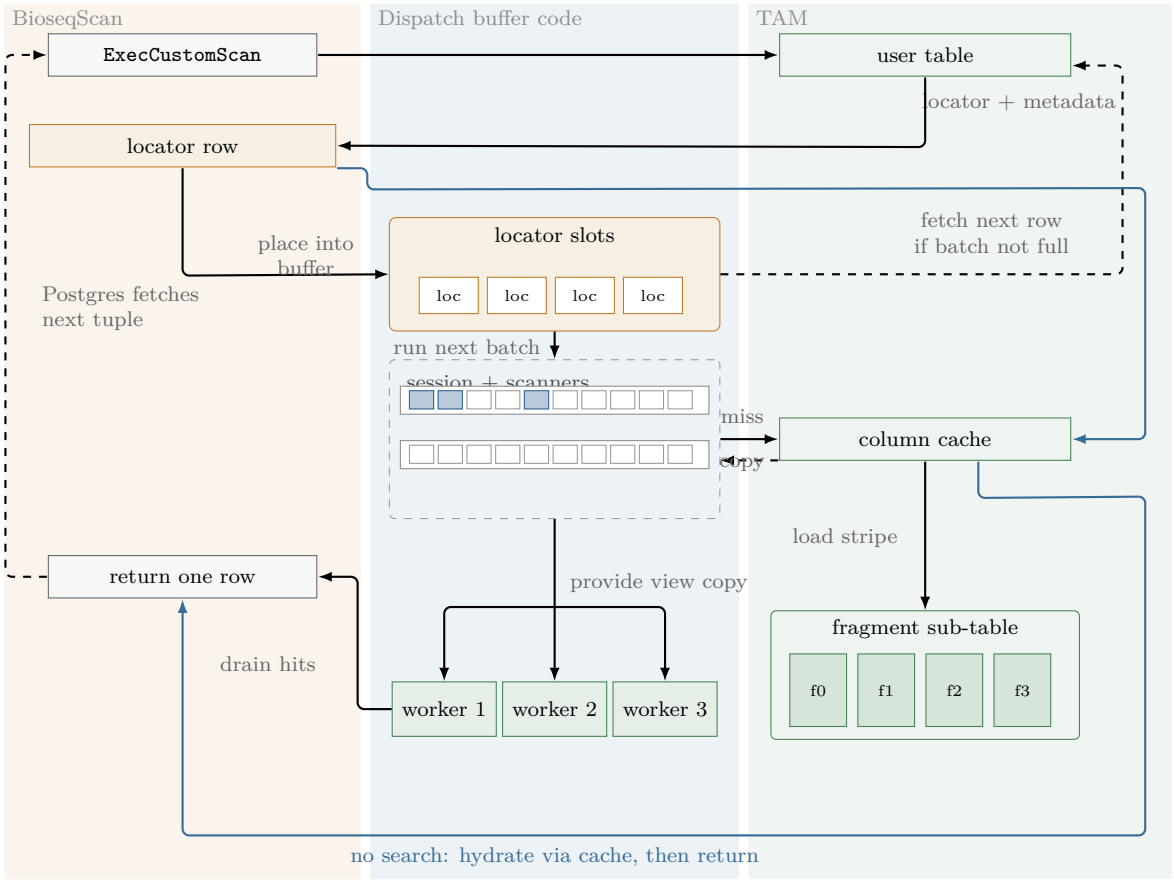


Figure 2.3: Inside `ExecCustomScan`, the following things happen: `BioseqScan` holds dispatch buffer and fetches a locator handles it to the buffer logic and retrieves results from it. The dispatch buffer stores locators, then builds a session that holds scanners. A scanner miss reads the fragment sub-table through the TAM column cache and copies the fragment into a slot. Workers receive a view copy from the scanners. Black arrows show the normal alignmen execution path, however it might happen that we don't perform alignment and just want to return the sequences itself. In that case the flow uses blue arrows (there's no `mmsearch` in the query) and `BioseqScan` hydrates the sequence row through cache and just returns it without interacting with dispatch buffer. Dashed arrows repesent loops here.

Algorithm 1 How to return tuple

1. Restore the original columns.
2. For every requested BioseqDB function like `score(seq)`, `target_start(seq)`, `cigar(seq)` and so on, write its field from this hit into the corresponding typed value node.
3. Rebuild the evaluator, then evaluate the output expressions.
4. Return the resulting SQL row

2.3.2. Supporting joins

A join can lift a helper such as `bioseq.score(t.pk)` out of the scan that produced the hit and an outer join can also emit a null row when one input has no match.

The `Const` rewrite from the previous subsection is nice execution trick inside `BioseqScan`, but the join planner does not see those constants. It still sees an ordinary function call, so PostgreSQL may copy that call onto the other input of a `LEFT JOIN`, or evaluate it again in a `Join` or `Aggregate` node.

PostgreSQL's `PlaceHolderVar` exactly to resolve this issue, however it's suprisingly low volume of documentation about it. I only found old mailing list archive, where it's described as an anti-COALESCE: the expression keeps its value on a matching row and is forced to `NULL` on a missing row.[14] The `PlaceHolderVar` records the relation at which the expression must be computed and the value then rises through the plan like a column. When an outer join nulls that relation, the placeholder becomes `NULL` (a `Const` cannot do this as it's always the same value).

PostgreSQL expression placement
PlaceHolderVar 2750

Without that wrapper the following could happen:

- The inner scan of a `LEFT JOIN` receives `bioseq.cigar(t1.pk)` even though the helper belongs to `t1` and that scan rewrites a function it does not own.
- A `Join` or `Aggregate` node calls the C++ stub. As I mentioned the function does not compute an alignment (the actual value is only scan-provided), so we get an error.
- Mishandling of `NULL` values:An unmatched outer-join row keeps a filled score instead of SQL `NULL` or `ORDER BY bioseq.score(t.pk)` after a join can sort on all-null keys.

ParadeDB uses the same mechanism for `pdb.score` and its planner function wraps the call in a `PlaceHolderVar` when the query has a join or an aggregate, so the custom scan keeps emitting the score and upper nodes do not re-run the function.

ParadeDB score placement
placeholder_support 314

In BioSeqDB I do this using `SupportRequestSimplify` hook as `bioseq_placeholder_support`, where we kip placeholders when there's no joins or aggregates.

2.3.3. Dispatch buffer internals

The main component I want to discuss here is the dispatch buffer: a container abstraction reponsible for retrieving, storing, distributing and collecting of the tuples with results. This requires managing memory and lifetime of Postgres tuples (both incoming from table and outgoing to the user).

Handling tuples

Postgres offsers `TupleTableSlot*` type, which represents reusable holder for a tuple. The Postgres executor keeps one scan slot on the expression node and asks the table access method to fill that same holder on every fetch.

This holder consist of a tuple descriptor, current column values, null flags, and a small operations (functions pointers) table that knows how to read or copy the physical layout currently stored inside it (PostgreSQL Global Development Group, PostgreSQL 17 source: tuple table slots). The tuple descriptor is the struct holding information about the columns schema (how many attributes it has, names of the attributes and their types with additonal metadata).

[14] Lane, Re: what exactly is a PlaceHolderVar?

PostgreSQL offsers the following types of slots Builtin tuple table slot types TTSOpsMinimalTuple 38:

- Buffer heap tuple (`TTSOpsBufferHeapTuple`). The slot points at a row that still lives in a shared buffer page.
- Heap tuple (`TTSOpsHeapTuple`). The slot owns a `palloc` copy of a full on-disk row with system columns (added by Postgres) (PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Database Page Layout).
- Minimal tuple (`TTSOpsMinimalTuple`). The slot owns a compact copy of the user columns only without system columns added by Postgres. Sort, hash, and materialize nodes use this type because it is smaller than a heap tuple.
- Virtual tuple (`TTSOpsVirtual`). Instead of storing entire row it stores arbitrary data in array (usually pointers to the actual values, so we can reference existing tuple).

The TAM has one slot that is used to return tuples and it's overwriten with each tuple fetch (`getnextslot` call). This means that dispatch buffer has to store the tuple somewhere. In a dispatch buffer method called `submit_scan` I allocate a fresh slot with `TTSOpsMinimalTuple`, copy the current user columns with `ExecCopySlotMinimalTuple`, and store it with `ExecStoreMinimalTuple`. The third argument is `shouldFree=true`, so the copied slot owns the bytes and will `pfree` them when the buffer later drops the slot.

Sequence tuples in the dispatch buffer needs to represent lazy-loadable seqences, so during their lifetime they can become of of the 3 types (I need to distinguish them, because in code they are all just minimal tuples, but semantically express different concepts):

- Locator: This object is what is usually stored in the user table and represents entire sequence stored in separate internal table. This is "unhydrated" sequence.
- Expanded: Postgres has a flag `VARATT_IS_EXTERNAL_EXPANDED` (I call this in a code "expanded recipe") to mark the value is not a flat byte array yet, but complex structure in memory. This happens when sequence holds piece-tree (slice graph) 1.4, so we have multiple slice operations to perform if we want to convert this value to flat sequence. This is also different form of "unhydrated" sequence.
- Varlena: Representing "hydrated" sequence fragment, so the slices or locator were converted to some flat byte array.

The alignment pipeline uses first two representations directly and as mentioned before when discussing sequence abstractions 1.1 the sequence will be kept as a locator or piece-tree until the actual data is requested 2.3. Then I load requested bytes and return them directly. However in some cases we need to convert the tuple into 3rd form. If user requests slice projection directly it goes through separate branch of BioseqDB handling where that conversion is performed:

Listing 2.7: ParadeDB pg_search examples

```
SELECT seq[2:3] FROM table;
```

Buffer API

Table 2.2 lists the most important methods of the dispatch buffer that are used during `ExecCustomScan` call. `Snapshot` here means the set of row versions this statement is allowed to see (this is mechanism for MVCC support in PostgreSQL) (PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Introduction to Concurrency Control) and is discussed later.

Table 2.2: Dispatch-buffer methods used by the Bioseq custom scans.

Method group	When the custom scan calls it	What it does
`configure_params`, `replace_executor_snapshot`, `bind_sequence_source_provider`	`BeginCustomScan` after the scan has a relation and a snapshot. PostgreSQL parallel workers (only for queries that do not use alignment 1.3) restore a copy of that snapshot and call `replace_executor_snapshot`. The worker must use its own registered `Snapshot` pointer for later fragment reads.	Stores aligner settings, the user table identity, the borrowed snapshot, and the statement-owned sequence source that scanners will read.
`configure_mmsearch`, `configure_mmsearch_per_row`, `configure_mmsearch_per_row_evaluated`	`BeginCustomScan` once the planner-private list has named the search, or on `ReScan` when a join parameter changes the query.	Records which columns are the target and (if needed) the per-row query, copies a constant query, and turns the buffer into the writing stage.
`submit_scan`, `submit_scan_evaluated`	Writing stage of `ExecCustomScan`, after a TAM row passed the filter conditions. The "evaluated" method is called when parameter in `mmsearch` is an expression that needs to be evaluated.	Copies that row into a buffer-owned MinimalTuple 2.3.3 slot and records its row number, primary key, and sequence locator (or evaluated Datums). Does not run alignment yet.
`ready_to_dispatch`, `has_finished_writing`, `mmsearch_active`	The `ExecCustomScan` loop, after each submit and after each drained hit.	Returns flags indicating whether the physical buffer has reached 8192 rows, whether the current batch has already been aligned, and whether this scan is an `mmsearch` scan at all.
`run_batch_and_build_cursors`	When the buffer is full, or when the TAM source is exhausted.	Streams the buffered locators into the alignment pipeline, then builds an output cursor for retrieving hits.
`read_next_output`	Reading stage of `ExecCustomScan`, once writing has finished.	Returns the next hit together with a pointer to the copied scan slot. The scan then fills the match-field `Const` nodes from Section 2.3 and projects.
`clear_slots`	After a full 8192-row batch has been drained while the TAM source still has rows, and on `ReScan` or `EndCustomScan`.	Frees every copied MinimalTuple slot and the payload arena so the next physical batch can start empty.
Support for joins: `configure_mmsearch_join`, `begin_join_target`, `prepare_join_target_queries`, `set_join_query_batch`, `run_join_query_batch_and_build_cursors`, `finish_join_target`, `reset_join_runtime_for_rescan`, `abandon_join_runtime_after_pg_error_noexcept`	Methods called only by `BioseqBatchJoin` custom scan for custom join (discussed in next section 2.4). PostgreSQL `ReScan` restarts a plan node from its first row, often with new parameters (for example the inner side of a nested loop for the next outer row, or a cursor rewind). `ReScanCustomScan` therefore calls the reset helper so leftover join targets and hits are dropped.	Holds one join target in the buffer, streams inner queries in batches of at most 8192, runs alignment against that stored target, then drops the target. The reset and abandon helpers clear join state without asking the caller to redo `submit_scan`.

The code in `ExecCustomScan` pulls one TAM row, runs filtering conditions (these can reject the row based on metdata) and on success calls `submit_scan` to let buffer copy the tuple. When the copies in buffer reach the set capacity `dispatch_batch_capacity` $8192 = 2^{13}$, `ready_to_dispatch` flag changes to true so the scan calls `run_batch_and_build_cursors`. This method is also invoked when there are no more tuples in TAM source.

`run_batch_and_build_cursors` is where the batch starts and scanners and workers from Figure 2.3 run. It groups the locators already stored in the MinimalTuple copies, lets scanners fetch fragment bytes through the TAM under the Postgres table snapshot object and then distributes the views to workers.

Then the scan needs to read results. Reading stage calls `read_next_output` on every later `ExecCustomScan` to retrieve the next result (this means that in this semantic first call to `ExecCustomScan` takes very long as it needs to align first batch, while later calls return tuples one-after-another from the output cursor until they are exhausted and process repeats). This is the place where we need to set values of nodes and project the tuple to let Postgres construct final results 2.3. After Postgres consumes that projected tuple, the next call drops the previous result slot and asks for the next hit. When the cursor is empty and the TAM source is not yet exhausted, the node calls `clear_slots` and returns to the writing stage (this corresponds to finishing batch and starting collecting rows again)

Execution statistics. Each completed call to `run_batch_and_build_cursors` writes a `pipeline_execution` aggregate onto the buffer, where I collect debugging statistics. `EXPLAIN ANALYZE` reads then through `execution_stats()`. Table 2.3 presents some of them as the list is long and not every one is relevant to the common user.

Table 2.3: Grouped execution statistics collected by the dispatch buffer.

Group	Fields (batched)	What they tell you
Backend choice	requested versus actual backend, backend declines, backend fallbacks	Whether the scan asked for GPU or CPU, and whether the runtime had to use fallback (hardware unavailable or alogirhtm is not supported).
Alignment jobs	attempted, completed, failed	How many target and query pairs the pipeline tried, how many finished.
Target index	target rows, persisted index rows, missing persisted rows, recomputed rows	Whether minimizer indexes were read from storage or rebuilt for this batch.
GPU dispatch	seed, chain, backtrack and other stages	How much of the aligner actually ran on the device
Stage timings	per-stage nanosecond counters (query sketch, index build, seed, chain, extend, …)	Which pipeline stage dominated wall time on this statement.
Integrity flags	mixed environment, mixed persisted identity, saturated counters	Additional flags indicating overflow or aggregation of counters and ”mixing” which means that different stages of pipeline run in different modes (incompatible k/w or GPU/CPU mixing) - this is used for debugging

2.3.4. MVCC support

PostgreSQL holds its current input row in temporary storage called a tuple slot, but another transaction may also update that row. `Snapshot` in Postgres is set of row versions some statement is allowed to see as it needs to perform visibility checks before doing anything (PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Introduction to Concurrency Control). `configure_params` borrows this snapshot object so later fragment reads and bulk fetches observe the same MVCC view as the original TAM fetch. Let's see if this is compatible with the BisoeqDB model:

- Copying values: Dispatch buffer always copies the original tuple into an owned MinimalTuple slot, so Postgres won't overwirte this data.
- Saving Postgres snapshot object: PostgreSQL uses a snapshot to decide which row versions a command can see. The sequence saves scan's Snapshot and checks a stored reference against the row (visibility check). Stored source fragments are immutable. An update publishes a new version, while an older snapshot can still read its earlier version. Delaying a read does not switch it to the latest committed sequence.[15]
- Saving row numbers: While the primary key is important for user, the row internally holds a row number that is used to indentify results.
- Opening relation: In scan I use `ACCESS SHARE` relation lock to prevent conflicting operations such as dropping or rewriting a relation while it is in use (concurrent updates and deletes are still possible, but avoided due to snapshot mechanism)[16]

I use existing Postgres mechanisms to support MVCC so it didn't require any additional custom code.

2.3.5. Filter placement

As this mentioned here, before saving tuple into the buffer, scan checks if the filter predicate on non-alignment dependent values evaluates to true. This is done to avoid aligning rows that will be discarded anyway. In BioseqDB there are two types of filter conditions:

- A row condition such as `t.id > 20` depends only on stored columns, so is evaluated before alignment. Sometimes columns that are comparable (like Postgres primitive types) can allow min/max chunk prunning to be used. This is special strategy to skip entire columnar chunks with data 2
- A hit condition This is special case of alignment prefilter. I have a separate stage in the pipeline that can be invoked to run cheaper estimation of edit distance for each sequence. If I detect special predicate of form `bioseq.edit(t.seq) < 10` I run pipeline with prefilter enabled. This is different to the case above, because it still requires batching tuples and running pipeline, but it allows for faster termination. If the distance estimation yields false, then the rest of stages are skipped (there's no seeding, chaining, backtracking etc.) so we avoid even the cost of fetching the minimizers from the saved index. This results in great speedups, but at this time is yet an experimental feature using code from libraries that weren't developed by me.

[15] PostgreSQL 17, Transaction Isolation.
[16] PostgreSQL 17, Explicit Locking.

Listing 2.8: Filter input rows and then filter alignment hits.

```
SELECT t.id, bioseq.dp_score(t.seq) AS score
FROM targets AS t
WHERE t.id > 20
  AND mmsearch(t.seq, $1::bioseq.seq('dna5', 64))
  AND bioseq.edit(t.seq) < 10
LIMIT 5;
```

Potential features

Currently I do not optimise limits so `LIMIT 5` will still evaluate all tuples in batch even after more than 5 results are found. I also do not provide any optimisations for score ordering, so `ORDER BY score DESC LIMIT 5` doesn't result in any specialised algorithm. However there are some very speicific cases that would be interesting to optimise. We can use faster chaining if we know that there's only one good hit per sequence. This isn't expressable in SQL with this API, but you can imagine providing custom limit per one row: `WHERE bioseq.count(t.seq) = 1`.

2.4. Joins support

Asssime we have a search between two tables `reads` and `targets`:

Listing 2.9: Search sequence pairs from two tables.

```
SELECT t.id AS target_id, q.id AS query_id,
       bioseq.dp_score(t.seq) AS score,
       bioseq.cigar(t.seq) AS cigar
FROM targets AS t
JOIN reads AS q ON mmsearch(t.seq, q.seq)
WHERE t.id > 20 AND q.id <= 100;
```

There are several possible plans for searches:

- Search before a join: A base `BioseqScan` can search a target against a constant query. This is the most straightforward operation, but isn't possible for the described join.
- Search with an outer parameter: Postgres can create a nested-loop plan that used `BioseqScan` inside. This was discussed in scan list 2.2.3 and causes the target to be loaded multiple times, which reduces performance heavily.
- Search at the join: `BioseqBatchJoin` replaces the nested-loop plan and allows to keep the target index loaded only once, while iterating over queries. This is the most efficient plan for the described join.

Interesting thing happens if we want to make two searches for the same row. Consider the example below:

Listing 2.10: Two searches with one ambiguous target binding.

```
-- Unsupported: this will cause an error
SELECT t.id, bioseq.dp_score(t.seq)
FROM targets AS t
WHERE mmsearch(t.seq, $1::bioseq.seq('dna5', 64))
  AND mmsearch(t.seq, $2::bioseq.seq('dna5', 64));
```

In this case due to my choise of SQL API, we don't know which score `bioseq.dp_score(t.seq)` refers to as we have two potential results with different scores. To resolve this issue we can do a self-join on the same table:

Listing 2.11: Give each independent search its own target binding.

```
-- Supported: this is valid use-case within BioseqDB
SELECT a.id,
       bioseq.dp_score(a.seq) AS score_a,
       bioseq.dp_score(b.seq) AS score_b
FROM targets AS a
JOIN targets AS b ON a.id = b.id
WHERE mmsearch(a.seq, $1::bioseq.seq('dna5', 64))
  AND mmsearch(b.seq, $2::bioseq.seq('dna5', 64));
```

Now, let a_i be hit i of the search bound to alias `a`, and let b_j be hit j of the search bound to alias `b`. Figure 2.4 shows their combinations for one target row. We can see that natural extension of the used syntax would be to generate cross-product of hits. It's worth noting that currently we are not able to express search-specific limits 2.3.5 meaning that there's no optimisation for example if we wanted to have only best-hit per each sequence (only tuple with best score on both searches). In that case all tuples needs to be evaluated and user can construct a query manually to select the highest-scoring hits:

Listing 2.12: Keep one highest score from each of the two searches, per stored row.

```
SELECT a.id,
       MAX(bioseq.dp_score(a.seq)) AS score_a,
       MAX(bioseq.dp_score(b.seq)) AS score_b
FROM targets AS a
JOIN targets AS b ON a.id = b.id
WHERE mmsearch(a.seq, $1::bioseq.seq('dna5', 64))
  AND mmsearch(b.seq, $2::bioseq.seq('dna5', 64))
GROUP BY a.id;
```

The intereesting part is that self-joins in relational databases are commonly used for hierarchical structures (trees), detect duplicates, extract consecutive elements (and other comparison operations). In this case it's used to create new alias for the column without any additional operations.

There is however one interesting exception that is a bug in the BioseqDB code (initially I supported conflicting calls under one alias and there is leftover branch of the code that still supports it). Users should be aware that there still can be some undiscovered edge cases that are not correctly detected. I initially planned to support same-alias searches if no feature is used (we don't specify CIGAR, score etc.), but this results in usability issue that adding these properties suddently requires user to rewrite the query completely. For this reason I later decided to disable any conflicting alias searches even if we don't specify any properties in the `SELECT` list.

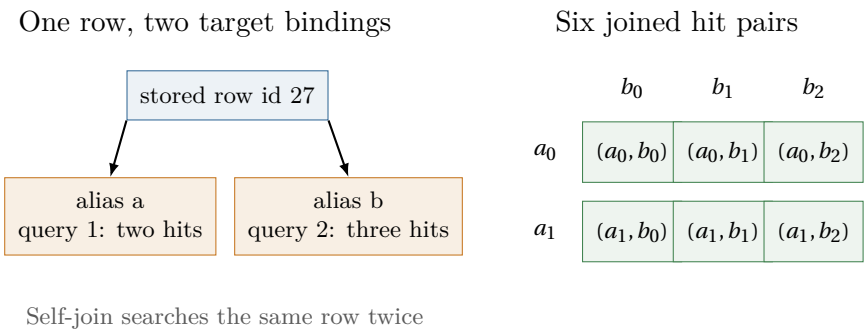


Figure 2.4: Two hits from the first search and three from the second produce six combinations. Additional hit conditions may select subset of this cross-product matrix, but this is responsibility of the user as I don't provide any specialised optimisation for example to extract only one best-hit for each search.

Listing 2.13: Low priority bug that still exists in the code

```
-- This shouldn't be possible, but currently works (bug)
SELECT t.id, bioseq.dp_score(t.seq) AS score
FROM queries AS q, targets AS t
-- index_read_requirement is a special debug parameter to disable BioseqDB from
    loading index
-- for some reason enabling it allows you to trigger specific BioseqJoin path
    that doesn't correctly detect the conficting calls
-- The queries table needs to be normal table, not defined using "USING bioseq"
WHERE mmsearch(t.seq, q.seq, aligner_options(index_read_requirement => 1))
  AND mmsearch(t.seq, 'ACGTACGTACGTACGT'::bioseq.seq('dna5', 64));
```

2.5. Caching results

As mentioned before I provide layers of caching:

- Chunk (strip) cache that stores rows of columnar stored (compressed blobs that can potentially store multiple sequences) 2.3. Described more in detail in next chapters 3.1
- Lazy interface cache implemented by scanner slots 2.3
- I also provide long-term experimental cache layer for alignments results described here.

An alignment query may run for hours while returning a small result. If two sessions submit the same request, running the alignment twice wastes work. I created separate small `bioseq_cache` extension that is provided along with BioseqDB.

Experimental support

This chapter doesn't go into details of chache implementation, because it's still experimental. I didn't perform enough testing to be confident enough about all edge cases correctness as I needed to be very careful how to lock queries and split work (it needs comparing expression trees to detect query identity or overlap, serializing them to store them, parallel-safe coordination of locking). The subextension isn't loaded by default and user need to execute `CREATE EXTENSION bioseq_cache` manually and restard Postgres server. There are some problems with this approach as the MVCC Postgres visiblity can change during execution (there could be insert when alignment is running). This problem wasn't addressed so the user would see old version of the data. That's why I decided to disable caching now. Moreover the "excact results" setting is fully supported now and can be enabled if user loads extension, while "reusable fragments" have most of the code ready except that it was only tested on scalar types (INTs, FLOATs etc.) and sequence support will require some alterations (planner will fallback to "exact results" anyway)

The cache subextension coordinates requests and stores completed results. The extension provides two forms of reuse:

- Exact results: If user executes two identical queries at the same time, we provide special custom scan in plan that waits for currently running job and loads results from it. It needs to happen during planning stage - planner needs to detect that alignment pipeline is running and then during execution stage block execution until results are available.
- Reusable fragments: This is more complicated part of the code that doesn't completely block execution of the query. If similar query is currently running, planner will try to alter current query to still execute it, but try to cover region that is not covered by running jobs.

producer is the query runner that currently executes alignment job. In that timeframe another Postgres-spawned process with other user query waits for results. Submission of partial results would be possible, although very complicated to implement correctly. sealing here means that we store last tuple, collect some metadata like tuple counts and create new UUID for the result. Figure 2.5 shows what happens during the execution of two queries. The cache has expiration TTL time after which the results cannot be reused anymore.

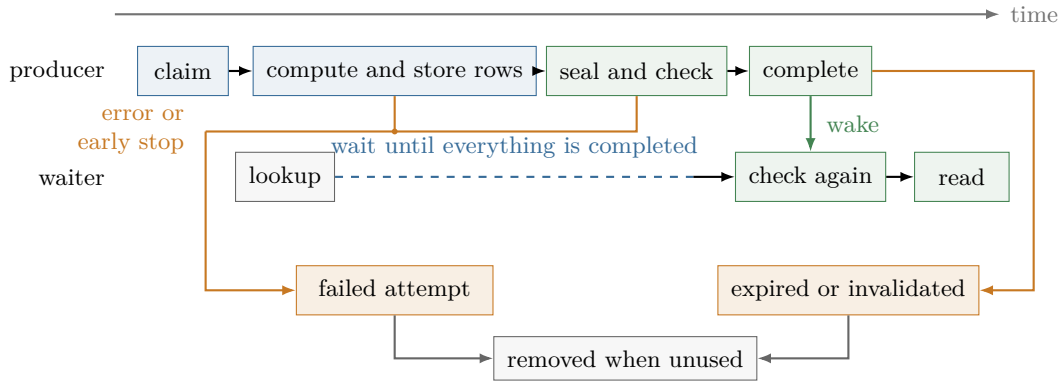


Figure 2.5: Consumer just waits for all results from producer and then reads them.

There's one lock that is taken by any consumer during execution just to access shared state. The consumer sleeps and preiodically acquires and releases the lock to check if it can continue execution (polling). If the producer fails or is cancelled I wake all consumers that registered for this job and then each of them will try manually run the query (so the consumer won't fail due to producer error or cancellation).

The interesting problem is that matching SQL text alone is insufficient. I needed to include analyzed query, parameter values, result types, relevant execution settings, and the database context objects to make sure we reliably detect identical queries. Additionally I keep track of query dependencies (all relations) and if the relation changes (row insert) during processing, then cache will be invalidated. This means that insert during consumer's wait time will cause the consumer to execute query by themselves (although this require more testing to confirm correctness)

2.5.1. Reusing fragments of queries

Suppose that table is kept unchanged, but user slightly changes some metadata condition (for example instead of aligning all sequences with dates between some range, we extend the range to include few more dates). For indentiy comparison I store the analyzed query plan as well so we can say what filter conditions were used for that cache entry.

The general idea of describing what is cached, reading useful part and computing remainer was very loosely inspired by semantic caching paper[17]

For example, select input rows with $40 \le id < 100$. Completed sources cover $0 \le id < 60$ and $50 \le id < 80$. We read $[40, 60)$ from the first source, $[60, 80)$ from the second, and compute $[80, 100)$. Let's define it formally via predicates:

- $T(t)$: t is an input tuple
- $A(t, h)$: h is tuple with alignment of tuple t
- $p(t)$: the new SQL predicate P evaluates to `TRUE` on t
- $c_i(t)$: cached source i has tuple with alginment for which the original predicateP_i evaluates to `TRUE` on t.

[17] Dar, Franklin, Jónsson, Srivastava, and Tan, Semantic Data Caching and Replacement, VLDB 1996, pp. 330–341.

Order the selected cached sources from 1 to m, source i supplies part B_i, while the reminder we have to compute is D:

$$B_i = \{(t,h) \mid T(t) \wedge A(t,h) \wedge p(t) \wedge c_i(t) \wedge \forall j < i : \neg c_j(t)\}, \tag{2.1}$$

$$D = \{t \mid T(t) \wedge p(t) \wedge \forall i \in \{1,\ldots,m\} : \neg c_i(t)\}. \tag{2.2}$$

Each requested hit is returned from the first valid source covering its input, or computed from D:

$$\{(t,h) \mid T(t) \wedge A(t,h) \wedge p(t)\} = B_1 \uplus \cdots \uplus B_m \uplus \{(t,h) \mid t \in D \wedge A(t,h)\}. \tag{2.3}$$

Here $\uplus$ joins disjoint sets of occurrences. This simple formal description can be implemented as large `UNION ALL` operation of the entries in cache. Figure 2.6 shows the same example, including the overlap between the two sources.

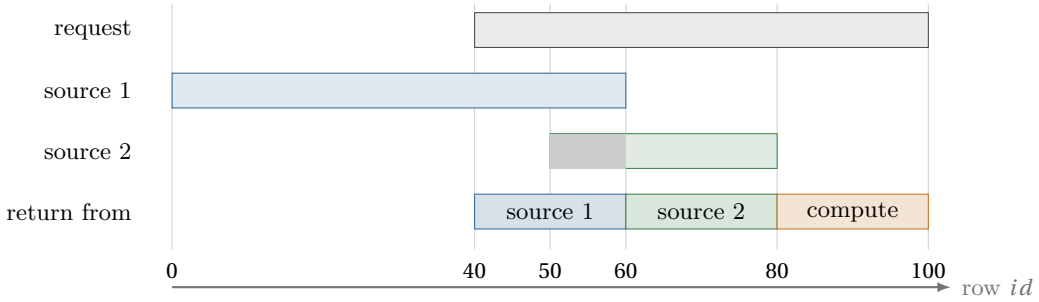


Figure 2.6: Example of resuable query with one predicate on the primary ID column.The new request uses source 1 for $[40,60)$, source 2 for $[60,80)$, and computes $[80,100)$. The gray overlap in source 2 is omitted because source 1 already contains those occurrences. Cached rows outside the new request are also omitted.

With both sources valid, the missing inputs have this equivalent SQL description:

Listing 2.14: Input rows on which the expensive operation is still needed.

```
SELECT id, seq
FROM sequences
WHERE id >= 40 AND id < 100 -- New query we want to compute
  AND (id >=  0 AND id < 60) IS NOT TRUE -- Minus source 1 results
  AND (id >= 50 AND id < 80) IS NOT TRUE; -- Minus source 2 results
```

There are few important details about the implementation:

- I need to correctly handle NULLs For a nullable predicate, `NOT P` also becomes unknown when P is unknown. `P IS NOT TRUE` includes both false and unknown, so it keeps every input excluded from the earlier cache.[18]
- Handle source failures If source 1 fails, c_1 is falsethen D should become $[40,50) \cup [80,100)$ so on error we don't abort execution and just adjust computed region (currently this is realised by adding barrier for each query and computing D at the end)
- Column differences There could be a case in which source supplies only subset of columns. I don't optimise this scenario as computing new columns still requires running entire pipeline again, so we assume here that the column list is always the same.

[18] PostgreSQL 17, Comparison Functions and Operators, Table 9.2.

Chapter 3

Physical sequence storage in PostgreSQL

3.1. Store sequences together with their metadata

I already mentioned before for example when discussing sequence abstractions 1.1 that I usually operate on locators, that just point to location where actual sequence is stored.

PostgreSQL already has mechanism for that: it stores large values out of line (outside actual row) through TOAST.[1] This mechanism however, has it's limitations - it's storage is limited to 1GB per row, which is an effect of maximum width of integer storing row's OID (object indentifier is internal numbering scheme in Postgres). If we assume that DNA4 sequence is bit-packed and it's entire human genome (lengths of around 3.1 - 3.2 Gbps) this gives us conservatively $3 * 10^9 * \frac{2}{8}B = 750MB$ of storage per row. If we want to store both pairs of chromosomes we can pesimisitcally require $1.6GB$ of storage per row. In most cases researchers split the sequence into smaller chunks, but this is manual process requiring additional tools.

Moreover, most of the workflows are OLAP-type.: user is expected to do bulk inserts, do not perform frequent updates and deletes when running alignment. Usual TOAST wouldn't take advantage of that specific workflow assumption as we cannot read TOAST in bulk.

That's why I decided to implement custom TOAST-like mechanism in BioseqDB and use columnar storage appropiate for OLAP workflows:

- User table holds all the data in columnar format, where the sequence column itself is replaced with small locator object that points to sequence fragment.
- Sequence fragments are stored in multiple sharded tables that are normally invisible to the user. All of these tables also use columnar storage, so short sequences are treated the same way as metadata in columnar context - they will be merged into one batch. I call these batches of rows "strips" in the code.
- Persistent indexes are separate entities stored as tables that are normally not visible to the user as well. They are responsible for providing pre-computed objects for sequences. There's small abstraction layer in BioseqDB which allows to extend it by adding different types of indexes that provide different information.

[1] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: TOAST.

Adapted from Citus and Hydra Columnar

BioSeqDB's columnar layer adapts code from both Citus Columnar and Hydra Columnar.[a] Hydra extended the Citus code with features including very primitive vectorized SQL execution and a cache of stripes. In BioSeqDB I modernized the adapted C code by rewriting it completely into C++26 modules with automatic code generation and added support for newer PostgreSQL 19.0 (which in case of Hydra original codebase was not supported). Only small of the original code was left untouched. This code was used as a low-level storagebase, where concepts like sequence locators, fragment readers, and sequence indexes are purely my innovations.

[a] Citus Data contributors, Citus Columnar storage implementation; Hydra contributors, Hydra Columnar: PostgreSQL columnar storage extension.

The linked Hydra columnar documentation was unavailable as the theam building it moved to create a different database system. Although I was basing my implementation in the Hydra codebase, understanding the chunk operations required multiple manual investigations to understand it completely.[2] Current BioseqDB storage design is an effect of this process.

It's worth noting that I mention both Citus[3] and Hydra sources here as Hydra was fork of Citus that diverged at some point to later be abandoned. Citus columnar is still actively maintained but it lacks features like row deletion, vectorization and it's very simple columnar interface. Meanwhile Hydra offered these features, but due to being abandoned didn't correctly build for newer Postgres version and lacked bugfixes done by Citus team since the forking moment. Modernizing the Hydra codebase thus involved not only manually inspecting the code, but also tracking commits between Hydra and Citus. This work thus was complicated and very time-consuming involving also detection of new bugs in Hydra code that were undected after retirement of the project.

3.2. Columnar storage

ClickBench is a benchmark for analytical DBMS and it also reported results for both Hydra and Citus. Figure 3.1 uses historical benchmark submissions on AWS machines.[4] The benchmark consisted of around 43 queries over a dataset of around 100 million rows (3 runs with minum of second two taken as into the sum).

Both columnar submissions store about 19 GB, compared with PostgreSQL's 77.8 GB. This was due to support of compression on stored columns. Hydra[5] had much better performance due to introduction of vectorized SQL execution for aggregates and column cache that wasn't present in Citus.[6] These results do not translate directly to BioseqDB as the benchmarks tests some arbitrarily-built queries, however it motivated me to put big effort into modernizing Hydra codebase and incorporating it into BioseqDB as mentioned in previous section 3.1.

[2] Hydra contributors, Hydra Columnar: PostgreSQL columnar storage extension.
[3] Cubukcu et al., "Citus: Distributed PostgreSQL for Data-Intensive Applications."
[4] ClickHouse contributors, ClickBench: analytical database benchmark.
[5] Hydra contributors, Hydra Columnar: PostgreSQL columnar storage extension.
[6] Cubukcu et al., "Citus: Distributed PostgreSQL for Data-Intensive Applications."

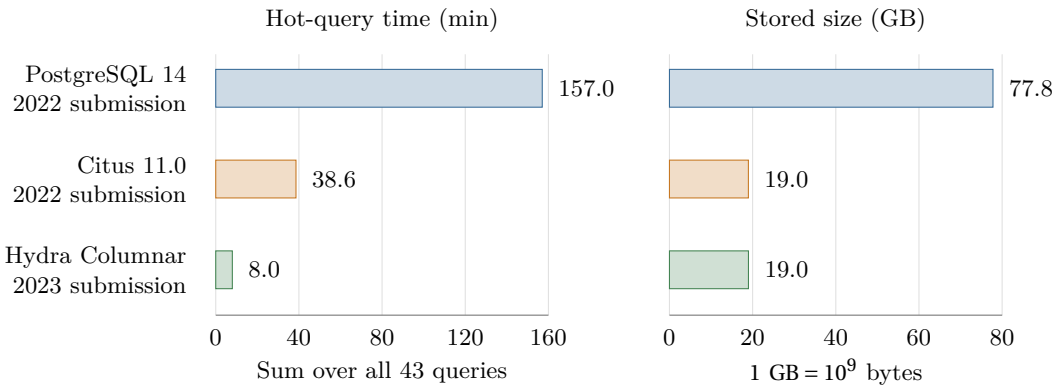


Figure 3.1: Historical ClickBench, calculated from the repository snapshot of 20 September 2023. Each submission uses `c6a.4xlarge` machine with 16 vCPUs, 32 GiB RAM, and 500 GB of GP2 storage. PostgreSQL and Citus results are dated 1 July 2022, and Hydra results 19 September 2023. The times use the sum of shorter of runs two and three for every query. Note that these are "best-fit" results so the configuration was different. It's good however for quick comparison of performance.

3.2.1. Columnar layout

The columnar layout has three levels:

- A stripe is a batch of chunks coresponding to set of rows. Normally one stripe holds maximum is 150,000 rows.
- A chunk group is a group of chunks for the same rows inside a stripe. By default one chunk group holds maximum of 10,000 rows.
- A column chunk is the most basic entity that just holds values for one group as a one column. It also stores aggregated values (count, min, max) to enable faster aggregate operations (vectorization) and the bitmask (marking which values are NULLs).

Stripe and chunk-group sizes are options per table, where larger ones allows more efficient compression and smaller ones allows more fine-grained read access.[7]

Figure 3.2 shows this model for the six rows. By default Postgres row-based storage (heap tuple) stores `id`, `sample_id`, and the sequence locator together. The stripe is columnar representation of these rows. Each column occupies its own sequence of chunks. A chunk group is only an abstract concept for chunks that are related to the same rows. The last group of a stripe can be shorter if total row count isn't a multiple of chunk group size.

[7] Davis, Citus 10 brings columnar compression to Postgres.

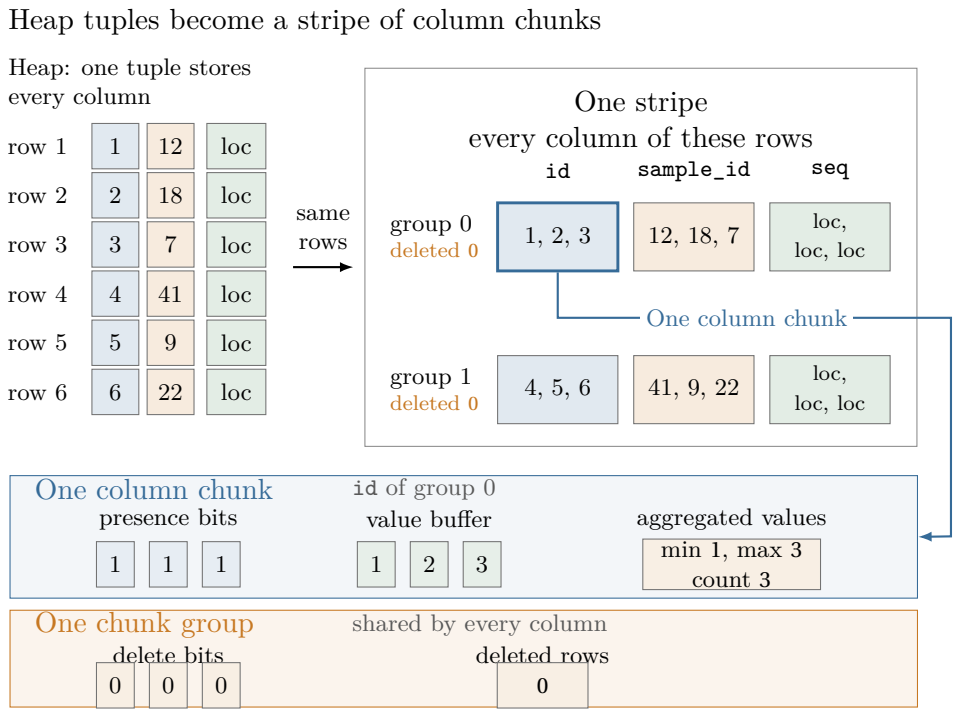


Figure 3.2: The same six rows as heap tuples and as one stripe. Matching colours mark the same column. The blue callout is one column chunk: presence bits, present values, min/max, and slot count. The orange callout is the chunk group: a deletion mask and `deleted_rows`

When I want to store row, each time the writer append this row to the current chunk group (in memory). After sufficient number of row is reached (max chunk group size) the code serializes each column chunk to a stripe buffer that still is in memory If the number of rows per stripe is reached (max stripe size) then stripe is "flushed", which means writing the stripe. Commonly the maximums of stripe row counts will be hit, but there's additional check if the current chunk group for specific column buffer exceeds $1GB$ limit, then it forces flush of the entire stripe. This means that if we store large BLOB along very small column, the BLOB will force the stripe to end i.e stripes always need to account for the same number of rows in each column (this also implies that the decision to store sequences via locators increases efficiency of that storage model as more tuples with non-sequence fields can be packed into single stripe). Additionally there's hard limit on maximum size of $256MiB$ for a single value in row.

> **Storing large BLOBs**
>
> Users should avoid storing large BLOBs along small columns - either store BLOBs of similar sizes together or only store smaller payloads in the same table. This is potential optimisation to be implemented in the future, although it's very low priority.

The figure 3.3 shows 3 main stages: we save tuples first to accumulate enough of them, then serialize them to a buffer and when stripe's buffer eaches $1GB$ or the strip max row count, only then it's flushed. Algorithm 2 presents pseudocode for that process as well.

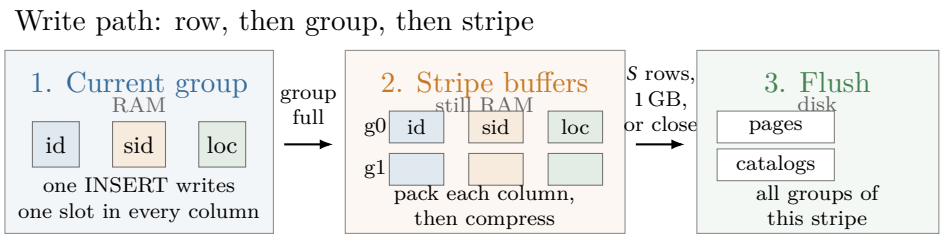


Figure 3.3: How we pack chunks groups into stripe buffers, then to flush them if necessary.

Algorithm 2: Appending rows

Require: columns C, group limit G (default 10,000), stripe limit S (default 150,000), comparators Compare_c (possibly NULL)

$G_{\max} \leftarrow \lceil S/G \rceil$
$s \leftarrow \text{NULL}$ open stripe, or none

procedure AllocateNewStripe
  $s.n \leftarrow 0$ rows already in this stripe
  $s.P \leftarrow 0^{|C| \times G}$ presence bits of the current group
  $s.B \leftarrow \varepsilon^{|C|}$ serialized present values of the current group
  $s.K \leftarrow \{\min : \bot, \max : \bot, cnt : 0\}^{|C| \times G_{\max}}$ skip min, max, and slot count of every column chunk
  $s.\text{chunk} \leftarrow \{P : \varepsilon, V : \varepsilon\}^{|C| \times G_{\max}}$ packed streams of every serialized group
  $s.N \leftarrow 0^{G_{\max}}$ row count of each serialized group
  return s

procedure WriteRow(v)
  if $\text{IsOpen}(s) \wedge \exists c \in C : \text{Size}(s.B_c \| v_c) > 1\,\text{GB}$ then
    FlushStripe(s)
  if $\neg\text{IsOpen}(s)$ then
    $s \leftarrow$ AllocateNewStripe
  $g \leftarrow \lfloor s.n/G \rfloor, \quad r \leftarrow s.n \bmod G$
  for all $c \in C$ do
    if $\text{IsNull}(v_c)$ then
      $s.P_{c,r} \leftarrow 0$
    else
      if $\text{Align}(v_c) \geq 256\,\text{MiB}$ then
        error
      $s.P_{c,r} \leftarrow 1$
      $s.B_c \leftarrow s.B_c \| \text{Serialize}(v_c)$
      if $\text{Compare}_c \neq \text{NULL}$ then
        $s.K_{c,g}.\min \leftarrow \min_{\text{Compare}_c}(s.K_{c,g}.\min, v_c)$
        $s.K_{c,g}.\max \leftarrow \max_{\text{Compare}_c}(s.K_{c,g}.\max, v_c)$ $\bot$ is the identity
    $s.K_{c,g}.cnt \leftarrow s.K_{c,g}.cnt + 1$
  if $r = G - 1$ then
    SerializeGroup(s, g, G)
  $s.n \leftarrow s.n + 1$
  if $s.n \geq S$ then
    FlushStripe(s)

procedure SerializeGroup(s, g, k)

35: $s.N_g \leftarrow k$
36: for all $c \in C$ do
37: $s.\text{chunk}_{c,g}.P \leftarrow \text{PackBits}(\langle s.P_{c,r} \rangle_{r=0}^{k-1})$
38: for all $c \in C$ do
39: $s.\text{chunk}_{c,g}.V \leftarrow \text{Compress}(s.B_c)$ identity if incompressible
40: $s.B_c \leftarrow \varepsilon$

41: procedure FlushStripe(s)
42: $k \leftarrow s.n \bmod G$
43: if $k > 0$ then
44: SerializeGroup(s, $\lfloor s.n/G \rfloor$, k)
45: $m \leftarrow \lceil s.n/G \rceil$
46: for all $c \in C$ do
47: Write($\langle s.\text{chunk}_{c,g}.P \rangle_{g=0}^{m-1}$)
48: Write($\langle s.\text{chunk}_{c,g}.V \rangle_{g=0}^{m-1}$)
49: WriteCatalog(s)
50: $s \leftarrow \text{NULL}$

51: procedure WriteCatalog(s)
52: Write($\langle s.N, s.K, 0^{s.n} \rangle$) Saves row count for group, aggegate stats and deletion mask

In's important that closing the writer also flushes any leftover stripe, so all in-memory buffers are saved to disk when transaction is committed.

As presented on figure 3.2 column chunk stores a presence bitmap, a value buffer of only the present values, and aggregated values used later for delettion and vectorized aggregation: the smallest and largest non-NULL values (when the type has a comparison function), and a row count (`value_count`) counting all values in the chunk including NULLs.

To support deletion, the chunk group stores `deleted_rows`, along with separate row mask representing which rows were deleted. `DELETE` only sets the matching mask bit in the chunk group's mask and increments `deleted_rows`. This allows BioseqDB to offer fast deletion without the need to rewrite the columns themselves. The wasted storage can be however reclaimed:

- `VACUUM (TRUNCATE)` may rewrite last stripes in table whose undeleted rows still fit in one stripe and whose decompressed size is below 1 GB. A single last stripe is rewritten only when more than 20% of its rows are deleted. The rewrite copies undeleted rows into a new stripe and drops the old ones.
- The internal tables storing sequence fragments cannot be rewritten as each locator needs to store the row number. Supporting VACUUM for sequence fragments would need to be dispatched to all tables including user table. Currently VACUUM is only supported on the user table.
- `VACUUM FULL` can rewrite the whole table to reclaim wasted space.

3.2.2. Smart reads

In BioseqDB it's unnecessary to read all chunks all the time. The two optimisations are:

1. Projection means that only selected columns from `SELECT` query are red. This is trivial optimisation.

2. Min/max pruning is more interesting one. For an ordered column, the writer records the smallest and largest non-NULL values in each chunk (in aggregates fields seen in figure 3.2). The BioseqDB TAM presented in figure 2.2 can determine if none of the values in the chunk group satisfy the filter. If that happens, entire chunk group is skipped. For example, a group with sample identifiers from 1 to 100 cannot satisfy `BETWEEN 500 AND 600`.

Figure 3.4 shows both optimisations. Group that satisfies both of these conditions still needs to apply filters to each row individually, but I save time but not touching groups that are not revelant to the search. The figure assumes query `SELECT id FROM table WHERE sample_id BETWEEN 500 AND 600` is executed.

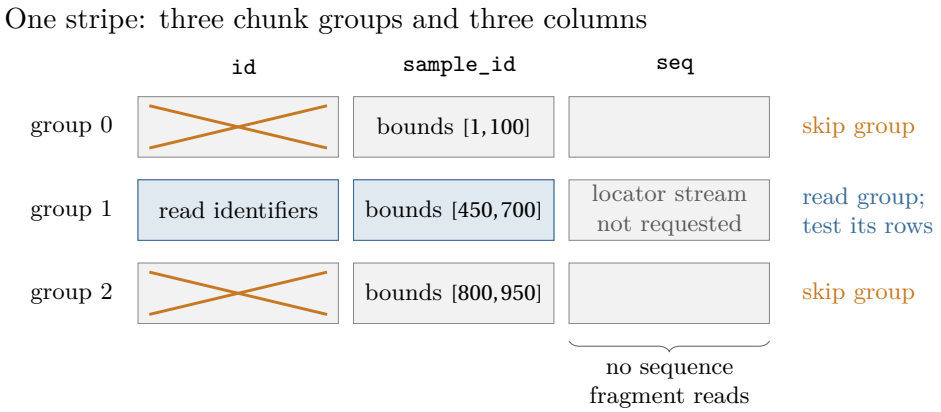


Figure 3.4: Column selection and group pruning for simple query `SELECT id FROM table WHERE sample_id BETWEEN 500 AND 600`. Blue cells are read, orange crosses mark groups whose aggregates exclude all the rows. Unrequested sequence locators remain unread, because they were not specified in the `SELECT` clause. The drawing shows only logical representation of chunks, not actual disk layout.

Prunning for sequence columns

Sequence columns with locators are excluded for min/max pruning, but it's possible to implement that feature theoretically. I decided not to do it, because storing min/max values would be simple as locator isn't big object, however computing these values for very large sequences will be computionally expensive and would negatively impact table inserts.

3.2.3. Compression

Figure 3.5 shows in detail how presence bitmap is computed. If the NULLs are very frequent, I will save the space by not storing them.

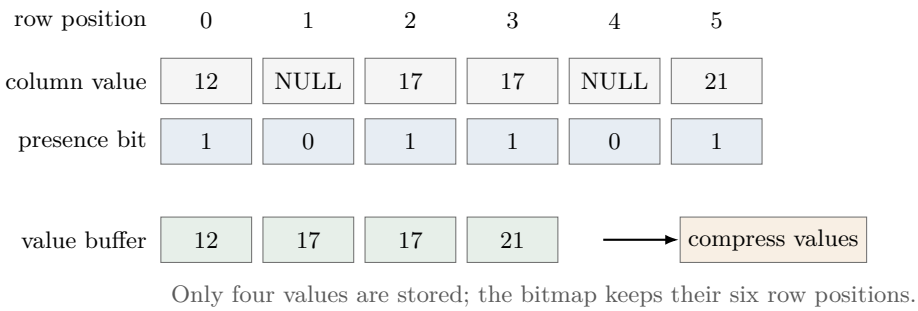


Figure 3.5: Presence bitmap is useful to enable better NULL compression

The Hydra code that BioseqDB columnar TAM was based on supported PGLZ, LZ4, or Zstandard, depending on the build and table options. In BioseqDB all compression is disabled by default as I would like to benchmark in the future best (and more modern) compression algorithms decicated for example to sequence storage. Abadi et al. show that the choice of compression method depends on both the data and the query workload,[8] so I leave it as future extension.

3.3. Raw byte storage

After stripe is serialized and flushed it needs to be written to disk - function $Write$ and $WriteCatalog$ in the pseudocode 2

- Physically first written is the presence bitmap, then the value buffer for a column, then each column follow. These are placed in the user relation database pages.
- Catalog (row counts, aggregates and deletion mask) is written separately from the stripe data and it's placed in usual heap rows i.e BioseqDB introduces separate relations for storing chunks information.

Catalog is represented by four relations inside the BioseqDB:

[8] Abadi, Madden, and Ferreira, "Integrating compression and execution in column-oriented database systems."

Listing 3.1: Metadata catalogs for flushed stripes.

```
CREATE TABLE bioseq_internal.stripe (
  storage_id bigint,
  stripe_num bigint,
  file_offset bigint,
  data_length bigint,
  column_count integer,
  chunk_row_count integer,
  row_count bigint,
  chunk_group_count integer,
  first_row_number bigint
);

CREATE TABLE bioseq_internal.chunk_group (
  storage_id bigint,
  stripe_num bigint,
  chunk_group_num integer,
  row_count bigint,
  deleted_rows bigint
);

CREATE TABLE bioseq_internal.chunk (
  storage_id bigint,
  stripe_num bigint,
  attr_num integer,
  chunk_group_num integer,
  minimum_value bytea,
  maximum_value bytea,
  value_stream_offset bigint,
  value_stream_length bigint,
  exists_stream_offset bigint,
  exists_stream_length bigint,
  value_compression_type integer,
  value_compression_level integer,
  value_decompressed_length bigint,
  value_count bigint
);

CREATE TABLE bioseq_internal.row_mask (
  id bigint,
  storage_id bigint,
  stripe_id bigint,
  chunk_id integer,
  start_row_number bigint,
  end_row_number bigint,
  deleted_rows integer,
  mask bytea
);
```

When writing stripe data I need to take care about page boundaries. Let BLCKSZ be the PostgreSQL page size in bytes (this is compile-time constant provided by Postgres), H the page-header size (technically sizeof(PageHeaderData)),[9] and $P = \text{BLCKSZ} - H$ the remaining

[9] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Database Page Layout.

page capacity. If $x \geq 0$ is the total offset of the x-th byte, then page coordinates are defined as:

$$\mathbf{block}(x) = 2 + \lfloor x/P \rfloor, \qquad \mathbf{position}(x) = H + (x \bmod P).$$

BioseqDB will write multiple pages for each stripe, so I need to take additional care about buffers going through multiple page boundaries as seen on figure 3.6.

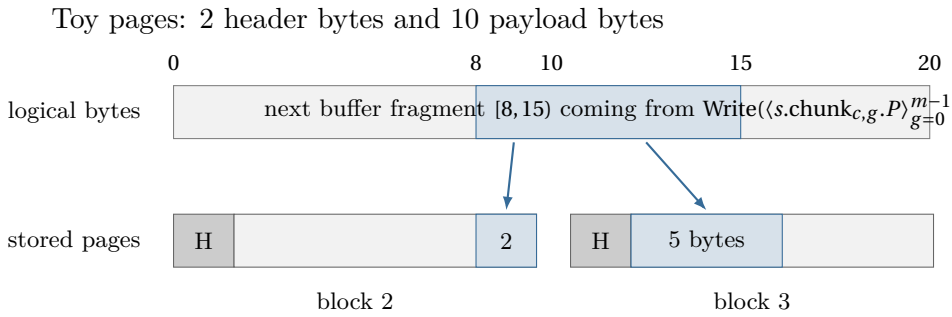


Figure 3.6: I use logical offsets $\mathbf{block}(x)$ and $\mathbf{position}(x)$ to hide page boundaries. Diagram shows that we want to write some fragment of in-memory buffer $[8, 15)$ to the disk, but it needs to be split across multiple pages. In reality the dimensions are larger than just 20 bytes, but it illustrates the idea.

The original idea comes from Citus page writer that it's avaialble in their public repository

Citus columnar page mapping
columnar_storage.c 118

To allow concurrent access (queries during inserts) I need to note that the writer does the following things:

- Reservation: Identifiers for chunks and groups is done under the relation extension lock.
- Write: Writes to pages are done using PostgreSQL buffer locks and generic write-ahead logging (WAL).[10]
- Record completed data: The logic uses Postgres' Snapshots so data will be visible only after the transaction is committed.

3.4. Locator implementation

Sequence locator are implemented as 40-byte struct in C++. The struct must also include the Postgres header so it's effectively 56 bytes in total. The Postgres header stores alphabet code and base storage unit as `TYPMOD` field (custom field for storing type modifier). I don't provide small-sequence optimisation in BioseqDB, because that would complicate the implementation (instead of storing locator for small sequences we can potentially store sequence directly in the user table. This would require storing it's padding so potential maximum sequence length that could be stored inline would be $(40B - 1B) * 8/2 = 156$ symbols for DNA4).

[10] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Generic WAL Records.

Listing 3.2: The 40-byte locator stored after the 16-byte sequence header.

```
struct sequence_locator {
  std::uint8_t  payload_version;   // format version used by me to change the locator format
    (currently it's just set to 3)
  std::uint8_t  bucket_index;      // length bucket 0..34
  std::uint16_t reserved_16;       // pad so shard_index is 4-byte aligned, must be 0
  std::uint32_t shard_index;       // fragment relation inside the bucket
  std::uint32_t fragment_count;    // consecutive fragment rows
  std::uint32_t reserved_32;       // pad so logical_length is 8-byte aligned, must be 0
  std::uint64_t logical_length;    // sequence length in symbols
  std::uint64_t first_fragment_row;// row of fragment 0
  std::uint64_t source_epoch;      // immutable source version, must be > 0
};
```

- the internal tables are divided into groups by sequence length and these groups are divided into shards (so I record both of these value to identify sequence)
- the number of row for sequence first fragment and number of fragments (i.e number of rows occupied by sequence)
- sequence length in symbols (logical length)
- version of the sequence in case it's changed (source epoch)
- version of locator format (not relevant for users)

Locator format optimisation

Currently locators size could be reduces as I don't need to store version (it was used for developent) and I waste too much space for padding. It's potential oppornunity to optimsie in the future (it wasn't a priority as locators occupy overall small space in comparison to sequences themselves).

Each sequence as mentioned in section desribing sequence interfaces 1.2 has it's alphabet type and base storage unit. Base store unit is configurable from SQL. User can declare any power of 2 that is width of supported machine integer:

Listing 3.3: Examples of different base storage units

```
SELECT 'ACTGTGCAACCCCTG'::bioseq.seq('dna4', 64);
SELECT 'ACTGTGCAACCCCTG'::bioseq.seq('dna4', 32);
SELECT 'ACTGTGCAACCCCTG'::bioseq.seq('dna4', 16);
SELECT 'ACTGTGCAACCCCTG'::bioseq.seq('dna4', 8);
-- Example for DNA5 (floor(64/3) = 21 symbols per unit)
SELECT 'ACTGTGCAACCCCTG'::bioseq.seq('dna5', 64);
-- This is illegal as there's no uint128_t. 128-bit unit is just equivalent to
    two 64-bit units.
SELECT 'ACTGTGCAACCCCTG'::bioseq.seq('dna4', 128);
```

Sequences are divided into fragments to enable lazy loading. A fragment is 1024 packed storage units, so actual bytes occupied depend by the width of storage supplied by the user. The maximum possible fragment size is for 64-bit storage unit and it's $1024 * 8B = 8192B$ (note

that this goes beyond default postgres page size of $8192B$, because page needs to include the page header as well)

Fragment abstraction is a separate layer above columnar storage. Shards for fragments are internally held as `CREATE TABLE ... (seq bioseq.seq(...)) USING bioseq`, so each fragment is one TAM row in that single sequence column.

Storage considerations

Fragments always storing 1024 storage units which works well for long sequences, however if the sequence is short then only small number of storage units will be occupied, while the rest is filled with zeros. Compression (default is pglz, but zstd is avialable as well and others for manual building) will take care of this problem and shrink the fragment. However it still has measureable effects on processing in the pipeline as BioseqDB will waste space in memory for very short sequences. This along with locator format optimisation is a task for future work. If we assume occupancy rate of 50% then minimal sequence size for 32-bit storageshould be $1024 * \frac{32}{2} * 0.5 = 8192$ symbols and for smallest possible storage: $1024 * \frac{8}{2} * 0.5 = 2048$ symbols.

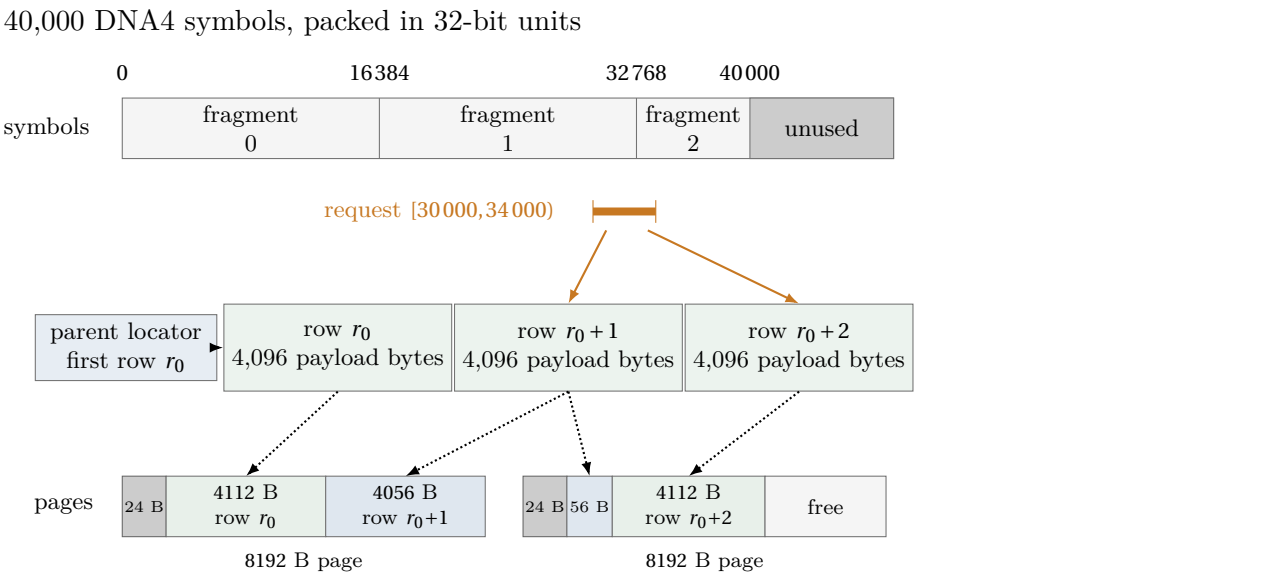


Figure 3.7: A 40000-symbol DNA4 value with 32-bit units uses $1024 \times 32/2 = 16384$ symbols per fragment, so two full fragments and one partial fragment. Request $[30000, 34000)$ reads fragments 1 and 2. Each stored row is 4112 bytes (4096 payload plus the 16-byte sequence header). The TAM concatenates those in column buffers via writer code 2 and during flush, writes them on 8192-byte pages, skipping the 24-byte `PageHeaderData` (8168 usable bytes). One fragment fits in a page, but two concatenated fragments do not ($8224 > 8168$), so fragment 1 continues after the next header. The compression will shrink the last fragment as it's mostly zeroed, but the illustration assumes no compression happened.

3.4.1. Two levels of sharding

Postgres tables have limits on the number of stored rows, so in the most extreme cases the amount of sequence data can be more than one table can hold, especially if we store many full human genomes. A (single copy i.e haploid) human genome is about 3.1 to 3.2 billion bases (3.2×10^9 symbols and the 32-bit DNA4 unit from the figure above). PostgreSQL documents a maximum table size of 32 TB at $8192B$ default page size.[11] That bound is a result of internal Postgres 32-bit page index.

$$
\begin{aligned}
F &= 1024 \times \frac{32}{2} = 16384 && \text{symbols per fragment} \\
n_{\text{frag}} &= \left\lceil \frac{3.2 \times 10^9}{F} \right\rceil = 195313 && \text{fragment rows per genome} \\
B &= 16 + 1024 \times 4 = 4112 && \text{bytes per fragment row} \\
P &= 8192 - 24 = 8168 && \text{usable bytes per page} \\
n_{\text{page}} &= \left\lceil \frac{n_{\text{frag}} B}{P} \right\rceil = 98327 && \text{pages per genome} \\
n_{\text{seq}} &= \left\lfloor \frac{4294{,}967{,}294}{n_{\text{page}}} \right\rfloor = 43680 && \text{genomes in one table}
\end{aligned}
\tag{3.1}
$$

Without compression, one fragment table can therefore hold about 43,680 human-genome sequences before it runs out of pages. Practically compression rates for human genomes were already measured Kryukov et al., "Nucleotide Archival Format (NAF) enables efficient lossless reference-free compression of DNA sequences" and zstd is capable of compressing a human genome to effectively 790 MB (7.9×10^8 bytes). With that size, each genome occupies 96,719 pages, so the same limit allows 44,406 genomes in one table.

Table limits

I present this calculation to present that hitting the table limit isn't very improbable. There exist datasets bigger than this limit. To prvent users from hitting limit and requiring them to manually spli tables, I decided to provide internal sharding.

The internal sharding allows fragments to spill over into multiple tables. If the underlying table achieves $1TB$ limit (which is lower than theoretical 32 Postgres limit), fragements are stored in newly created columnar relations. The sharding is two-level:

- length bucket: groups sequences by their logical number of symbols - precisely $max(0, min(\lfloor log_2(|s|) \rfloor - 8, 34))$ is the bucket index.
- shard: inside one length bucket BioseqDB can create multiple tables when existing tables reach $1TB$ limit (potentially to store extremally large datasets)

The current layout has 35 buckets. The first covers lengths $[0, 2^8)$ and each next bucket covers $[2^i, 2^{i+1})$ for $i = 8..40$. The largest bucket is designated to store sequences longer than 2^{41} symbols.

[11] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Appendix K. PostgreSQL Limits.

Sharding limitations

It's very important to note that current during stream access policy 1.5 the sequence size is not known until its end is reached, thus my code in BioseqDB decides to store the sequence in the largest bucket (34) even when the sequence is short. This code path is utilised by IO submodule that loads sequences from FASTA file. This means that each sequence loaded from FASTA currently will be stored in the largest bucket. I plan to resolve this problem in the future - solution could be adding ability to IO submodule to search for first sequence end and assume all sequences are similar sizes. Alternatively I could write one sequence and then adjust the bucket after size is known. Both of these solutions however add additional overhead and complexity to the FASTA loader module. Additionally length-based sharding can allow better utilisation of fragments memory (so fragments are smaller even without compression 3.4 and they require fewer bytes when allocated in RAM)

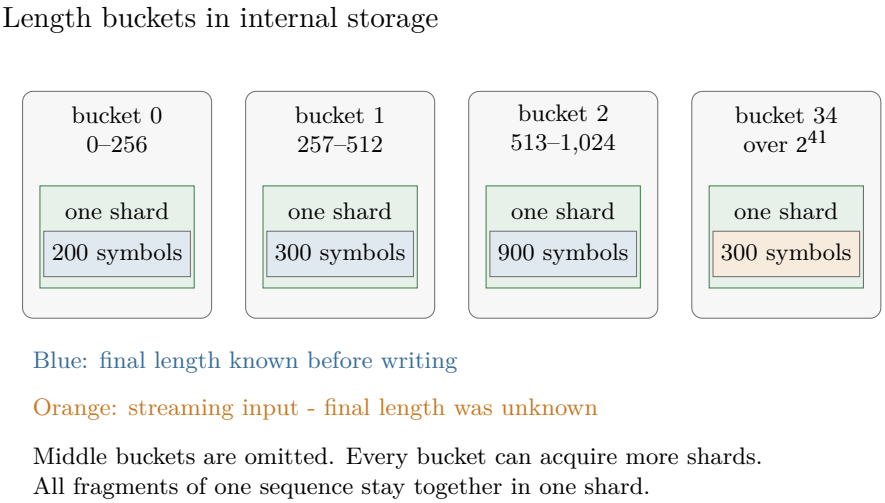


Figure 3.8: Length-based buckets are choosen based on the sequence length log 2, however sequences coming from FASTA/FASTQ have no known length and are streamed, thus internal code always store them in the largest bucket (34). There are multiple tables (called shards) per one bucket depending if we reach $1TB$ limit.

3.5. MVCC and deletion for sequence fragments

I discussed MVCC support when talking about Postgres Snapshots in chapter about BioseqDB custom scans 2.3.4. In custom scan case support is rather simple and provided by the Postgres engine itself. It only required me to correctly use it. Storing fragments is entirely separate problem to visibility in columnar storage, because multiple queries can operate on locators, while some other process can modify sequence data in the same time.

Deleting sequence require doing two operations:

- Deleting row for fragment, which under the hood will mark the row as deleted in the row-mask metadata.

- As fragment is separate abstraction layer than columnar storage, I also store versions of rows in locators, so deletion just needs to invalidate that version (epoch)

Each epoch has stored separate row in the catalog table (so it's one entry per sequence version):

Listing 3.4: Heap catalog for one immutable sequence version. Invalidation updates `state`.

```
CREATE TABLE bioseq_internal.sequence_source_version (
  source_epoch bigint PRIMARY KEY, -- immutable version id, must be > 0
  user_relid oid,                  -- parent user table
  database_oid oid,                -- parent database
  tablespace_oid oid,              -- parent tablespace
  main_relfilenode oid,            -- parent file node
  main_storage_id bigint,          -- parent TAM storage generation
  column_attno smallint,           -- 1-based sequence column
  locator_version smallint,        -- locator format, currently 3
  bucket_idx smallint,             -- length bucket 0..34
  shard_idx integer,               -- shard inside the bucket
  fragment_relid oid,              -- fragment table, NULL if empty
  fragment_storage_id bigint,      -- fragment TAM storage generation
  first_sub_row_no bigint,         -- row of fragment 0
  num_fragments bigint,            -- consecutive fragment rows
  total_bp_size bigint,            -- sequence length in symbols
  alphabet_code smallint,          -- packed alphabet
  storage_unit_byte_width smallint,-- unit width in bytes
  padding_letters integer,         -- unused letters in last packed unit
  source_content_checksum bigint,  -- CRC-64 of decoded symbols
  state smallint                   -- 0 unpublished, 1 ready, 2 invalid
);
```

The specific epoch's `state` follows very simple linear transitions:

- Unpublished (0). An insert has allocated an epoch (sequence version) and written fragments. Locator is then filled with state 0 which is default one. It means that the locator in the user table is not yet written (so the fragment is there, but there's no saved locator that points to it)
- Ready (1). Locator was written to a table, so I update the epoch state to mark that this sequence version is used somewhere (not doing that might lead to dangling fragments that occupy storage but have no existing locator). This state is crucial to prevent a situation in which locator write fails or is rolled back, but fragment was written.
- Invalid (2). Locator cannot read this row version anymore, the underlying fragment shouldn't be accessed. This happens when user either calls `DELETE` or `UPDATE`.

Note that the catalog contains far more information that is neccessary. I decided to save CRC hashes to make sure the fragment is not malformed during operations or shard coordinates shared by the locator itself. As each row version requires separate entry this limits the maximum number of sequences stored in the BioseqDB, however entries are cheap so I didn't incorporate it into estimate calculations 3.1.

Listing 3.5 gives SQL code for this example:

3.5.1. BioseqDB all cache layers

As presented in previous figures as "chunk cache" 1.5 1.5 and as "column cache" 2.3, there's cache for column decompression.

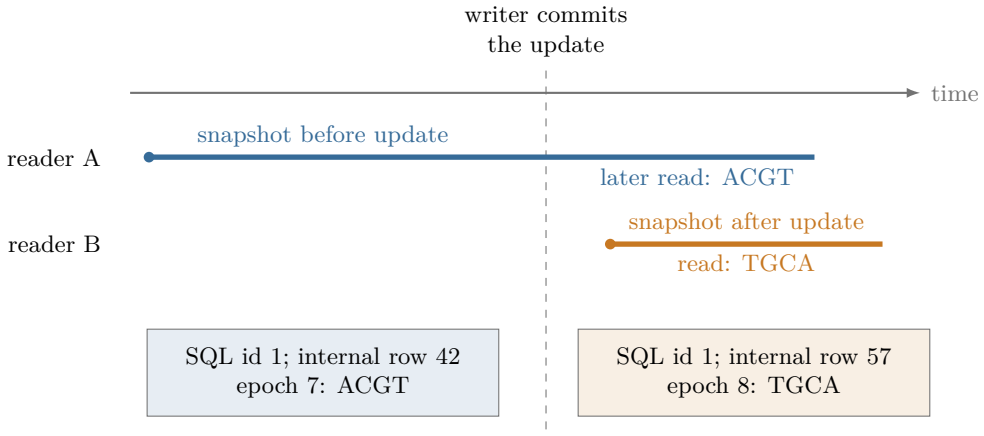


Figure 3.9: Postgres snaphot mechanism allows to check visivility of rows in MVCC model, but it's not sufficient on its own. Let's assume that reader A and reader B are reading the same sequence before and after other process commited update. Using fragments epochs versioning system means that read A will see older locator via snapshot, while reader B will get newer locator row with changed epoch. Both of them can complete reads as both epochs are still valid.

Listing 3.5: A repeatable-read transaction keeps the old sequence version. Initially, row id 1 begins with ACGTACGT.

```
-- Session A
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT bioseq.to_text(seq[0:8]) FROM reads WHERE id = 1;
-- ACGTACGT

-- Session B, while A remains open
BEGIN;
UPDATE reads
SET seq = repeat('TGCA', 4096)::bioseq.seq('dna4', 32)
WHERE id = 1;
COMMIT;

-- Session A
SELECT bioseq.to_text(seq[0:8]) FROM reads WHERE id = 1;
-- ACGTACGT
COMMIT;
SELECT bioseq.to_text(seq[0:8]) FROM reads WHERE id = 1;
-- TGCATGCA
```

The cache is disabled by default, but can be enabled by setting `bioseq.column_cache.enabled` to `true`. It stores decoded compressed chunks inside one PostgreSQL backend process. Its key contains relation, stripe, chunk group, and column identifiers (the configured capacity when enabled defaults to 200 MiB)

The cache can return borrowed pointer to a decompression buffer for a chunk, which allows to prevent additional chunk loads wasting time on decompression multiple times. As I discussed all methods of caching within BisoeqDB already, I present table with comparison of all cache layers:

Table 3.1: Stored bytes and reusable memory have different owners.

Object	Retained data	Scope
PostgreSQL buffer	Relation page bytes	Internal buffer manager in Postgres pins each page and loads/unloads pages on demand so multiple accesses to the same page are efficient
Column cache 3.5.1	Decoded column value buffer	Can cache decompressed chunks, so loading and decompression of the same chunks can be efficient
Fragment slots 1.5	Copy of one packed fragment	When pipeline runs sequence scanners collect fragments in available slots and loads/unloads fragments on demand so multiple accesses to the same fragment are efficient (disabled by default)
Query-result cache 2.5.1	Completed SQL result rows	Separate `bioseq_cache` subextension can store and share results after pipeline execution is finished so same or similar queries can be efficient (experimental and disabled by default)

Chapter 4

Persistent sequence indexes

4.1. Keep the lookup work between queries

In previous chapter 3.1 I describe how columnar and sequence storage layers are implemented in BioseqDB. There's however also aspect for storing indexes for sequences. We would like to be able to store computed minimizers (seeds/sketches) ?? in a way that enables performant computations of alignments. They should be computed once when sequence is loaded from file or updated and then reused for subsequent select queries.

I want to discuss existing aligners' indexing first.

4.2. What existing sequence indexes store

4.2.1. Indexing: minimap2

After computing minimizers they need to be grouped by their hash, minimap2 choose to use the lowest b hash bits (by default $b = 14$) to choose one of 2^b buckets.[1] Minimap2 builds a hash table per bucket whose keys are the remaining hash bits. This buckets with hasmaps are serialized into `.mmi` file. I performed in-depth analysis of how the `.mmi` format is specified based on available Github old issues[2] and original code from minimap2's `mm_idx_dump` and `mm_idx_load` functions.

Writing and loading an index part
mm_idx_dump, mm_idx_load 475

The index file is concatenation of blocks presented in the Figure 4.1:

- Magic bytes The bytes are `MMI` followed by `0x02` (`MM_IDX_MAGIC`).
- Metadata Minimap2 saves w, the k-mer length k, used bitcount b, the number of sequences n_{seq} and a flag word (for various minimap2-specific bitflags)
- Sequence table For each of the n_{seq} sequences the file stores 1-byte name length ℓ followed by ℓ ASCII characters representing the sequence name (header taken from original FASTA file), and a 32-bit sequence length
- Buckets The buckets store the hash-map, where key is remaining hash and bit indicating if there's single value for that key or not. The hash-map is stored as two components however. First is the value list p which is used when there are more than 1 value per

[1] Li, "Minimap2: pairwise alignment for nucleotide sequences"; Li, minimap2.

[2] minimap2 contributors, How to interpret the mmi index file? Issue 820; Lanius, parse_mmi.py: Python parser and lookup functions for minimap2 indexes.

key. Second is the hash array itself, which stores key/value pairs. If there are more than one value per key (last bit of key set to 1), then the value represents offset and count into the value list p The values, either as values in the key/value pairs or entries in p store sequence id, position and strand bit.

- Bit-acked bases (optional) If the `MM_I_NO_SEQ` flag is unset manually by user, bit-packed reference bases are saved at the end.

The depiction of the index file layout is presented in the Figure 4.1 below.

The idea of storing hash-map as two lists inside bucket is also presented in the Figure 4.2 below to allow easier understanding of the index concept.

4.2.2. Indexing: mmseqs2

There are alternative approaches in literature to building an index. MMseqs2 stores one index file that contains counts and offsets, and then writes larger index files that contain all occurence lists instead of bucketing them like minimap2 does. It's worth noting that 2017 MMseqs2[3] paper uses spaced k-mers which wasn't the focus of this thesis, so I didn't investigate more how the alignment is precisely done.

MMseqs2 offers great performance in clsutering, which is different problem to alignment, however connected as it involves estimation of edit distances for large datasets. The software also allows sets of taxonomy operations and alignment-based searches as well.[4][5][6][7]

Count and fill the MMseqs2 index
IndexBuilder::fillDatabase 56

Read a word's entry range
getDBSeqList 178

Keep distinct word codes within one target
addSequence 350

I illustrate the idea of MMseqs2 index on a small example in the Figure 4.3 below, where I just assume that we use 2-letter k-mers of natural language for simplicity.

4.3. BioseqDB indexing abstractions

Indexes are stored separately in BioseqDB 3.1 to all internal sequence fragmented tables 3.8 and they are separate entities to the usual indexes in Postgres which provide generic access paths to the tables.

The index bytes are not stored in the fragment rows. I keep them in one of three layouts. A chunk is one stored piece of those bytes. One chunk must stay at or below 960 MiB, which is 1 GiB minus 64 MiB, so the row still fits in Postgres.

- **`CHUNKED_PER_FRAGMENT`**. One sequence can have many chunks. They are grouped by a sub-key, an extra number chosen by the index. For `minimizers_per_fragment` that number is the fragment, so one fragment can fill several chunks. `minimizers_per_sequence`

[3] Steinegger and Söding, "MMseqs2: sensitive protein sequence searching for the analysis of massive data sets"; Steinegger and Söding, "MMseqs2 enables sensitive protein sequence searching for the analysis of massive data sets."

[4] Steinegger and Söding, "Clustering huge protein sequence sets in linear time."

[5] M. Mirdita et al., "Fast and sensitive taxonomic assignment to metagenomic contigs."

[6] Milot Mirdita, Steinegger, and Söding, "MMseqs2 desktop and local web server app for fast, interactive sequence searches."

[7] Kallenborn et al., "GPU-accelerated homology search with MMseqs2."

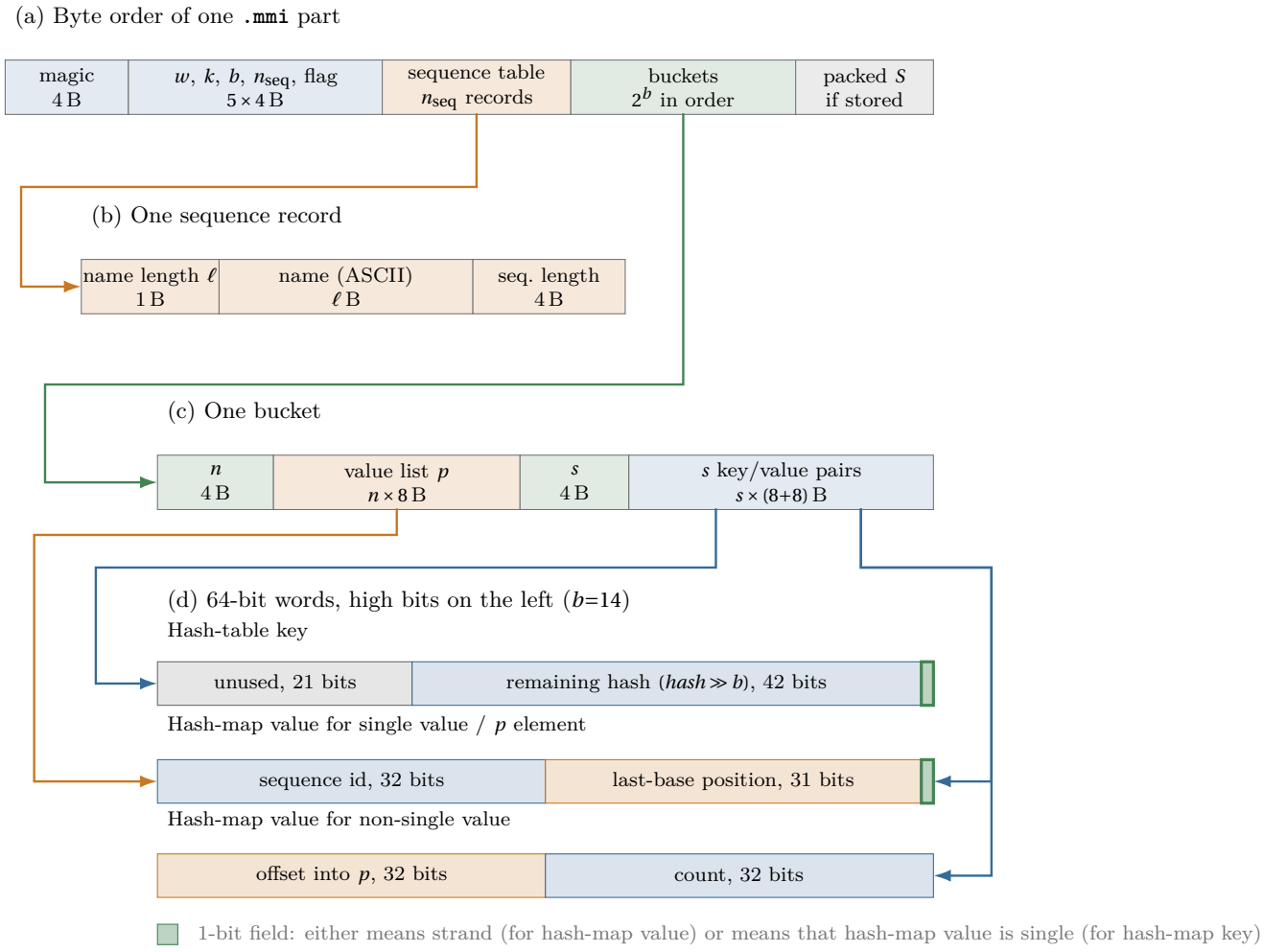


Figure 4.1: Interpretation of one `.mmi` index. For very large datasets index files are split into multiple parts where each of them follows the same format. Minimap2 needs to store sequence ID (RID ??), because the index parts are generated for a set of sequences, not just one, so each part needs to identify which reference sequence the minimizer comes from. Arrows represent how specific elements of arrays look like in close inspection. Panel (a) depicts high-level structure of the index 4.2.1. Panel (b) presents records storing sequence information. Panel (c) shows each bucket of minimizers, which stores its own hash-map via two lists. Panel (d) shows how elements of p looks like and key/values in the hash-map list. Hash map key is always remaining 42 bits with bit indicating that there's single value for that key. If the bit is set, then value in the key/value pair is (sequence id, position, strand bit), if not, then value points via offset/count pair into the list p. The list p stores the same (sequence id, position, strand bit) values.

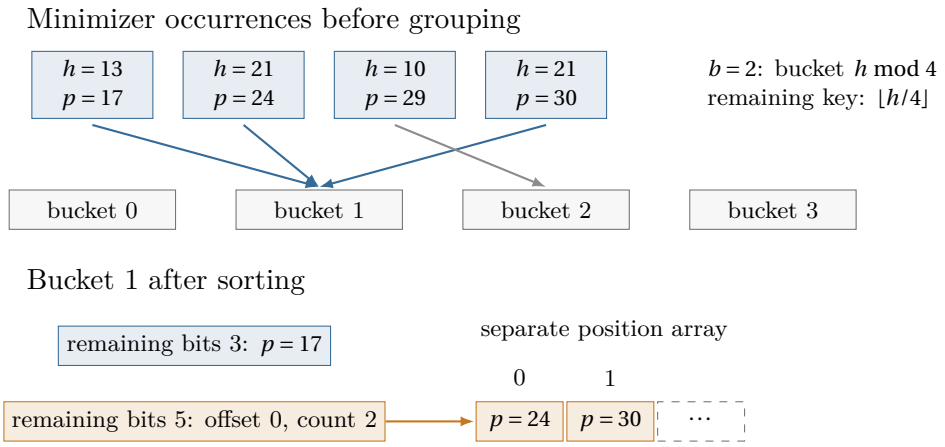


Figure 4.2: A simplified minimap2 bucket: hashes 13 and 21 have the same lowest two bits, but different remaining keys, so they land in bucket 1 together. The two occurrences of hash 21 become adjacent in the bucket's position array.

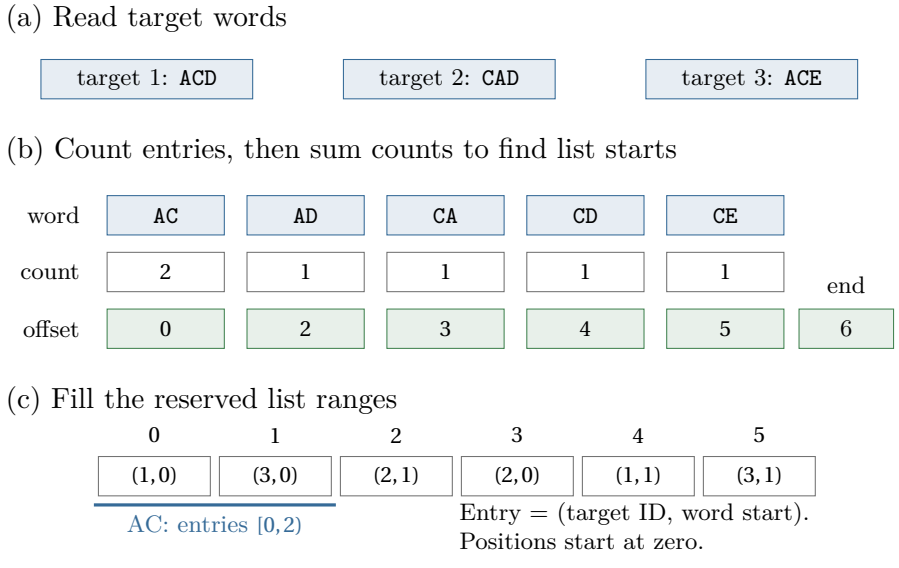


Figure 4.3: A small example of MMseqs2's lists construction, where word/count/offsets triplets are stored in single file, file the actual (very large) lists of entries are stored separately

uses this same layout. Its sub-key is the number of one hash group, not a fragment number.

- `INLINE_PER_SEQUENCE`. One sequence has a single chunk. There is no sub-key and no second chunk. If the bytes exceed 960 MiB, the write stops with an error. `symbol_counts` uses this layout. It stores how many times each symbol occurs in that sequence.
- `CHUNKED_PER_TABLE`. The chunks belong to the whole user table. A build can visit every sequence. It is not maintained automatically when a row is inserted. No production index uses it yet. A test index uses it when tests are compiled in.

Figure 4.4 draws the three layouts under one user table and the fragment rows of one sequence.

The three layouts go through one storage interface, so a later index can pick a layout without a new file format. The check is the C++ concept `is_index_chunked_storage_for`. The fragment calls take a sub-key and a chunk number. The inline calls do not take a sub-key or a chunk number. The table calls take a chunk number, and a build can walk every sequence. Postgres implements one class for each layout:

- `fragment_index_storage` for many chunks under a sub-key
- `sequence_index_storage` for the single chunk
- `table_index_storage` for chunks of the whole table

These classes create the relations. A new index does not need a new table definition. The first use of its name creates the tables in the `bioseq_internal` schema.

Each index name has its own family of tables.

- A per-fragment or per-sequence relation is named `seqidx_`, then a short name, then a hash, then the length bucket and the shard.
- `minimizers_per_fragment` is shortened to `mpf`. `minimizers_per_sequence` is shortened to `mps`. `symbol_counts` is shortened to `sc`.
- The hash mixes the database, the tablespace, the user table's relfilenode, the sequence column, and the full index name. Two index names therefore do not share a relation.
- The bucket and the shard come from the sequence locator. They are the same length bucket and shard as that sequence's fragment rows.

`mps` does not add a new table shape. It uses the same per-fragment storage class as `mpf`, so the columns match, but the relations are a separate family. The inline layout uses the same naming scheme. Its primary key is only the sequence row number, because that sequence has one chunk. The per-fragment primary key also contains the sub-key and the chunk number. The table-wide layout uses the prefix `seqidxtab_` instead of `seqidx_`. Its primary key is only the chunk number.

These relations do not use the columnar access method. They are created with `USING heap`. The user table is the columnar table. A minimizer generation relation is a heap table as well, and that one is created `UNLOGGED`. The generation check refuses the relation when it is not the heap access method.

`pg_supported_indexes` connects those classes to the registered indexes. Listing 4.1 is the check used by minimizers per fragment. The compiler rejects that index unless the storage provides the fragment-layout calls. The listing leaves out the build functions.

Listing 4.1: Minimizers per fragment require fragment-chunk storage.

```
template <class p_storage>
struct minimizers_per_fragment_index {
  static_assert(is_index_chunked_storage_for<
      p_storage, sequence_index_layout::fragment_chunks>);
  static constexpr sequence_index_layout index_type =
      sequence_index_layout::fragment_chunks;
  using storage_type = p_storage;
};
```

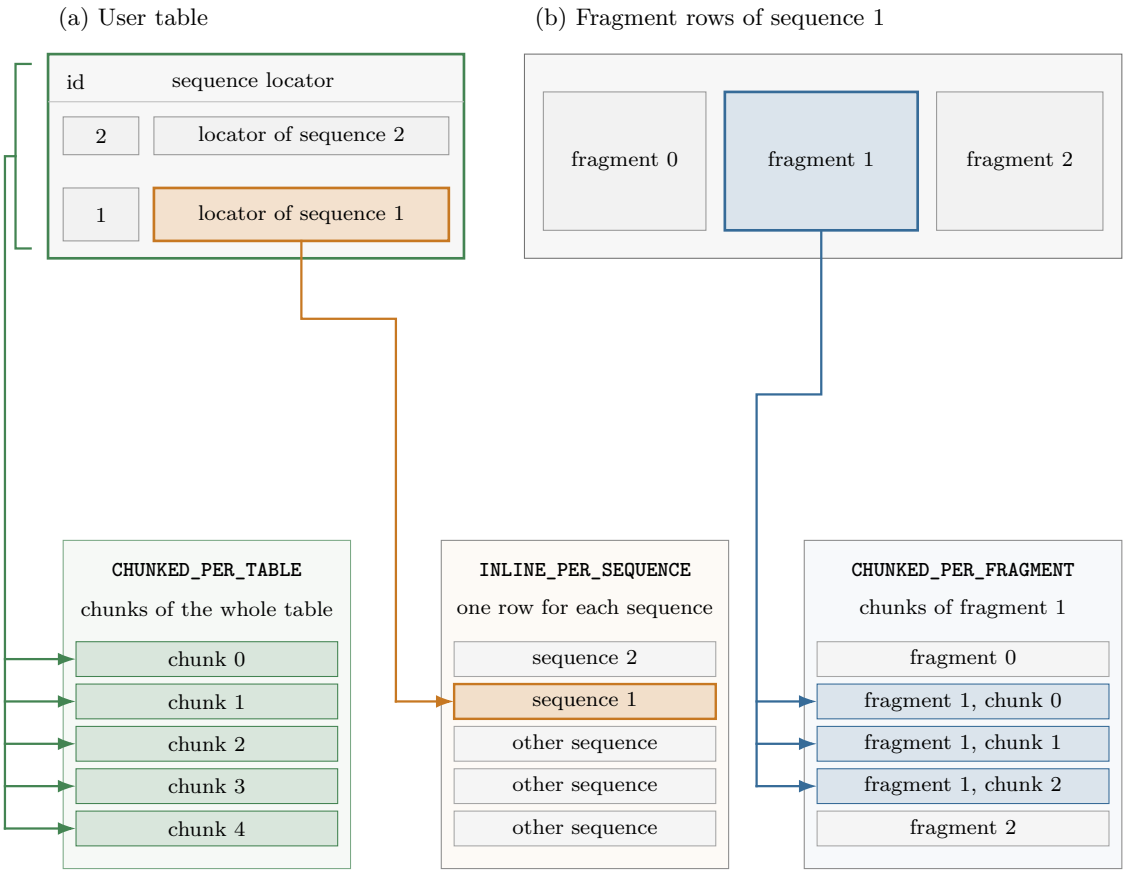


Figure 4.4: Each index is drawn as five rows. The green brace marks the whole user table, and its line splits into every chunk row. The orange arrow leaves the locator of sequence 1 and ends on that sequence's one row. That row cannot grow into a second row. The blue arrow leaves fragment 1 and splits into that fragment's three chunk rows. Grey rows belong to other sequences or other fragments. This drawing shows the fragment-number use of `CHUNKED_PER_FRAGMENT`. The same layout can use another sub-key. For `minimizers_per_sequence` that sub-key is the number of one hash group.

4.4. BioseqDB supported indexing

BioSeqDB supports minimizers per fragment (MPF) and minimizers per sequence (MPS). MPF groups entries by source interval; MPS groups them by hash across one logical sequence.

Both preserve positions needed to form anchors. Figure 4.5 shows the same six occurrences in the two organizations, and the heap rows that store the lists. Panel (c) uses the per-fragment layout of Figure 4.4. The lists are stored. A header and a control list sit in front of each position list. A directory row stores metadata and does not store positions. Here a minimizer position is its last-symbol coordinate, using zero-based positions.

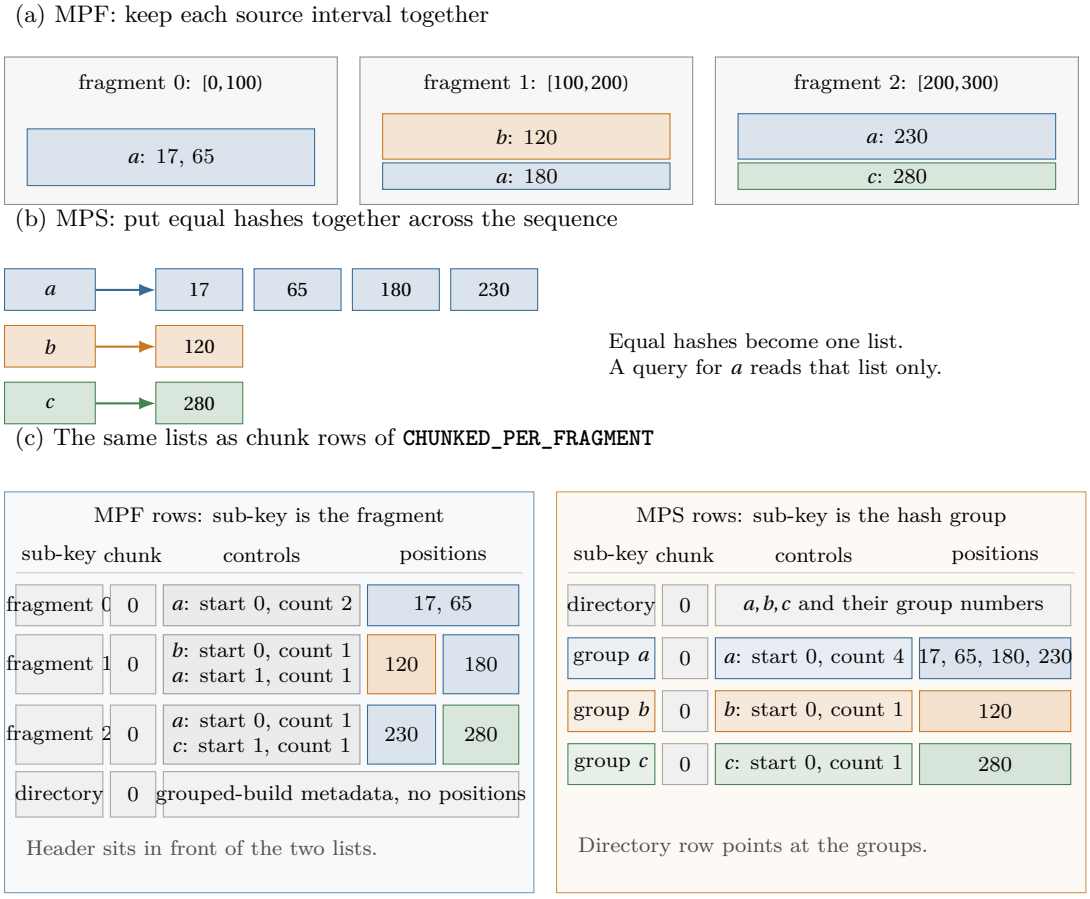


Figure 4.5: Panels (a) and (b) show the same six minimizers in two groupings. Panel (c) writes those lists into heap rows of the per-fragment layout in Figure 4.4. The sub-key is the fragment for MPF and the hash group for MPS. Every payload row begins with a header, then a control list, then the position list. A control stores the hash, the start of its positions, and the count. The position list is the coordinates from panels (a) and (b). The directory row is metadata and holds no positions. MPF writes that directory row for a grouped build. MPS always writes it, and each entry names the hash range stored at a group sub-key. These small lists fit in chunk 0. A longer list continues as the next chunk of the same sub-key. The 100-symbol intervals are illustrative, and the drawn positions are global coordinates.

4.4.1. MPF follows source intervals

An MPF lookup has a hash table or ordered hash controls and a separate array of positions. A control records the hash and the location and length of its posting list. Panel (c) of Figure 4.5 draws a header, the controls, and the position list as one chunk row. The sub-key of the row is the fragment. The directory row in that panel belongs to a grouped build. Local

construction avoids sorting the complete logical sequence at once. A target query can read or assemble the necessary fragment objects, then use their positions to construct anchors.

The ordinary MPF builder uses fixed 32,768-symbol cores. A core is the interval whose occurrences a construction task owns. A generation is a separately identified index build, which becomes usable only after completion and publication. The optional grouped generation builder groups positions according to physical source fragment capacity, while collecting them through cores of at most 32,768 symbols. These construction units should not be confused with a PostgreSQL page or a compressed column chunk. Both builders need symbols around their boundaries so that a minimizer near an edge is chosen from a complete window. Section 4.5 explains this overlap.

The grouped MPF layout stores hashes, list offsets, counts, and positions in separate arrays, also called a structure of arrays (SoA). This gives the reader explicit ranges to check and allows a prepared lookup to use arrays without recreating a C++ hash table. An additional read-only index file (Section 4.7.2) contains a sequence-wide hash-to-positions section built during publication. It uses membership bits and compact list records, explained in Section 4.7.2. The lookup setting chooses this section or the fragment controls in the same file. Building the section moves the merge of equal hashes across fragments into index construction. Grouped layouts and their readers are optional, rather than implied merely by selecting the MPF index name.

4.4.2. MPS follows hash ranges

MPS combines the sequence's minimizers into hash order. Its directory maps hash ranges to stored data. A query can search this directory before loading positions. The current selective layout separates three streams:

- the directory identifies the hash ranges and their controls;
- controls give hashes, exact occurrence counts, and posting locations;
- postings give the sequence positions and strands.

Panel (c) of Figure 4.5 draws the directory as its own chunk row. Each hash group is another sub-key. Its chunk holds the controls and the position list. All indexed occurrences remain stored. A query's occurrence threshold controls which posting lists are read; it does not remove them permanently during construction.

MPS spends more work on sorting and merging so later queries can skip unrelated hash ranges. MPF keeps its base representation closer to source intervals. The current default active minimizer index is MPF. MPS is an explicit choice for workloads that justify a sequence-wide build. The registered persistent minimizer profile uses $k = 15, w = 10$; the selective MPS stream route additionally requires the supported DNA4 and hash-layout identity. Not every minimizer setting for a lookup built during the query has a corresponding persistent reader.

4.4.3. A smaller index answers symbol counts

Not every sequence index stores seed positions. The `symbol_counts` index keeps a histogram for one sequence: one 64-bit counter for each encoded symbol rank. For DNA4, the counters are A, C, G, T. The sequence `AACGTA` therefore gives $(3, 1, 1, 1)$, occupying 32 counter bytes before row and value metadata.

Listing 4.2: Request a symbol count for each stored sequence.

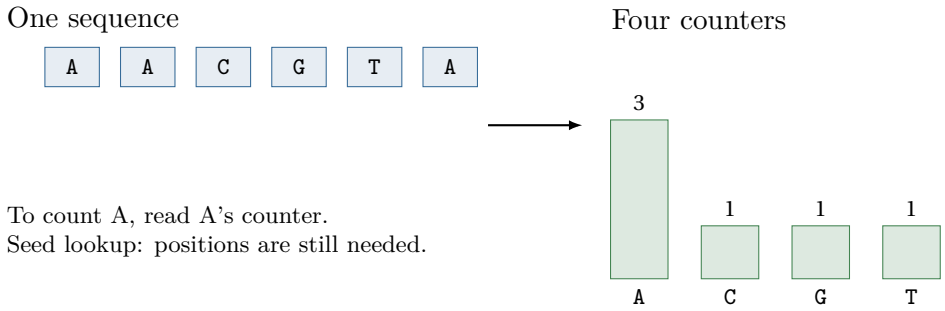


Figure 4.6: A histogram preserves sequence composition but loses position and order. It can answer a count directly, but cannot locate an alignment seed.

```
SELECT id, bioseq.symbol_count(seq, 'A') AS a_count
FROM reads;
```

For a supported scan projection on a typed sequence column and a constant letter, the executor can read the current row's saved histogram and supply the selected counter. If the eligible histogram is unavailable, it counts the lazy sequence's symbols. This index avoids a full sequence read for such projections. It is not an implemented rule for discarding alignment candidates, and it does not preserve enough information to answer position queries. The histogram is stored in an internal relation keyed by the parent's internal row number.

4.5. Build a large index with bounded work buffers

A simple MPS builder could collect every $(hash, position, strand)$ record and sort the complete vector. Its memory use would then grow with the number of target minimizers. BioSeqDB instead sorts smaller runs: ordered groups of records that can be merged without holding their complete contents in memory.

The source scan must first handle boundaries correctly. A halo is the extra context read on either side of a core. For the current grouped builder, a halo of $k+w-2$ symbols covers the span needed around a minimizer window. An occurrence belongs to the core containing its last-symbol coordinate. Thus overlap supplies context without giving the same occurrence to two owners.

MPS turns the resulting global records into a sequence-wide order:

1. Fill one work buffer, sort it by hash, then position and strand, and remove duplicate records.
2. Write the sorted run to PostgreSQL temporary-file storage, then reuse the work buffer.
3. Merge a bounded number of runs at a time. Reading a small prefix from each input is enough to choose the next output record.
4. Continue merging until one ordered stream remains. Remove duplicates across runs as well.
5. Read that stream to count each hash's occurrences, then replay it to encode controls and positions with the correct counts.

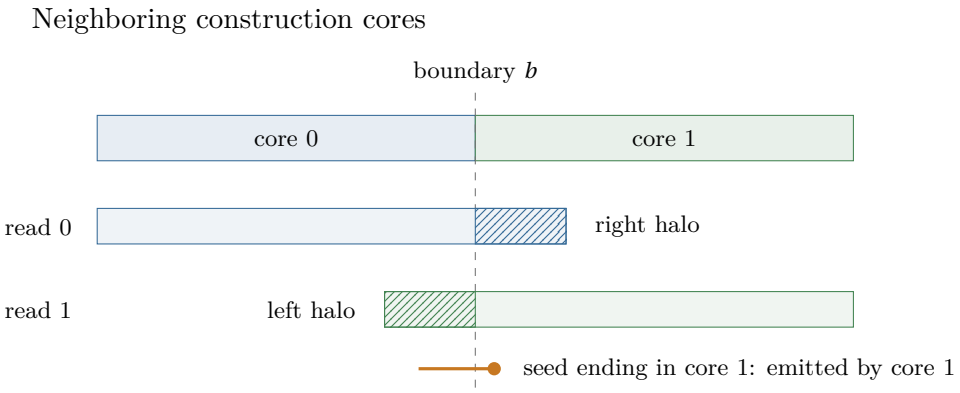


Figure 4.7: The readers overlap, but their output ownership does not. The orange seed crosses the boundary; its last position belongs to core 1. The drawing shows only the internal halos, with lengths exaggerated.

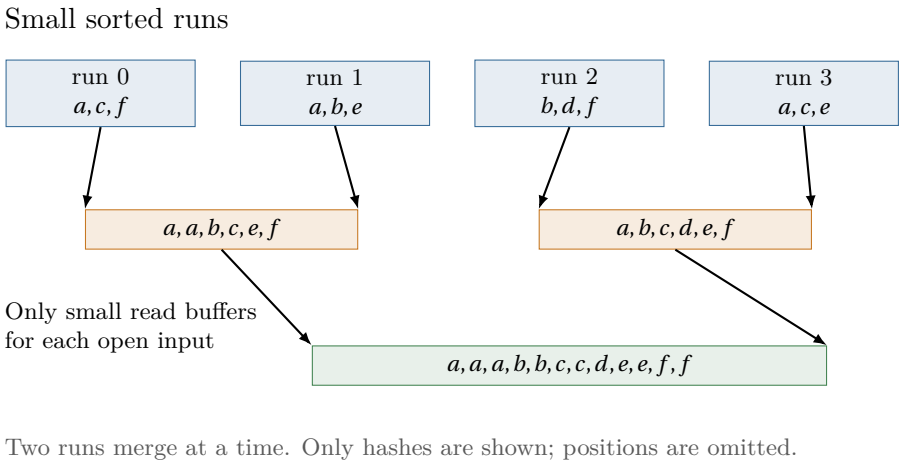


Figure 4.8: External merging bounds the number of simultaneously open input runs. Equal letters represent equal hashes with different positions, so their occurrences are preserved. Only duplicate full records are removed.

The spill implementation uses PostgreSQL's `BufFile` temporary-file interface, rather than giving each run an unmanaged permanent file. The number of merge inputs is limited independently of the total run count. One unusually frequent hash can also have a posting list larger than a work buffer. Its positions are streamed through bounded buffers, so one long list does not require one equally large vector. Temporary disk use can still grow with the input and the merge passes. A memory bound does not imply constant disk space or constant build time.

Construction writes the complete payload before its final navigation metadata becomes usable. In the sequence-wide stream layout, directory entries may only refer to finished controls and postings. Publication of the new generation is a separate final step, described in Section 4.7.1.

4.6. Read counts before large position lists

Suppose the query contains hashes 12, 35, and 78. Reading every target posting would waste work on keys that cannot produce an anchor for this query. It can also waste work on a highly repeated key that the occurrence filter will reject. The selective MPS format, called Format 6, separates the information needed to make these two decisions.

The reader performs four steps:

1. Read and check the complete directory, then find hash ranges intersecting the distinct query hashes.
2. Read the selected controls to find each key's exact global occurrence count.
3. Apply the occurrence threshold before requesting positions.
4. Read accepted posting ranges and form the lookup needed by the alignment batch.

"Global" here means over the indexed target sequence. Looking at one loaded piece alone could underestimate repetition and wrongly admit a frequent key. The stored compact counts therefore have additional exact-count information when their small field cannot hold the value.

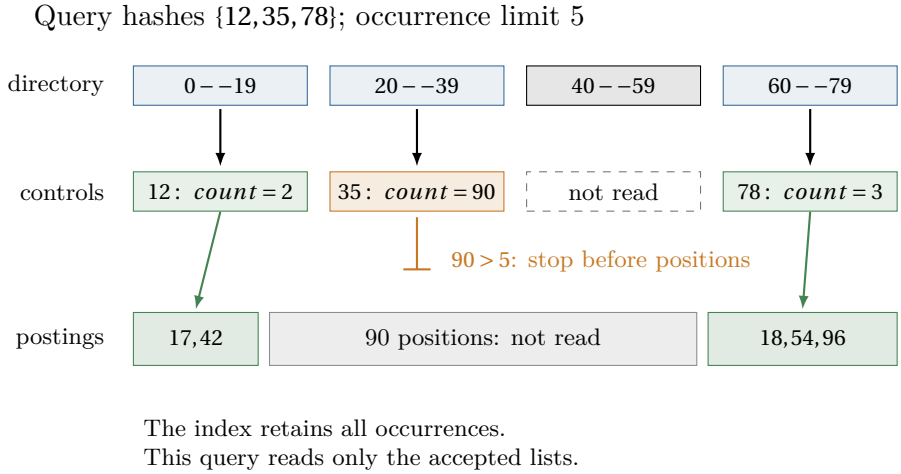


Figure 4.9: Two decisions save different reads. All directory entries are read; blue ranges select the controls. Counts reject a frequent hash before its long list is loaded. Positions and thresholds are illustrative.

Adjacent accepted ranges can be read together through a bounded I/O buffer. The PostgreSQL representation stores payloads in heap rows, so a requested range may fetch a complete row containing some additional bytes. Selective logical ranges therefore reduce unnecessary reads without making the physical read granularity equal to one posting.

The selective MPS reader prepares a lookup for the complete stored target identified by its locator. This reader does not support transformed targets or arbitrary SQL slices. Other index readers can clip and rebase positions for supported intervals. The conditions for using each reader remain separate from the lazy sequence interface itself.

4.7. A saved index needs a source version

The same table and SQL primary key can survive an update that changes a sequence. An index must therefore identify more than its owner by name. The versioned index metadata binds the payload to a source epoch and to the rules that produced its minimizers. Invalidation changes the old epoch's catalog state. A replacement source receives a new epoch.

The important checks have simple reasons:

- Source identity. The relation, column, source locator, epoch, and generation must name the intended source version.
- Seed meaning. The alphabet, k, w, hash scheme, and scheme version must agree with the query. Equal parameter names alone do not make different hash schemes compatible.
- Position meaning. Width and coordinate encoding must represent the target without truncation, including its strand bits.
- Array meaning. The reader must recognize the format and every addressed range. A device reader also needs a compatible array interface.

Lengths, offsets, and checksums help detect damaged or misaddressed data. Checks happen before the reader forms a span or allocates a destination from an untrusted stored count. A checksum is a corruption check; it does not replace source-version or algorithm checks. Ordinary serialized minimizer formats record layout revisions and array sizes, but do not store a source epoch or hash-scheme identifier. Generation readers also check the stored source version and minimizer hash scheme.

4.7.1. Publish complete generations

A small descriptor is the record that identifies a generation and its payload. Building a new generation does not require changing the old payload in place:

1. Allocate a new generation and write its payload.
2. Check the complete layout and its source and algorithm identities.
3. Replace the catalog descriptor transactionally, after the payload is ready.

The descriptor update uses PostgreSQL's tuple visibility. An older snapshot can continue to see its older descriptor; a later eligible snapshot can see the replacement. Failure before publication must not make a partial build the current index.

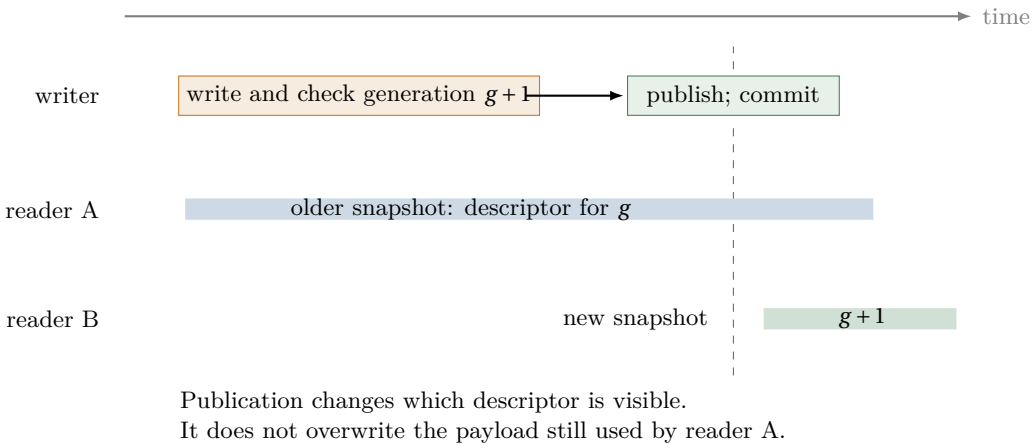


Figure 4.10: Versioned publication with two snapshots. Reader A resolves the old descriptor and retains its read resources. Reader B starts after commit and can resolve the new one. Physical reclamation is a separate operation.

A snapshot that sees a parent sequence's deletion must also reject its old source epoch. These changes belong to the same transaction. Updating a sequence similarly invalidates the old source for newer snapshots and appends a replacement. Neither operation means that every fragment and index byte is immediately erased. An old reader may still need the former version, and the stored objects need their own cleanup policy. The current MPS build path does not automatically delete replaced payloads; reclaiming them requires separate cleanup.

4.7.2. From stored index rows to a CPU reader file

The optional grouped MPF path keeps two stored copies of its index data. It first writes index rows in PostgreSQL, then creates a file for CPU reads. The motivation is repeated searches over a large target. Saving minimizers avoids computing them again, but a reader may still need to sort positions and rebuild a lookup structure. Grouped arrays make the saved layout usable directly. The additional file gives CPU workers stable array ranges without extracting heap tuples or copying the full payload into each reader. To understand why, distinguish three objects:

- The sequence. The original packed symbols remain in the sequence-storage relations from the previous chapter.
- The index relation. Separate heap rows hold the computed minimizer controls and positions, split into byte chunks, with generation metadata. This is already an index, not another copy of the sequence.
- The reader file. This file contains the index payload bytes in consecutive ranges, their directory and metadata, and an additional hash-to-positions section. We call this fixed layout a reader image. Once published, the image is read-only.

The conversion copies the encoded index payload out of the rows. It does not copy PostgreSQL pages or recompute the minimizers. Heap pages contain page headers, row headers and other fields around each stored value.[8] Mapping the heap file would still leave

[8] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Database Page Layout.

those fields between payload chunks. It would not expose one contiguous index array. The reader image removes that obstacle once during index construction.

Construction happens once per generation. The current relation-backed builder takes these steps:

1. Build the index rows. Compute the minimizers and save a new generation's payload, directory and metadata in the index relation.

2. Write the reader file. Reread and check that generation's rows in index order. Append their payload bytes to a private construction file. Add the directory, metadata and hash-to-positions section, then finish and check the file header.

3. Finish the file before publishing the index. Make the file read-only, synchronize it, give it a new final name without replacing an existing file, and remove the temporary name. Then update the database descriptor and active-generation record in the transaction. Other transactions can select the new generation after commit.

The index relation remains after this export. The extra copy costs disk space and a complete payload reread during construction. The hash-to-positions section also adds construction work and bytes. In the current writer this section is always built; the lookup setting chooses how a later query reads the image. Figure 4.11 shows generation g and its completed file F_g.

Direct lookup uses a hash as a bit address. The extra section combines equal hashes from all fragments of one target sequence. It has three kinds of data:

- Membership bits. Bit $B[h]$ is one when hash h has a position list stored in this section. The hash is the bit's address, so it need not be stored again as a key beside that list.

- List records. Records follow increasing hash order. The number of set bits before h, called its rank, gives its record number. Saved counts at each 256-hash boundary make this calculation local: add the saved count to the number of set bits before h in that group. A record gives the list's start, total count, and number of strand-zero positions. Strand-zero positions precede strand-one positions.

- Positions. The list start is an entry number in a shared position array, not a file-byte offset. Positions include a strand bit. When a list has one 32-bit position, the record stores it directly.

In Figure 4.11, hash 5 has one earlier set bit. Its rank is therefore 1. Record 1 gives start 2 and count 2, so the reader uses posting entries $[2,4)$, which contain positions 40 and 230. This is direct addressing followed by a short bit count, with no hash-table collision search or reconstruction of a target-wide C++ map.

An unset bit means no list is retained in this extra section. It can mean either that the hash is absent or that its sequence-wide count exceeds the saved cap, currently 65,534. The original fragment payload remains stored. A query can impose a lower positive occurrence limit, but cannot exceed the saved cap on this direct route. The $k = 15$ profile used below has 2^{30} possible hash values. Its membership bitmap occupies 128 MiB and its saved counts occupy 16 MiB, before list records and positions. This fixed space cost favors large targets. It is a file-size cost, not a requirement to keep every mapped page in RAM at once.

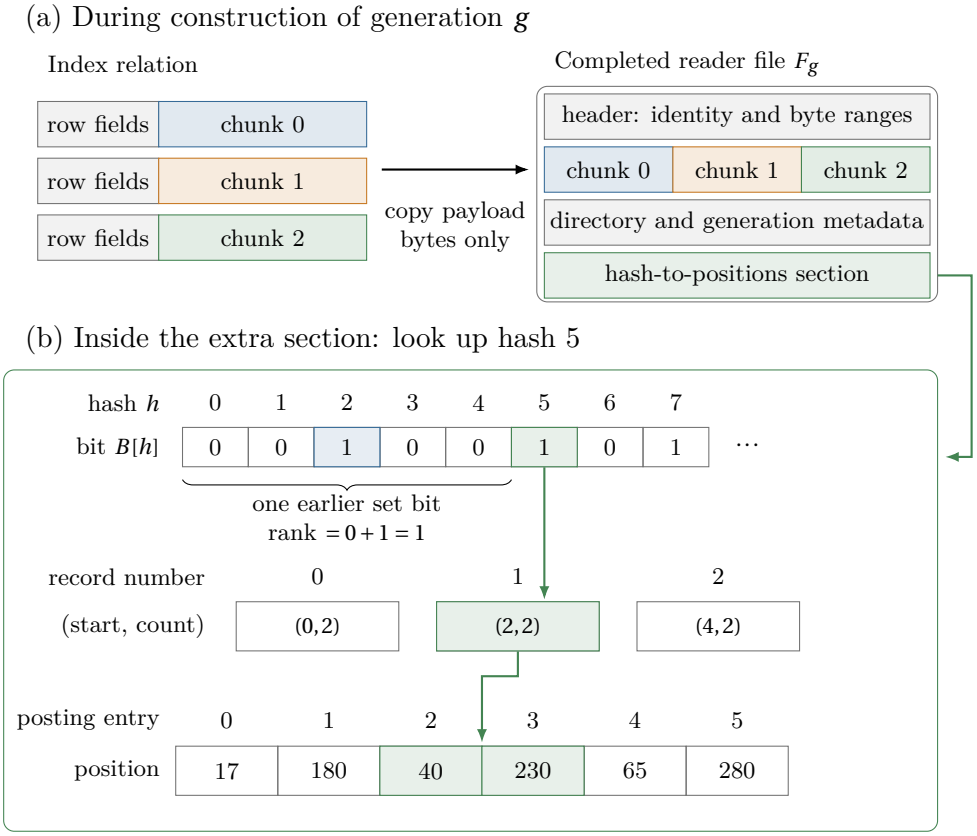


Figure 4.11: Building the reader image and using its direct lookup. The upper panel copies chunk payloads and adds a lookup across the target's fragments. The lower panel uses illustrative hashes 2, 5 and 7, each with two positions. Only the bitmap's first eight bits are shown. Its first 256-hash group has saved count zero. Hash 5 selects bit 5, whose rank selects record 1; that record selects posting entries 2 and 3. Green marks this lookup. Positions are decoded sequence coordinates; all strands are zero, and the record's strand count is omitted here.

Queries open the file rather than exporting it again. The CPU reader first validates the source and reads the generation descriptor under the query snapshot. It opens the matching file read-only, checks its identity and size, and maps it with `mmap`. Mapping gives the process an address range through which it can read the file. After header and range checks, the reader uses views of the mapped index arrays. It still allocates metadata and query work buffers, but does not copy the complete index payload into a new array. One prepared reader can serve several query sequences in a batch. A later prepared reader can reopen the same file.

Algorithm 3 separates the two operations. Here s is the complete, unchanged stored target sequence, R_g denotes its index rows for generation g, and σ is the query snapshot. The named operations are pseudocode, not an API. File and database errors stop the operation; registered cleanup releases partly opened resources. The build runs within the caller's transaction.

Algorithm 3 Build an MPF reader image once; open it for later queries.

```
procedure BuildMappedMPF(s)
    g ← a new generation number
    R_g ← BuildAndStoreMPF(s, g)
    F ← CreatePrivateFile
    for all payload chunks c of R_g, in stored index order do
        CheckChunk(c)
        AppendPayload(F, c)
    AddMetadataAndDirectLookup(F, R_g)
    F_g ← CheckAndPublishReadOnlyFile(F, g)
    UpdateDescriptorAndActiveGeneration(s, g, F_g)
    return g  the caller commits the transaction

procedure OpenMappedMPF(s, σ)
    D ← ReadAndCheckVisibleDescriptor(s, σ)
    F_g ← OpenAndCheckFileReadOnly(D)  check identity and size
    M ← MapFileReadOnly(F_g)
    CheckHeaderAndRanges(M, D)
    return a reader using M while the read session owns the mapping and file
```

Which SQL queries use the image? The public function `bioseq.update_index` builds the file when grouped MPF construction is enabled. Later SQL can reach the mapped CPU reader through an eligible `BioseqBatchJoin`. This custom execution node handles an ordinary inner join whose `mmsearch` call compares a stored target column with a query column. The first argument names the target; the second supplies the queries. In the mapped MPF case, the executor:

1. Keeps the query rows. These can include lazy sequence values; retaining a row need not load all its symbols.

2. Takes one target row. Its stored locator identifies the target whose compatible index generation must be opened.

3. Opens the existing image. The reader validates and maps the file built earlier by `update_index`. Search does not repeat that construction.

4. Searches the retained queries. Query minimizer hashes find target positions in the mapped arrays. Those positions supply anchors for chaining and alignment. The same reader serves all query groups for this target.

5. Moves to the next target. The target's reader is released, while its file stays available for later SQL searches.

A query input with one row is allowed. Two aliases of the same table can also supply the two inputs. Thus "batch" describes how the executor reuses target preparation, not a separate SQL interface or a requirement to search two different tables.

The mapped branch currently requires a complete, unchanged stored target, a compatible CPU profile and generation, and `index_read_requirement = 2`: require a compatible stored index. An ordinary single-table search such as `WHERE mmsearch(r.seq, 'ACGT...')` does not use this mapped reader. Requiring an index does not turn that scan into the two-input join; an unsupported required read fails. Even the `BioseqBatchJoin` name in `EXPLAIN` is insufficient, because a separate variant for transformed operands has the same name and rejects this requirement. The example below uses plain columns and returns identifiers, leaving the stored target in the form needed by the mapped reader.

Both approaches still read the input tables. Figure 4.12 compares two ways to prepare the target lookup inside a CPU batch join. In both, `queries` supplies query rows and `refs` supplies target rows. The difference is the source of the target's minimizer lookup:

- Ordinary MPF: read and decode the stored fragment indexes, then assemble a private lookup for this join, containing the hashes needed by the retained queries. Reuse that lookup across the target's query groups. A cache of stored byte chunks can help the reads, but this join still assembles its own lookup.
- Grouped MPF: open the saved image and use views of its arrays. This avoids assembling the private target lookup above.

Both still fetch sequence symbols needed for base alignment from sequence storage. The mapped image supplies the seed index. If no compatible stored index is used, a third choice is to compute a transient lookup from the target sequence during the query.

The figure assumes that the two indexes were built beforehand: ordinary construction with grouped generation disabled on the left, and grouped construction enabled on the right. The right-hand query must also select the supported mapped-reader path. Changing the construction setting alone does not switch the join's reader.

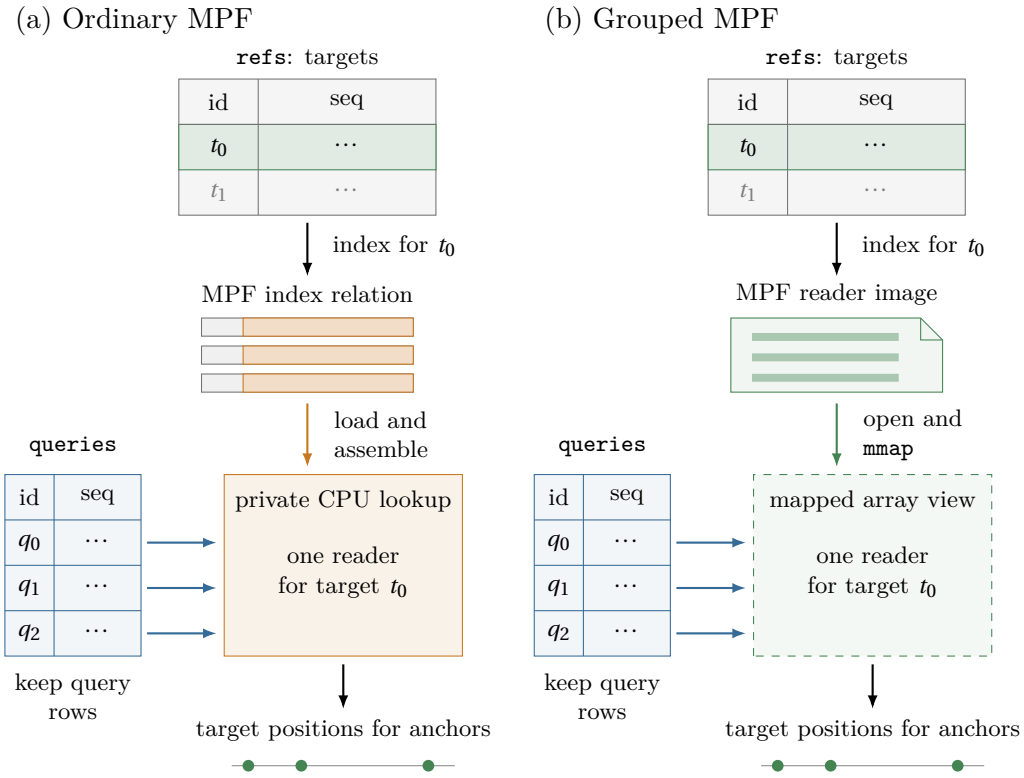


Figure 4.12: The same SQL inputs, with different target-index preparation. Target t_0 is selected while t_1 waits. The blue arrows represent query hash lookups through one reused reader. On the left, stored fragment indexes become a private CPU lookup. On the right, the dashed view refers to the file's existing arrays; it is not another complete payload copy. Both indexes were built before the query. Generation checks and later sequence-symbol reads are omitted.

Construction settings and read settings have different jobs. A GUC is a PostgreSQL configuration setting changed with `SET`. Three settings control the construction choices used here:

- `bioseq.aligner.grouped_soa_generation.enabled`
  Default: `off`. Makes `update_index` build a grouped generation. For MPF this writes both relation rows and the reader image. For MPS it writes the separate relation streams. Turning the setting on does not itself build an index, convert an older layout, or select a query reader. Turning it off later does not remove a completed generation.

- `bioseq.index.async_persistence.enabled`
  Default: `on`. Permits eligible background persistence work. It does not make an explicit `update_index` call asynchronous.

- `bioseq.index.recompute_async_persistence.enabled`
  Default: `off`. Additionally permits a query that recomputes a lookup to request persistence for later queries. It needs the preceding setting too. Turning it off does not stop a query from building a transient lookup to answer the current search.

The current deferred implementation rebuilds eligible ordinary MPS indexes. It stages source identifiers and parameters, then attempts to submit work when the transaction com-

mits. Abort discards the request. A background worker reopens the source and checks its policy before rebuilding it. Queue availability and these checks can prevent persistence, so enabling both settings does not promise that every recomputed lookup will be saved. This mechanism does not build the grouped MPF image. Explicit grouped construction rejects either asynchronous-persistence setting being on; both must therefore be off in the example.

Listing 4.3 shows the public calls. It assumes committed DNA4 sequences with 32-bit packed units in a new permanent `refs(id,seq)` table using the Bioseq table access method, and query sequences with the same alphabet and packing in `queries(id,seq)`. Load `refs` with `bioseq.index.insert_maintenance.enabled = off`. This setting disables insertion-time index maintenance, including symbol counts; it avoids creating an ordinary MPF relation during this load. It does not remove one that already exists. Grouped construction is not an automatic conversion of that older layout. Other settings not shown retain their ordinary defaults.

The remaining settings select compatible algorithms and execution:

- `bioseq.aligner.minimizer.algorithm = 'two_stack_nthash'`
  Selects the SIMD implementation with the minimap2 integer hash listed in Table ??. Construction and search must agree on that implementation. The accompanying `kmer_size` and `window_size` settings set the query's k and w. The current grouped builder uses the fixed $k = 15, w = 10$ profile, so the query must match it.

- `bioseq.aligner.chaining.algorithm = 'simd_minimap2'`
  Selects CPU chaining. Setting `bioseq.aligner.base_alignment.algorithm` to `'block_aligner'` selects base alignment. These choices govern later alignment stages; they do not change the stored minimizer arrays.

- `bioseq.aligner.vulkan.enabled = off`
  Marks Vulkan as unavailable to the aligner. Together with the CPU algorithms and the other default settings, this gives a CPU example. It does not rewrite any GPU algorithm choices previously made in the session.

- `max_parallel_workers_per_gather = 0`
  Prevents PostgreSQL parallel query workers.[9] The current batch path needs an eligible serial query input. BioSeqDB can still use its own CPU workers and SIMD instructions.

- `bioseq.index.sequence_read.enabled = on`
  Allows stored reads for permissive searches. The per-call requirement 2 is stronger: it forces stored reading and reports a missing compatible index as an error. It is an `aligner_options` argument, not a GUC.

- `bioseq.aligner.fragment_minimizer.lookup_route`
  Selects the grouped MPF lookup method. The default `control_block_scan` reads fragment controls. The example selects `direct_compact_hash_lookup`, which uses the bitmap and records in Figure 4.11. Both methods map the same image; changing between them does not require rebuilding it.

[9] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Resource Consumption.

Listing 4.3: Explicitly build and use the mapped MPF path.

```
BEGIN;
SET max_parallel_workers_per_gather = 0;
SET bioseq.aligner.vulkan.enabled = off;
SET bioseq.aligner.minimizer.algorithm = 'two_stack_nthash';
SET bioseq.aligner.minimizer.kmer_size = 15;
SET bioseq.aligner.minimizer.window_size = 10;
SET bioseq.aligner.chaining.algorithm = 'simd_minimap2';
SET bioseq.aligner.base_alignment.algorithm = 'block_aligner';
SET bioseq.index.async_persistence.enabled = off;
SET bioseq.index.recompute_async_persistence.enabled = off;
SET bioseq.aligner.grouped_soa_generation.enabled = on;

SELECT bioseq.update_index('refs'::regclass,
                           'minimizers_per_fragment');
COMMIT;

SET bioseq.index.sequence_read.enabled = on;
SET bioseq.aligner.fragment_minimizer.lookup_route =
  'direct_compact_hash_lookup';
SELECT bioseq.set_active_read_index('refs'::regclass,
                                    'minimizers_per_fragment');
SELECT r.id, q.id
FROM refs AS r JOIN queries AS q
  ON mmsearch(r.seq, q.seq,
              aligner_options(index_read_requirement => 2));
```

Plain `SET` survives a successful `COMMIT`, so the later query keeps the chosen algorithms and parameters. `SET LOCAL`, used in the MPS example below, instead restores the previous session value when the transaction ends.[10] The function `set_active_read_index` records which index the column's searches should use; it is a catalog change, not a GUC or an index build.

Grouped MPS reads its relation-backed streams. The grouped MPF GPU path reads the complete relation payload into device arrays. Neither uses this CPU reader file.

The mapped path is implemented and callable through public SQL, but remains an optional, experimental path: construction is disabled by default, supported query shapes and profiles are restricted, and automatic removal of retired images and complete recovery support remain unfinished.

RAM is an alternative, not the opposite of a mapped file. An index kept in RAM can be fast while it remains available. BioSeqDB's ordinary index caches already retain prepared CPU objects. However, a new backend may have to read and reconstruct its own copy, and an index may be larger than the memory available for that copy.

A mapped file still uses RAM: the operating system keeps file pages in its page cache. Read-only mappings in different processes can use the same cached file pages. Pages can be loaded as needed and reclaimed under memory pressure without making each backend retain the whole index in a private array. This is the performance reason for the extra representation: avoid repeated tuple extraction and full-payload copies at query preparation. It is not a guarantee of faster search. A cold query can wait for page reads, and a broad

[10] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: SET.

lookup can touch much of the file. Building the image also costs an extra write, checks and synchronization. A warm in-memory index remains a useful comparison.

Releasing a reader and deleting its file are different steps. The file has the following lifecycle:

- During construction: a private temporary name identifies the unfinished file. Publication installs a distinct final name.
- On transaction abort: registered file actions attempt to remove the transaction's unfinished or newly published files after checking their identity. On commit, the completed file is kept.
- When a reader finishes: its PostgreSQL resource owner unmaps the address range and closes the file descriptor. The file remains available for another reader. Error cleanup releases these resources too.
- After replacement or source deletion: visibility and source checks decide whether a reader may use the old generation. The current query and rebuild paths do not automatically reclaim its retired image.

A missing or damaged image causes the required mapped read to fail. The query does not recreate it from the relation. An explicit rebuild produces a new generation and image. The file is derived data outside ordinary relation WAL. Synchronizing it does not establish a recovery protocol or ensure that a backup contains a matching file and database descriptor. Its relation-backed generation also uses unlogged payload relations. Crash recovery, backup and removal of orphaned files therefore need explicit support rather than an assumption that PostgreSQL manages the image like a heap relation.

4.8. Choose reuse without changing the search meaning

Update policy decides when construction happens. BioSeqDB has two policies:

Table 4.1: Maintenance policy for one registered index name.

Policy	When work runs	Useful case
`UPDATE_ON_INSERT`	Build the new row's index while inserting it; other rows are not rebuilt	Small additions that should be indexed immediately
`UPDATE_MANUALLY`	Skip that index's insertion-time build; `update_index` requests an explicit rebuild	Load rows first and schedule index construction separately

For example, ordinary MPF maintenance can index one new reference without rebuilding the table's older references. The following uses the storage chapter's DNA4 column with 32-bit packed units. It assumes ordinary defaults: insertion maintenance enabled and grouped generation construction disabled.

Listing 4.4: Maintain MPF when one new sequence row is inserted.

```
SELECT bioseq.set_index_policy(
  'reads'::regclass, 'minimizers_per_fragment', 'UPDATE_ON_INSERT');

INSERT INTO reads (id, sample_id, seq)
VALUES (101, 7, repeat('ACGT', 2000)::bioseq.seq('dna4', 32));
```

An update appends a replacement row, so the same insertion policy applies to its new sequence. Changing a policy does not build missing indexes for older rows, clear existing payloads, or select the index that later searches use. The SQL policy call applies the chosen index name to all current sequence columns in the given table.

Read policy answers a separate question: a query can allow stored reads, require a compatible stored index, or require a lookup built for the current query. We call the last choice a transient lookup.

The important choices can be stated without listing every format setting:

- Ordinary MPF. MPF is the default minimizer index and uses insertion maintenance by default. Its relation adapters read serialized objects and prepare a runtime lookup.
- Grouped MPF. Optional generation construction and its prepared CPU reader use the explicitly described arrays and immutable image. The grouped setting is disabled by default. This CPU reader requires the complete stored target without slicing or transformation, and a policy that requires a compatible stored index. Load and commit source rows before enabling this construction mode.
- MPS. Maintenance is manual by default, though ordinary MPS also supports opting into `UPDATE_ON_INSERT`. The selective generation form uses an explicit rebuild and the separate streams described above.
- Symbol counts. A small per-sequence histogram is maintained on insertion by default and can supply supported count projections.

For a bulk load, an explicit policy keeps a selected index's construction outside the insertion transaction:

Listing 4.5: Choose manual MPS maintenance before loading source rows.

```
SELECT bioseq.set_index_policy('reference_sequences'::regclass,
  'minimizers_per_sequence', 'UPDATE_MANUALLY');
```

This call changes MPS only. MPF and symbol-count maintenance can still run under their own policies. Manual maintenance does not remove the work needed for a later rebuild; it controls when that work runs.

After loading and committing `reference_sequences`, the next example requests an MPS generation build. The construction settings have the roles explained in Section 4.7.2: grouped arrays are enabled and both background-persistence permissions are disabled. This example starts without an older ordinary MPS index; changing its maintenance policy would not remove one. Here `bioseq.aligner.minimizer.algorithm` is set to `'scalar'`, selecting the scalar CPU minimizer implementation for DNA4 sequences. The build uses the registered $k = 15, w = 10$ profile. `SET LOCAL` restores the prior session settings at transaction end; later searches must still select a compatible minimizer implementation and parameters.

Listing 4.6: Build a sequence-wide minimizer index after loading the source.

```
BEGIN;
SET LOCAL bioseq.index.async_persistence.enabled = off;
SET LOCAL bioseq.index.recompute_async_persistence.enabled = off;
SET LOCAL bioseq.aligner.grouped_soa_generation.enabled = on;
SET LOCAL bioseq.aligner.minimizer.algorithm = 'scalar';

SELECT bioseq.update_index('reference_sequences'::regclass,
                           'minimizers_per_sequence');
COMMIT;
```

If a query requires compatible persistence, an absent or incompatible object is an error. Silently rebuilding it would violate that operational request. A permissive query can instead construct a transient lookup. Requiring a transient lookup bypasses stored-index reuse even when a stored index exists. The result's seed semantics still depend on the selected algorithm and its parameters, not on whether the same lookup was loaded or rebuilt.

The deferred ordinary MPS work described in Section 4.7.2 also respects PostgreSQL ownership. A background worker uses its own transaction and memory; it does not borrow the submitting backend's snapshot, relation pointer, or memory context.

4.8.1. Keep useful bytes while their owners remain valid

There are several kinds of reuse, with different lifetimes:

- Serialized chunks. Ordinary relation readers can use a bounded backend-local cache of stored byte chunks. A read handle may still copy a cached chunk into its own buffer.

- Prepared lookups. Ordinary MPF and MPS paths can retain assembled CPU objects under keys describing the source, index, algorithm, and selected interval. Eviction removes the cache's reference; an active reader can still own the object.

- Mapped pages. The operating system caches touched pages of an immutable image. The grouped generation route does not populate the ordinary serialized-chunk cache.

- Device arrays. A compatible GPU lookup can remain resident across work in a batch. Its allocation and memory allowance remain held while any active handle needs it.

The ordinary chunk cache has both an entry cap and a byte cap: currently 256 entries and 2 GiB. The assembled CPU cache has caps of 32 entries and 4 GiB. These cap retained cache entries, not the entire backend's peak memory. Other live buffers, active references to evicted objects, and device owners can overlap. Memory permits account for significant owned allocations while they remain live. A mapped file's virtual length and the operating system's page-cache residency are separate from that accounting.

The chunk cache marks entries written by the current transaction so abort cleanup can discard them. Relation and index invalidation also remove affected cache entries. Reuse requires the matching source and index identity. Releasing a lookup or a mapping is not the same as deleting the stored index: the next eligible query can reopen its persistent representation.

4.9. What the database integration adds

Like minimap2 and MMseqs2, BioSeqDB reuses seed indexes. It connects this reuse to PostgreSQL sequence values through fragment or hash grouping, exact counts before position reads, bounded construction, and source-version checks. The costs are MPS merging, generation metadata, and old payloads awaiting safe cleanup. Unlogged payloads and mapped images need recovery or regeneration. Reuse still requires a compatible source version, representation, and reader lifetime.

Chapter 5

BioSeqDB architecture

5.1. Connecting sequence processing to the database

Consider a search between stored reference sequences and query sequences. Both tables below have an identifier `id` and a sequence column `seq`. One statement supplies comparisons to the alignment system:

Listing 5.1: A sequence join connects SQL input to the processing core.

```
SELECT r.id AS target_id, q.id AS query_id
FROM refs AS r JOIN queries AS q
  ON mmsearch(r.seq, q.seq);
```

PostgreSQL supplies the rows and owns their visibility. BioSeqDB selects compatible implementations, obtains sequence and index views, schedules the comparisons, and returns matches for SQL projection. This chapter explains how those responsibilities fit together.

A batch groups input comparisons for processing. A wave is a subset admitted together under the memory policy. A task is a unit of work submitted to the task scheduler and can process one or more comparisons. These terms describe how work is grouped, not different alignment algorithms.

The preceding chapters describe the work performed by a sequence search. Minimizers produce sparse seed keys (Section ??), chaining groups matching positions (Section ??), and final alignment compares the selected intervals at base level (Section ??). These stages must also fit into a database execution. The sequence store determines how symbols can be read, persistent indexes retain lookup work, and the custom scan turns matches into SQL rows.

BioSeqDB connects these components through a reusable C++ core and PostgreSQL-specific adapters. The core expresses biological operations over logical sequences and index views. The adapters connect those operations to database resources. This division gives three kinds of code different duties:

- Algorithms consume symbols, positions and scores. Their interfaces do not require a PostgreSQL tuple or relation.
- Adapters construct suitable views and keep their backing resources valid. They also convert returned results into database values.
- PostgreSQL execution controls planning, snapshots, locks and the lifetime of a statement. These mechanisms remain native to PostgreSQL.

The build reflects this division. A shared object-library target owns the PostgreSQL-free modules, and the extension consumes their compiled objects and module interfaces. PostgreSQL-dependent modules are compiled separately. The common code is therefore shared with non-server consumers without recompiling it under PostgreSQL-specific definitions. This is a source and compilation boundary. It does not by itself define a separately distributed library or remove PostgreSQL discovery from the project's build configuration.

5.1.1. Reuse at the alphabet boundary

SeqAn provides an established example of composing efficient sequence-analysis software from generic C++ components. Its library includes sequence representations, indexes and configurable alignment algorithms, with SIMD alignment already described in the 2017 overview.[1] BioSeqDB uses SeqAn3's alphabet layer within its own representation. A rank is a small integer code for a symbol, and the alphabet's cardinality gives the number of possible ranks. Rank and character conversions give those ranks their biological meaning, and complement operations apply where the alphabet supports them.[2] The implementation also uses related type traits and a few mathematical utilities.

BioSeqDB defines the packed storage, lazy views and search composition around these alphabet semantics. This lets the same logical symbol model participate in storage reads and in the local alignment kernels. It also lets index preparation produce the lookup form needed by a particular execution. Keeping this composition local gives BioSeqDB control over when storage is read, which stages run, and how long their data survives. SeqAn's own generic interfaces and optimized algorithms remain a precedent for that design. The choice of a local pipeline does not establish a performance advantage over a pipeline assembled from SeqAn components.

Two applications illustrate why composition matters. Masai describes seed search over conceptual trees but chooses concrete index structures for each role and combines the search with banded bit-vector verification.[3] DREAM-Stellar groups query segments by reference region and uses bounded batches with shared sequence ownership before local verification.[4] These are examples of separating algorithms, representations and scheduling. BioSeqDB makes those choices around its database inputs and result contracts.

5.2. Contracts describe the operations a stage needs

The implementation is built in C++26 mode. It uses templates, concepts and named modules to express these boundaries, although concepts and modules entered the language in C++20. A concept states which operations a type must provide. A template constrained by that concept can accept different representations without requiring inheritance from one common base class.

The following reduced API excerpt shows the contiguous minimizer-lookup contract. It keeps the current operation and return type, while shortening the template parameter names. A lookup receives a hash and returns a borrowed span of encoded positions.

[1] Reinert et al., "The SeqAn C++ template library for efficient sequence analysis: A resource for programmers"; SeqAn Project, SeqAn Manual.

[2] SeqAn Project, Alphabets in SeqAn.

[3] Siragusa, Weese, and Reinert, Fast and sensitive read mapping with approximate seeds and multiple backtracking.

[4] Aasna et al., "DREAM-Stellar: parallel and space efficient exact local alignment."

Listing 5.2: The contiguous lookup API constrains the operation visible to a consumer.

```
template<class I>
concept contiguous_minimizer_lookup_index =
    requires(const I& index, std::uint64_t hash) {
        { index.lookup(hash) }
            -> std::same_as<std::span<const std::uint64_t>>;
    };
```

This is one supported lookup shape. Other consumers can receive segmented position spans or canonical anchors prepared for a query. The consumer needs the meaning and lifetime of those positions, rather than the PostgreSQL relation from which they came. In particular, the total occurrence count remains authoritative when a selective reader omits the postings of a high-frequency seed. The storage choice must preserve the pruning decision.

Index construction uses a separate contract. It describes typed chunk reads and writes, enumeration and temporary sorted-run storage. Concrete PostgreSQL types satisfy the contracts for per-fragment chunks, a single per-sequence value, and table-wide chunks. Compile-time assertions check the required interfaces. Separating these storage operations from the query's lookup operation lets each side use the representation suited to its work. The persistence mechanisms are developed in Section 4.1.

Sequence access follows the same principle. A scanner supplies size and fragment-access operations. Its PostgreSQL implementation reads the fragments described in Section 3.1, while an in-memory implementation exposes caller-owned packed storage. Algorithms normally use the logical views introduced in Section 1.1, so they do not need to repeat either scanner's storage logic. A smaller example appears in column comparison: a result-sink concept asks for result allocation and writable value and null masks. The PostgreSQL sink supplies those buffers using database allocation rules. Character comparison that requires PostgreSQL collation remains in the PostgreSQL adapter.

These are selective abstractions. A minimizer lookup needs occurrence positions, but it does not need an abstract join operator. A sequence reader needs valid source access, but it does not implement an alternative MVCC system. Keeping the database mechanisms in PostgreSQL preserves their existing ownership and visibility rules. The common contracts instead expose the operations needed by the biological stages and their test callers.

5.3. Ownership completes the interface

A successful concept check proves that an expression has a usable type. It does not prove that a returned span remains valid or that its source may be read from another thread. Figure 5.1 separates the operation interface from the resources that make an execution valid.

The custom scan from Section 2.1 owns its locked relation and registered snapshot. Read adapters borrow them within the statement's lifetime. A lazy sequence view can therefore be PostgreSQL-free as a C++ interface while still requesting data through a PostgreSQL-backed source. The source rejects calls from the wrong thread before entering PostgreSQL error-handling machinery. The sharing rules in Section ?? apply here: worker eligibility and source lifetime are separate requirements. Depending on the route, preparation may copy data into admitted resident storage or retain an immutable mapped index until its readers finish.

Memory also has several owners. The core context separates temporary alignment storage from longer-lived pipeline storage. Numerical worker tasks install a context for their execut-

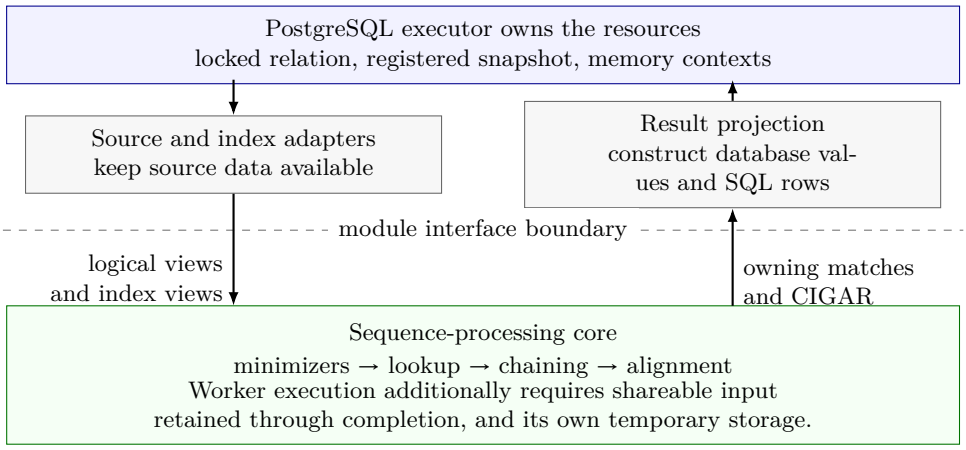


Figure 5.1: Data crosses a narrow interface while the PostgreSQL executor retains database ownership. Arrows show runtime data flow. Static module dependencies point from the adapters to the core contracts. A logical view may still request a PostgreSQL read on the owning backend thread. Worker execution requires an additional check of input sharing and lifetime.

ing thread, while database buffers remain under PostgreSQL memory contexts. A memory permit accounts for a reservation but does not own the allocation itself. The bytes must be released before that reservation becomes available to another task. Returned matches and their CIGAR operations use owning C++ vectors, which the PostgreSQL adapter can consume without retaining a temporary alignment arena.

Error handling follows the same division. PostgreSQL failures are translated at checked calls so that surrounding C++ owners can unwind normally. Database resource cleanup also has to remain valid on PostgreSQL's own error path. PostgreSQL interrupt checks occur at supported adapter boundaries, including between rows, rather than inside every native numerical loop. Thus a generic function signature alone makes no promise about cancellation latency.

The remaining shared assumptions are concrete. Both sides must agree on alphabet ranks, coordinate conventions and index encodings. A prepared index must describe the source version and minimizer rules expected by the query. The build must contain the selected implementation. These agreements make the stages composable, while the adapters keep each execution within the database's resource and visibility rules.

5.4. Runtime choices and compiled implementations

An alignment request names biological work, but it does not identify one machine-code function. BioSeqDB must also account for the sequence alphabet, packed representation, available indexes, and execution hardware. These decisions select an already compiled implementation and connect the algorithms described earlier to the storage and execution layers.

The logical flow remains sketching, seed lookup, chaining, region selection, and optional base alignment. Sections ??, ??, and ?? explain those algorithms. Here the concern is how the system selects their implementations and supplies compatible inputs.

5.4.1. Turn SQL settings into a request

PostgreSQL configuration parameters, usually called GUCs, provide session defaults. The three settings in Listing 5.3 select minimizer extraction, chaining and base alignment, respectively. Their defaults are `scalar`, `scalar_minimap2` and `block_aligner`. The available CPU minimizer choices are described in Section ?? and Table ??.

A constant `aligner_options` record overrides individual settings for one search. Unspecified or NULL fields inherit the session values. The following example instead selects all three algorithm families through named GUC values: scalar sketching, scalar chaining, and KSW2 base alignment. Both `refs` and `queries` are stored sequence tables with an `id` column and a `seq` column of type `bioseq.seq('dna4',32)`.

Listing 5.3: Select algorithm families before planning a sequence join.

```
BEGIN;
SET LOCAL bioseq.aligner.minimizer.algorithm = 'scalar';
SET LOCAL bioseq.aligner.chaining.algorithm = 'scalar_minimap2';
SET LOCAL bioseq.aligner.base_alignment.algorithm = 'ksw2';
EXPLAIN
SELECT r.id, q.id
FROM refs AS r JOIN queries AS q ON mmsearch(r.seq, q.seq);
COMMIT;
```

An ordinary `BioseqScan` stores the defaults and overrides in its plan. A `BioseqBatchJoin` reads the defaults again when execution starts and reapplies the constant overrides. Changes to the corresponding alignment GUCs invalidate cached plans. The options reference in Section ?? gives the complete mapping. The request also fixes the selected chaining policy: PostgreSQL SIMD chaining uses `bounded_batch_approximate`, whereas the C++ interface additionally permits `exact_visited`. They are distinct policies, as described in Section ??. Likewise, the internal batched KSW2 implementation is not an additional PostgreSQL base-aligner setting.

Required results also determine which stages run. A coordinate-only request can avoid base alignment. A CIGAR or exact edit-distance request requires additional alignment work. Divergence and mapping quality have their own requirements. Thus selecting an aligner does not imply that every search executes it. A prebuilt query sketch also lets execution enter after sketching.

5.4.2. Separate algorithm selection from hardware selection

Figure 5.2 separates the decisions that constrain execution. A requested combination must have an implementation and satisfy its input contract. The system does not treat every combination of algorithm names as interchangeable or supported.

For CPU kernels, Highway selects an instruction-set target that is both compiled into the extension and supported by the running processor. BioSeqDB binds the resulting kernel table once at the alignment-batch entry and passes it to the selected implementation. It does not repeat CPU feature detection for each seed or choose a different biological algorithm because another instruction set is available. PostgreSQL vectorized tuple processing is a separate execution choice from the SIMD kernels inside an aligner.

Here a specialization is a compiled variant of a C++ template. Runtime selectors choose among these variants. The dispatch chain fixes a representation bucket and algorithm traits before calling a typed pipeline. Representation-width choices depend on the selected k-mer bucket, avoiding a full independent product of both axes. When an outer caller has already

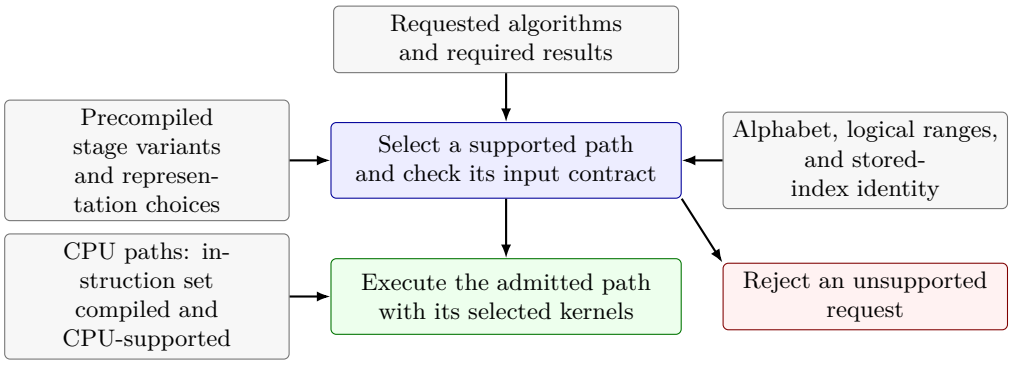


Figure 5.2: Algorithm selection, input compatibility, and CPU target selection have different responsibilities. The diagram shows their dependencies, not a universal order of planner callbacks. GPU execution has additional admission requirements. A runtime failure can still occur after admission.

selected the representation, the inner pipeline retains that choice. When sketches are supplied, it also removes the unnecessary sketch-algorithm dispatch dimension.

Not every numerical parameter needs its own specialization. Compile profiles control which specialized forms are instantiated. Other values use a compiled generic implementation that reads the requested k and w at runtime. This specialization fallback preserves the biological parameters. It is different from the explicit short-pair adjustment of untouched defaults, which can choose smaller k and w before execution. An indexed path prepares query sketches with the parameters required by the stored lookup.

5.4.3. Require compatible indexes and execution routes

An index is usable only when its identity matches the selected operation. The relevant information includes the source generation, alphabet, minimizer scheme, producer identity, and k, w values. Matching the latter two numbers alone is insufficient. In particular, the resident and indexed continuation routes within the two-stack family cannot be mixed merely because they share an algorithm name. The indexed query uses the selection route required by its index. Section 4.8 describes the separate policies for allowing or requiring persisted reuse.

Admission also limits acceleration:

- CPU SIMD. Selected kernels check their supported alphabet, representation, parameter range, and vector shape. Bounded ntHash requires DNA4 with $k = 15, w = 10$. SIMD chaining requires a vector target and valid limits for its selected policy. A rejected request is not retried with another chaining policy or a scalar implementation.
- Biological Vulkan stages. PostgreSQL requires the complete supported resident combination, including Vulkan sketching, seed lookup, chaining, fused execution, and permission to build GPU indexes when needed. It requires usable hardware and a batch containing at least two jobs. Unsupported mixtures and device failures produce errors instead of entering the general CPU pipeline.
- GPU output. The persisted grouped-SoA route currently supports coordinates only. Transient resident execution also has a bounded CIGAR route, which retains host packing and result construction. Admission to a GPU route therefore does not mean that every substage runs on the device.

These rules have specific exceptions and should not be reduced to a blanket ban on fallback. The reference SIMD minimizer explicitly uses scalar rescan for alphabets without its two-bit representation. A Vulkan rank prefilter can accompany CPU biological stages, as explained in Section ??. A permissive index policy can rebuild an absent or inapplicable lookup. Finally, a rejected parallel work split can run serially with the same algorithm. Each alternative belongs to a particular layer and preserves that layer's stated contract.

5.5. Schedule independent work within a backend

The ownership boundary in Section 5.3 also determines where parallel execution is possible. Two queries can read the same immutable target, but they must not overwrite the same alignment scratch space. A database-backed reader may expose the right sequence interface while still requiring access from its creating PostgreSQL thread. The scheduler therefore needs more than a collection of independent mathematical problems. It needs inputs that remain readable, separate output locations, and enough memory for the work that overlaps.

BioSeqDB expresses this work through oneAPI Threading Building Blocks (oneTBB). A task is a unit of executable work, such as processing a range of independent queries. A worker thread is an operating-system thread that executes such tasks. An arena is oneTBB's scheduling domain with a limit on how many threads may participate simultaneously. Tasks describe available parallelism without assigning a permanent thread to each operation. This is the distinction emphasized by oneTBB's task-oriented and data-parallel interface.[5]

5.5.1. PostgreSQL processes and oneTBB threads

PostgreSQL executes a client session in a backend process. Its original server process, traditionally called the postmaster, starts backends and remains responsible for accepting connections. It does not execute each client's alignment tasks.[6] Inside an executing backend, BioSeqDB has one process-local shared scheduling interface. It owns a oneTBB arena used by alignment and other supported work. The name "global pool" refers to this process-local scope.

The extension also registers a separate PostgreSQL background supervisor when it is preloaded. The supervisor maintains shared scheduling configuration and service state. It can perform background work such as servicing index-persistence requests. It does not own one central CPU arena to which every client sends its sequence pairs. Figure 5.3 separates these roles.

Initialization follows the same boundary. During preload, BioSeqDB registers PostgreSQL hooks and background-worker metadata. It delays arena initialization until execution in a backend or the background process. At a scheduling boundary, the backend adopts the current configuration generation and configures its own arena. A policy restart can advance that generation and recreate the caller's arena. Other backends adopt it at a later synchronization point. This is not an immediate stop and restart of every query in the database.

oneTBB's arena limit includes the application thread when it participates. An arena limit of four therefore does not promise four extra workers in addition to the backend main thread. The scheduling interface's range operations enter the arena through `task_arena::execute`,

[5] UXL Foundation Contributors, oneTBB Benefits.
[6] PostgreSQL 17, Architectural Fundamentals.

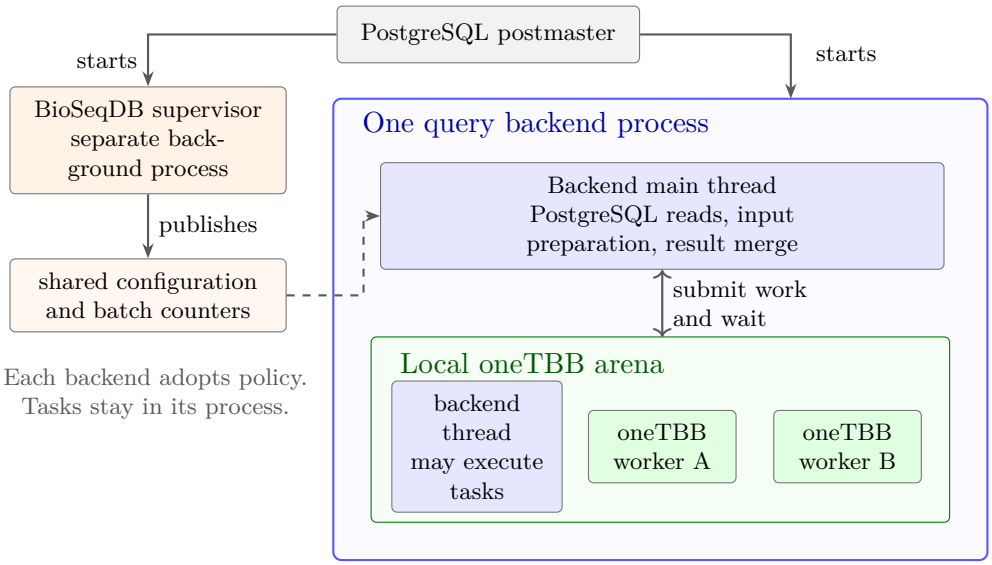


Figure 5.3: The supervisor and the query backend are different PostgreSQL processes. The arena belongs to the query backend. Its worker threads share that process's address space, and the backend main thread may participate in arena computation. Dashed communication carries shared policy, not a cross-process queue of alignment tasks. The three arena participants are illustrative.

which permits the caller to execute work. The configured concurrency is also distinct from a live census of threads that happen to be busy.[7]

PostgreSQL's parallel-query workers are different resources. A `Gather` plan obtains additional PostgreSQL processes, subject to PostgreSQL's worker-process limits.[8] A oneTBB thread inside a backend does not become one of those processes and does not consume a PostgreSQL parallel-worker slot. Nevertheless, both forms of concurrency use the same machine's processors and memory. BioSeqDB clamps its arena width to one when running inside a PostgreSQL parallel worker unless composition is explicitly enabled. This is a defensive policy for such callers. It does not make the alignment custom scan parallel-safe, nor does it allocate a common CPU quota across all client sessions. Several ordinary backends can still compete for the same cores.

5.5.2. Choose the unit of parallel work

The scheduling interface uses oneTBB range operations, task groups, and explicit task submission. The biological pipeline's stages remain the operations described in the earlier algorithm chapters. They are not assigned to a row of permanent threads, with one thread sketching and another chaining. Instead, the implementation exposes independent work where its data and algorithm dependencies permit it.

At dispatch level, eligible in-memory sequence pairs can run independently. Each item writes to its own result collection. The backend combines those collections after the range has completed. A join waits for submitted work to finish before its results are combined. A second useful shape is several queries against one shared target. The backend first validates the query and target identities and prepares the supported target reader. When the required

[7] UXL Foundation Contributors, oneTBB Specification: task_arena.

[8] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Resource Consumption.

resident target can be admitted, queries borrow the same immutable target state and retained query sketches. They still own separate alignment scratch and results.

Figure 5.4 shows why the number of tasks need not equal the number of threads. Query costs vary with the number of anchors and the extension work they induce. oneTBB can subdivide ranges and schedule ready tasks as participants become available. Its scheduler favors recently created local tasks and can steal older ready work from another participant's deque. This balances available work while seeking locality, without guaranteeing a particular task order.[9]

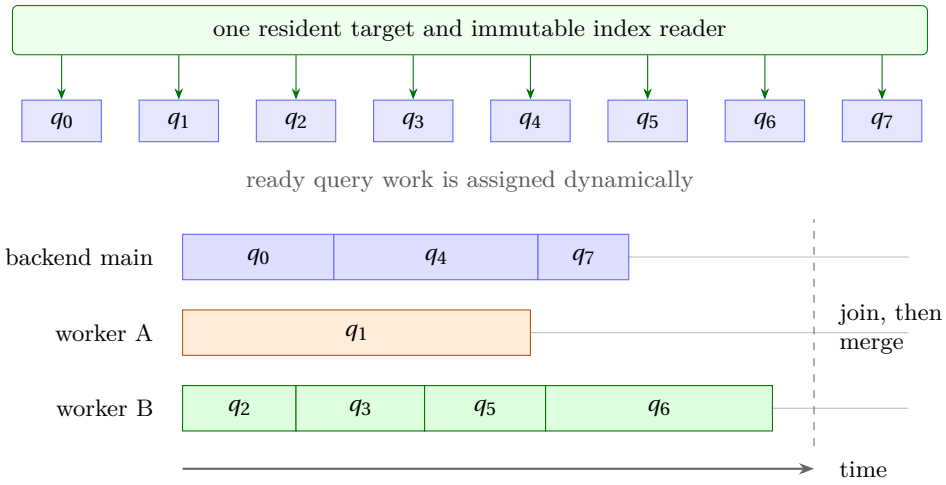


Figure 5.4: Eight independent queries can share three arena participants. Each query has its own result slot, regardless of which thread executes it. Colors distinguish participants. Block lengths and assignments illustrate uneven costs, not measurements or a guaranteed oneTBB schedule. A range task may also contain several queries. Shared target bytes remain owned until all their consumers finish.

Within one query, sufficiently large groups of independent alignment regions can also run in parallel. Large-target recomputation can expose several admitted segments. These choices depend on the selected algorithm and on whether the relevant sequence views can safely cross threads. A PostgreSQL reader does not become shareable because its C++ interface matches a core concept. Such work must retain a supported resident view or use the backend's serial reading path, as described in Section 5.3.

The scheduler provides three modes:

- Single-thread: run dispatch and coarse pipeline regions on the caller.
- Dispatch-parallel: admit eligible pairs from a batch.
- Task-parallel: also allow supported query, region, segment and index-fragment work.

These are runtime scheduling policies, not another dimension of biological algorithm selection. Running a supported region serially does not by itself choose a different minimizer, chainer, or aligner.

Coarse parallel regions share a process-wide guard. Once one such region has acquired it, another coarse region executes serially instead of opening another level of parallel work. For example, a parallel query batch does not also multiply its active workers by independently

[9] UXL Foundation Contributors, How Task Scheduler Works.

parallelizing every query's alignment regions. The guard applies to these coarse regions, rather than prohibiting all nested oneTBB operations. Workload thresholds also avoid submitting very small ranges whose scheduling overhead could exceed their useful work.

Each participating thread needs its own installed computation context for temporary pipeline storage. For context-aware pipeline ranges, the pool supplies contexts scoped to the parallel region and reuses an already installed context when possible. The results outlive temporary scratch through their own owners. Consequently, work stealing changes which thread executes ready work, but does not grant a task access to another thread's mutable alignment scratch storage.

5.5.3. Admit memory before increasing overlap

A CPU limit alone cannot bound memory use. Four simultaneous target-index builds can require roughly four times the temporary storage of one build, even if each task uses only one processor. BioSeqDB therefore combines the arena's concurrency limit with a process-local memory-admission controller. Admission means obtaining a reservation, represented by a move-only permit, before starting work whose retained allocation is covered by that permit.

The controller uses the common memory policy's view of the process footprint and available host memory. It also applies an optional index-memory cap and accounts for bytes already promised to live permits. Reservations are updated atomically, so simultaneous tasks in the same process cannot each treat the same remaining reservation capacity as their own. A rejected request returns without waiting for another task to free memory. The caller can reduce the admitted wave, use a supported serial or streaming path, or report that the selected route cannot proceed.

A wave is the group of work items whose input and temporary storage may overlap. It is smaller than the logical batch when the whole batch would retain too much state. Preparation belongs inside this bound. Waiting until after every sequence has been copied would leave the copies outside the intended memory limit. Conversely, a lazy PostgreSQL source need not be flattened merely because its locator belongs to a batch. The selected reader and compute path determine which bytes must become resident.

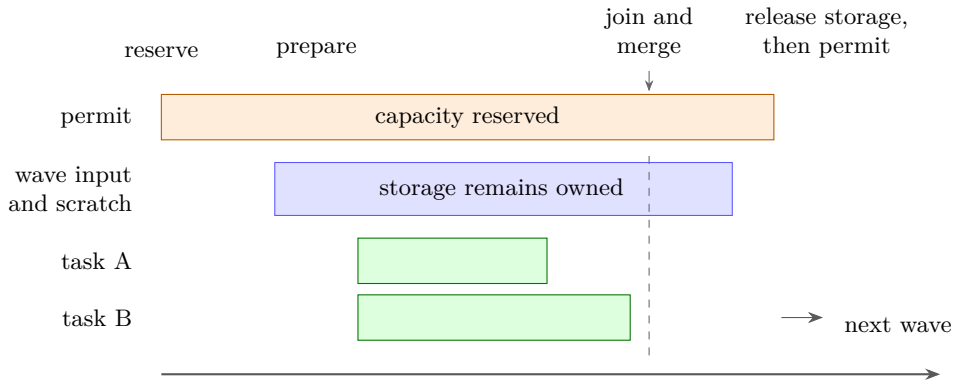


Figure 5.5: An admitted wave retains its input and temporary storage through the join. Completing one task is insufficient to release bytes another task still reads. In the ordinary permit-retaining path shown here, covered storage is released before its reservation becomes available to subsequent work. Output that must survive the wave has a separate owner.

Figure 5.5 distinguishes reservation from allocation. A permit records admitted capacity. It does not own the bytes or prove that the operating system has reserved physical pages.

The ordinary release order is therefore to destroy the covered allocation and then return its permit. Paths that reconcile an allocation with the measured process footprint can adjust reservation accounting, but still need an owner for the bytes and their readers.

Algorithm 4 summarizes the CPU scheduling boundary. It deliberately leaves biological implementation selection outside the loop. An unsupported requested implementation is handled by its admission policy, rather than silently replaced because fewer CPU tasks can run.

Algorithm 4 CPU work under backend ownership and bounded admission

```
Require: supported CPU request and a bounded batch
Backend adopts its scheduling configuration
for each next row or query wave do
    Validate source and index identities on the backend
    Obtain admission for required resident input and scratch
    if mode, work size, ownership, and admission allow parallel work then
        Prepare retained, worker-shareable input
        Run independent items in the local arena into separate result slots
        Join the submitted range
    else
        Execute the supported serial or streaming path
    Check PostgreSQL interrupts at backend-controlled boundaries
    Merge completed results into their continuing owners
    Release the wave's input and temporary storage
    Return remaining admission
```

These decisions bound controlled overlap within one process. They do not provide a cluster-wide memory reservation. Another backend can allocate after a host-memory sample, and unavailable host probes cannot establish how much physical memory remains. Persistent results and other database state also have their own lifetimes. The useful architectural guarantee is the explicit ownership and admission of controlled work, not a claim that any number of sessions or any input can never exhaust memory.

5.5.4. Completion, cancellation, and progress

Submission does not transfer PostgreSQL's API to a worker. Relation access, PostgreSQL allocation, and interrupt checks stay on the backend main thread. The worker runs the permitted computation and returns results or a failure to that thread. For explicitly tracked asynchronous tasks, the owner registers cleanup resources before submission. A bounded error record carries worker failure, and completion is published before joining can release the borrowed storage. The CustomScan lifetime rules in Section ?? supply the enclosing statement cleanup. They need not be implemented again by every numeric algorithm.

This boundary limits cancellation responsiveness. The ordinary parallel range completes before the backend resumes its per-row interrupt checks. The native aligner does not continuously call PostgreSQL's cancellation API while computing an alignment. The explicitly tracked task wrapper can observe a cancellation request before invoking its callback, but that is not evidence of interruption at every point inside an already running kernel. A join protects ownership even when the caller no longer needs a result. It does not imply that already executing CPU work stops instantly.

The scheduler must also retain enough runnable work to make progress. oneTBB is intended primarily for computation that does not spend long periods blocked on IO or mu-

texes.[10] Nonblocking memory admission and the coarse-region guard avoid important forms of nested waiting and resource multiplication. They do not prove general deadlock freedom. BioSeqDB's input path, for example, can overlap parsing with backend writes through a bounded queue, so a claim that every pool task is nonblocking would be incorrect. Its producer and consumer require their own close-and-join protocol. The common arena supplies execution resources, while each operation retains responsibility for a valid dependency graph and bounded ownership.

5.6. Load sequence files through a reusable parser

Public sequence archives distribute FASTA or FASTQ text, often compressed with gzip. BioSeqDB's input layer turns that text into compact sequence chunks and raw headers. A PostgreSQL sink then turns the headers into typed columns and stores the chunks. Separating these jobs lets the same parser serve database loading and ordinary C++ consumers without importing PostgreSQL into the file-reading code.

5.6.1. Separate byte sources from record framing

A byte source fills a caller-owned buffer and reports bytes read, end of input, or an error. The parser is specialized for the source type, so refill calls need no virtual dispatch. Plain files can use a memory mapping or sequential reads. The SQL loading path uses zlib for gzip input. Biological format and compression are selected independently: the filename suffix chooses FASTA or FASTQ, while gzip magic bytes identify compression. The loader does not infer the biological grammar from an arbitrary first record.

The parser design was inspired by Helicase's vectorized parsing and compact DNA output, and by StringZilla's vectorized byte-search operations.[11] BioSeqDB implements its own kernels through Highway. These libraries are not runtime dependencies of the input path. In FASTA, the header begins with `>`, followed by a sequence body. For DNA4 packed into 64-bit units, the optimized body loop searches for the next `>` while encoding symbols. Boundary detection and packing thus share an input pass. Other alphabet and unit combinations use the same record states with their corresponding encoder and line scanner.

FASTQ adds a `+` separator and quality bytes. Quality lines can contain `@`, `+`, or `>`, so those characters cannot independently identify the next record. The parser instead counts quality bytes against the raw sequence length after removing line endings. This count is distinct from the number of symbols retained by the compact encoder.

5.6.2. Deliver chunks while a record remains open

The parser invokes sink functions, called callbacks, when a record begins, when packed chunks are available, and when the record ends. It keeps its current state and an unfinished storage word across refills. Complete words can be delivered before the record ends. The final chunk supplies the remaining word and its valid symbol count. Figure 5.6 shows why an input-buffer boundary need not be a record boundary.

The SQL sink runs on the PostgreSQL backend thread. Its default path consumes callbacks directly. Optional overlap moves owned events through a FIFO queue of fixed capacity: a worker parses, and the backend performs database writes. A full queue blocks the producer.

[10] UXL Foundation Contributors, When Task-Based Programming Is Inappropriate.

[11] Martayan et al., "Helicase: Vectorized parsing and bitpacking of genomic sequences"; Vardanian, StringZilla: Fast SIMD, SWAR, and GPGPU String Processing.

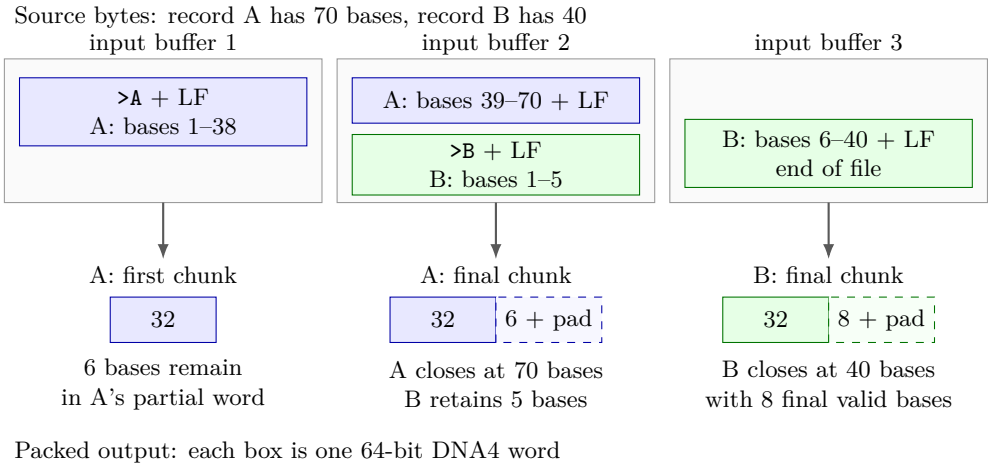


Figure 5.6: Two single-line FASTA records parsed with illustrative 41-byte input buffers. LF denotes a line-feed byte. Header and LF bytes occupy input space but are not sequence symbols. The first refill ends inside A: 32 bases are delivered, while six remain in its partial word. The next buffer finishes A and starts B. Source and packed widths are schematic. Word labels count valid bases, with padding in dashed words. Chunk views are borrowed for their callback and copied by the backend before reuse.

Closing it wakes blocked participants, and the worker is joined before a consumer error is propagated.

The reusable batch API has a different contract. It owns complete batches and can use direct mapped views, parallel parser shards, or a parallel gzip source. Those options do not describe SQL streaming, which refills a slab and uses the single-threaded zlib source for gzip. In particular, a batch parser's throughput is not a measurement of database loading.

5.6.3. Route header fields into SQL columns

Headers can be preserved verbatim or parsed using a named convention, such as GenBank, RefSeq, or UniProt. Their conventions distinguish identifiers from annotations carried in the same definition line.[12] Automatic selection samples a bounded decompressed prefix once per file. The loader also resolves field-to-column routes once, using destination names and any explicit mappings. Per-record parsing then passes field slices to the PostgreSQL sink, which applies the destination types. A header dialect describes metadata structure. It is not a strict validator for the sequence alphabet.

For example, a gzipped FASTA file may contain the header `>dna_new [organism=Homo sapiens] [chromosome=17] [moltype=mRNA]`. The following table gives those fields their relational types:

[12] National Center for Biotechnology Information, FASTA Format for Nucleotide Sequences; UniProt Consortium, FASTA headers.

Listing 5.4: Load a FASTA file with GenBank-style header fields.

```
CREATE TABLE sequences (
  id text,
  organism text,
  chromosome integer,
  moltype text,
  seq bioseq.seq('dna4', 64)
) USING bioseq;

SELECT bioseq.load('sequences'::regclass,
                   '/data/example.fasta.gz');
```

Here automatic routing produces identifier `dna_new`, organism `Homo sapiens`, integer chromosome `17`, and molecule type `mRNA`. The return value is the number of loaded records. Compression changes the byte source, while the SQL interface remains the same.

The sequence column selects the compact alphabet and storage-unit width before parsing starts. Once a record's fragments are ready, the sink writes the parent row and publishes its sequence locator. PostgreSQL controls transaction visibility and rollback. The sink evaluates defaults and `NOT NULL`/`CHECK` constraints. It does not run ordinary `INSERT` triggers or maintain ordinary secondary indexes. Normal trigger-based foreign-key checks and index-based unique or exclusion checks therefore do not run.

The memory guarantee concerns streamed packed bodies. Headers and FASTQ quality storage can grow with record length, and paths that retain unfinished physical lines remain subject to the input-buffer limit. Alphabet handling is permissive, and the declared strict-alphabet option is not enforced by the parser. FASTQ length checks also have an exception for excess quality bytes on an unterminated final line. These limits preclude treating successful loading as complete input validation.

5.7. Remote sequences and local persistence

An installation built with external-provider support exposes metadata, complete nucleotide records and sequence chunks through relations in the `bioseq_external` schema. These relations obtain their rows from remote services. An application can select a versioned accession and insert its sequence into a local BioSeqDB table before repeated analysis.

Metadata discovery and sequence retrieval use distinct requests. Generated HTTPS clients perform bounded metadata requests, and provider adapters normalize their responses. For NCBI, the catalog reads Datasets assembly reports.[13] A nucleotide accession such as `NC_012920.1` instead requests plain-text FASTA from the EFetch nucleotide endpoint.[14] Assembly accessions beginning with `GCF_` or `GCA_` select a Datasets ZIP package containing sequence files. An assembly accession and a constituent sequence accession therefore identify different retrieval units.

The remote relation returns an unbound sequence literal: sequence characters without a selected storage alphabet or packed layout. In Listing 5.5, assignment to the destination `bioseq.seq('dna5',64)` column validates and encodes those characters for the declared alphabet and storage width.

[13] National Center for Biotechnology Information, NCBI Datasets v2 REST API documentation.

[14] Sayers, "The E-utilities In-Depth: Parameters, Syntax and More."

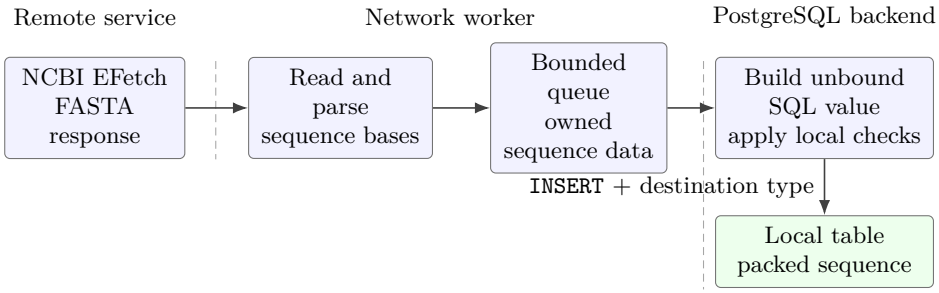


Figure 5.7: An exact NCBI sequence request followed by local insertion. Remote delivery and persistent sequence encoding have separate owners.

Listing 5.5: Importing one versioned NCBI sequence into local storage.

```
CREATE TABLE ncbi_sequences (
    accession text,
    sequence bioseq.seq('dna5', 64),
    retrieved_at timestamptz
) USING bioseq;

INSERT INTO ncbi_sequences
SELECT accession, sequence, retrieved_at
FROM bioseq_external.sequences
WHERE provider = 'ncbi' AND accession = 'NC_012920.1';

SELECT accession, bioseq.len(sequence) AS bases
FROM ncbi_sequences;
```

Ordinary SQL qualifications remain local checks even when equality predicates also constrain the remote request. Array expansion is capped at 64 query combinations, with a warning when combinations are omitted. A provider union may return partial results if one provider fails, and metadata scans do not automatically follow every page. Explicit accessions and a single provider make the import's requested scope clear. Selecting fewer columns from a sequence relation does not currently avoid sequence retrieval.

For the NCBI streaming route, a worker passes owned sequence data through a queue bounded by both bytes and item count. Chunk delivery limits temporary buffering, whereas a complete-record value still requires record-sized memory. Archive retrieval buffers a size-limited ZIP before decoding. The backend checks cancellation between bounded queue waits. Transient FASTA requests can be retried only before the first row is emitted, preventing replay of visible chunks.

Remote responses are not a durable cache, and request rates are not coordinated across backends. E-utilities usage requirements therefore remain relevant to the caller.[15] A transaction controls the local INSERT, but does not make separate HTTP requests one database snapshot. Retaining the accession version and retrieval time records which import supplied the stored sequence. Subsequent queries of the local table need no remote request.

[15] Sayers, "A General Introduction to the E-utilities."

5.8. Generation and validation

The shared computational core needs several interfaces: typed C++ calls, PostgreSQL functions, configuration settings, and benchmark adapters. Keeping these interfaces consistent is part of the architecture. Generation removes some repeated declarations, while assigning compilation work to modules limits its repetition.

5.8.1. Deriving the PostgreSQL boundary

A PostgreSQL routine repeats one contract in several places. Installation SQL declares its argument types, result and volatility. A C entry point must expose the corresponding symbol, decode those arguments and encode that result. BioSeqDB derives these artifacts from annotated C++ declarations. The following existing annotation fixture illustrates the API. Its declaration supplies the name and C++ types, while attributes supply SQL metadata:

Listing 5.6: A typed declaration with SQL metadata.

```
[[bioseq_sql::function({
    .volatility = bioseq::sql::volatility::immutable,
})]]
inline constexpr std::int32_t identity(
    [[bioseq_sql::param({.sql_type = bioseq::sql::pg::integer})]]
    std::int32_t value) {
    return value;
}
```

The generator uses Clang's C++ LibTooling and abstract syntax tree (AST) interfaces through `libclang-cpp`. Compiler actions inspect registered module owners under an explicit compilation profile.[16] AST facts describe declarations and types. Compiler token ranges associate annotations and documentation with those declarations. BioSeqDB then parses the annotation options as a restricted language and validates their admitted forms. Unknown options, conflicting annotations, duplicate defaults and unsupported C++/SQL conversions stop generation. Volatility and parallel safety remain author-supplied contracts: the generator does not prove them from an arbitrary function body.

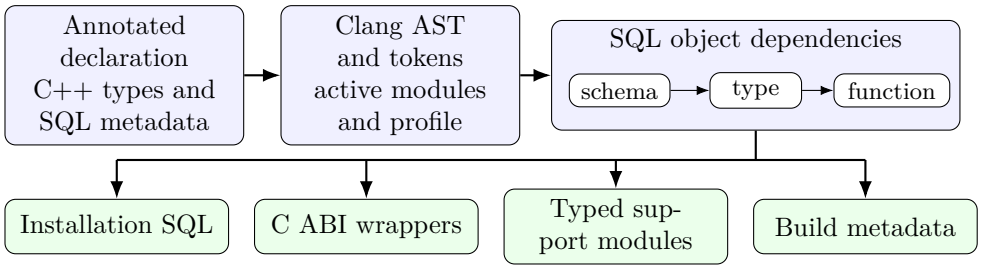


Figure 5.8: One checked interface produces related artifacts. The small dependency chain is illustrative. Typed support includes row codecs, configuration bindings and catalog handles where the declarations require them.

The graph records dependencies among declared SQL objects, each with a stable identity. Tarjan's algorithm identifies strongly connected components, after which a stable topological traversal orders the admitted component graph.[17] Unsupported cycles are rejected. Base

[16] The Clang Team, LibTooling; The Clang Team, Choosing the Right Interface for Your Application.
[17] Tarjan, "Depth-First Search and Linear Graph Algorithms."

types additionally need statement-level ordering between their shell declaration, input/output functions and completed definition.

Generated support code must exist before its consumers compile. The build therefore produces row codecs, configuration bindings and catalog handles before final SQL and wrappers. Final generation checks these intermediate outputs against the active declarations. Separate build profiles keep test-only declarations out of the production interface.

The generated C ABI also centralizes exception translation. C++ exceptions must not escape into PostgreSQL, while a PostgreSQL error can jump past C++ destructors.[18] The boundary preserves captured database diagnostics and leaves C++ exception scopes before raising the SQL error. Calls into PostgreSQL below that boundary still require an explicit bridge or a region without live nontrivial C++ objects.

Language reflection is a related alternative, but it is not the mechanism used here. Reflection was adopted for C++26.[19] Upstream Clang still lists it as unsupported.[20] BioSeqDB therefore uses an external compiler pass, including source locations, preprocessing state and module provenance, together with its own annotation validation.

5.8.2. Controlling specialization cost

Templates specialize kernels for concrete alphabets, storage layouts and algorithm choices. Independent parameters multiply the potential number of instantiations (Figure 5.9). A parameter can create new compiler work even when it does not change a downstream algorithm. Link-time removal of equivalent code cannot recover all the earlier parsing, instantiation and optimization work.

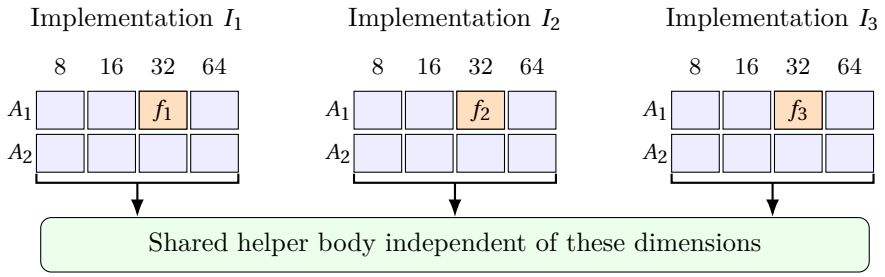


Figure 5.9: Illustrative specialization product: two alphabets, four storage widths in bits, and three implementations produce 24 cells. Each cell denotes one possible function body. Highlighted cells keep alphabet and width fixed while changing implementation. This is not a production support matrix.

BioSeqDB limits this cost by selecting length buckets after fixing the k-mer size, passing already selected input representations into scheduled work, and assigning expensive specializations to concrete module owners. Shared computational modules can be compiled once for compatible consumers. An ordinary benchmark and the extension can consequently reuse the same core without each owning its compilation again. This does not eliminate specialization: it gives the build graph explicit work units and avoids repeating irrelevant choices.

The cost has been substantial during development. Recorded May 2026 runs included a 32-minute debug benchmark build and PostgreSQL translation-unit compilations that exceeded a 50-minute timeout. These are historical observations, not current build timings or

[18] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Using C++ for Extensibility.
[19] Köppe et al., Editors' Report: Programming Languages—C++.
[20] The Clang Team, C++ Support in Clang.

measurements of SQL generation alone. Later investigations also found expensive chains of imported modules behind small function bodies. Controlling both template dimensions and module ownership is therefore necessary.

5.8.3. Benchmark specifications and validation

Benchmarking uses a generic framework and adapters for individual workloads. The framework owns configuration validation, matrix expansion, scheduling, measurement and report generation. An adapter defines the experiment: how to prepare its input, invoke an algorithm or SQL statement, evaluate the result and release temporary resources. The same orchestration serves in-process alignment, sequence input and output, and PostgreSQL experiments.

A JSON specification declares named inputs and algorithm parameter schemas. Within each benchmark, `matrix.for` assigns repetition counts to inputs, while `matrix.over` lists algorithms and parameter values. The framework expands the combinations separately for each algorithm and fills omitted parameters from declared defaults. Schema and reference checks reject inconsistent specifications before execution. JSON selects implementations already registered by the adapters.

Listing 5.7: Input and algorithm selection fields from an alignment benchmark specification.

```
"matrix": {
  "for": {"RMQ_REPRO_LONG_CCS": 100},
  "over": {
    "bioseq_ksw2_rmq_scalar_mm": {},
    "minimap2_rmq": {}
  }
}
```

This excerpt selects 100 repetitions of each registered alignment route on the same named input. The empty objects use the declared parameter defaults. Input definitions, metric selection and other run settings are omitted.

The measurement boundary is explicit. The framework times the execution callback, then passes its input and output to the evaluator. This permits reference computations outside that timer. However, adapters may also check results while constructing their output, so elapsed time must be interpreted together with the measured region. Alignment experiments through PostgreSQL check expected matches, coordinates within the configured tolerance and strand. Selected cases additionally validate CIGAR replay. These checks complement authored integration tests of SQL behavior.

Reports retain metric summaries and rule outcomes. Adapters can additionally emit durable raw samples. Rules compare selected configurations within one run, while regression checks use eligible historical captures. Eligibility requirements determine which runs can enter a historical comparison. The configured matrix still determines coverage: a successful timing run does not establish correctness for every query, setting or executor path.

5.9. Authorship, development and AI assistance

I developed BioSeqDB over approximately three years. Early work included Python aligner prototypes written approximately two years ago. I designed the system and implemented the BioSeqDB-specific algorithms, representations and optimizations presented as my contributions in the preceding chapters. Moving from those prototypes to a PostgreSQL extension also required substantial integration, testing and performance work.

These contributions build on published algorithms and existing software. The minimap2 mapping structure, prior SIMD minimizer work and adaptive block alignment are important foundations.[21] The implementation includes ports and adaptations, as well as new work. The basic columnar store comes from Citus and Hydra and retains code written by their authors and maintainers, as described in Section 3.1.[22] PostgreSQL, SeqAn, Highway and oneTBB also provide substantial components. The original contribution lies in the specific algorithms, changes and integration identified in this thesis. It does not imply authorship of the underlying published algorithms or of all source code in the repository.

5.9.1. Where AI tools were used

I used AI coding tools under requirements and designs that I supplied. Their work included:

- Source reorganization. Long files were divided into modules, functions were renamed, and code was adjusted for Clang-based checks. Many present file and function names were produced during this work.
- Development tools. AI helped implement benchmarking utilities and code generators according to my designs, and assisted with implementation and optimization tasks.
- Technical writing. AI generated working documentation.

The large reorganizations were intended to preserve the algorithms while changing names and file boundaries. They nevertheless introduced bugs. In my assessment, diagnosing and correcting regressions from AI-assisted work consumed approximately two months. Numerical code was particularly sensitive: a seemingly equivalent change to a mask, integer width or coordinate calculation could change results. A refactoring that compiles is therefore insufficient to establish numerical correctness.

AI assistance accelerated parts of development despite these costs. The research questions, project-specific designs and decisions about which ideas to pursue were mine. I directed and corrected the assisted work and retain responsibility for the final implementation and the claims made here.

Automatically generated documentation is not an authoritative specification. Some descriptions were incorrect, and others became stale after later changes. For this chapter, generated descriptions were checked against the implementation, the cited technical sources and the stated tests. Generated names and reports alone do not establish how a feature works.

5.10. Validation limits of the complete system

Validation of query-result caching, described in Section 2.5, remains incomplete. It has targeted tests, including comparisons with caching disabled, but these do not cover all combinations of plans, settings and transaction histories. Incorrect reuse and therefore incorrect query results remain a risk. The cache should be treated as experimental.

[21] Li, "Minimap2: pairwise alignment for nucleotide sequences"; Groot Koerkamp and Martayan, "SimdMinimizers: Computing Random Minimizers, fast"; Liu and Steinegger, "Block Aligner: an adaptive SIMD-accelerated aligner for sequences and position-specific scoring matrices."

[22] Citus Data contributors, Citus Columnar storage implementation; Hydra contributors, Hydra Columnar: PostgreSQL columnar storage extension.

Chapter 6

Benchmarking and performance evaluation

6.1. Benchmarking and performance evaluation

There will be some content here.

6.1.1. End-to-end results and their interpretation

The following tables present the approved results available while the benchmark campaign is still running. They compare the BioseqDB mapping pipeline with minimap2 on one local Apple Silicon machine and four AWS machines. Each timing covers a batch of 1,024 queries. A missing measurement remains visibly missing, so the tables describe completed observations rather than an estimated full matrix. The machine inventory precedes the result tables.

- Index. A persisted index is constructed before measurement. Each repeat starts a fresh process and measures reading the index and mapping the queries. Construction of the persisted index is excluded from the current mapping interval. The index is available on disk, but is not retained in the previous process's memory. Operating-system page caches are not purged, so this is process-cold execution rather than a guaranteed cold disk read.

- No index. No pre-existing user index is available. The current measured interval includes transient target-index construction and mapping. The transient index can be reused within the batch, including by the remaining 1,023 queries after its initial construction. Fixture generation and process restart setup lie outside the current measured interval.

- Timing scopes. Historical outer-E2E timings are treated as mismeasurements and excluded from the tables, coverage counts and comparisons. Standard observations use the current framework mapping-iteration interval. FAST fallbacks retain their distinct single-sample timing scope and carry an ^{s} marker. They are never pooled with standard repeats. The final external-aligner table identifies its separate timing boundary explicitly. In particular, "mapping interval" still includes transient construction in no-index mode.

- Two measurements per cell. The first duration, A, is the nearest-rank P50 of the complete measured interval. The second, B, is the nearest-rank P50 after subtracting the physical union of instrumented CIGAR and MAPQ intervals separately in each repeat. With five repeats the P50 is the third sorted observation. For FAST entries, P50 is the single observed value. If U_i is that interval union, the definitions are

  $$A_{50} = \mathrm{P50}(A_i), \qquad B_{50} = \mathrm{P50}(A_i - |U_i|).$$

  Overlapping intervals are subtracted only once. The published medians are used directly, without subtracting independently aggregated medians. On both BioseqDB KSW2 and Block Aligner paths, the CIGAR interval contains fused base alignment, trace storage, traceback and result construction. B therefore does not universally represent seeding and chaining alone. Likewise, $A_{50} - B_{50}$ is not a measured median CIGAR/MAPQ cost.

Input sequences and repetitions. The campaign configuration defines eight deterministic target fixtures, covering both target sizes, both alphabets and both minimizer parameter pairs. Each fixture contains one reference sequence and 1,024 derived queries. The same fixture is reused across the relevant algorithm configurations and repeats. The configured parameters are as follows.

- Target sizes and minimizers. References contain exactly 2×10^9 or 4×10^9 base pairs, denoted 2 and 4 decimal Gbp. The parameter pairs are $(k, w) = (15, 10)$ and $(20, 15)$, where k is the k-mer length and w is the number of k-mers in a minimizer window.

- Alphabets. DNA4 uses equiprobable canonical bases. The DNA5 generator assigns probability 1% to N and 24.75% to each canonical base.

- Query lengths and strand. The sampled pre-edit length has Gaussian mean 20,000 bp and standard deviation 500 bp, with rounding and clamping in the generator. Insertions and deletions can change the final length. A query is reverse complemented with probability 0.5.

- Divergence parameters. Substitution probability is 0.001. Insertion-event and deletion-event probabilities are each 0.0005. Their nominal sum is 0.002, or 0.2%, rather than a measured edit-distance rate. Indel lengths have a configured mean parameter of 1 bp and the generator's default standard-deviation parameter of 1 bp, followed by rounding and clamping. They are not fixed single-base edits.

- Occurrences. Each query originates from one known sampled reference interval. The fixture does not specify an occurrence-count distribution or an occurrence standard deviation. This provenance does not imply that an aligner returns exactly one hit. The original five summaries certify exact baseline match-count equality but intentionally omit the actual counts.

- Repetitions and variability. Standard observations have five accepted repeats. FAST observations have one measured batch and cannot establish repeat-to-repeat variability. A repeat measures the complete query batch, not a single query. The original five summaries contain medians and repeat counts, but no raw timing samples, so timing standard deviations and confidence intervals cannot be recovered from these files.

- FAST procedure. Compatible persisted indexes are reused across index-mode configurations. PostgreSQL restarts before measurement, and the operating-system filesystem cache is not cleared. Index preparation and restart are outside the timer. No-index measurements still include transient index construction. FAST remote runs omit the additional plan and alternative-setting controls, while retaining measured-statement correctness and timing checks.

Execution rows. BioseqDB uses task-parallel execution with prefiltering disabled and both CIGAR and MAPQ projection enabled. MPF and MPS denote minimizers per fragment and per sequence. Each row identifies the requested minimizer, chaining algorithm and base aligner in addition to this index representation. Scalar-DP and SIMD-DP refer to the scalar and literal SIMD minimap2 chaining routes. RMQ denotes the separate range-minimum-query algorithm. SIMD-ref is the reference minimizer, Two-stack is the two-stack ntHash minimizer, and Bounded is bounded ntHash SIMD. Block denotes Block Aligner. Vulkan rows can include CPU index handling, backtracking and base alignment, and do not imply an entirely GPU pipeline. If none of the Vulkan configurations has a measurement for the workloads and machines in a table, those configurations share one "Vulkan (all modes)" row. Recorded GPU unavailability is marked explicitly. A missing timing alone is not taken as evidence of unavailable hardware.

Reading comparisons and highlights. Each cell places the A duration above its comparison, followed by the B duration and its own comparison. Durations are rounded to tenths of a second for display. All comparisons and minima use the original integer nanoseconds.

- A BioseqDB value is compared with minimap2 for the same machine class, fixture and index mode. Marked FAST fallbacks may be compared with standard observations, which have a different repeat count and execution procedure. The minimap2 row compares with the fastest measured eligible BioseqDB effective path in that same column, independently for A and B. Thus its two reference algorithms can differ. Baselines are repeated on continuation pages solely to assist reading.

- A green upward triangle means less elapsed time. A red downward triangle means more elapsed time. A gray rightward triangle indicates a tie when the ratio of the larger to the smaller duration is at most 1.05. This is a display convention, not a statistical significance test.

- For current duration t and reference r, the percentage is $100|t-r|/r$, describing relative elapsed-time reduction or increase. Differences above 30% use ratio notation instead: the larger duration divided by the smaller, rounded to one decimal place. A green 2.5× therefore means the reference takes 2.5 times as long, while a red 2.5× means the displayed execution takes 2.5 times as long. Differences through 30% retain percentage notation. Direction is always conveyed by the triangle.

- A blue background marks one smallest A and one smallest B down each machine/workload column, including minimap2 and all measured BioseqDB rows across continuation pages. Exact ties select the first displayed row. Repeated minimap2 baselines receive blue marks only on the first page, so each column has two winner marks across the complete table when measurements are available. Each AWS workload has four machine subcolumns separated by vertical rules.

- Light gray identifies every minimap2 baseline row. Pale yellow identifies the preferred BioseqDB configuration: MPF / Bounded / SIMD-DP / Block. This is the author's selected configuration for comparison with the external aligners. The yellow tint does not assert that it wins every workload. Blue timing highlights identify the measured column winners and remain visible within either row tint.

- A dot means there is no approved observation in the snapshot. A horizontal dash with ^{g} identifies an explicitly recorded unavailable GPU configuration, which was not executed. Neither marker is a zero duration. The comparison reads n/a when no measured counterpart exists. Only execution rows with an approved observation or an explicit unavailable cell in a panel are printed. Entirely unmeasured row families remain omitted.

Limits of the comparison. These observations were accumulated across builds. Some retain an older sealed minimap2 correctness control, which is not a new contemporaneous performance run. A correctness-only index control is never substituted for a no-index performance baseline. Comments in the source authorities also disclose missing process-priority attestation, recovery after post-measurement wrapper failures and historical callback-lifetime caveats. The ^{p} marker identifies the local observation retained by an explicit preservation decision without a run receipt. Approval of its measurement does not certify the safety of its historical build. No uniform elevated-priority or noise-free execution claim is made.

The remote C7G observations include severe paging, and other observations also report resource pressure. Cross-machine differences therefore combine hardware, resource availability and execution conditions. They are not an isolated test of instruction-set performance.

The table records physical RAM and provisioned storage capacity. Selected row comments contain sampled available memory, but there is no uniform per-repeat series of free-memory and free-disk measurements. Missing C7G BioseqDB timings cannot establish either a win or a loss.

Reproducible table generation. The standalone Python generator reads only the normative `approved_results` rows and the remote `hardware_unavailable_cells`, with their embedded machine metadata. It validates identities and timing scopes, rejects duplicates within a source, and records the SHA-256 of each input in the generated LaTeX source. Coverage totals are recomputed from approved rows rather than copied from historical progress summaries. Regeneration is manual and is not part of CMake. The fixture description was checked against the campaign JSON and its sequence generator, together with DOC0432 and DOC0433. The full row-specific qualifications remain in the source YAML authorities. For each remote cell, the generator selects the first available approved observation from the default summary, then TEST3, TEST4, TEST7, TEST2 and finally FAST for the same machine class and workload. Local cells use the default local summary followed by FAST LOCAL. Both A and B come from the same selected observation. The generator checks that every unique eligible tuple in these sources is rendered once logically, excluding the historical mismeasurements and accounting for repeated baseline rows on continuation pages. Missing fallback files are skipped. This selection also supplies the minimap2 and BioseqDB rows in the final comparison table. The final table additionally reads the four external-aligner authorities and explains their separate timing and validation rules.

Table 6.1: Machine inventory captured with the benchmark summaries. Capacities are not free resources at execution time.

Property	Local	M7A	M7I	M4	C7G
Machine	MacBook Pro	m7a.xlarge	m7i.xlarge	m4.xlarge	c7g.xlarge
CPU	Apple M1 Pro	AMD EPYC 9R14	Xeon Platinum 8488C	Xeon E5-2686 v4 @ 2.30GHz	Neoverse-V1
Architecture	arm64	x86_64	x86_64	x86_64	aarch64
OS version / kernel	macOS 15.6 / Darwin 24.6.0	Linux / 7.0.0-1012-aws	Linux / 7.0.0-1012-aws	Linux / 7.0.0-1012-aws	Linux / 7.0.0-1012-aws
OS build / distribution	24G5054d	Distribution not recorded	Distribution not recorded	Distribution not recorded	Distribution not recorded
Logical / physical cores	8 / 8 (6P + 2E)	4 / 4	4 / 2	4 / 2	4 / 4
Physical RAM	16 GiB unified	16 GiB	16 GiB	16 GiB	8 GiB
GPU	Apple M1 Pro, 14 cores	None available	None available	None available	None available
Storage	APPLE SSD AP0512R	EBS gp3	EBS gp3	EBS gp3	EBS gp3
Disk capacity	500.28 GB	500 GiB	500 GiB	500 GiB	500 GiB
Provisioned disk I/O	Not recorded	6,000 IOPS / 500 MiB/s	6,000 IOPS / 500 MiB/s	6,000 IOPS / 500 MiB/s	6,000 IOPS / 500 MiB/s
Free disk / free RAM	No uniform per-repeat record	No uniform per-repeat record	No uniform per-repeat record	No uniform per-repeat record	No uniform per-repeat record

The AWS summaries record the Linux kernel but do not seal an OS distribution version. The local legacy capture does not seal the MoltenVK implementation identity. Unrecorded values are not inferred from the current machine.

Remote cells use the first available approved result in this order: the default summary, TEST3, TEST4, TEST7, TEST2, then FAST. Local cells use the default summary, then FAST LOCAL. A and B come from the same observation. The inventory describes the default hosts. Fallback measurements may come from other instances of the same machine class, and comparisons may combine campaigns.

Table 6.2: Published snapshot coverage after excluding mismeasured historical timings.

Host	Logical tuples	Observations	Repeats	BioseqDB obs.	GPU unavailable
Local	111	111	523	95	0
M7A	261	261	1305	245	512
M7I	256	256	1280	240	512
M4	90	90	450	74	512
C7G	84	84	420	68	512

The published BioseqDB campaign snapshot contains 802 timing observations and 3,978 accepted repeats. FAST entries retain their separate single-sample timing scope and are marked with a superscript s. The 66 historical outer-E2E observations are treated as mismeasurements and excluded from coverage and comparisons. Standard observations have five accepted repeats. FAST observations have one. The planned local matrix has 1,296 logical cells and 6,480 repeats. It remains incomplete. Unavailable-GPU counts refer only to the remote authorities' recorded eligible subsets.

Engine	Execution	Index								No index							
		4 Gbp				2 Gbp				4 Gbp				2 Gbp			
		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15	
		DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5
minimap2	Baseline	39.7s ▼2.2× 36.8s ▼2.2×	25.2s ▼17% 22.7s ▼12%	1m 56.9s ▼8.0× 1m 53.2s ▼8.2×	24.9s ▼19% 22.2s ▼12%	28.6s ▼11.5× 25.8s ▼15.6×	15.3s ▼2.8× 12.7s ▼3.3×	29.3s ▼15.0× 26.2s ▼22.9×	8.7s ▼1.7× 8.1s ▼2.3×	2m 52.4s ▲19% 2m 48.5s ▲15%	2m 17.1s ▲3.9× 2m 15.9s ▲1.7×	2m 55s ▼6% 2m 51s ▼12%	55.9s ▲8.6× 55.1s ▲3.1×	43.3s ▲2.6× 42.5s ▲2.4×	38.9s ▲8.0× 38.2s ▲2.8×	53.1s ▲1.6× 48.2s ▲1.5×	28.7s ▲9.8× 28.2s ▲2.9×
BioseqDB	MPF / Bounded RMQ / Block	·	·	·	·	·	5.9s ▲2.6×[s] 4.6s ▲2.8×[s]	9.5s ▲3.1× 9s ▲2.9×	·	·	·	·	·	·	·	·	·
	MPF / Bounded RMQ / KSW2	·	·	·	·	·	5.8s ▲2.6×[s] 3.9s ▲3.3×[s]	·	·	·	·	·	·	·	·	·	·
	MPF / Bounded Scalar-DP / Block	·	·	·	·	·	5.4s ▲2.8×[s] 4.3s ▲3.0×[s]	2.4s ▲12.0× 1.9s ▲13.8×	·	·	·	·	·	·	·	·	·
	MPF / Bounded Scalar-DP / KSW2	·	·	·	·	·	5.8s ▲2.6×[s] 4.2s ▲3.1×[s]	2.4s ▲12.3× 1.1s ▲22.9×	·	·	·	·	·	·	·	·	·
	MPF / Bounded SIMD-DP / Block	17.7s ▲2.2× 17s ▲2.2×	21.5s ▲15% 20.2s ▲11%	14.6s ▲8.0× 13.8s ▲8.2×	21s ▲16% 19.8s ▲11%	2.5s ▲11.5× 1.7s ▲15.6×	5.4s ▲2.8× 4.2s ▲3.1×	2s ▲15.0× 1.3s ▲20.4×	5.3s ▲1.6× 3.9s ▲2.1×	3m 48.1s ▼1.3× 3m 35s ▼28%	8m 57.6s ▼3.9× 3m 49.2s ▼1.7×	2m 52s ▶2% 2m 39.3s ▲7%	8m 13.1s ▼8.8× 2m 54.5s ▼3.2×	1m 54.3s ▼2.6× 1m 45.5s ▼2.5×	5m 17.3s ▼8.2× 1m 54.9s ▼3.0×	1m 28.3s ▼1.7× 1m 16.8s ▼1.6×	4m 45.7s ▼9.9× 1m 26.6s ▼3.1×
	MPF / Bounded SIMD-DP / KSW2	20.8s ▲1.9× 19.1s ▲1.9×	·	19.8s ▲5.9× 18.2s ▲6.2×	30.1s ▼21% 28.5s ▼28%	3.7s ▲7.8× 2.4s ▲10.7×	6s ▲2.5× 4.5s ▲2.9×	2.5s ▲11.7× 1.2s ▲22.6×	5s ▲1.7× 3.6s ▲2.3×	3m 32.7s ▼23% 3m 17.7s ▼17%	·	2m 44.9s ▲6% 2m 32.2s ▲11%	·	1m 52.7s ▼2.6× 1m 40s ▼2.4×	5m 12.1s ▼8.0× 1m 46.7s ▼2.8×	1m 26.1s ▼1.6× 1m 13.9s ▼1.5×	4m 41.4s ▼9.8× 1m 21.9s ▼2.9×

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Row order: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index								No index							
		4 Gbp				2 Gbp				4 Gbp				2 Gbp			
		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15	
		DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5
minimap2	Baseline	39.7s ▼2.2× 36.8s ▼2.2×	25.2s ▼17% 22.7s ▼12%	1m 56.9s ▼8.0× 1m 53.2s ▼8.2×	24.9s ▼19% 22.2s ▼12%	28.6s ▼11.5× 25.8s ▼15.6×	15.3s ▼2.8× 12.7s ▼3.3×	29.3s ▼15.0× 26.2s ▼22.9×	8.7s ▼1.7× 8.1s ▼2.3×	2m 52.4s ▲19% 2m 48.5s ▲15%	2m 17.1s ▲3.9× 2m 15.9s ▲1.7×	2m 55s ▼6% 2m 51s ▼12%	55.9s ▲8.6× 55.1s ▲3.1×	43.3s ▲2.6× 42.5s ▲2.4×	38.9s ▲8.0× 38.2s ▲2.8×	53.1s ▲1.6× 48.2s ▲1.5×	28.7s ▲9.8× 28.2s ▲2.9×
BioseqDB	MPF / Bounded Vulkan-DP / Block	·	·	·	·	37.9s ▼1.3×[s] 25.3s ▶ 2%[s]	4m 4.2s ▼16.0×[s] 38.6s ▼3.0×[s]	·	·	·	·	·	·	·	·	·	·
	MPF / Bounded Vulkan-DP / KSW2	·	·	·	·	42s ▼1.5×[s] 27.9s ▼8%[s]	3m 59.2s ▼15.7×[s] 30.8s ▼2.4×[s]	·	·	·	·	·	·	·	·	·	·
	MPF / Scalar RMQ / Block	·	·	·	·	·	·	1m 50s ▼3.7× 1m 49.6s ▼4.2×	·	·	·	·	·	·	·	·	·
	MPF / Scalar Scalar-DP / Block	·	·	·	·	·	·	1m 48.8s ▼3.7× 1m 48.4s ▼4.1×	·	·	·	·	·	·	·	·	·
	MPF / Scalar SIMD-DP / Block	·	·	·	·	·	·	1m 52.3s ▼3.8× 1m 51.9s ▼4.3×	·	·	·	·	·	·	·	·	·
	MPF / Scalar Vulkan-DP / Block	·	·	·	·	·	·	12m 37.6s ▼25.8× 12m 26.5s ▼28.5×	·	·	·	·	·	·	·	·	·
	MPF / SIMD-ref RMQ / Block	·	·	·	·	·	·	1m 40.4s ▼3.4× 1m 40s ▼3.8×	·	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Row order: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index								No index							
		4 Gbp				2 Gbp				4 Gbp				2 Gbp			
		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15	
		DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5
minimap2	Baseline	39.7s ▼2.2× 36.8s ▼2.2×	25.2s ▼17% 22.7s ▼12%	1m 56.9s ▼8.0× 1m 53.2s ▼8.2×	24.9s ▼19% 22.2s ▼12%	28.6s ▼11.5× 25.8s ▼15.6×	15.3s ▼2.8× 12.7s ▼3.3×	29.3s ▼15.0× 26.2s ▼22.9×	8.7s ▼1.7× 8.1s ▼2.3×	2m 52.4s ▲19% 2m 48.5s ▲15%	2m 17.1s ▲3.9× 2m 15.9s ▲1.7×	2m 55s ▼6% 2m 51s ▼12%	55.9s ▲8.6× 55.1s ▲3.1×	43.3s ▲2.6× 42.5s ▲2.4×	38.9s ▲8.0× 38.2s ▲2.8×	53.1s ▲1.6× 48.2s ▲1.5×	28.7s ▲9.8× 28.2s ▲2.9×
BioseqDB	MPF / SIMD-ref Scalar-DP / Block	·	·	·	·	·	·	1m 34.2s ▼3.2× 1m 34s ▼3.6×	·	·	·	·	·	·	·	·	·
	MPF / SIMD-ref SIMD-DP / Block	·	·	·	·	·	·	1m 42.9s ▼3.5× 1m 42.6s ▼3.9×	·	·	·	·	·	·	·	·	·
	MPF / Two-stack RMQ / Block	·	·	·	·	·	·	1m 35.2s ▼3.2× 1m 34.5s ▼3.6×	·	·	·	·	·	·	·	·	·
	MPF / Two-stack RMQ / KSW2	·	·	·	·	·	·	1m 12.2s ▼2.5× 1m 10.5s ▼2.7×	·	·	·	·	·	·	·	·	·
	MPF / Two-stack Scalar-DP / Block	·	·	·	·	·	·	1m 21s ▼2.8× 1m 20.6s ▼3.1×	·	·	·	·	·	·	·	·	·
	MPF / Two-stack Scalar-DP / KSW2	·	·	·	·	·	·	1m 21.1s ▼2.8× 1m 19.3s ▼3.0×	·	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Row order: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index								No index							
		4 Gbp				2 Gbp				4 Gbp				2 Gbp			
		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15	
		DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5
minimap2	Baseline	39.7s ▼2.2× 36.8s ▼2.2×	25.2s ▼17% 22.7s ▼12%	1m 56.9s ▼8.0× 1m 53.2s ▼8.2×	24.9s ▼19% 22.2s ▼12%	28.6s ▼11.5× 25.8s ▼15.6×	15.3s ▼2.8× 12.7s ▼3.3×	29.3s ▼15.0× 26.2s ▼22.9×	8.7s ▼1.7× 8.1s ▼2.3×	2m 52.4s ▲19% 2m 48.5s ▲15%	2m 17.1s ▲3.9× 2m 15.9s ▲1.7×	2m 55s ▼6% 2m 51s ▼12%	55.9s ▲8.6× 55.1s ▲3.1×	43.3s ▲2.6× 42.5s ▲2.4×	38.9s ▲8.0× 38.2s ▲2.8×	53.1s ▲1.6× 48.2s ▲1.5×	28.7s ▲9.8× 28.2s ▲2.9×
BioseqDB	MPF / Two-stack SIMD-DP / Block	·	·	·	·	·	·	1m 45s ▼3.6× 1m 44.4s ▼4.0×	·	·	·	·	·	·	·	·	·
	MPF / Two-stack SIMD-DP / KSW2	·	·	·	·	·	·	1m 21.3s ▼2.8× 1m 19.5s ▼3.0×	·	·	·	·	·	·	·	·	·
	MPF / Two-stack Vulkan-DP / Block	·	·	·	·	·	·	12m 53.5s ▼26.4× 12m 25.3s ▼28.4×	·	·	·	·	·	·	·	·	·
	MPF / Two-stack Vulkan-DP / KSW2	·	·	·	·	·	·	13m 56.9s ▼28.5× 13m 25.1s ▼30.7×	·	·	·	·	·	·	·	·	·
	MPF / Vulkan Scalar-DP / Block	·	·	·	·	·	·	3m 41.9s ▼7.6× 3m 31.8s ▼8.1×	·	·	·	·	·	·	·	·	·
	MPF / Vulkan Scalar-DP / KSW2	·	·	·	·	·	·	3m 25.8s ▼7.0× 3m 14.4s ▼7.4×	·	·	·	·	·	·	·	·	·
	MPF / Vulkan SIMD-DP / Block	·	·	·	·	·	·	4m 42.2s ▼9.6× 4m 30.5s ▼10.3×	·	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Row order: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index								No index							
		4 Gbp				2 Gbp				4 Gbp				2 Gbp			
		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15	
		DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5
minimap2	Baseline	39.7s ▼2.2× 36.8s ▼2.2×	25.2s ▼17% 22.7s ▼12%	1m 56.9s ▼8.0× 1m 53.2s ▼8.2×	24.9s ▼19% 22.2s ▼12%	28.6s ▼11.5× 25.8s ▼15.6×	15.3s ▼2.8× 12.7s ▼3.3×	29.3s ▼15.0× 26.2s ▼22.9×	8.7s ▼1.7× 8.1s ▼2.3×	2m 52.4s ▲19% 2m 48.5s ▲15%	2m 17.1s ▲3.9× 2m 15.9s ▲1.7×	2m 55s ▼6% 2m 51s ▼12%	55.9s ▲8.6× 55.1s ▲3.1×	43.3s ▲2.6× 42.5s ▲2.4×	38.9s ▲8.0× 38.2s ▲2.8×	53.1s ▲1.6× 48.2s ▲1.5×	28.7s ▲9.8× 28.2s ▲2.9×
BioseqDB	MPF / Vulkan SIMD-DP / KSW2	·	·	·	·	·	·	3m 33.4s ▼7.3× 3m 19s ▼7.6×	·	·	·	·	·	·	·	·	·
	MPF / Vulkan Vulkan-DP / Block	9m 36.5s ▼14.5× 9m 22.3s ▼15.3×	13m 49.8s ▼32.9× 8m 45.2s ▼23.2×	4m 57.7s ▼2.5× 4m 42.7s ▼2.5×	10m 50.3s ▼26.1× 5m 42.9s ▼15.4×	6m 46.4s ▼14.2× 6m 34.1s ▼15.3×	8m 46.4s ▼34.5× 5m 22.9s ▼25.4×	3m 11.5s ▼6.5× 3m 2.2s ▼6.9×	6m 30.8s ▼45.0× 3m 13.4s ▼23.8×	9m 50.5s ▼3.4× 9m 37.9s ▼3.4×	14m 21.8s ▼6.3× 9m 10.7s ▼4.1×	4m 10.1s ▼1.4× 4m 1s ▼1.4×	9m 51.7s ▼10.6× 4m 35.5s ▼5.0×	6m 16.9s ▼8.7× 6m 6.9s ▼8.6×	9m 20.4s ▼14.4× 5m 57.2s ▼9.4×	3m 10.7s ▼3.6× 3m 0.2s ▼3.7×	6m 41.2s ▼14.0× 3m 21.5s ▼7.1×
	MPF / Vulkan Vulkan-DP / KSW2	·	·	·	·	·	·	3m 46.3s ▼7.7× 3m 33.1s ▼8.1×	·	·	·	·	·	·	·	·	·
	MPS / Bounded Scalar-DP / KSW2	·	·	·	·	·	·	5.5s ▲5.3× 3.9s ▲6.7×	·	·	·	·	·	·	·	·	·
	MPS / Bounded SIMD-DP / Block	·	·	·	·	·	·	4.5s ▲6.6× 3.8s ▲6.9×	·	3m 53.6s ▼1.4× 3m 40.3s ▼1.3×	8m 50.7s ▼3.9× 3m 46s ▼1.7×	2m 51.3s ▶2% 2m 38.1s ▲8%	7m 58.5s ▼8.6× 2m 48.9s ▼3.1×	1m 59.6s ▼2.8× 1m 49s ▼2.6×	5m 15.7s ▼8.1× 1m 54.7s ▼3.0×	1m 28s ▼1.7× 1m 18.3s ▼1.6×	4m 46.9s ▼10.0× 1m 25.3s ▼3.0×
	MPS / Bounded SIMD-DP / KSW2	·	·	·	·	5.2s ▲5.5× 3.2s ▲8.1×	·	5.3s ▲5.5× 3.8s ▲7.0×	·	3m 54s ▼1.4× 3m 39s ▼30%	·	2m 52.6s ▶1% 2m 36.8s ▲8%	·	2m 3.1s ▼2.8× 1m 50.4s ▼2.6×	5m 13.3s ▼8.1× 1m 50.9s ▼2.9×	1m 30.6s ▼1.7× 1m 17.7s ▼1.6×	4m 44.7s ▼9.9× 1m 25.3s ▼3.0×
	MPS / Two-stack Scalar-DP / KSW2	·	·	·	·	·	·	21.8s ▲26% 4.8s ▲5.5×	·	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Row order: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 4 Gbp k=15, w=10								No index 4 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
minimap2	Baseline	36.2s ▼6.9× 35s ▼8.0×	38.1s ▼7.8× 36s ▼9.3×	1m 33.5s ▼10.8× 1m 29.7s ▼13.7×	6m 31.5s n/a 6m 26.3s n/a	31.6s n/a 30.3s n/a	34.1s n/a 32s n/a	1m 9.5s n/a 1m 6.1s n/a	3m 1s n/a 2m 51.5s n/a	2m 55.5s ▲20% 2m 54s ▲17%	2m 50.4s ▲23% 2m 48.2s ▲20%	6m 12.2s ▲9% 6m 8.2s ▶ 5%	44m 20.5s n/a 44m 15.6s n/a	1m 47.1s ▲5.5× 1m 45.8s ▲2.1×	1m 47.2s ▲4.6× 1m 45.1s ▲2.2×	3m 40.3s ▲4.2× 3m 36.6s ▲1.7×	29m 43.4s n/a 29m 38.5s n/a
BioseqDB	MPF / Bounded RMQ / Block	5.4s ▲6.8× 4.6s ▲7.6×	9.9s ▲3.9× 9.1s ▲4.0×	·	·	·	·	·	·	4m 2.1s ▼1.4× 3m 53.3s ▼1.3×	3m 42.6s ▼1.3× 3m 31.1s ▼25%	·	·	·	·	·	·
	MPF / Bounded RMQ / KSW2	6.9s ▲5.2× 4.5s ▲7.9×	9.6s ▲3.9× 7.2s ▲5.0×	·	·	·	·	·	·	3m 41.3s ▼26% 3m 30.7s ▼21%	3m 59.5s ▼1.4× 3m 45.5s ▼1.3×	·	·	·	·	·	·
	MPF / Bounded Scalar-DP / Block	5.3s ▲6.8× 4.6s ▲7.7×	5.5s ▲6.9× 4.6s ▲7.8×	·	·	·	·	·	·	3m 40.6s ▼26% 3m 31.8s ▼22%	4m 25.9s ▼1.6× 4m 13.9s ▼1.5×	·	·	·	·	17m 59.1s ▼4.9× 9m 9.4s ▼2.5×	·
	MPF / Bounded Scalar-DP / KSW2	6.9s ▲5.3× 4.4s ▲7.9×	7.6s ▲5.0× 5.2s ▲6.9×	·	·	·	·	·	·	3m 56.3s ▼1.3× 3m 44.9s ▼29%	4m 6.8s ▼1.4× 3m 52.3s ▼1.4×	·	·	·	·	·	·
	MPF / Bounded SIMD-DP / Block	9.9s ▲3.6× 9s ▲3.9×	13s ▲2.9× 11.9s ▲3.0×	9.4s ▲9.9× 7s ▲12.8×	·	·	·	·	·	3m 44.5s ▼28% 3m 35.9s ▼24%	3m 57.4s ▼1.4× 3m 45.7s ▼1.3×	6m 50.6s ▼10% 6m 30.7s ▼6%	·	9m 45.5s ▼5.5× 3m 41.5s ▼2.1×	8m 25.6s ▼4.7× 3m 56.7s ▼2.3×	15m 24.9s ▼4.2× 6m 0.2s ▼1.7×	·
	MPF / Bounded SIMD-DP / KSW2	14.7s ▲2.5× 12.1s ▲2.9×	8.1s ▲4.7× 5.5s ▲6.5×	·	·	·	·	·	·	3m 57.7s ▼1.4× 3m 46.9s ▼1.3×	4m 0.1s ▼1.4× 3m 46s ▼1.3×	·	·	·	·	·	·
	MPF / Scalar RMQ / Block	5.5s ▲6.6× 4.6s ▲7.6×	5s ▲7.6× 4.1s ▲8.8×	·	·	·	·	·	·	5m 26.3s ▼1.9× 5m 17.1s ▼1.8×	5m 17.7s ▼1.9× 5m 5.9s ▼1.8×	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Row order: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 4 Gbp k=15, w=10								No index 4 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
minimap2	Baseline	36.2s ▼6.9× 35s ▼8.0×	38.1s ▼7.8× 36s ▼9.3×	1m 33.5s ▼10.8× 1m 29.7s ▼13.7×	6m 31.5s n/a 6m 26.3s n/a	31.6s n/a 30.3s n/a	34.1s n/a 32s n/a	1m 9.5s n/a 1m 6.1s n/a	3m 1s n/a 2m 51.5s n/a	2m 55.5s ▲20% 2m 54s ▲17%	2m 50.4s ▲23% 2m 48.2s ▲20%	6m 12.2s ▲9% 6m 8.2s ▶ 5%	44m 20.5s n/a 44m 15.6s n/a	1m 47.1s ▲5.5× 1m 45.8s ▲2.1×	1m 47.2s ▲4.6× 1m 45.1s ▲2.2×	3m 40.3s ▲4.2× 3m 36.6s ▲1.7×	29m 43.4s n/a 29m 38.5s n/a
BioseqDB	MPF / Scalar RMQ / KSW2	7s ▲5.1× 4.5s ▲7.7×	6.5s ▲5.9× 3.9s ▲9.1×	·	·	·	·	·	·	·	·	·	·	11m 18.9s ▼6.3× 5m 12.1s ▼2.9×	9m 52.6s ▼5.5× 5m 20s ▼3.0×	·	·
	MPF / Scalar Scalar-DP / Block	5.4s ▲6.7× 4.6s ▲7.6×	4.9s ▲7.7× 4s ▲8.9×	·	·	·	·	·	·	·	·	·	·	11m 9.4s ▼6.3× 5m 3.9s ▼2.9×	9m 52.5s ▼5.5× 5m 21.2s ▼3.1×	·	·
	MPF / Scalar Scalar-DP / KSW2	15s ▲2.4× 12.5s ▲2.8×	10.6s ▲3.6× 8.2s ▲4.4×	·	·	·	·	·	·	·	·	·	·	11m 14.1s ▼6.3× 5m 5.9s ▼2.9×	9m 54.4s ▼5.5× 5m 20.4s ▼3.0×	·	·
	MPF / Scalar SIMD-DP / Block	5.5s ▲6.6× 4.6s ▲7.6×	5.1s ▲7.5× 4.1s ▲8.8×	·	·	·	·	·	·	5m 24.9s ▼1.9× 5m 16s ▼1.8×	·	·	·	·	9m 48.5s ▼5.5× 5m 17.7s ▼3.0×	·	·
	MPF / Scalar SIMD-DP / KSW2	8.2s ▲4.4× 5.6s ▲6.2×	6.5s ▲5.9× 3.9s ▲9.1×	·	·	·	·	·	·	5m 27.2s ▼1.9× 5m 16.6s ▼1.8×	5m 50s ▼2.1× 5m 36.1s ▼2.0×	·	·	11m 14.3s ▼6.3× 5m 8.1s ▼2.9×	9m 48.4s ▼5.5× 5m 16.4s ▼3.0×	·	·
	MPF / SIMD-ref RMQ / Block	5.4s ▲6.7× 4.5s ▲7.8×	5s ▲7.7× 4.1s ▲8.8×	·	·	·	·	·	·	·	·	·	·	·	·	·	·
	MPF / SIMD-ref RMQ / KSW2	6.9s ▲5.3× 4.4s ▲8.0×	6.5s ▲5.9× 3.9s ▲9.3×	·	·	·	·	·	·	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Row order: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 4 Gbp k=15, w=10 DNA4 M7A	Index 4 Gbp k=15, w=10 DNA4 M7I	Index 4 Gbp k=15, w=10 DNA4 M4	Index 4 Gbp k=15, w=10 DNA4 C7G	Index 4 Gbp k=15, w=10 DNA5 M7A	Index 4 Gbp k=15, w=10 DNA5 M7I	Index 4 Gbp k=15, w=10 DNA5 M4	Index 4 Gbp k=15, w=10 DNA5 C7G	No index 4 Gbp k=15, w=10 DNA4 M7A	No index 4 Gbp k=15, w=10 DNA4 M7I	No index 4 Gbp k=15, w=10 DNA4 M4	No index 4 Gbp k=15, w=10 DNA4 C7G	No index 4 Gbp k=15, w=10 DNA5 M7A	No index 4 Gbp k=15, w=10 DNA5 M7I	No index 4 Gbp k=15, w=10 DNA5 M4	No index 4 Gbp k=15, w=10 DNA5 C7G
minimap2	Baseline	36.2s ▼6.9× 35s ▼8.0×	38.1s ▼7.8× 36s ▼9.3×	1m 33.5s ▼10.8× 1m 29.7s ▼13.7×	6m 31.5s n/a 6m 26.3s n/a	31.6s n/a 30.3s n/a	34.1s n/a 32s n/a	1m 9.5s n/a 1m 6.1s n/a	3m 1s n/a 2m 51.5s n/a	2m 55.5s ▲20% 2m 54s ▲17%	2m 50.4s ▲23% 2m 48.2s ▲20%	6m 12.2s ▲9% 6m 8.2s ▶ 5%	44m 20.5s n/a 44m 15.6s n/a	1m 47.1s ▲5.5× 1m 45.8s ▲2.1×	1m 47.2s ▲4.6× 1m 45.1s ▲2.2×	3m 40.3s ▲4.2× 3m 36.6s ▲1.7×	29m 43.4s n/a 29m 38.5s n/a
BioseqDB	MPF / SIMD-ref Scalar-DP / Block	5.3s ▲6.9× 4.5s ▲7.8×	4.9s ▲7.8× 4s ▲9.0×	·	·	·	·	·	·	·	·	·	·	·	·	·	·
	MPF / SIMD-ref Scalar-DP / KSW2	13.8s ▲2.6× 11.3s ▲3.1×	6.3s ▲6.0× 3.9s ▲9.3×	·	·	·	·	·	·	·	·	·	·	·	·	·	·
	MPF / SIMD-ref SIMD-DP / Block	5.4s ▲6.7× 4.5s ▲7.8×	17.6s ▲2.2× 16.5s ▲2.2×	8.7s ▲10.8× 6.5s ▲13.7×	·	·	·	·	·	·	·	·	·	·	·	·	·
	MPF / SIMD-ref SIMD-DP / KSW2	6.9s ▲5.2× 4.4s ▲8.0×	6.5s ▲5.9× 3.9s ▲9.2×	·	·	·	·	·	·	·	·	·	·	·	·	·	·
	MPF / Two-stack RMQ / Block	5.3s ▲6.8× 4.4s ▲7.9×	5s ▲7.7× 4.1s ▲8.9×	·	·	·	·	·	·	4m 31.5s ▼1.5× 4m 22.4s ▼1.5×	4m 45s ▼1.7× 4m 33.2s ▼1.6×	·	·	·	·	·	·
	MPF / Two-stack RMQ / KSW2	6.9s ▲5.2× 4.4s ▲8.0×	6.4s ▲5.9× 3.9s ▲9.3×	·	·	·	·	·	·	·	·	·	·	·	·	·	·
	MPF / Two-stack Scalar-DP / Block	5.3s ▲6.8× 4.5s ▲7.7×	4.9s ▲7.8× 4s ▲8.9×	·	·	·	·	·	·	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Row order: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 4 Gbp k=15, w=10								No index 4 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
minimap2	Baseline	36.2s ▼6.9× 35s ▼8.0×	38.1s ▼7.8× 36s ▼9.3×	1m 33.5s ▼10.8× 1m 29.7s ▼13.7×	6m 31.5s n/a 6m 26.3s n/a	31.6s n/a 30.3s n/a	34.1s n/a 32s n/a	1m 9.5s n/a 1m 6.1s n/a	3m 1s n/a 2m 51.5s n/a	2m 55.5s ▲20% 2m 54s ▲17%	2m 50.4s ▲23% 2m 48.2s ▲20%	6m 12.2s ▲9% 6m 8.2s ▶ 5%	44m 20.5s n/a 44m 15.6s n/a	1m 47.1s ▲5.5× 1m 45.8s ▲2.1×	1m 47.2s ▲4.6× 1m 45.1s ▲2.2×	3m 40.3s ▲4.2× 3m 36.6s ▲1.7×	29m 43.4s n/a 29m 38.5s n/a
BioseqDB	MPF / Two-stack Scalar-DP / KSW2	20.1s ▲1.8× 17.6s ▲2.0×	16.8s ▲2.3× 14.4s ▲2.5×	·	·	·	·	·	·	·	·	·	·	10m 1.7s ▼5.6× 3m 53.1s ▼2.2×	8m 16.7s ▼4.6× 3m 46.2s ▼2.2×	·	·
	MPF / Two-stack SIMD-DP / Block	5.4s ▲6.7× 4.5s ▲7.8×	5s ▲7.6× 4s ▲8.9×	·	·	·	·	·	·	·	·	·	·	·	·	·	·
	MPF / Two-stack SIMD-DP / KSW2	6.9s ▲5.3× 4.4s ▲8.0×	7.3s ▲5.2× 4.8s ▲7.5×	12.5s ▲7.5× 6.8s ▲13.3×	·	·	·	·	·	4m 37.5s ▼1.6× 4m 25.9s ▼1.5×	4m 48.3s ▼1.7× 4m 34.5s ▼1.6×	·	·	·	·	·	·
	MPS / Bounded RMQ / Block	11s ▲3.3× 10.3s ▲3.4×	11.5s ▲3.3× 10.7s ▲3.4×	·	·	·	·	·	·	4m 10.2s ▼1.4× 4m 1s ▼1.4×	4m 11.6s ▼1.5× 3m 59.9s ▼1.4×	6m 51.1s ▼10% 6m 31.1s ▼6%	·	·	·	·	·
	MPS / Bounded RMQ / KSW2	12.6s ▲2.9× 9.7s ▲3.6×	13.7s ▲2.8× 9.5s ▲3.8×	·	·	·	·	·	·	4m 20.8s ▼1.5× 4m 9.9s ▼1.4×	3m 58.2s ▼1.4× 3m 44.9s ▼1.3×	6m 49.2s ▼10% 6m 25.6s ▶ 5%	·	·	·	·	·
	MPS / Bounded Scalar-DP / Block	10.9s ▲3.3× 10.3s ▲3.4×	11.1s ▲3.4× 10.3s ▲3.5×	·	·	·	·	·	·	4m 17.5s ▼1.5× 4m 8.2s ▼1.4×	4m 7.1s ▼1.4× 3m 55.3s ▼1.4×	·	·	·	·	·	·
	MPS / Bounded Scalar-DP / KSW2	12.3s ▲2.9× 9.7s ▲3.6×	13.5s ▲2.8× 9.6s ▲3.7×	·	·	·	·	·	·	4m 19.1s ▼1.5× 4m 8.7s ▼1.4×	4m 29.2s ▼1.6× 4m 15s ▼1.5×	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Row order: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 4 Gbp k=15, w=10								No index 4 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
minimap2	Baseline	36.2s ▼6.9× 35s ▼8.0×	38.1s ▼7.8× 36s ▼9.3×	1m 33.5s ▼10.8× 1m 29.7s ▼13.7×	6m 31.5s n/a 6m 26.3s n/a	31.6s n/a 30.3s n/a	34.1s n/a 32s n/a	1m 9.5s n/a 1m 6.1s n/a	3m 1s n/a 2m 51.5s n/a	2m 55.5s ▲20% 2m 54s ▲17%	2m 50.4s ▲23% 2m 48.2s ▲20%	6m 12.2s ▲9% 6m 8.2s ▶5%	44m 20.5s n/a 44m 15.6s n/a	1m 47.1s ▲5.5× 1m 45.8s ▲2.1×	1m 47.2s ▲4.6× 1m 45.1s ▲2.2×	3m 40.3s ▲4.2× 3m 36.6s ▲1.7×	29m 43.4s n/a 29m 38.5s n/a
BioseqDB	MPS / Bounded SIMD-DP / Block	11.1s ▲3.3× 10.5s ▲3.3×	11.4s ▲3.4× 10.5s ▲3.4×	·	·	·	·	·	·	4m 19.3s ▼1.5× 4m 10.3s ▼1.4×	4m 29.2s ▼1.6× 4m 17.1s ▼1.5×	·	·	·	·	·	·
	MPS / Bounded SIMD-DP / KSW2	12.3s ▲3.0× 9.7s ▲3.6×	12.1s ▲3.1× 9.1s ▲4.0×	·	·	·	·	·	·	4m 19.7s ▼1.5× 4m 8.9s ▼1.4×	3m 58.7s ▼1.4× 3m 45.4s ▼1.3×	10m 17.4s ▼1.7× 9m 56s ▼1.6×	·	·	·	·	·
	MPS / Scalar RMQ / Block	11.5s ▲3.2× 10.9s ▲3.2×	11.9s ▲3.2× 11s ▲3.3×	·	·	·	·	·	·	5m 40.1s ▼1.9× 5m 31s ▼1.9×	6m 11.3s ▼2.2× 5m 59.5s ▼2.1×	·	·	·	·	·	·
	MPS / Scalar RMQ / KSW2	13.3s ▲2.7× 10.8s ▲3.3×	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·
	MPS / Scalar Scalar-DP / Block	·	·	·	·	·	·	·	·	·	·	·	·	11m 44.1s ▼6.6× 5m 20.6s ▼3.0×	10m 13.1s ▼5.7× 5m 32.9s ▼3.2×	·	·
	MPS / Scalar SIMD-DP / Block	11.3s ▲3.2× 10.7s ▲3.3×	11.4s ▲3.3× 10.7s ▲3.4×	·	·	·	·	·	·	·	5m 51.7s ▼2.1× 5m 39.9s ▼2.0×	·	·	11m 45.1s ▼6.6× 5m 20.1s ▼3.0×	10m 10.3s ▼5.7× 5m 30.2s ▼3.1×	·	·
	MPS / Scalar SIMD-DP / KSW2	12.7s ▲2.9× 9.9s ▲3.6×	13.2s ▲2.9× 9.7s ▲3.7×	·	·	·	·	·	·	5m 39.7s ▼1.9× 5m 28.2s ▼1.9×	5m 42.5s ▼2.0× 5m 29s ▼2.0×	·	·	11m 45s ▼6.6× 5m 19.1s ▼3.0×	10m 13.5s ▼5.7× 5m 31.5s ▼3.2×	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Row order: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 4 Gbp k=15, w=10								No index 4 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
minimap2	Baseline	36.2s ▼6.9× 35s ▼8.0×	38.1s ▼7.8× 36s ▼9.3×	1m 33.5s ▼10.8× 1m 29.7s ▼13.7×	6m 31.5s n/a 6m 26.3s n/a	31.6s n/a 30.3s n/a	34.1s n/a 32s n/a	1m 9.5s n/a 1m 6.1s n/a	3m 1s n/a 2m 51.5s n/a	2m 55.5s ▲20% 2m 54s ▲17%	2m 50.4s ▲23% 2m 48.2s ▲20%	6m 12.2s ▲9% 6m 8.2s ▶ 5%	44m 20.5s n/a 44m 15.6s n/a	1m 47.1s ▲5.5× 1m 45.8s ▲2.1×	1m 47.2s ▲4.6× 1m 45.1s ▲2.2×	3m 40.3s ▲4.2× 3m 36.6s ▲1.7×	29m 43.4s n/a 29m 38.5s n/a
BioseqDB	MPS / SIMD-ref SIMD-DP / Block	·	·	·	·	·	·	·	·	·	·	8m 12.5s ▼1.3× 7m 55.2s ▼29%	·	·	·	·	·
	MPS / Two-stack RMQ / Block	11.4s ▲3.2× 10.9s ▲3.2×	11.8s ▲3.2× 10.9s ▲3.3×	·	·	·	·	·	·	4m 44.1s ▼1.6× 4m 35.1s ▼1.6×	4m 42.8s ▼1.7× 4m 30.9s ▼1.6×	·	·	·	·	·	·
	MPS / Two-stack RMQ / KSW2	13.3s ▲2.7× 10.8s ▲3.2×	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·
	MPS / Two-stack Scalar-DP / KSW2	·	·	·	·	·	·	·	·	·	·	·	·	10m 24.5s ▼5.8× 4m 4.1s ▼2.3×	8m 30.6s ▼4.8× 3m 55.3s ▼2.2×	·	·
	MPS / Two-stack SIMD-DP / Block	11.5s ▲3.1× 10.9s ▲3.2×	11.2s ▲3.4× 10.4s ▲3.5×	·	·	·	·	·	·	4m 39.5s ▼1.6× 4m 30.1s ▼1.6×	4m 24.7s ▼1.6× 4m 13s ▼1.5×	·	·	·	·	·	·
	MPS / Two-stack SIMD-DP / KSW2	13.1s ▲2.8× 10.5s ▲3.3×	13.4s ▲2.8× 9.7s ▲3.7×	·	·	·	·	·	·	4m 54.4s ▼1.7× 4m 44.1s ▼1.6×	4m 39.9s ▼1.6× 4m 26.4s ▼1.6×	·	·	·	·	·	·
BioseqDB	Vulkan (all modes)	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Row order: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 4 Gbp k=20, w=15								No index 4 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
minimap2	Baseline	2m 48.1s ▼ 20.4× 2m 41.7s ▼ 22.6×	2m 50.3s ▼ 19.8× 2m 43s ▼ 22.2×	4m 54.3s ▼ 43.2× 4m 39s ▼ 54.6×	25m 5.5s n/a 25m 0.7s n/a	33.9s ▼ 1.9× 32.9s ▼ 2.1×	34.4s ▼ 2.6× 32.7s ▼ 2.9×	1m 8.2s ▼ 3.7× 1m 5.1s ▼ 4.6×	3m 9.8s n/a 3m 3.7s n/a	3m 28.3s ▼ 1.3× 3m 22.8s ▼ 1.4×	3m 53.2s ▼ 1.4× 3m 43.5s ▼ 1.4×	7m 14.6s ▼ 1.5× 6m 57.2s ▼ 1.6×	51m 22s n/a 51m 17.2s n/a	1m 10.3s ▲ 7.6× 1m 9s ▲ 2.4×	1m 8.6s ▲ 6.4× 1m 6.9s ▲ 2.5×	2m 21.5s ▲ 6.0× 2m 18.4s ▲ 2.2×	15m 56.8s n/a 15m 51.9s n/a
BioseqDB	MPF / Bounded RMQ / Block	8.3s ▲ 20.4× 7.6s ▲ 21.3×	8.6s ▲ 19.8× 7.8s ▲ 20.8×	·	·	·	·	·	·	2m 53.8s ▲ 17% 2m 43.7s ▲ 19%	3m 4.5s ▲ 21% 2m 52.1s ▲ 23%	·	·	·	·	·	·
	MPF / Bounded RMQ / KSW2	10.6s ▲ 15.9× 8.3s ▲ 19.5×	14.6s ▲ 11.7× 12.5s ▲ 13.0×	·	·	·	·	·	·	2m 51.8s ▲ 18% 2m 40.6s ▲ 21%	2m 50.2s ▲ 27% 2m 36.4s ▲ 1.4×	·	·	·	·	·	·
	MPF / Bounded Scalar-DP / Block	10.7s ▲ 15.7× 10.1s ▲ 16.1×	11.5s ▲ 14.8× 10.8s ▲ 15.2×	·	·	·	·	·	·	2m 50s ▲ 18% 2m 41s ▲ 21%	2m 48.3s ▲ 28% 2m 35.9s ▲ 1.4×	4m 47.5s ▲ 1.5× 4m 26.9s ▲ 1.6×	·	·	·	·	·
	MPF / Bounded Scalar-DP / KSW2	14.5s ▲ 11.6× 12.2s ▲ 13.3×	14.2s ▲ 12.0× 11.9s ▲ 13.7×	·	·	·	·	·	·	2m 40.6s ▲ 23% 2m 29.9s ▲ 26%	3m 2.9s ▲ 22% 2m 48.3s ▲ 25%	·	·	·	·	·	·
	MPF / Bounded SIMD-DP / Block	9.2s ▲ 18.3× 8.5s ▲ 19.0×	11.4s ▲ 15.0× 10.7s ▲ 15.3×	6.8s ▲ 43.2× 5.1s ▲ 54.6×	·	18.2s ▲ 1.9× 16s ▲ 2.1×	13.2s ▲ 2.6× 11.4s ▲ 2.9×	18.6s ▲ 3.7× 14.3s ▲ 4.6×	·	2m 38.5s ▲ 24% 2m 29.8s ▲ 26%	2m 49s ▲ 28% 2m 37s ▲ 30%	5m 3.3s ▲ 1.4× 4m 42.7s ▲ 1.5×	·	8m 52.3s ▼ 7.6× 2m 45.5s ▼ 2.4×	7m 22.2s ▼ 6.4× 2m 48.2s ▼ 2.5×	14m 43.3s ▼ 6.2× 5m 9.5s ▼ 2.2×	·
	MPF / Bounded SIMD-DP / KSW2	9.6s ▲ 17.6× 7.1s ▲ 22.6×	9.6s ▲ 17.7× 7.3s ▲ 22.2×	·	·	·	·	·	·	2m 51.2s ▲ 18% 2m 39.2s ▲ 21%	3m 0s ▲ 23% 2m 45.6s ▲ 26%	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Row order: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 4 Gbp k=20, w=15								No index 4 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
minimap2	Baseline	2m 48.1s ▼20.4× 2m 41.7s ▼22.6×	2m 50.3s ▼19.8× 2m 43s ▼22.2×	4m 54.3s ▼43.2× 4m 39s ▼54.6×	25m 5.5s n/a 25m 0.7s n/a	33.9s ▼1.9× 32.9s ▼2.1×	34.4s ▼2.6× 32.7s ▼2.9×	1m 8.2s ▼3.7× 1m 5.1s ▼4.6×	3m 9.8s n/a 3m 3.7s n/a	3m 28.3s ▼1.3× 3m 22.8s ▼1.4×	3m 53.2s ▼1.4× 3m 43.5s ▼1.4×	7m 14.6s ▼1.5× 6m 57.2s ▼1.6×	51m 22s n/a 51m 17.2s n/a	1m 10.3s ▲7.6× 1m 9s ▲2.4×	1m 8.6s ▲6.4× 1m 6.9s ▲2.5×	2m 21.5s ▲6.0× 2m 18.4s ▲2.2×	15m 56.8s n/a 15m 51.9s n/a
BioseqDB	MPF / Scalar RMQ / Block	11m 22.5s ▼4.1× 11m 22s ▼4.2×	24m 52.9s ▼8.8× 24m 52.4s ▼9.2×	·	·	·	·	·	·	5m 16.6s ▼1.5× 4m 33.9s ▼1.4×	4m 53.5s ▼26% 4m 17.9s ▼15%	·	·	·	·	·	·
	MPF / Scalar SIMD-DP / Block	·	·	·	·	·	·	·	·	4m 51s ▼1.4× 4m 8.2s ▼22%	4m 40s ▼20% 4m 5.7s ▼10%	·	·	·	·	·	·
	MPF / Scalar SIMD-DP / KSW2	11m 14.3s ▼4.0× 11m 12.2s ▼4.2×	24m 37.5s ▼8.7× 24m 35.6s ▼9.1×	·	·	·	·	·	·	4m 53s ▼1.4× 4m 8.2s ▼22%	5m 9.3s ▼1.3× 4m 31.7s ▼22%	·	·	·	·	·	·
	MPF / Two-stack RMQ / Block	9m 11s ▼3.3× 9m 10.5s ▼3.4×	13m 20.9s ▼4.7× 13m 20.1s ▼4.9×	·	·	·	·	·	·	·	·	·	·	·	·	·	·
	MPF / Two-stack RMQ / KSW2	9m 9.8s ▼3.3× 9m 7.7s ▼3.4×	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·
	MPF / Two-stack SIMD-DP / Block	9m 11.2s ▼3.3× 9m 10.7s ▼3.4×	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·
	MPF / Two-stack SIMD-DP / KSW2	9m 11.4s ▼3.3× 9m 9.5s ▼3.4×	13m 22s ▼4.7× 13m 19.3s ▼4.9×	·	·	·	·	·	·	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Row order: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 4 Gbp k=20, w=15								No index 4 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
minimap2	Baseline	2m 48.1s ▼20.4× 2m 41.7s ▼22.6×	2m 50.3s ▼19.8× 2m 43s ▼22.2×	4m 54.3s ▼43.2× 4m 39s ▼54.6×	25m 5.5s n/a 25m 0.7s n/a	33.9s ▼1.9× 32.9s ▼2.1×	34.4s ▼2.6× 32.7s ▼2.9×	1m 8.2s ▼3.7× 1m 5.1s ▼4.6×	3m 9.8s n/a 3m 3.7s n/a	3m 28.3s ▼1.3× 3m 22.8s ▼1.4×	3m 53.2s ▼1.4× 3m 43.5s ▼1.4×	7m 14.6s ▼1.5× 6m 57.2s ▼1.6×	51m 22s n/a 51m 17.2s n/a	1m 10.3s ▲7.6× 1m 9s ▲2.4×	1m 8.6s ▲6.4× 1m 6.9s ▲2.5×	2m 21.5s ▲6.0× 2m 18.4s ▲2.2×	15m 56.8s n/a 15m 51.9s n/a
BioseqDB	MPS / Bounded RMQ / Block	11.7s ▲14.3× 11s ▲14.6×	11.8s ▲14.5× 11.1s ▲14.7×	·	·	·	·	·	·	2m 43.8s ▲21% 2m 34.7s ▲24%	2m 52.1s ▲26% 2m 39.6s ▲29%	·	·	·	·	14m 16s ▼6.0× 5m 26.4s ▼2.4×	·
	MPS / Bounded RMQ / KSW2	13.5s ▲12.4× 11s ▲14.7×	12.8s ▲13.3× 10.6s ▲15.4×	·	·	·	·	·	·	2m 56.8s ▲15% 2m 44.8s ▲19%	3m 11.4s ▲18% 2m 56.8s ▲21%	·	·	·	·	·	·
	MPS / Bounded Scalar-DP / Block	11.8s ▲14.2× 11.1s ▲14.6×	12.7s ▲13.4× 11.9s ▲13.7×	·	·	·	·	·	·	2m 52.6s ▲17% 2m 42.8s ▲20%	2m 58.9s ▲23% 2m 46.9s ▲25%	·	·	·	·	·	·
	MPS / Bounded Scalar-DP / KSW2	13.4s ▲12.6× 10.9s ▲14.9×	13s ▲13.1× 10.7s ▲15.2×	·	·	·	·	·	·	2m 59.1s ▲14% 2m 47.7s ▲17%	2m 50.8s ▲27% 2m 36.9s ▲30%	·	·	·	·	·	·
	MPS / Bounded SIMD-DP / Block	11.7s ▲14.3× 11s ▲14.7×	11.9s ▲14.3× 11.2s ▲14.6×	·	·	·	·	·	·	2m 43.8s ▲21% 2m 34.7s ▲24%	2m 58.9s ▲23% 2m 46.3s ▲26%	·	·	·	·	·	·
	MPS / Bounded SIMD-DP / KSW2	13.3s ▲12.7× 10.8s ▲15.0×	13.6s ▲12.5× 11.1s ▲14.7×	·	·	·	·	·	·	2m 59s ▲14% 2m 48.5s ▲17%	2m 52.5s ▲26% 2m 38.8s ▲29%	·	·	·	·	·	·
	MPS / Scalar RMQ / Block	57.4s ▲2.9× 14.6s ▲11.1×	51.1s ▲3.3× 15.9s ▲10.3×	·	·	·	·	·	·	5m 10.6s ▼1.5× 4m 28.1s ▼1.3×	5m 37s ▼1.4× 5m 0.7s ▼1.3×	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Row order: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 4 Gbp k=20, w=15								No index 4 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
minimap2	Baseline	2m 48.1s ▼ 20.4× 2m 41.7s ▼ 22.6×	2m 50.3s ▼ 19.8× 2m 43s ▼ 22.2×	4m 54.3s ▼ 43.2× 4m 39s ▼ 54.6×	25m 5.5s n/a 25m 0.7s n/a	33.9s ▼ 1.9× 32.9s ▼ 2.1×	34.4s ▼ 2.6× 32.7s ▼ 2.9×	1m 8.2s ▼ 3.7× 1m 5.1s ▼ 4.6×	3m 9.8s n/a 3m 3.7s n/a	3m 28.3s ▼ 1.3× 3m 22.8s ▼ 1.4×	3m 53.2s ▼ 1.4× 3m 43.5s ▼ 1.4×	7m 14.6s ▼ 1.5× 6m 57.2s ▼ 1.6×	51m 22s n/a 51m 17.2s n/a	1m 10.3s ▲ 7.6× 1m 9s ▲ 2.4×	1m 8.6s ▲ 6.4× 1m 6.9s ▲ 2.5×	2m 21.5s ▲ 6.0× 2m 18.4s ▲ 2.2×	15m 56.8s n/a 15m 51.9s n/a
BioseqDB	MPS / Scalar SIMD-DP / Block	57s ▲ 3.0× 14.3s ▲ 11.3×	50.6s ▲ 3.4× 15.1s ▲ 10.8×	·	·	·	·	·	·	5m 10.1s ▼ 1.5× 4m 27.6s ▼ 1.3×	5m 3.5s ▼ 1.3× 4m 28.9s ▼ 20%	·	·	·	·	·	·
	MPS / Scalar SIMD-DP / KSW2	58.6s ▲ 2.9× 14.4s ▲ 11.2×	52.3s ▲ 3.3× 15.4s ▲ 10.6×	·	·	·	·	·	·	5m 10.5s ▼ 1.5× 4m 25.5s ▼ 1.3×	4m 45.4s ▼ 22% 4m 9.3s ▼ 12%	·	·	·	·	·	·
	MPS / SIMD-ref SIMD-DP / Block	·	·	·	·	·	·	·	·	·	·	6m 45.8s ▲ 7% 5m 46.6s ▲ 17%	·	·	·	·	·
	MPS / Two-stack RMQ / Block	50.7s ▲ 3.3× 9.4s ▲ 17.2×	44.9s ▲ 3.8× 9.9s ▲ 16.5×	·	·	·	·	·	·	·	·	·	·	·	·	·	·
	MPS / Two-stack SIMD-DP / Block	51.6s ▲ 3.3× 9.8s ▲ 16.6×	44.7s ▲ 3.8× 9.6s ▲ 17.0×	1m 16.1s ▲ 3.9× 15.7s ▲ 17.7×	·	·	·	·	·	·	·	·	·	·	·	·	·
	MPS / Two-stack SIMD-DP / KSW2	55.3s ▲ 3.0× 9.6s ▲ 16.8×	48.6s ▲ 3.5× 9.6s ▲ 17.0×	·	·	·	·	·	·	·	·	·	·	·	·	·	·
BioseqDB	Vulkan (all modes)	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Row order: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 2 Gbp k=15, w=10 DNA4 M7A	DNA4 M7I	DNA4 M4	DNA4 C7G	DNA5 M7A	DNA5 M7I	DNA5 M4	DNA5 C7G	No index 2 Gbp k=15, w=10 DNA4 M7A	DNA4 M7I	DNA4 M4	DNA4 C7G	DNA5 M7A	DNA5 M7I	DNA5 M4	DNA5 C7G
minimap2	Baseline	11.2s ▼3.1× 10s ▼3.6×	10.7s ▼3.2× 8.6s ▼3.5×	22.5s ▼3.6× 18.8s ▼4.4×	1m 13.1s ▼22.5× 1m 5.7s ▼27.7×	9.9s ▼21% 8.8s ▼1.5×	9.5s ▼1.4× 7.6s ▼1.6×	16.5s ▼1.7× 12.9s ▼2.0×	28.2s ▼2.1× 26.2s ▼2.4×	53s ▲2.2× 51.9s ▲2.1×	56.6s ▲2.1× 54.6s ▲1.9×	1m 55.5s ▲1.8× 1m 51.8s ▲1.5×	6m 1.3s n/a 5m 55.2s n/a	48.1s ▲7.3× 47s ▲2.4×	44.5s ▲6.5× 42.6s ▲2.7×	1m 30.2s ▲6.4× 1m 26.7s ▲2.2×	2m 46.4s n/a 2m 42.7s n/a
BioseqDB	MPF / Bounded RMQ / Block	3.7s ▲3.0× 2.9s ▲3.4×	3.5s ▲3.0× 2.7s ▲3.2×	6.5s ▲3.5× 4.5s ▲4.2×	3.4s ▲21.2× 2.5s ▲26.1×	·	·	·	·	1m 55.5s ▼2.2× 1m 47.1s ▼2.1×	2m 21s ▼2.5× 2m 10.3s ▼2.4×	·	·	·	·	·	·
	MPF / Bounded RMQ / KSW2	5.3s ▲2.1× 2.8s ▲3.6×	4.8s ▲2.2× 2.5s ▲3.5×	9.8s ▲2.3× 4.2s ▲4.4×	7.7s ▲9.5× 4.6s ▲14.2×	·	·	·	·	2m 17.7s ▼2.6× 2m 7s ▼2.4×	1m 59.4s ▼2.1× 1m 47.5s ▼2.0×	·	·	·	·	·	·
	MPF / Bounded Scalar-DP / Block	3.6s ▲3.1× 2.8s ▲3.5×	3.4s ▲3.1× 2.6s ▲3.3×	6.4s ▲3.5× 4.4s ▲4.2×	3.2s ▲22.5× 2.4s ▲27.7×	·	·	·	·	1m 56.2s ▼2.2× 1m 47.8s ▼2.1×	2m 7.4s ▼2.3× 1m 56.8s ▼2.1×	·	·	·	·	·	·
	MPF / Bounded Scalar-DP / KSW2	5.2s ▲2.1× 2.8s ▲3.6×	4.7s ▲2.3× 2.5s ▲3.5×	9.7s ▲2.3× 4.3s ▲4.4×	7.2s ▲10.1× 4.3s ▲15.4×	·	·	·	·	2m 8.2s ▼2.4× 1m 58.3s ▼2.3×	2m 3.7s ▼2.2× 1m 51.2s ▼2.0×	·	·	·	·	9m 39.3s ▼6.4× 3m 13.2s ▼2.2×	·
	MPF / Bounded SIMD-DP / Block	3.7s ▲3.0× 3s ▲3.4×	3.5s ▲3.0× 2.7s ▲3.2×	6.7s ▲3.3× 4.5s ▲4.2×	8.8s ▲8.3× 7.9s ▲8.3×	8.2s ▲17% 5.8s ▲1.5×	6.6s ▲1.4× 4.6s ▲1.6×	13.7s ▲17% 8.8s ▲1.5×	·	1m 56.4s ▼2.2× 1m 48.1s ▼2.1×	1m 56.2s ▼2.1× 1m 46.2s ▼1.9×	3m 42.9s ▼1.9× 3m 25s ▼1.8×	·	5m 52.1s ▼7.3× 1m 53.5s ▼2.4×	4m 49.2s ▼6.5× 1m 56.8s ▼2.7×	9m 39.5s ▼6.4× 3m 18.1s ▼2.3×	·
	MPF / Bounded SIMD-DP / KSW2	5.2s ▲2.1× 2.8s ▲3.6×	4.7s ▲2.3× 2.5s ▲3.5×	9.9s ▲2.3× 4.3s ▲4.4×	7.4s ▲9.9× 4.4s ▲15.1×	·	·	14.1s ▲14% 8.5s ▲1.5×	·	2m 11.6s ▼2.5× 2m 1.1s ▼2.3×	2m 7.5s ▼2.3× 1m 54.8s ▼2.1×	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Row order: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 2 Gbp k=15, w=10 DNA4 M7A	DNA4 M7I	DNA4 M4	DNA4 C7G	DNA5 M7A	DNA5 M7I	DNA5 M4	DNA5 C7G	No index 2 Gbp k=15, w=10 DNA4 M7A	DNA4 M7I	DNA4 M4	DNA4 C7G	DNA5 M7A	DNA5 M7I	DNA5 M4	DNA5 C7G
minimap2	Baseline	11.2s ▼3.1× 10s ▼3.6×	10.7s ▼3.2× 8.6s ▼3.5×	22.5s ▼3.6× 18.8s ▼4.4×	1m 13.1s ▼22.5× 1m 5.7s ▼27.7×	9.9s ▼21% 8.8s ▼1.5×	9.5s ▼1.4× 7.6s ▼1.6×	16.5s ▼1.7× 12.9s ▼2.0×	28.2s ▼2.1× 26.2s ▼2.4×	53s ▲2.2× 51.9s ▲2.1×	56.6s ▲2.1× 54.6s ▲1.9×	1m 55.5s ▲1.8× 1m 51.8s ▲1.5×	6m 1.3s n/a 5m 55.2s n/a	48.1s ▲7.3× 47s ▲2.4×	44.5s ▲6.5× 42.6s ▲2.7×	1m 30.2s ▲6.4× 1m 26.7s ▲2.2×	2m 46.4s n/a 2m 42.7s n/a
BioseqDB	MPF / Scalar RMQ / Block	4s ▲2.8× 3.1s ▲3.2×	3.7s ▲2.9× 2.8s ▲3.1×	7.1s ▲3.2× 4.8s ▲3.9×	3.5s ▲20.6× 2.6s ▲25.3×	·	·	9.5s ▲1.7× 6.5s ▲2.0×	·	2m 51.5s ▼3.2× 2m 29.1s ▼2.9×	3m 4.8s ▼3.3× 2m 44.5s ▼3.0×	·	·	·	·	·	·
	MPF / Scalar RMQ / KSW2	5.5s ▲2.0× 2.9s ▲3.4×	5.1s ▲2.1× 2.7s ▲3.2×	10.3s ▲2.2× 4.6s ▲4.1×	5.6s ▲13.1× 2.5s ▲26.3×	·	·	·	·	3m 1.5s ▼3.4× 2m 37.7s ▼3.0×	2m 54.6s ▼3.1× 2m 33.3s ▼2.8×	·	·	·	·	·	·
	MPF / Scalar Scalar-DP / Block	3.8s ▲3.0× 3.1s ▲3.3×	3.5s ▲3.1× 2.7s ▲3.2×	6.5s ▲3.4× 4.8s ▲3.9×	3.4s ▲21.8× 2.5s ▲26.2×	·	·	·	·	2m 51.4s ▼3.2× 2m 29.1s ▼2.9×	2m 59.3s ▼3.2× 2m 39.2s ▼2.9×	·	·	·	·	·	·
	MPF / Scalar Scalar-DP / KSW2	5.2s ▲2.1× 2.9s ▲3.5×	4.8s ▲2.2× 2.6s ▲3.3×	9.5s ▲2.4× 4.5s ▲4.2×	8.6s ▲8.5× 5.7s ▲11.6×	·	·	·	·	2m 52.3s ▼3.3× 2m 28.6s ▼2.9×	3m 4.9s ▼3.3× 2m 43.4s ▼3.0×	·	·	·	·	·	·
	MPF / Scalar SIMD-DP / Block	3.9s ▲2.9× 3.1s ▲3.3×	4.5s ▲2.4× 3.6s ▲2.4×	6.6s ▲3.4× 4.7s ▲4.0×	9.1s ▲8.1× 8.1s ▲8.1×	·	·	·	14.3s ▲2.0× 12.1s ▲2.2×	3m 8.5s ▼3.6× 2m 46.3s ▼3.2×	3m 1.2s ▼3.2× 2m 41.3s ▼3.0×	·	·	·	·	·	·
	MPF / Scalar SIMD-DP / KSW2	5.4s ▲2.1× 2.9s ▲3.5×	4.9s ▲2.2× 2.6s ▲3.3×	10.3s ▲2.2× 4.6s ▲4.1×	7.8s ▲9.3× 5s ▲13.2×	·	·	·	·	3m 4.9s ▼3.5× 2m 40.5s ▼3.1×	3m 7.6s ▼3.3× 2m 46s ▼3.0×	·	·	·	·	·	·
	MPF / SIMD-ref RMQ / Block	3.7s ▲3.0× 3s ▲3.4×	3.5s ▲3.0× 2.7s ▲3.2×	6.7s ▲3.4× 4.6s ▲4.1×	3.5s ▲21.0× 2.6s ▲25.5×	·	·	·	·	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Row order: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 2 Gbp k=15, w=10								No index 2 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
minimap2	Baseline	11.2s ▼3.1× 10s ▼3.6×	10.7s ▼3.2× 8.6s ▼3.5×	22.5s ▼3.6× 18.8s ▼4.4×	1m 13.1s ▼22.5× 1m 5.7s ▼27.7×	9.9s ▼21% 8.8s ▼1.5×	9.5s ▼1.4× 7.6s ▼1.6×	16.5s ▼1.7× 12.9s ▼2.0×	28.2s ▼2.1× 26.2s ▼2.4×	53s ▲2.2× 51.9s ▲2.1×	56.6s ▲2.1× 54.6s ▲1.9×	1m 55.5s ▲1.8× 1m 51.8s ▲1.5×	6m 1.3s n/a 5m 55.2s n/a	48.1s ▲7.3× 47s ▲2.4×	44.5s ▲6.5× 42.6s ▲2.7×	1m 30.2s ▲6.4× 1m 26.7s ▲2.2×	2m 46.4s n/a 2m 42.7s n/a
BioseqDB	MPF / SIMD-ref RMQ / KSW2	5.3s ▲2.1× 2.8s ▲3.6×	4.9s ▲2.2× 2.5s ▲3.4×	9.9s ▲2.3× 4.3s ▲4.4×	5.5s ▲13.3× 2.4s ▲26.8×	·	·	·	·	·	·	·	·	·	·	·	·
	MPF / SIMD-ref Scalar-DP / Block	3.6s ▲3.1× 2.9s ▲3.5×	3.4s ▲3.2× 2.6s ▲3.3×	6.5s ▲3.5× 4.4s ▲4.2×	3.3s ▲22.1× 2.5s ▲26.2×	·	·	·	·	·	·	·	·	·	·	·	·
	MPF / SIMD-ref Scalar-DP / KSW2	5.1s ▲2.2× 2.8s ▲3.6×	4.7s ▲2.3× 2.5s ▲3.4×	9.8s ▲2.3× 4.3s ▲4.3×	7.2s ▲10.1× 4.4s ▲15.1×	·	·	·	·	·	·	·	·	·	·	·	·
	MPF / SIMD-ref SIMD-DP / Block	3.7s ▲3.0× 2.9s ▲3.4×	3.5s ▲3.0× 2.7s ▲3.2×	6.9s ▲3.3× 4.5s ▲4.2×	3.4s ▲21.4× 2.5s ▲26.3×	·	·	·	26.2s ▲7% 24.4s ▲7%	·	2m 32.6s ▼2.7× 2m 12.9s ▼2.4×	·	·	·	·	·	·
	MPF / SIMD-ref SIMD-DP / KSW2	5.3s ▲2.1× 2.8s ▲3.6×	4.8s ▲2.2× 2.5s ▲3.4×	9.7s ▲2.3× 4.3s ▲4.4×	9s ▲8.1× 6.2s ▲10.6×	·	·	·	·	·	·	3m 23.3s ▼1.8× 2m 50.3s ▼1.5×	·	·	·	·	·
	MPF / Two-stack RMQ / Block	3.8s ▲2.9× 3s ▲3.3×	3.6s ▲3.0× 2.7s ▲3.2×	6.8s ▲3.3× 4.7s ▲4.0×	3.6s ▲20.1× 2.7s ▲24.2×	·	·	12.8s ▲23% 8.6s ▲1.5×	·	2m 38.4s ▼3.0× 2m 16.3s ▼2.6×	2m 38s ▼2.8× 2m 17.8s ▼2.5×	·	·	·	·	·	·
	MPF / Two-stack RMQ / KSW2	5.3s ▲2.1× 2.9s ▲3.5×	5s ▲2.1× 2.5s ▲3.4×	10.1s ▲2.2× 4.5s ▲4.2×	5.6s ▲12.9× 2.6s ▲25.5×	·	·	·	·	2m 37s ▼3.0× 2m 12.9s ▼2.6×	2m 34.6s ▼2.7× 2m 12.9s ▼2.4×	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Row order: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 2 Gbp k=15, w=10 DNA4 M7A	M7I	M4	C7G	DNA5 M7A	M7I	M4	C7G	No index 2 Gbp k=15, w=10 DNA4 M7A	M7I	M4	C7G	DNA5 M7A	M7I	M4	C7G
minimap2	Baseline	11.2s ▼3.1× 10s ▼3.6×	10.7s ▼3.2× 8.6s ▼3.5×	22.5s ▼3.6× 18.8s ▼4.4×	1m 13.1s ▼22.5× 1m 5.7s ▼27.7×	9.9s ▼21% 8.8s ▼1.5×	9.5s ▼1.4× 7.6s ▼1.6×	16.5s ▼1.7× 12.9s ▼2.0×	28.2s ▼2.1× 26.2s ▼2.4×	53s ▲2.2× 51.9s ▲2.1×	56.6s ▲2.1× 54.6s ▲1.9×	1m 55.5s ▲1.8× 1m 51.8s ▲1.5×	6m 1.3s n/a 5m 55.2s n/a	48.1s ▲7.3× 47s ▲2.4×	44.5s ▲6.5× 42.6s ▲2.7×	1m 30.2s ▲6.4× 1m 26.7s ▲2.2×	2m 46.4s n/a 2m 42.7s n/a
BioseqDB	MPF / Two-stack Scalar-DP / Block	3.6s ▲3.1× 2.9s ▲3.4×	3.4s ▲3.2× 2.6s ▲3.3×	6.3s ▲3.6× 4.6s ▲4.1×	7.9s ▲9.2× 7.1s ▲9.3×	·	·	·	·	·	·	·	·	·	·	·	·
	MPF / Two-stack Scalar-DP / KSW2	5.2s ▲2.1× 2.8s ▲3.5×	4.7s ▲2.3× 2.5s ▲3.4×	9.5s ▲2.4× 4.5s ▲4.2×	7.2s ▲10.1× 4.4s ▲14.9×	·	·	·	·	·	·	·	·	·	·	·	·
	MPF / Two-stack SIMD-DP / Block	3.7s ▲3.0× 3s ▲3.4×	3.5s ▲3.0× 2.6s ▲3.3×	6.7s ▲3.4× 4.7s ▲4.0×	3.4s ▲21.5× 2.5s ▲25.8×	·	·	·	13.5s ▲2.1× 11s ▲2.4×	2m 38.5s ▼3.0× 2m 16s ▼2.6×	2m 34.1s ▼2.7× 2m 13.1s ▼2.4×	·	·	·	·	·	·
	MPF / Two-stack SIMD-DP / KSW2	5.3s ▲2.1× 2.9s ▲3.5×	4.8s ▲2.2× 2.5s ▲3.4×	9.8s ▲2.3× 4.5s ▲4.2×	7.1s ▲10.3× 4.2s ▲15.6×	·	·	·	·	2m 27.4s ▼2.8× 2m 3.1s ▼2.4×	2m 26.8s ▼2.6× 2m 5.2s ▼2.3×	·	·	·	·	·	·
	MPS / Bounded RMQ / Block	6.5s ▲1.7× 5.7s ▲1.8×	6.5s ▲1.6× 5.8s ▲1.5×	·	6.1s ▲12.0× 5.1s ▲12.8×	·	·	·	·	2m 31.3s ▼2.9× 2m 9.3s ▼2.5×	2m 16.4s ▼2.4× 1m 56.6s ▼2.1×	·	·	·	·	·	·
	MPS / Bounded RMQ / KSW2	7.9s ▲29% 4.9s ▲2.0×	8.5s ▲20% 4.9s ▲1.7×	·	8.1s ▲9.1× 4.6s ▲14.2×	·	·	·	·	2m 21.9s ▼2.7× 1m 57.6s ▼2.3×	2m 27.9s ▼2.6× 2m 5.4s ▼2.3×	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Row order: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 2 Gbp k=15, w=10								No index 2 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
minimap2	Baseline	11.2s ▼3.1× 10s ▼3.6×	10.7s ▼3.2× 8.6s ▼3.5×	22.5s ▼3.6× 18.8s ▼4.4×	1m 13.1s ▼22.5× 1m 5.7s ▼27.7×	9.9s ▼21% 8.8s ▼1.5×	9.5s ▼1.4× 7.6s ▼1.6×	16.5s ▼1.7× 12.9s ▼2.0×	28.2s ▼2.1× 26.2s ▼2.4×	53s ▲2.2× 51.9s ▲2.1×	56.6s ▲2.1× 54.6s ▲1.9×	1m 55.5s ▲1.8× 1m 51.8s ▲1.5×	6m 1.3s n/a 5m 55.2s n/a	48.1s ▲7.3× 47s ▲2.4×	44.5s ▲6.5× 42.6s ▲2.7×	1m 30.2s ▲6.4× 1m 26.7s ▲2.2×	2m 46.4s n/a 2m 42.7s n/a
BioseqDB	MPS / Bounded Scalar-DP / Block	6.4s ▲1.7× 5.7s ▲1.8×	6.6s ▲1.6× 5.7s ▲1.5×	·	5.9s ▲12.3× 4.9s ▲13.4×	·	·	·	·	2m 24.9s ▼2.7× 2m 2.4s ▼2.4×	2m 39.7s ▼2.8× 2m 18.8s ▼2.5×	·	·	·	·	·	·
	MPS / Bounded Scalar-DP / KSW2	7.9s ▲29% 4.9s ▲2.0×	8.6s ▲20% 5s ▲1.7×	·	8s ▲9.2× 4.6s ▲14.2×	·	·	·	·	2m 37s ▼3.0× 2m 12s ▼2.5×	2m 30.6s ▼2.7× 2m 7.1s ▼2.3×	·	·	·	·	·	·
	MPS / Bounded SIMD-DP / Block	6.7s ▲1.7× 6s ▲1.7×	6.9s ▲1.5× 6s ▲1.4×	12.5s ▲1.8× 10.4s ▲1.8×	6s ▲12.1× 5s ▲13.0×	·	·	·	·	2m 20.8s ▼2.7× 1m 57.4s ▼2.3×	2m 25.9s ▼2.6× 2m 5.5s ▼2.3×	·	·	·	·	·	·
	MPS / Bounded SIMD-DP / KSW2	8.2s ▲26% 5.4s ▲1.8×	8.4s ▲21% 4.9s ▲1.8×	·	9.6s ▲7.6× 6s ▲10.9×	·	·	·	·	2m 20.7s ▼2.7× 1m 56.8s ▼2.3×	2m 19.7s ▼2.5× 1m 57.9s ▼2.2×	·	·	·	·	·	·
	MPS / Scalar RMQ / Block	6.9s ▲1.6× 6.2s ▲1.6×	7s ▲1.5× 6.2s ▲28%	·	·	·	·	·	·	3m 15.8s ▼3.7× 2m 53.2s ▼3.3×	3m 13.2s ▼3.4× 2m 53s ▼3.2×	·	·	·	·	·	·
	MPS / Scalar RMQ / KSW2	8.2s ▲26% 5.4s ▲1.9×	8.7s ▲18% 5.4s ▲1.6×	·	·	·	·	·	·	3m 14.2s ▼3.7× 2m 49.7s ▼3.3×	·	·	·	·	·	·	·
	MPS / Scalar Scalar-DP / Block	7.5s ▲1.5× 6.3s ▲1.6×	7.2s ▲1.5× 6.4s ▲26%	·	·	·	·	·	·	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Row order: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 2 Gbp k=15, w=10 DNA4 M7A	Index 2 Gbp k=15, w=10 DNA4 M7I	Index 2 Gbp k=15, w=10 DNA4 M4	Index 2 Gbp k=15, w=10 DNA4 C7G	Index 2 Gbp k=15, w=10 DNA5 M7A	Index 2 Gbp k=15, w=10 DNA5 M7I	Index 2 Gbp k=15, w=10 DNA5 M4	Index 2 Gbp k=15, w=10 DNA5 C7G	No index 2 Gbp k=15, w=10 DNA4 M7A	No index 2 Gbp k=15, w=10 DNA4 M7I	No index 2 Gbp k=15, w=10 DNA4 M4	No index 2 Gbp k=15, w=10 DNA4 C7G	No index 2 Gbp k=15, w=10 DNA5 M7A	No index 2 Gbp k=15, w=10 DNA5 M7I	No index 2 Gbp k=15, w=10 DNA5 M4	No index 2 Gbp k=15, w=10 DNA5 C7G
minimap2	Baseline	11.2s ▼3.1× 10s ▼3.6×	10.7s ▼3.2× 8.6s ▼3.5×	22.5s ▼3.6× 18.8s ▼4.4×	1m 13.1s ▼22.5× 1m 5.7s ▼27.7×	9.9s ▼21% 8.8s ▼1.5×	9.5s ▼1.4× 7.6s ▼1.6×	16.5s ▼1.7× 12.9s ▼2.0×	28.2s ▼2.1× 26.2s ▼2.4×	53s ▲2.2× 51.9s ▲2.1×	56.6s ▲2.1× 54.6s ▲1.9×	1m 55.5s ▲1.8× 1m 51.8s ▲1.5×	6m 1.3s n/a 5m 55.2s n/a	48.1s ▲7.3× 47s ▲2.4×	44.5s ▲6.5× 42.6s ▲2.7×	1m 30.2s ▲6.4× 1m 26.7s ▲2.2×	2m 46.4s n/a 2m 42.7s n/a
BioseqDB	MPS / Scalar Scalar-DP / KSW2	8.3s ▲26% 5.4s ▲1.8×	8.7s ▲18% 5.3s ▲1.6×	·	·	·	·	·	·	·	·	·	·	·	·	·	·
	MPS / Scalar SIMD-DP / Block	6.8s ▲1.6× 6.2s ▲1.6×	7.1s ▲1.5× 6.2s ▲28%	·	6.6s ▲11.1× 5.6s ▲11.7×	·	·	·	·	3m 9.8s ▼3.6× 2m 47.5s ▼3.2×	3m 6.4s ▼3.3× 2m 46.4s ▼3.0×	·	·	·	·	·	·
	MPS / Scalar SIMD-DP / KSW2	8.3s ▲26% 5.5s ▲1.8×	8.3s ▲22% 5.2s ▲1.6×	·	·	·	·	·	·	3m 1.5s ▼3.4× 2m 37.6s ▼3.0×	3m 21.1s ▼3.6× 2m 58.7s ▼3.3×	·	·	·	·	·	·
	MPS / SIMD-ref RMQ / Block	·	·	·	·	·	·	·	·	·	·	3m 24.8s ▼1.8× 2m 59.9s ▼1.6×	·	·	·	·	·
	MPS / SIMD-ref Scalar-DP / KSW2	·	·	16.4s ▲27% 8.9s ▲2.1×	·	·	·	·	·	·	·	·	·	·	·	·	·
	MPS / SIMD-ref SIMD-DP / Block	·	·	·	6.4s ▲11.4× 5.5s ▲12.0×	·	·	·	·	·	·	4m 15.9s ▼2.2× 3m 41.9s ▼2.0×	·	·	·	·	·
	MPS / SIMD-ref SIMD-DP / KSW2	·	·	15.7s ▲1.4× 9.3s ▲2.0×	·	·	·	·	·	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Row order: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 2 Gbp k=15, w=10								No index 2 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
minimap2	Baseline	11.2s ▼3.1× 10s ▼3.6×	10.7s ▼3.2× 8.6s ▼3.5×	22.5s ▼3.6× 18.8s ▼4.4×	1m 13.1s ▼22.5× 1m 5.7s ▼27.7×	9.9s ▼21% 8.8s ▼1.5×	9.5s ▼1.4× 7.6s ▼1.6×	16.5s ▼1.7× 12.9s ▼2.0×	28.2s ▼2.1× 26.2s ▼2.4×	53s ▲2.2× 51.9s ▲2.1×	56.6s ▲2.1× 54.6s ▲1.9×	1m 55.5s ▲1.8× 1m 51.8s ▲1.5×	6m 1.3s n/a 5m 55.2s n/a	48.1s ▲7.3× 47s ▲2.4×	44.5s ▲6.5× 42.6s ▲2.7×	1m 30.2s ▲6.4× 1m 26.7s ▲2.2×	2m 46.4s n/a 2m 42.7s n/a
BioseqDB	MPS / Two-stack RMQ / Block	6.9s ▲1.6× 6.2s ▲1.6×	7.5s ▲29% 6.6s ▲23%	·	·	·	·	·	·	2m 47.3s ▼3.2× 2m 24.6s ▼2.8×	2m 30.3s ▼2.7× 2m 10.8s ▼2.4×	4m 45s ▼2.5× 4m 11.3s ▼2.2×	·	·	·	·	·
	MPS / Two-stack RMQ / KSW2	8.3s ▲26% 5.3s ▲1.9×	8.1s ▲23% 5.1s ▲1.7×	·	·	·	·	·	·	2m 51.1s ▼3.2× 2m 26.7s ▼2.8×	·	·	·	·	·	·	·
	MPS / Two-stack Scalar-DP / Block	6.9s ▲1.6× 6.2s ▲1.6×	7.2s ▲1.5× 6.3s ▲26%	·	·	·	·	·	·	·	·	·	·	·	·	·	·
	MPS / Two-stack Scalar-DP / KSW2	8.4s ▲25% 5.4s ▲1.8×	9.4s ▲12% 5.5s ▲1.6×	·	·	·	·	·	·	·	·	·	·	·	·	·	·
	MPS / Two-stack SIMD-DP / Block	6.8s ▲1.6× 6.1s ▲1.7×	6.8s ▲1.6× 6s ▲1.4×	12.6s ▲1.8× 10.6s ▲1.8×	6.5s ▲11.2× 5.7s ▲11.6×	·	·	·	·	2m 46.7s ▼3.1× 2m 24.6s ▼2.8×	2m 47.4s ▼3.0× 2m 27.3s ▼2.7×	·	·	·	·	·	·
	MPS / Two-stack SIMD-DP / KSW2	8.3s ▲26% 5.4s ▲1.8×	8.4s ▲21% 5.2s ▲1.7×	16.2s ▲28% 9.2s ▲2.0×	·	·	·	·	·	2m 40.2s ▼3.0× 2m 15.4s ▼2.6×	2m 42.5s ▼2.9× 2m 20.9s ▼2.6×	·	·	·	·	·	·
BioseqDB	Vulkan (all modes)	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Row order: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 2 Gbp k=20, w=15 DNA4 M7A	Index 2 Gbp k=20, w=15 DNA4 M7I	Index 2 Gbp k=20, w=15 DNA4 M4	Index 2 Gbp k=20, w=15 DNA4 C7G	Index 2 Gbp k=20, w=15 DNA5 M7A	Index 2 Gbp k=20, w=15 DNA5 M7I	Index 2 Gbp k=20, w=15 DNA5 M4	Index 2 Gbp k=20, w=15 DNA5 C7G	No index 2 Gbp k=20, w=15 DNA4 M7A	No index 2 Gbp k=20, w=15 DNA4 M7I	No index 2 Gbp k=20, w=15 DNA4 M4	No index 2 Gbp k=20, w=15 DNA4 C7G	No index 2 Gbp k=20, w=15 DNA5 M7A	No index 2 Gbp k=20, w=15 DNA5 M7I	No index 2 Gbp k=20, w=15 DNA5 M4	No index 2 Gbp k=20, w=15 DNA5 C7G
minimap2	Baseline	19.6s ▼6.7× 18.6s ▼8.4×	19.4s ▼7.2× 17.5s ▼9.1×	1m 12.7s ▼13.6× 1m 8.1s ▼20.2×	7m 25.4s ▼106.2× 6m 50.8s ▼212.9×	9.3s ▼24% 8.4s ▼1.6×	8.7s ▼1.5× 7s ▼1.7×	15.8s ▼26% 12.7s ▼1.6×	18.5s ▼1.6× 17s ▼1.8×	42.4s ▲2.0× 41.4s ▲1.9×	42.9s ▲2.0× 41.1s ▲1.8×	1m 26.9s ▲1.9× 1m 23.7s ▲1.7×	4m 51.6s n/a 4m 46.6s n/a	35s ▲8.6× 34s ▲2.5×	35.9s ▲6.7× 34.2s ▲2.5×	1m 14.4s ▲6.9× 1m 11.3s ▲2.2×	1m 0.1s n/a 58.6s n/a
BioseqDB	MPF / Bounded RMQ / Block	2.9s ▲6.7× 2.2s ▲8.3×	2.7s ▲7.1× 2s ▲8.8×	·	4.7s ▲94.5× 3.9s ▲106.2×	·	·	·	11.6s ▲1.6× 9.4s ▲1.8×	1m 34s ▼2.2× 1m 24.9s ▼2.1×	1m 33s ▼2.2× 1m 22.3s ▼2.0×	·	·	·	·	·	·
	MPF / Bounded RMQ / KSW2	4.5s ▲4.3× 2.2s ▲8.4×	4.2s ▲4.6× 2s ▲8.9×	·	6.2s ▲72.1× 3.4s ▲120.4×	·	·	·	12.6s ▲1.5× 9.8s ▲1.7×	1m 32.9s ▼2.2× 1m 23.1s ▼2.0×	1m 41.2s ▼2.4× 1m 28.4s ▼2.2×	·	·	·	·	·	·
	MPF / Bounded Scalar-DP / Block	3s ▲6.6× 2.3s ▲8.1×	2.8s ▲6.9× 2s ▲8.7×	·	4.2s ▲106.2× 3.3s ▲122.9×	·	·	·	12.8s ▲1.4× 10.6s ▲1.6×	1m 32.9s ▼2.2× 1m 23.4s ▼2.0×	1m 31.2s ▼2.1× 1m 21s ▼2.0×	·	·	·	·	·	·
	MPF / Bounded Scalar-DP / KSW2	4.6s ▲4.3× 2.2s ▲8.3×	4.1s ▲4.7× 1.9s ▲9.1×	·	5.8s ▲76.9× 3s ▲137.6×	·	·	·	13.3s ▲28% 10.5s ▲1.6×	1m 27.2s ▼2.1× 1m 17s ▼1.9×	1m 39.6s ▼2.3× 1m 26.7s ▼2.1×	·	·	·	·	·	·
	MPF / Bounded SIMD-DP / Block	3s ▲6.5× 2.3s ▲8.1×	2.7s ▲7.2× 2s ▲8.9×	5.4s ▲13.6× 3.4s ▲20.2×	4.5s ▲99.2× 3.6s ▲112.7×	7.5s ▲19% 5.2s ▲1.6×	5.8s ▲1.5× 4s ▲1.7×	12.6s ▲21% 7.9s ▲1.6×	12.9s ▲1.4× 10.6s ▲1.6×	1m 24.8s ▼2.0× 1m 16.9s ▼1.9×	1m 25.4s ▼2.0× 1m 15.7s ▼1.8×	2m 44s ▼1.9× 2m 26.2s ▼1.7×	·	5m 1.5s ▼8.6× 1m 24.2s ▼2.5×	4m 0.6s ▼6.7× 1m 24.3s ▼2.5×	8m 31.7s ▼6.9× 2m 34.1s ▼2.2×	·
	MPF / Bounded SIMD-DP / KSW2	4.6s ▲4.3× 2.2s ▲8.3×	4.1s ▲4.8× 1.9s ▲9.0×	·	4.7s ▲95.2× 1.9s ▲212.9×	·	·	·	12.5s ▲1.5× 9.7s ▲1.8×	1m 34.1s ▼2.2× 1m 24s ▼2.0×	1m 32.2s ▼2.2× 1m 20s ▼1.9×	·	·	·	·	·	·
	MPF / Scalar RMQ / Block	5m 39.9s ▼17.3× 5m 39.5s ▼18.2×	12m 19.6s ▼38.2× 12m 19.1s ▼42.3×	22m 41.7s ▼18.7× 22m 40.7s ▼20.0×	12m 57s ▼1.7× 12m 56.4s ▼1.9×	·	·	·	·	2m 49.9s ▼4.0× 2m 13.4s ▼3.2×	2m 29.9s ▼3.5× 2m 1.1s ▼2.9×	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Row order: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 2 Gbp k=20, w=15								No index 2 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
minimap2	Baseline	19.6s ▼6.7× 18.6s ▼8.4×	19.4s ▼7.2× 17.5s ▼9.1×	1m 12.7s ▼13.6× 1m 8.1s ▼20.2×	7m 25.4s ▼106.2× 6m 50.8s ▼212.9×	9.3s ▼24% 8.4s ▼1.6×	8.7s ▼1.5× 7s ▼1.7×	15.8s ▼26% 12.7s ▼1.6×	18.5s ▼1.6× 17s ▼1.8×	42.4s ▲2.0× 41.4s ▲1.9×	42.9s ▲2.0× 41.1s ▲1.8×	1m 26.9s ▲1.9× 1m 23.7s ▲1.7×	4m 51.6s n/a 4m 46.6s n/a	35s ▲8.6× 34s ▲2.5×	35.9s ▲6.7× 34.2s ▲2.5×	1m 14.4s ▲6.9× 1m 11.3s ▲2.2×	1m 0.1s n/a 58.6s n/a
BioseqDB	MPF / Scalar RMQ / KSW2	5m 46.6s ▼17.7× 5m 44.5s ▼18.5×	12m 26.1s ▼38.5× 12m 24.3s ▼42.6×	·	11m 45.8s ▼1.6× 11m 43.3s ▼1.7×	·	·	·	·	2m 53.6s ▼4.1× 2m 15.9s ▼3.3×	2m 44s ▼3.8× 2m 12.1s ▼3.2×	·	·	·	·	·	·
	MPF / Scalar Scalar-DP / Block	5m 39.5s ▼17.3× 5m 39s ▼18.2×	12m 20.4s ▼38.2× 12m 19.8s ▼42.3×	22m 35.5s ▼18.7× 22m 34.5s ▼19.9×	12m 31.2s ▼1.7× 12m 30.6s ▼1.8×	·	·	·	·	·	·	·	·	·	·	·	·
	MPF / Scalar Scalar-DP / KSW2	5m 39.7s ▼17.3× 5m 37.2s ▼18.1×	12m 8.9s ▼37.7× 12m 7.2s ▼41.6×	·	10m 39.3s ▼1.4× 10m 36.8s ▼1.5×	·	·	·	·	·	·	·	·	·	·	·	·
	MPF / Scalar SIMD-DP / Block	5m 39.7s ▼17.3× 5m 39.2s ▼18.2×	12m 5.2s ▼37.5× 12m 4.6s ▼41.4×	22m 32.6s ▼18.6× 22m 31.6s ▼19.8×	12m 48.6s ▼1.7× 12m 47.9s ▼1.9×	·	·	·	·	2m 45.2s ▼3.9× 2m 8.9s ▼3.1×	2m 44.3s ▼3.8× 2m 14.4s ▼3.3×	·	·	·	·	·	·
	MPF / Scalar SIMD-DP / KSW2	5m 42.8s ▼17.5× 5m 40.6s ▼18.3×	12m 19.8s ▼38.2× 12m 18s ▼42.2×	22m 43.4s ▼18.8× 22m 39.5s ▼20.0×	11m 32.1s ▼1.6× 11m 29.4s ▼1.7×	·	·	·	·	2m 52.8s ▼4.1× 2m 15.7s ▼3.3×	2m 32.2s ▼3.6× 2m 0.8s ▼2.9×	·	·	·	·	·	·
	MPF / SIMD-ref RMQ / KSW2	·	·	·	10m 52.1s ▼1.5× 10m 49.7s ▼1.6×	·	·	·	·	·	·	·	·	·	·	·	·
	MPF / SIMD-ref SIMD-DP / Block	·	·	·	13m 2.2s ▼1.8× 13m 1.5s ▼1.9×	·	·	·	·	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Row order: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 2 Gbp k=20, w=15								No index 2 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
minimap2	Baseline	19.6s ▼6.7× 18.6s ▼8.4×	19.4s ▼7.2× 17.5s ▼9.1×	1m 12.7s ▼13.6× 1m 8.1s ▼20.2×	7m 25.4s ▼106.2× 6m 50.8s ▼212.9×	9.3s ▼24% 8.4s ▼1.6×	8.7s ▼1.5× 7s ▼1.7×	15.8s ▼26% 12.7s ▼1.6×	18.5s ▼1.6× 17s ▼1.8×	42.4s ▲2.0× 41.4s ▲1.9×	42.9s ▲2.0× 41.1s ▲1.8×	1m 26.9s ▲1.9× 1m 23.7s ▲1.7×	4m 51.6s n/a 4m 46.6s n/a	35s ▲8.6× 34s ▲2.5×	35.9s ▲6.7× 34.2s ▲2.5×	1m 14.4s ▲6.9× 1m 11.3s ▲2.2×	1m 0.1s n/a 58.6s n/a
BioseqDB	MPF / Two-stack RMQ / Block	4m 37.6s ▼14.2× 4m 37.1s ▼14.9×	7m 30s ▼23.2× 7m 29.2s ▼25.7×	·	·	·	·	·	·	·	·	·	·	·	·	·	·
	MPF / Two-stack RMQ / KSW2	4m 34.2s ▼14.0× 4m 32.2s ▼14.6×	6m 52.3s ▼21.3× 6m 49.6s ▼23.4×	·	·	·	·	·	·	·	·	·	·	·	·	·	·
	MPF / Two-stack Scalar-DP / Block	4m 36s ▼14.1× 4m 35.5s ▼14.8×	6m 53.8s ▼21.4× 6m 52.9s ▼23.6×	·	·	·	·	·	·	·	·	·	·	·	·	·	·
	MPF / Two-stack Scalar-DP / KSW2	4m 38.1s ▼14.2× 4m 36.2s ▼14.8×	7m 34.7s ▼23.5× 7m 31.6s ▼25.8×	·	·	·	·	·	·	·	·	·	·	·	·	·	·
	MPF / Two-stack SIMD-DP / Block	4m 36.1s ▼14.1× 4m 35.5s ▼14.8×	7m 3.9s ▼21.9× 7m 3s ▼24.2×	·	5m 27s ▲27% 5m 26s ▲21%	·	·	·	4m 0.4s ▼13.0× 3m 59s ▼14.0×	·	·	·	·	·	·	·	·
	MPF / Two-stack SIMD-DP / KSW2	4m 37.8s ▼14.2× 4m 35.9s ▼14.8×	7m 37.6s ▼23.6× 7m 34.4s ▼26.0×	·	·	·	·	·	·	·	·	·	·	·	·	·	·
	MPS / Bounded RMQ / Block	6.6s ▲3.0× 5.9s ▲3.2×	6.5s ▲3.0× 5.8s ▲3.0×	·	6.4s ▲69.8× 5.5s ▲75.0×	·	·	·	·	1m 43.2s ▼2.4× 1m 21s ▼2.0×	1m 41.8s ▼2.4× 1m 21.9s ▼2.0×	2m 55s ▼2.0× 2m 21s ▼1.7×	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Row order: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 2 Gbp k=20, w=15								No index 2 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
minimap2	Baseline	19.6s ▼6.7× 18.6s ▼8.4×	19.4s ▼7.2× 17.5s ▼9.1×	1m 12.7s ▼13.6× 1m 8.1s ▼20.2×	7m 25.4s ▼106.2× 6m 50.8s ▼212.9×	9.3s ▼24% 8.4s ▼1.6×	8.7s ▼1.5× 7s ▼1.7×	15.8s ▼26% 12.7s ▼1.6×	18.5s ▼1.6× 17s ▼1.8×	42.4s ▲2.0× 41.4s ▲1.9×	42.9s ▲2.0× 41.1s ▲1.8×	1m 26.9s ▲1.9× 1m 23.7s ▲1.7×	4m 51.6s n/a 4m 46.6s n/a	35s ▲8.6× 34s ▲2.5×	35.9s ▲6.7× 34.2s ▲2.5×	1m 14.4s ▲6.9× 1m 11.3s ▲2.2×	1m 0.1s n/a 58.6s n/a
BioseqDB	MPS / Bounded RMQ / KSW2	8.2s ▲2.4× 5.7s ▲3.3×	8s ▲2.4× 5.6s ▲3.1×	·	8.3s ▲53.4× 5.3s ▲77.4×	·	·	·	·	1m 48.4s ▼2.6× 1m 24.7s ▼2.0×	1m 39.1s ▼2.3× 1m 18s ▼1.9×	·	·	·	·	·	·
	MPS / Bounded Scalar-DP / Block	6.6s ▲3.0× 5.9s ▲3.2×	6.6s ▲2.9× 5.9s ▲2.9×	·	6.3s ▲70.5× 5.4s ▲75.7×	·	·	·	·	1m 42.1s ▼2.4× 1m 20s ▼1.9×	1m 37.4s ▼2.3× 1m 18s ▼1.9×	·	·	·	·	·	·
	MPS / Bounded Scalar-DP / KSW2	8.2s ▲2.4× 5.7s ▲3.3×	8.1s ▲2.4× 5.6s ▲3.1×	·	8.4s ▲53.2× 5.3s ▲77.0×	·	·	·	·	1m 50.8s ▼2.6× 1m 27.1s ▼2.1×	1m 43.6s ▼2.4× 1m 21.8s ▼2.0×	·	·	·	·	·	·
	MPS / Bounded SIMD-DP / Block	6.5s ▲3.0× 5.8s ▲3.2×	6.5s ▲3.0× 5.8s ▲3.0×	·	6.4s ▲69.2× 5.5s ▲74.5×	·	·	·	·	·	·	·	·	·	·	8m 50.5s ▼7.1× 2m 35s ▼2.2×	·
	MPS / Bounded SIMD-DP / KSW2	8.4s ▲2.3× 5.7s ▲3.3×	8s ▲2.4× 5.6s ▲3.1×	·	8.2s ▲54.1× 5.3s ▲77.9×	·	·	·	·	1m 43.1s ▼2.4× 1m 19.2s ▼1.9×	1m 39.2s ▼2.3× 1m 17.9s ▼1.9×	·	·	·	·	·	·
	MPS / Scalar RMQ / Block	43.6s ▼2.2× 8.8s ▲2.1×	37.4s ▼1.9× 8.8s ▲2.0×	·	·	·	·	·	·	2m 53.3s ▼4.1× 2m 17.7s ▼3.3×	2m 48s ▼3.9× 2m 18.9s ▼3.4×	3m 24s ▼2.3× 2m 47.8s ▼2.0×	·	·	·	·	·
	MPS / Scalar RMQ / KSW2	45.4s ▼2.3× 8.8s ▲2.1×	39.7s ▼2.1× 9s ▲2.0×	·	·	·	·	·	·	2m 52.9s ▼4.1× 2m 15.4s ▼3.3×	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Row order: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 2 Gbp k=20, w=15 DNA4 M7A	Index 2 Gbp k=20, w=15 DNA4 M7I	Index 2 Gbp k=20, w=15 DNA4 M4	Index 2 Gbp k=20, w=15 DNA4 C7G	Index 2 Gbp k=20, w=15 DNA5 M7A	Index 2 Gbp k=20, w=15 DNA5 M7I	Index 2 Gbp k=20, w=15 DNA5 M4	Index 2 Gbp k=20, w=15 DNA5 C7G	No index 2 Gbp k=20, w=15 DNA4 M7A	No index 2 Gbp k=20, w=15 DNA4 M7I	No index 2 Gbp k=20, w=15 DNA4 M4	No index 2 Gbp k=20, w=15 DNA4 C7G	No index 2 Gbp k=20, w=15 DNA5 M7A	No index 2 Gbp k=20, w=15 DNA5 M7I	No index 2 Gbp k=20, w=15 DNA5 M4	No index 2 Gbp k=20, w=15 DNA5 C7G
minimap2	Baseline	19.6s ▼6.7× 18.6s ▼8.4×	19.4s ▼7.2× 17.5s ▼9.1×	1m 12.7s ▼13.6× 1m 8.1s ▼20.2×	7m 25.4s ▼106.2× 6m 50.8s ▼212.9×	9.3s ▼24% 8.4s ▼1.6×	8.7s ▼1.5× 7s ▼1.7×	15.8s ▼26% 12.7s ▼1.6×	18.5s ▼1.6× 17s ▼1.8×	42.4s ▲2.0× 41.4s ▲1.9×	42.9s ▲2.0× 41.1s ▲1.8×	1m 26.9s ▲1.9× 1m 23.7s ▲1.7×	4m 51.6s n/a 4m 46.6s n/a	35s ▲8.6× 34s ▲2.5×	35.9s ▲6.7× 34.2s ▲2.5×	1m 14.4s ▲6.9× 1m 11.3s ▲2.2×	1m 0.1s n/a 58.6s n/a
BioseqDB	MPS / Scalar Scalar-DP / Block	43.3s ▼2.2× 8.7s ▲2.2×	37.5s ▼1.9× 8.9s ▲2.0×	·	·	·	·	·	·	·	·	·	·	·	·	·	·
	MPS / Scalar Scalar-DP / KSW2	45.2s ▼2.3× 8.7s ▲2.1×	39.2s ▼2.0× 8.8s ▲2.0×	1m 6.9s ▲8% 14.8s ▲4.6×	·	·	·	·	·	·	·	·	·	·	·	·	·
	MPS / Scalar SIMD-DP / Block	43.8s ▼2.2× 8.6s ▲2.2×	38s ▼2.0× 8.7s ▲2.0×	·	36.6s ▲12.2× 8.2s ▲49.9×	·	·	·	·	2m 52.9s ▼4.1× 2m 16.2s ▼3.3×	2m 34.3s ▼3.6× 2m 5.6s ▼3.1×	·	·	·	·	·	·
	MPS / Scalar SIMD-DP / KSW2	45.1s ▼2.3× 8.4s ▲2.2×	37.9s ▼2.0× 8.2s ▲2.1×	1m 6.6s ▲8% 14.5s ▲4.7×	40.5s ▲11.0× 8.3s ▲49.8×	·	·	·	·	2m 54.8s ▼4.1× 2m 18.1s ▼3.3×	2m 39.7s ▼3.7× 2m 8.9s ▼3.1×	4m 27.6s ▼3.1× 3m 34.6s ▼2.6×	·	·	·	·	·
	MPS / SIMD-ref RMQ / Block	·	·	·	·	·	·	·	·	·	·	3m 51.4s ▼2.7× 3m 1.9s ▼2.2×	·	·	·	·	·
	MPS / SIMD-ref SIMD-DP / Block	·	·	·	34.8s ▲12.8× 8.2s ▲50.2×	·	·	·	·	·	·	·	·	·	·	·	·
	MPS / SIMD-ref SIMD-DP / KSW2	·	·	1m 6.7s ▲8% 14.4s ▲4.7×	·	·	·	·	·	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Row order: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 2 Gbp k=20, w=15								No index 2 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
minimap2	Baseline	19.6s ▼6.7× 18.6s ▼8.4×	19.4s ▼7.2× 17.5s ▼9.1×	1m 12.7s ▼13.6× 1m 8.1s ▼20.2×	7m 25.4s ▼106.2× 6m 50.8s ▼212.9×	9.3s ▼24% 8.4s ▼1.6×	8.7s ▼1.5× 7s ▼1.7×	15.8s ▼26% 12.7s ▼1.6×	18.5s ▼1.6× 17s ▼1.8×	42.4s ▲2.0× 41.4s ▲1.9×	42.9s ▲2.0× 41.1s ▲1.8×	1m 26.9s ▲1.9× 1m 23.7s ▲1.7×	4m 51.6s n/a 4m 46.6s n/a	35s ▲8.6× 34s ▲2.5×	35.9s ▲6.7× 34.2s ▲2.5×	1m 14.4s ▲6.9× 1m 11.3s ▲2.2×	1m 0.1s n/a 58.6s n/a
BioseqDB	MPS / Two-stack RMQ / Block	40.6s ▼2.1× 6.8s ▲2.7×	35.5s ▼1.8× 7s ▲2.5×	·	·	·	·	·	·	·	·	·	·	·	·	·	·
	MPS / Two-stack RMQ / KSW2	43.9s ▼2.2× 6.8s ▲2.7×	39.6s ▼2.0× 6.7s ▲2.6×	·	·	·	·	·	·	·	·	·	·	·	·	·	·
	MPS / Two-stack Scalar-DP / Block	39.8s ▼2.0× 6.8s ▲2.7×	35.8s ▼1.8× 6.5s ▲2.7×	·	·	·	·	·	·	·	·	·	·	·	·	·	·
	MPS / Two-stack Scalar-DP / KSW2	43.6s ▼2.2× 6.8s ▲2.7×	39.5s ▼2.0× 7.2s ▲2.4×	·	·	·	·	·	·	·	·	·	·	·	·	·	·
	MPS / Two-stack SIMD-DP / Block	39.5s ▼2.0× 6.8s ▲2.7×	35.7s ▼1.8× 7.2s ▲2.4×	·	33.9s ▲13.1× 6.9s ▲59.6×	·	·	·	·	·	·	·	·	·	·	·	·
	MPS / Two-stack SIMD-DP / KSW2	44s ▼2.2× 6.8s ▲2.8×	39.6s ▼2.0× 7.2s ▲2.4×	·	·	·	·	·	·	·	·	·	·	·	·	·	·
BioseqDB	Vulkan (all modes)	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Row order: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

6.1.3. Comparison with other aligners

The final table places minimap2 and one fixed BioseqDB configuration alongside blastn, Winnowmap and LexicMap. Both minimap2 and BioseqDB timings come from the corresponding machine's non-external remote summary, matched by machine class, target size, alphabet, (k, w) and index mode. External-campaign minimap2 timings are excluded. BioseqDB always uses minimizers per fragment (MPF), bounded ntHash SIMD minimizers, SIMD-DP chaining and Block Aligner. Missing observations are marked with a dot. The same default / TEST3 / TEST4 / TEST7 / TEST2 / FAST priority applies to both rows. FAST entries carry a single-sample footnote. No other BioseqDB configuration or historical timing scope is substituted.

The table contains 302 distinct observations and 1,510 accepted repeats. Only approved results are included. Qualification runs, interrupted attempts and the small-query checks used before the full runs are excluded.

Measurement procedure.

- Each external observation has five fresh-process repeats of 1,024 queries. A is the median wall time, including executable startup, index loading, mapping and writing output. Fixture generation and validation are outside the timer. The index mode prepares the native on-disk index beforehand. No-index mode creates it inside the measured interval, including tool-specific preparation such as counting frequent k-mers or partitioning the reference. Operating-system caches are not purged.

- The external campaign checks one primary occurrence per query, its position and strand, and an edit script reconstructed from the gapped output. These files include the five raw timing samples and returned-count vectors. This is additional evidence beyond the median-only summaries used for the earlier tables. MAPQ is checked only where the tool provides it. No MAPQ value is invented for blastn or LexicMap.

- B is computed separately in each repeat by subtracting the union of the tool's instrumented alignment and available MAPQ intervals, then taking the median. BLAST subtracts traceback-search regions. LexicMap subtracts wavefront-alignment calls. Winnowmap subtracts its native alignment and MAPQ intervals. These are different boundaries, so B is a tool-specific remaining time, not the duration of a common seeding-and-chaining stage.

How the tools were configured.

- blastn 2.17.0+. The word size equals the requested k. There is no minimizer-window setting, so w labels the fixture only. Low-complexity masking is disabled, and each reported alignment must cover at least 80% of the query to remove short chance matches. Match reward 2, mismatch penalty 4, gap-open penalty 4 and gap-extension penalty 2 reproduce the baseline's first affine-gap branch. BLAST has no second branch. The default E-value threshold of 10 is retained. DNA5 cells are marked because BLAST scores N differently.

- Winnowmap. Both k and w are retained. Frequent k-mers are counted with Meryl to build weighted minimizers. This preparation is included in no-index time. Baseline scoring and first-stage bandwidth are retained, while Winnowmap's additional

structural-variant-aware stage keeps its native settings. Secondary alignments are suppressed. The benchmark build restores persisted-index output and reconstructs the weighting filter on loading. It also initializes a repeat-length variable to prevent undefined MAPQ/output behaviour. Memory-limited and multipart runs carry cell-specific notes.

- LexicMap. The requested k is retained, but LexicHash masks replace minimizer windows, so w is inapplicable. The native 20,000 masks, gap filling between sparsely seeded regions and low-complexity rejection remain enabled. Query coverage is at least 80%. Its native wavefront-alignment costs are match 0, mismatch 4 and gap open/extension 6/2. Its separate reported-score calculation uses match 2, mismatch -3 and gaps 5/2. These settings are not rewritten to mimic minimap2. References are divided into records of at most 268,435,455 bases with overlap equal to the longest query, and output coordinates are restored to the logical reference. DNA5 cells are marked because its native reference encoding replaces N with A.

What the comparison establishes. The external runs used new instances of the same EC2 types, not the physical instances used for the retained minimap2 and BioseqDB results. External volumes are 300 GiB unencrypted gp3, compared with 500 GiB encrypted gp3 in the earlier campaign. Both provision 6,000 IOPS and 500 MiB/s. The external host bindings record four benchmark threads, nice -20 and Clang 18.1.3, whereas the earlier campaign used Clang 22.1.8. Compressed-RAM swap is recorded for many external runs. Its presence does not prove how much swapping occurred during each measurement.

The workload dimensions agree, but the timing boundaries, reference layouts, scoring and resource conditions are not identical. In particular, the retained framework timings exclude process-startup setup, and BioseqDB no-index mode constructs a transient index rather than the external tools' on-disk indexes. The summaries do not establish cross-campaign byte-for-byte fixture identity. Ratios therefore describe the published elapsed times under these qualified conditions, rather than an isolated causal effect of the aligner.

The table retains the green/red/gray triangles, 5% tie band and × notation above 30%. Every non-baseline row compares with minimap2 in the same machine-class/workload column. Minimap2 compares only with the displayed fixed BioseqDB configuration, not another faster BioseqDB row. Blue marks one lowest reported A and one lowest reported B down the five rows in each column. Exact ties select the first displayed row. Light gray identifies the minimap2 baseline. Pale yellow identifies the preferred MPF / Bounded / SIMD-DP / Block configuration, independently of the measured winners. For B, both the ratio and the highlight compare native residual times with different work removed, and do not establish superiority at an equivalent processing stage. Dots mean no approved timing, not failure or hardware unavailability.

Table 6.3: aligner comparison, 4 Gbp, k=15, w=10

Different campaigns and native timing boundaries. A/B ratios are descriptive, subject to the measurement notes.

Aligner	Configuration	Index 4 Gbp k=15, w=10 DNA4 M7A	Index 4 Gbp k=15, w=10 DNA4 M7I	Index 4 Gbp k=15, w=10 DNA4 M4	Index 4 Gbp k=15, w=10 DNA4 C7G	Index 4 Gbp k=15, w=10 DNA5 M7A	Index 4 Gbp k=15, w=10 DNA5 M7I	Index 4 Gbp k=15, w=10 DNA5 M4	Index 4 Gbp k=15, w=10 DNA5 C7G	No index 4 Gbp k=15, w=10 DNA4 M7A	No index 4 Gbp k=15, w=10 DNA4 M7I	No index 4 Gbp k=15, w=10 DNA4 M4	No index 4 Gbp k=15, w=10 DNA4 C7G	No index 4 Gbp k=15, w=10 DNA5 M7A	No index 4 Gbp k=15, w=10 DNA5 M7I	No index 4 Gbp k=15, w=10 DNA5 M4	No index 4 Gbp k=15, w=10 DNA5 C7G
minimap2	Retained baseline	36.2s ▼3.6× 35s ▼3.9×	38.1s ▼2.9× 36s ▼3.0×	1m 33.5s ▼9.9× 1m 29.7s ▼12.8×	6m 31.5s n/a[g] 6m 26.3s n/a	31.6s n/a 30.3s n/a	34.1s n/a 32s n/a	1m 9.5s n/a 1m 6.1s n/a	3m 1s n/a[g] 2m 51.5s n/a	2m 55.5s ▲22% 2m 54s ▲19%	2m 50.4s ▲28% 2m 48.2s ▲25%	6m 12.2s ▲9% 6m 8.2s ▲6%	44m 20.5s n/a[g] 44m 15.6s n/a	1m 47.1s ▲5.5× 1m 45.8s ▲2.1×	1m 47.2s ▲4.7× 1m 45.1s ▲2.3×	3m 40.3s ▲4.2× 3m 36.6s ▲1.7×	29m 43.4s n/a[g] 29m 38.5s n/a
BioseqDB	MPF / Bounded SIMD-DP / Block	9.9s ▲3.6× 9s ▲3.9×	13s ▲2.9× 11.9s ▲3.0×	9.4s ▲9.9× 7s ▲12.8×	·	·	·	·	·	3m 44.5s ▼28% 3m 35.9s ▼24%	3m 57.4s ▼1.4× 3m 45.7s ▼1.3×	6m 50.6s ▼10% 6m 30.7s ▼6%	·	9m 45.5s ▼5.5× 3m 41.5s ▼2.1×	8m 25.6s ▼4.7× 3m 56.7s ▼2.3×	15m 24.9s ▼4.2× 6m 0.2s ▼1.7×	·
blastn	Native[*]	11m 14.7s ▼18.6×[c] 11m 9.5s ▼19.1×	16m 40.6s ▼26.3×[c] 16m 29.1s ▼27.5×	22m 6.7s ▼14.2×[c] 21m 51.5s ▼14.6×	12m 25.4s ▼1.9×[c] 12m 19.3s ▼1.9×	11m 40.1s ▼22.2×[a,c] 9m 50.3s ▼19.5×	16m 52.8s ▼29.7×[a,c] 13m 48.2s ▼25.9×	23m 30.3s ▼20.3×[a,c] 18m 49.3s ▼17.1×	12m 50.3s ▼4.3×[a,c] 10m 46.6s ▼3.8×	11m 28s ▼3.9×[c] 11m 22.8s ▼3.9×	16m 55s ▼6.0×[c] 16m 43.4s ▼6.0×	22m 36.2s ▼3.6×[c] 22m 20.8s ▼3.6×	13m 1.6s ▲3.4×[c] 12m 55.8s ▲3.4×	12m 18.6s ▼6.9×[a,c] 10m 29.2s ▼5.9×	17m 25.7s ▼9.8×[a,c] 14m 22.8s ▼8.2×	24m 29.4s ▼6.7×[a,c] 19m 49.6s ▼5.5×	13m 43.9s ▲2.2×[a,c] 11m 39.5s ▲2.5×
winnowmap	Native[*]	52.9s ▼1.5×[c,e] 50.6s ▼1.4×	59.4s ▼1.6×[c,e] 55.2s ▼1.5×	1m 49.1s ▼17%[c,e] 1m 41.3s ▼13%	1m 16.1s ▲5.1×[c,d] 1m 8.2s ▲5.7×	31.6s ▶ 0%[c,e] 29.4s ▶ 3%	44.2s ▼30%[c,e] 40.3s ▼26%	59s ▲15%[c,e] 51.8s ▲22%	1m 8s ▲2.7×[c,d] 1m 0.3s ▲2.8×	8m 6s ▼2.8×[c,e] 8m 3.7s ▼2.8×	7m 59.6s ▼2.8×[c,e] 7m 55.3s ▼2.8×	15m 50.3s ▼2.6×[c,e] 15m 42.3s ▼2.6×	7m 57s ▲5.6×[c,d] 7m 48.9s ▲5.7×	6m 51.2s ▼3.8×[c,e] 6m 49.1s ▼3.9×	6m 41.1s ▼3.7×[c,e] 6m 37.2s ▼3.8×	12m 42.6s ▼3.5×[c,e] 12m 35.1s ▼3.5×	7m 10.6s ▲4.1×[c,d] 7m 3.1s ▲4.2×
lexicmap	Native[*]	4m 7.1s ▼6.8×[c] 4m 7s ▼7.0×	6m 29.9s ▼10.2×[c] 6m 29.7s ▼10.8×	9m 24.6s ▼6.0×[c] 9m 24.4s ▼6.3×	7m 16.7s ▼12%[c] 7m 16.4s ▼13%	4m 7.8s ▼7.9×[b,c] 4m 7.5s ▼8.2×	6m 29.2s ▼11.4×[b,c] 6m 28.8s ▼12.1×	11m 26.4s ▼9.9×[b,c] 11m 25.5s ▼10.4×	7m 15.7s ▼2.4×[b,c] 7m 13s ▼2.5×	7m 34.2s ▼2.6×[c] 7m 34.1s ▼2.6×	11m 38.9s ▼4.1×[c] 11m 38.7s ▼4.2×	17m 11.5s ▼2.8×[c] 17m 11.2s ▼2.8×	15m 28s ▲2.9×[c] 15m 27.2s ▲2.9×	7m 48.7s ▼4.4×[b,c] 7m 48.4s ▼4.4×	11m 34.4s ▼6.5×[b,c] 11m 34s ▼6.6×	17m 36.1s ▼4.8×[b,c] 17m 35.3s ▼4.9×	16m 6s ▲1.8×[b,c] 16m 3.7s ▲1.8×

Each cell: A, comparison, B, comparison. Batches of 1,024 queries. Five repeats unless marked s (FAST: one sample). Blue = one lowest A and one lowest B down each machine/workload column.

Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column. Exact ties select the first row.

Ratios use minimap2 as reference, except minimap2 itself uses the fixed BioseqDB row. · = no approved timing. n/a = no measured counterpart.

Both minimap2 and BioseqDB use the matching non-external remote measurements.

[*]External A includes process startup and output. B removes different native stages and is not an equivalent-work comparison. BLAST and LexicMap have no *w* or MAPQ equivalent.

[a]BLAST DNA5 uses an N score of -3, compared with -1 in the baseline.

[b]LexicMap stores reference N as A. Queries retain their bases, and edit-script checks use that native reference encoding.

[c]External input uses two reference records for the 4 Gbp sequence to stay within signed 32-bit coordinates.

[d]Winnowmap uses overlapping 250 Mb records and 499M index batches after an out-of-memory failure. Results are merged globally. This can affect MAPQ.

[e]Winnowmap uses 1999M index batches and merges the results from multiple index parts.

[g]Retained C7G minimap2 measurement comes from the earlier campaign with documented paging pressure.

Table 6.3: aligner comparison, 4 Gbp, k=20, w=15

Different campaigns and native timing boundaries. A/B ratios are descriptive, subject to the measurement notes.

Aligner	Configuration	Index 4 Gbp k=20, w=15 DNA4 M7A	Index 4 Gbp k=20, w=15 DNA4 M7I	Index 4 Gbp k=20, w=15 DNA4 M4	Index 4 Gbp k=20, w=15 DNA4 C7G	Index 4 Gbp k=20, w=15 DNA5 M7A	Index 4 Gbp k=20, w=15 DNA5 M7I	Index 4 Gbp k=20, w=15 DNA5 M4	Index 4 Gbp k=20, w=15 DNA5 C7G	No index 4 Gbp k=20, w=15 DNA4 M7A	No index 4 Gbp k=20, w=15 DNA4 M7I	No index 4 Gbp k=20, w=15 DNA4 M4	No index 4 Gbp k=20, w=15 DNA4 C7G	No index 4 Gbp k=20, w=15 DNA5 M7A	No index 4 Gbp k=20, w=15 DNA5 M7I	No index 4 Gbp k=20, w=15 DNA5 M4	No index 4 Gbp k=20, w=15 DNA5 C7G
minimap2	Retained baseline	2m 48.1s ▼18.3× 2m 41.7s ▼19.0×	2m 50.3s ▼15.0× 2m 43s ▼15.3×	4m 54.3s ▼43.2× 4m 39s ▼54.6×	25m 5.5s n/a[g] 25m 0.7s n/a	33.9s ▼1.9× 32.9s ▼2.1×	34.4s ▼2.6× 32.7s ▼2.9×	1m 8.2s ▼3.7× 1m 5.1s ▼4.6×	3m 9.8s n/a[g] 3m 3.7s n/a	3m 28.3s ▼1.3× 3m 22.8s ▼1.4×	3m 53.2s ▼1.4× 3m 43.5s ▼1.4×	7m 14.6s ▼1.4× 6m 57.2s ▼1.5×	51m 22s n/a[g] 51m 17.2s n/a	1m 10.3s ▲7.6× 1m 9s ▲2.4×	1m 8.6s ▲6.4× 1m 6.9s ▲2.5×	2m 21.5s ▲6.2× 2m 18.4s ▲2.2×	15m 56.8s n/a[g] 15m 51.9s n/a
BioseqDB	MPF / Bounded SIMD-DP / Block	9.2s ▲18.3× 8.5s ▲19.0×	11.4s ▲15.0× 10.7s ▲15.3×	6.8s ▲43.2× 5.1s ▲54.6×	·	18.2s ▲1.9× 16s ▲2.1×	13.2s ▲2.6× 11.4s ▲2.9×	18.6s ▲3.7× 14.3s ▲4.6×	·	2m 38.5s ▲24% 2m 29.8s ▲26%	2m 49s ▲28% 2m 37s ▲30%	5m 3.3s ▲1.4× 4m 42.7s ▲1.5×	·	8m 52.3s ▼7.6× 2m 45.5s ▼2.4×	7m 22.2s ▼6.4× 2m 48.2s ▼2.5×	14m 43.3s ▼6.2× 5m 9.5s ▼2.2×	·
blastn	Native[*]	4m 35.4s ▼1.6×[c] 4m 30.3s ▼1.7×	6m 21.2s ▼2.2×[c] 6m 10.8s ▼2.3×	8m 55.3s ▼1.8×[c] 8m 41.3s ▼1.9×	5m 9.4s ▲4.9×[c] 5m 4.2s ▲4.9×	5m 39.4s ▼10.0×[a,c] 4m 13.6s ▼7.7×	7m 32.4s ▼13.1×[a,c] 5m 27.1s ▼10.0×	11m 29.3s ▼10.1×[a,c] 8m 4s ▼7.4×	6m 10.2s ▼2.0×[a,c] 4m 39s ▼1.5×	4m 58.3s ▼1.4×[c] 4m 53.3s ▼1.4×	6m 40.8s ▼1.7×[c] 6m 30.3s ▼1.7×	9m 34s ▼1.3×[c] 9m 19.8s ▼1.3×	5m 40.5s ▲9.1×[c] 5m 35.1s ▲9.2×	6m 15.6s ▼5.3×[a,c] 4m 49.2s ▼4.2×	8m 11.1s ▼7.2×[a,c] 6m 2.7s ▼5.4×	12m 29.6s ▼5.3×[a,c] 9m 3.4s ▼3.9×	7m 4.7s ▲2.3×[a,c] 5m 34.7s ▲2.8×
winnowmap	Native[*]	55.6s ▲3.0×[c,e] 53.8s ▲3.0×	59.6s ▲2.9×[c,e] 56.2s ▲2.9×	1m 43s ▲2.9×[c,e] 1m 36.3s ▲2.9×	1m 0.7s ▲24.8×[c,d] 54.1s ▲27.7×	24.6s ▲28%[c,e] 22.8s ▲1.4×	23.7s ▲1.4×[c,e] 20.4s ▲1.6×	36.1s ▲1.9×[c,e] 29.6s ▲2.2×	41.4s ▲4.6×[c,d] 34.7s ▲5.3×	10m 41.8s ▼3.1×[c,e] 10m 40s ▼3.2×	11m 28.6s ▼3.0×[c,e] 11m 25.2s ▼3.1×	33m 6.8s ▼4.6×[c,e] 33m 0.1s ▼4.7×	12m 28.4s ▲4.1×[c,d] 12m 21.9s ▲4.1×	8m 28.7s ▼7.2×[c,e] 8m 26.9s ▼7.3×	8m 59.5s ▼7.9×[c,e] 8m 56.1s ▼8.0×	24m 58.6s ▼10.6×[c,e] 24m 52s ▼10.8×	10m 19s ▲1.5×[c,d] 10m 12.1s ▲1.6×
lexicmap	Native[*]	56.9s ▲3.0×[c] 56.7s ▲2.9×	1m 21.3s ▲2.1×[c] 1m 21.1s ▲2.0×	2m 1.2s ▲2.4×[c] 2m 0.9s ▲2.3×	1m 16.5s ▲19.7×[c] 1m 16.1s ▲19.7×	56.4s ▼1.7×[b,c] 55.8s ▼1.7×	1m 20.1s ▼2.3×[b,c] 1m 19.6s ▼2.4×	2m 9.7s ▼1.9×[b,c] 2m 8.5s ▼2.0×	1m 14.6s ▲2.5×[b,c] 1m 12.8s ▲2.5×	4m 52.5s ▼1.4×[c] 4m 52.3s ▼1.4×	6m 45.3s ▼1.7×[c] 6m 45.1s ▼1.8×	10m 57.1s ▼1.5×[c] 10m 56.8s ▼1.6×	9m 46.1s ▲5.3×[c] 9m 45.6s ▲5.3×	4m 52.2s ▼4.2×[b,c] 4m 51.8s ▼4.2×	6m 46.8s ▼5.9×[b,c] 6m 46.3s ▼6.1×	·	·

Each cell: A, comparison, B, comparison. Batches of 1,024 queries. Five repeats unless marked s (FAST: one sample). Blue = one lowest A and one lowest B down each machine/workload column.

Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column. Exact ties select the first row.

Ratios use minimap2 as reference, except minimap2 itself uses the fixed BioseqDB row. · = no approved timing. n/a = no measured counterpart.

Both minimap2 and BioseqDB use the matching non-external remote measurements.

[*]External A includes process startup and output. B removes different native stages and is not an equivalent-work comparison. BLAST and LexicMap have no *w* or MAPQ equivalent.

[a]BLAST DNA5 uses an N score of -3, compared with -1 in the baseline.

[b]LexicMap stores reference N as A. Queries retain their bases, and edit-script checks use that native reference encoding.

[c]External input uses two reference records for the 4 Gbp sequence to stay within signed 32-bit coordinates.

[d]Winnowmap uses overlapping 250 Mb records and 499M index batches after an out-of-memory failure. Results are merged globally. This can affect MAPQ.

[e]Winnowmap uses 1999M index batches and merges the results from multiple index parts.

[g]Retained C7G minimap2 measurement comes from the earlier campaign with documented paging pressure.

Table 6.3: aligner comparison, 2 Gbp, k=15, w=10

Different campaigns and native timing boundaries. A/B ratios are descriptive, subject to the measurement notes.

Aligner	Configuration	Index 2 Gbp k=15, w=10 DNA4 M7A	Index 2 Gbp k=15, w=10 DNA4 M7I	Index 2 Gbp k=15, w=10 DNA4 M4	Index 2 Gbp k=15, w=10 DNA4 C7G	Index 2 Gbp k=15, w=10 DNA5 M7A	Index 2 Gbp k=15, w=10 DNA5 M7I	Index 2 Gbp k=15, w=10 DNA5 M4	Index 2 Gbp k=15, w=10 DNA5 C7G	No index 2 Gbp k=15, w=10 DNA4 M7A	No index 2 Gbp k=15, w=10 DNA4 M7I	No index 2 Gbp k=15, w=10 DNA4 M4	No index 2 Gbp k=15, w=10 DNA4 C7G	No index 2 Gbp k=15, w=10 DNA5 M7A	No index 2 Gbp k=15, w=10 DNA5 M7I	No index 2 Gbp k=15, w=10 DNA5 M4	No index 2 Gbp k=15, w=10 DNA5 C7G
minimap2	Retained baseline	11.2s ▼3.0× 10s ▼3.4×	10.7s ▼3.0× 8.6s ▼3.2×	22.5s ▼3.3× 18.8s ▼4.2×	1m 13.1s ▼8.3×[g] 1m 5.7s ▼8.3×	9.9s ▼21% 8.8s ▼1.5×	9.5s ▼1.4× 7.6s ▼1.6×	16.5s ▼20% 12.9s ▼1.5×	28.2s n/a[g] 26.2s n/a	53s ▲2.2× 51.9s ▲2.1×	56.6s ▲2.1× 54.6s ▲1.9×	1m 55.5s ▲1.9× 1m 51.8s ▲1.8×	6m 1.3s n/a[g] 5m 55.2s n/a	48.1s ▲7.3× 47s ▲2.4×	44.5s ▲6.5× 42.6s ▲2.7×	1m 30.2s ▲6.4× 1m 26.7s ▲2.3×	2m 46.4s n/a[g] 2m 42.7s n/a
BioseqDB	MPF / Bounded SIMD-DP / Block	3.7s ▲3.0× 3s ▲3.4×	3.5s ▲3.0× 2.7s ▲3.2×	6.7s ▲3.3× 4.5s ▲4.2×	8.8s ▲8.3×[g] 7.9s ▲8.3×	8.2s ▲17% 5.8s ▲1.5×	6.6s ▲1.4× 4.6s ▲1.6×	13.7s ▲17% 8.8s ▲1.5×	·	1m 56.4s ▼2.2× 1m 48.1s ▼2.1×	1m 56.2s ▼2.1× 1m 46.2s ▼1.9×	3m 42.9s ▼1.9× 3m 25s ▼1.8×	·	5m 52.1s ▼7.3× 1m 53.5s ▼2.4×	4m 49.2s ▼6.5× 1m 56.8s ▼2.7×	9m 39.5s ▼6.4× 3m 18.1s ▼2.3×	·
blastn	Native[*]	5m 36.2s ▼30.1×[f] 5m 30.7s ▼33.0×	8m 14.3s ▼46.4×[f] 8m 2.4s ▼56.0×	10m 51.3s ▼28.9× 10m 35.3s ▼33.8×	5m 54.4s ▼4.9×[f] 5m 48.6s ▼5.3×	6m 37.9s ▼40.2×[a] 4m 57.9s ▼33.9×	9m 32.3s ▼60.3×[a] 6m 18s ▼49.6×	13m 27.4s ▼49.0×[a] 8m 52.1s ▼41.2×	6m 53.6s ▼14.7×[a] 4m 51.8s ▼11.1×	5m 39.9s ▼6.4× 5m 34.5s ▼6.4×	8m 22.6s ▼8.9× 8m 10.6s ▼9.0×	11m 6.9s ▼5.8× 10m 50.9s ▼5.8×	6m 6.4s ▶1% 6m 0.6s ▶2%	6m 51.8s ▼8.6×[a] 5m 12.2s ▼6.6×	9m 48.4s ▼13.2×[a] 6m 35.3s ▼9.3×	13m 55.3s ▼9.3×[a] 9m 23.3s ▼6.5×	7m 14.5s ▼2.6×[a] 5m 12.1s ▼1.9×
winnowmap	Native[*]	12.9s ▼15% 11.1s ▼11%	13.5s ▼27% 10.3s ▼20%	23.2s ▶3% 17.1s ▲9%	44.3s ▲1.7×[d] 37.8s ▲1.7×	12.3s ▼24% 10.6s ▼21%	13s ▼1.4× 10s ▼1.3×	23.8s ▼1.4× 17.6s ▼1.4×	37.3s ▼1.3×[d] 31.4s ▼20%	3m 33.2s ▼4.0× 3m 31.5s ▼4.1×	3m 22.6s ▼3.6× 3m 19.3s ▼3.7×	6m 21.3s ▼3.3× 6m 15s ▼3.4×	3m 56.6s ▲1.5×[d] 3m 50.3s ▲1.5×	3m 7.4s ▼3.9× 3m 5.8s ▼4.0×	3m 2.2s ▼4.1× 2m 59.3s ▼4.2×	6m 35.9s ▼4.4× 6m 29.8s ▼4.5×	3m 33.9s ▼29%[d] 3m 28.1s ▼28%
lexicmap	Native[*]	2m 14.1s ▼12.0× 2m 14s ▼13.4×	3m 26.7s ▼19.4× 3m 26.6s ▼24.0×	4m 40.1s ▼12.4× 4m 39.9s ▼14.9×	3m 24.3s ▼2.8× 3m 24.2s ▼3.1×	2m 12.9s ▼13.4×[b] 2m 12.5s ▼15.1×	3m 24.1s ▼21.5×[b] 3m 23.6s ▼26.7×	4m 56.4s ▼18.0×[b] 4m 55.6s ▼22.9×	3m 22.8s ▼7.2×[b] 3m 22.5s ▼7.7×	3m 18.9s ▼3.8× 3m 18.7s ▼3.8×	5m 10.5s ▼5.5× 5m 10.3s ▼5.7×	8m 19.4s ▼4.3× 8m 19.2s ▼4.5×	5m 43.1s ▲5% 5m 43s ▶3%	3m 16.5s ▼4.1×[b] 3m 16.1s ▼4.2×	5m 5s ▼6.9×[b] 5m 4.5s ▼7.2×	7m 40.8s ▼5.1×[b] 7m 39.9s ▼5.3×	5m 52.3s ▼2.1×[b] 5m 51.9s ▼2.2×

Each cell: A, comparison, B, comparison. Batches of 1,024 queries. Five repeats unless marked s (FAST: one sample). Blue = one lowest A and one lowest B down each machine/workload column.

Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column. Exact ties select the first row.

Ratios use minimap2 as reference, except minimap2 itself uses the fixed BioseqDB row. · = no approved timing. n/a = no measured counterpart.

Both minimap2 and BioseqDB use the matching non-external remote measurements.

[*]External A includes process startup and output. B removes different native stages and is not an equivalent-work comparison. BLAST and LexicMap have no *w* or MAPQ equivalent.

[a]BLAST DNA5 uses an N score of -3, compared with -1 in the baseline.

[b]LexicMap stores reference N as A. Queries retain their bases, and edit-script checks use that native reference encoding.

[d]Winnowmap uses overlapping 250 Mb records and 499M index batches after an out-of-memory failure. Results are merged globally. This can affect MAPQ.

[f]Swap configuration was not recorded before this external measurement. Later host snapshots do not establish its earlier swap state.

[g]Retained C7G minimap2 measurement comes from the earlier campaign with documented paging pressure.

Table 6.3: aligner comparison, 2 Gbp, k=20, w=15

Different campaigns and native timing boundaries. A/B ratios are descriptive, subject to the measurement notes.

Aligner	Configuration	Index 2 Gbp k=20, w=15								No index 2 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
minimap2	Retained baseline	19.6s ▼6.5× 18.6s ▼8.1×	19.4s ▼7.2× 17.5s ▼8.9×	1m 12.7s ▼13.6× 1m 8.1s ▼20.2×	7m 25.4s ▼99.2×[g] 6m 50.8s ▼112.7×	9.3s ▼24% 8.4s ▼1.6×	8.7s ▼1.5× 7s ▼1.7×	15.8s ▼26% 12.7s ▼1.6×	18.5s ▼1.4×[g] 17s ▼1.6×	42.4s ▲2.0× 41.4s ▲1.9×	42.9s ▲2.0× 41.1s ▲1.8×	1m 26.9s ▲1.9× 1m 23.7s ▲1.7×	4m 51.6s n/a[g] 4m 46.6s n/a	35s ▲8.6× 34s ▲2.5×	35.9s ▲6.7× 34.2s ▲2.5×	1m 14.4s ▲6.9× 1m 11.3s ▲2.2×	1m 0.1s n/a[g] 58.6s n/a
BioseqDB	MPF / Bounded SIMD-DP / Block	3s ▲6.5× 2.3s ▲8.1×	2.7s ▲7.2× 2s ▲8.9×	5.4s ▲13.6× 3.4s ▲20.2×	4.5s ▲99.2×[g] 3.6s ▲112.7×	7.5s ▲19% 5.2s ▲1.6×	5.8s ▲1.5× 4s ▲1.7×	12.6s ▲21% 7.9s ▲1.6×	12.9s ▲1.4× 10.6s ▲1.6×	1m 24.8s ▼2.0× 1m 16.9s ▼1.9×	1m 25.4s ▼2.0× 1m 15.7s ▼1.8×	2m 44s ▼1.9× 2m 26.2s ▼1.7×	·	5m 1.5s ▼8.6× 1m 24.2s ▼2.5×	4m 0.6s ▼6.7× 1m 24.3s ▼2.5×	8m 31.7s ▼6.9× 2m 34.1s ▼2.2×	·
blastn	Native[*]	2m 19.5s ▼7.1× 2m 14.5s ▼7.2×	3m 12.3s ▼9.9× 3m 1.8s ▼10.4×	4m 30.1s ▼3.7× 4m 16s ▼3.8×	2m 27.9s ▲3.0× 2m 22.7s ▲2.9×	3m 27.6s ▼22.3×[a] 2m 22.7s ▼17.0×	4m 37.9s ▼31.8×[a] 2m 47s ▼23.8×	7m 14.2s ▼27.5×[a] 4m 26.5s ▼20.9×	3m 30.4s ▼11.4×[a] 2m 14.4s ▼7.9×	2m 28.5s ▼3.5× 2m 23.4s ▼3.5×	3m 20.9s ▼4.7× 3m 10.4s ▼4.6×	4m 46.7s ▼3.3× 4m 32.6s ▼3.3×	2m 38.7s ▲1.8× 2m 33.5s ▲1.9×	3m 43.6s ▼6.4×[a] 2m 37.7s ▼4.6×	4m 51.4s ▼8.1×[a] 2m 59.6s ▼5.3×	7m 39.4s ▼6.2×[a] 4m 49.6s ▼4.1×	3m 50.8s ▼3.8×[a] 2m 36.2s ▼2.7×
winnowmap	Native[*]	28.4s ▼1.4× 26.9s ▼1.4×	27.6s ▼1.4× 24.8s ▼1.4×	1m 2.8s ▲14% 55s ▲19%	8m 35.8s ▼16% 8m 4.4s ▼18%	10.8s ▼16% 9.4s ▼12%	10.9s ▼24% 8s ▼14%	20.1s ▼27% 14.5s ▼14%	21s ▼14% 17.1s ▶ 0%	4m 16.8s ▼6.1× 4m 15.2s ▼6.2×	5m 0.6s ▼7.0× 4m 57.9s ▼7.2×	10m 23.6s ▼7.2× 10m 18s ▼7.4×	16m 47.8s ▼3.5× 16m 11s ▼3.4×	3m 21.4s ▼5.8× 3m 19.9s ▼5.9×	4m 0.5s ▼6.7× 3m 57.6s ▼7.0×	8m 56.2s ▼7.2× 8m 50.6s ▼7.4×	4m 43.2s ▼4.7× 4m 39.3s ▼4.8×
lexicmap	Native[*]	38.8s ▼2.0× 38.6s ▼2.1×	54.3s ▼2.8× 54s ▼3.1×	1m 29.8s ▼24% 1m 29.4s ▼1.3×	49.9s ▲8.9× 49.7s ▲8.3×	37.8s ▼4.1×[b] 37.3s ▼4.4×	53.5s ▼6.1×[b] 52.8s ▼7.5×	1m 18.6s ▼5.0×[b] 1m 17.5s ▼6.1×	49.7s ▼2.7×[b] 49.3s ▼2.9×	1m 41.8s ▼2.4× 1m 41.7s ▼2.5×	2m 35.6s ▼3.6× 2m 35.4s ▼3.8×	4m 6.7s ▼2.8× 4m 6.3s ▼2.9×	3m 25.5s ▲30% 3m 25.3s ▲28%	1m 43.4s ▼3.0×[b] 1m 42.6s ▼3.0×	2m 37.7s ▼4.4×[b] 2m 37.1s ▼4.6×	3m 45.3s ▼3.0×[b] 3m 43.9s ▼3.1×	3m 16.3s ▼3.3×[b] 3m 15.8s ▼3.3×

Each cell: A, comparison, B, comparison. Batches of 1,024 queries. Five repeats unless marked s (FAST: one sample). Blue = one lowest A and one lowest B down each machine/workload column.

Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column. Exact ties select the first row.

Ratios use minimap2 as reference, except minimap2 itself uses the fixed BioseqDB row. · = no approved timing. n/a = no measured counterpart.

Both minimap2 and BioseqDB use the matching non-external remote measurements.

[*]External A includes process startup and output. B removes different native stages and is not an equivalent-work comparison. BLAST and LexicMap have no *w* or MAPQ equivalent.

[a]BLAST DNA5 uses an N score of -3, compared with -1 in the baseline.

[b]LexicMap stores reference N as A. Queries retain their bases, and edit-script checks use that native reference encoding.

[g]Retained C7G minimap2 measurement comes from the earlier campaign with documented paging pressure.

Chapter 7

Conclusions

BioSeqDB demonstrates that sequence search can be represented as a PostgreSQL execution operation rather than as a scalar call to an external mapper. The system stores packed biological sequences, constructs persistent minimizer indexes, and runs a minimap2-derived pipeline for seeding, chaining, filtering, and final extension. Planner support and custom scans preserve the batch structure required by SIMD and GPU implementations while presenting typed SQL results that can participate in filters, projections, and joins.

The first research question concerned a shared representation for the algorithmic stages. Alphabet-generic compact ranges cover resident values, while lazy sequence views represent fragmented, sliced, and composed sources. Algorithms read the required region without first converting the complete value to a byte array. The second question concerned heterogeneous execution. Minimizer extraction and chaining provide regular work for the implemented SIMD and GPU routes. Dispatch follows explicit algorithm settings, input compatibility, index form, and resource availability. Final alignment uses banded or unbanded KSW2, adaptive Block Aligner, and GPU routes that may return specialized scripts before device dynamic programming. Optional prefilters may reject unsuitable candidates but do not replace alignment.

The third question concerned database execution. A custom scan owns the search expression, batches input rows, and substitutes match fields into planned expressions. This design also permits multiple alignments for one input row. The fourth question concerned the surrounding data system. Physical sequence storage, persistent indexes, bounded ingestion, external-provider relation definitions, cache coordination, and source-driven SQL generation give the kernels explicit storage and lifecycle contracts.

The remaining limits are operational and algorithmic. Some remote retrieval paths still require record-sized or archive-sized memory. Provider quotas are not coordinated across all PostgreSQL backends. Hardware-specific routes have batch-size thresholds and require broader evaluation on additional devices. GPU chaining may examine a bounded predecessor set, and seed and alignment backends retain route-specific contracts. The present mapper is centered on linear references rather than pangenome graphs. A relational database can nevertheless plan and execute batched sequence analysis while retaining typed data, bounded resources, persistent derived state, and explicit algorithm selection.

References

Aasna, Evelin et al. “DREAM-Stellar: parallel and space efficient exact local alignment.” In: BMC Bioinformatics 27.1, 67 (Feb. 18, 2026). doi: 10.1186/s12859-026-06389-0. url: https://link.springer.com/article/10.1186/s12859-026-06389-0.

Abadi, Daniel, Samuel Madden, and Miguel Ferreira. “Integrating compression and execution in column-oriented database systems.” In: Proceedings of the 2006 ACM SIGMOD International Conference on Management of Data. SIGMOD ’06. Association for Computing Machinery, 2006, pp. 671–682. doi: 10.1145/1142473.1142548. url: https://doi.org/10.1145/1142473.1142548.

Chernishev, George A. et al. “A Comprehensive Study of Late Materialization Strategies for a Disk-Based Column-Store.” In: Proceedings of the 24th International Workshop on Design, Optimization, Languages and Analytical Processing of Big Data. Vol. 3130. CEUR Workshop Proceedings. 2022, pp. 21–30. url: https://ceur-ws.org/Vol-3130/paper3.pdf.

Citus Data contributors. Citus Columnar storage implementation. Version 14.0.0. AGPL-3.0. Feb. 17, 2026. url: https://github.com/citusdata/citus (visited on 07/28/2026).

ClickHouse contributors. ClickBench: analytical database benchmark. Historical PostgreSQL, Citus, and Hydra submissions. Sept. 20, 2023. url: https://github.com/ClickHouse/ClickBench/tree/2e8146c397f81e3e98937205d09e331313b991e3 (visited on 09/09/2026).

Conway, Neil. Inside the PostgreSQL Query Optimizer. Slide deck. Undated online source. Fujitsu Australia Software Technology. url: https://www.neilconway.org/talks/optimizer/optimizer.pdf (visited on 09/17/2026).

cppreference.com contributors. C++ named requirements: StandardLayoutType. Undated online source. url: https://en.cppreference.com/cpp/named_req/StandardLayoutType (visited on 09/15/2026).

Cubukcu, Umur et al. “Citus: Distributed PostgreSQL for Data-Intensive Applications.” In: Proceedings of the 2021 International Conference on Management of Data. SIGMOD ’21. Association for Computing Machinery, 2021, pp. 2490–2502. doi: 10.1145/3448016.3457551. url: https://doi.org/10.1145/3448016.3457551.

Davis, Jeff. Citus 10 brings columnar compression to Postgres. Mar. 6, 2021. url: https://www.citusdata.com/blog/2021/03/06/citus-10-columnar-compression-for-postgres/ (visited on 09/08/2026).

Groot Koerkamp, Ragnar and Igor Martayan. packed-seq: Bitpacked sequence trait and implementation. Version 5.0.0. MIT license. Rust crate developed for SimdMinimizers (SEA 2025). url: https://github.com/rust-seq/packed-seq (visited on 09/15/2026).

Groot Koerkamp, Ragnar and Igor Martayan. packed_seq crate documentation. Version 5.0.0. July 7, 2026. url: https://docs.rs/packed-seq/latest/packed_seq/ (visited on 09/15/2026).
— "SimdMinimizers: Computing Random Minimizers, fast." In: 23rd International Symposium on Experimental Algorithms. Vol. 338. Leibniz International Proceedings in Informatics. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025, 20:1–20:19. doi: 10.4230/LIPIcs.SEA.2025.20. url: https://doi.org/10.4230/LIPIcs.SEA.2025.20.
Hydra contributors. Hydra Columnar: PostgreSQL columnar storage extension. Source snapshot d1f7657ee822. Feb. 10, 2025. url: https://github.com/hydradatabase/columnar (visited on 09/09/2026).
Kallenborn, Felix et al. "GPU-accelerated homology search with MMseqs2." In: Nature Methods 22.10 (Sept. 18, 2025), pp. 2024–2027. doi: 10.1038/s41592-025-02819-8. url: https://doi.org/10.1038/s41592-025-02819-8.
Editors' Report: Programming Languages—C++. WG21 paper N5015. ISO/IEC JTC 1/SC 22/WG 21, Aug. 14, 2025. url: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5015.html (visited on 09/09/2026).
Kryukov, Kirill et al. "Nucleotide Archival Format (NAF) enables efficient lossless reference-free compression of DNA sequences." In: Bioinformatics 35.19 (Oct. 2019), pp. 3826–3828. issn: 1367-4803. doi: 10.1093/bioinformatics/btz144. eprint: https://academic.oup.com/bioinformatics/article-pdf/35/19/3826/48977341/bioinformatics_35_19_3826.pdf. url: https://doi.org/10.1093/bioinformatics/btz144.
Lane, Tom. Re: what exactly is a PlaceHolderVar? Reply to Robert Haas. Message-ID: 23076.1277179362@sss.pgh.pa.us. PostgreSQL Hackers mailing list. June 22, 2010. url: https://www.postgresql.org/message-id/23076.1277179362@sss.pgh.pa.us (visited on 09/17/2026).
Lanius, Christian. parse_mmi.py: Python parser and lookup functions for minimap2 indexes. Sept. 27, 2021. url: https://gist.github.com/christian-lanius/8b8d6a38e35b93783ff7d6236211ff5e (visited on 09/08/2026).
Li, Heng. minimap2. Version 2.30. url: https://github.com/lh3/minimap2/tree/79c9cc186b95f50bd899f69b48eba995ced810c6 (visited on 07/28/2026).
— "Minimap2: pairwise alignment for nucleotide sequences." In: Bioinformatics 34.18 (2018), pp. 3094–3100. doi: 10.1093/bioinformatics/bty191. arXiv: 1708.01492 [q-bio.GN]. url: https://doi.org/10.1093/bioinformatics/bty191.
Liu, Daniel and Martin Steinegger. "Block Aligner: an adaptive SIMD-accelerated aligner for sequences and position-specific scoring matrices." In: Bioinformatics 39.8, btad487 (2023). doi: 10.1093/bioinformatics/btad487. url: https://doi.org/10.1093/bioinformatics/btad487.
Martayan, Igor et al. "Helicase: Vectorized parsing and bitpacking of genomic sequences." In: bioRxiv (Mar. 2026). Preprint. doi: 10.64898/2026.03.19.712912. url: https://github.com/imartayan/helicase (visited on 09/09/2026).
minimap2 contributors. How to interpret the mmi index file? Issue 820. Discussion updated in 2025. Sept. 26, 2021. url: https://github.com/lh3/minimap2/issues/820 (visited on 09/08/2026).

Mirdita, M. et al. “Fast and sensitive taxonomic assignment to metagenomic contigs.” In: Bioinformatics 37.18 (2021), pp. 3029–3031. doi: 10.1093/bioinformatics/btab184. url: https://doi.org/10.1093/bioinformatics/btab184.

Mirdita, Milot, Martin Steinegger, and Johannes Söding. “MMseqs2 desktop and local web server app for fast, interactive sequence searches.” In: Bioinformatics 35.16 (2019), pp. 2856–2858. doi: 10.1093/bioinformatics/bty1057. url: https://doi.org/10.1093/bioinformatics/bty1057.

National Center for Biotechnology Information. FASTA Format for Nucleotide Sequences. June 18, 2025. url: https://www.ncbi.nlm.nih.gov/genbank/fastaformat/ (visited on 09/09/2026).

— NCBI Datasets v2 REST API documentation. Undated online source. url: https://www.ncbi.nlm.nih.gov/datasets/docs/v2/api/ (visited on 07/28/2026).

Neon. The pg_search extension. Undated online source. Documents ParadeDB pg_search on Neon. url: https://neon.com/docs/extensions/pg_search (visited on 09/17/2026).

Pigsty. pg_search: full text search for PostgreSQL using BM25. Version 0.25.2. Undated online source. Last modified 2026-08-21. Aug. 21, 2026. url: https://pigsty.io/ext/e/pg_search/ (visited on 09/17/2026).

PostgreSQL Global Development Group. PostgreSQL 12 Release Notes. Version 12. url: https://www.postgresql.org/docs/12/release-12.html (visited on 09/16/2026).

— PostgreSQL 17 Documentation: Appendix K. PostgreSQL Limits. Version 17. url: https://www.postgresql.org/docs/17/limits.html (visited on 09/21/2026).

— PostgreSQL 17 Documentation: Architectural Fundamentals. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/tutorial-arch.html (visited on 09/16/2026).

— PostgreSQL 17 Documentation: Background Worker Processes. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/bgworker.html (visited on 09/16/2026).

— PostgreSQL 17 Documentation: Behavior in Threaded Programs. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/libpq-threading.html (visited on 09/16/2026).

— PostgreSQL 17 Documentation: Database Page Layout. Version 17. url: https://www.postgresql.org/docs/17/storage-page-layout.html (visited on 09/08/2026).

— PostgreSQL 17 Documentation: Foreign Data. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/ddl-foreign-data.html (visited on 09/16/2026).

— PostgreSQL 17 Documentation: Function Optimization Information. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/xfunc-optimization.html (visited on 07/28/2026).

— PostgreSQL 17 Documentation: Generic WAL Records. Version 17. url: https://www.postgresql.org/docs/17/generic-wal.html (visited on 09/08/2026).

— PostgreSQL 17 Documentation: How Parallel Query Works. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/how-parallel-query-works.html (visited on 09/16/2026).

— PostgreSQL 17 Documentation: Introduction to Concurrency Control. Version 17. url: https://www.postgresql.org/docs/17/mvcc-intro.html (visited on 09/08/2026).

— PostgreSQL 17 Documentation: *postgres_fdw*. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/postgres-fdw.html (visited on 09/16/2026).
PostgreSQL Global Development Group. PostgreSQL 17 Documentation: Resource Consumption. Version 17. url: https://www.postgresql.org/docs/17/runtime-config-resource.html#GUC-MAX-PARALLEL-WORKERS-PER-GATHER (visited on 09/09/2026).
— PostgreSQL 17 Documentation: SET. Version 17. url: https://www.postgresql.org/docs/17/sql-set.html (visited on 09/09/2026).
— PostgreSQL 17 Documentation: Set Returning Functions. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/functions-srf.html (visited on 07/28/2026).
— PostgreSQL 17 Documentation: Table Access Method Interface Definition. Version 17. url: https://www.postgresql.org/docs/17/tableam.html (visited on 09/08/2026).
— PostgreSQL 17 Documentation: TOAST. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/storage-toast.html (visited on 07/28/2026).
— PostgreSQL 17 Documentation: Using C++ for Extensibility. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/xfunc-c.html (visited on 07/28/2026).
— PostgreSQL 17 Documentation: Writing a Custom Scan Provider. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/custom-scan.html (visited on 07/28/2026).
— PostgreSQL 17 source: tuple table slots. Version 17. Header `src/include/executor/tuptable.h` on tag `REL_17_10`. url: https://github.com/postgres/postgres/blob/REL_17_10/src/include/executor/tuptable.h (visited on 09/18/2026).
— PostgreSQL 9.1 Release Notes. Version 9.1. url: https://www.postgresql.org/docs/9.1/release-9-1.html (visited on 09/16/2026).
— PostgreSQL 9.3 Release Notes. Version 9.3. url: https://www.postgresql.org/docs/9.3/release-9-3.html (visited on 09/16/2026).
— PostgreSQL 9.5 Release Notes. Version 9.5. url: https://www.postgresql.org/docs/9.5/release-9-5.html (visited on 09/16/2026).
— PostgreSQL Documentation: Planner/Optimizer. Undated online source. Section on generating possible plans, including the Genetic Query Optimizer. url: https://www.postgresql.org/docs/current/planner-optimizer.html#PLANNER-OPTIMIZER-GENERATING-POSSIBLE-PLANS (visited on 09/17/2026).
Python Software Foundation. Python 3 Documentation: Common Sequence Operations. Version 3.14.7. Slice clamping rules are given in note 4 of the operation table. Sept. 15, 2026. url: https://docs.python.org/3/builtins/stdtypes.html#common-sequence-operations (visited on 09/15/2026).
Reinert, Knut et al. "The SeqAn C++ template library for efficient sequence analysis: A resource for programmers." In: Journal of Biotechnology 261 (Nov. 10, 2017), pp. 157–168. doi: 10.1016/j.jbiotec.2017.07.017. url: https://www.sciencedirect.com/science/article/pii/S0168165617315420.
Sayers, Eric. "A General Introduction to the E-utilities." In: Entrez Programming Utilities Help. Updated 2022-11-17. Bethesda, Maryland: National Center for Biotechnology Information, May 26, 2009. url: https://www.ncbi.nlm.nih.gov/books/NBK25497/ (visited on 09/09/2026).
— "The E-utilities In-Depth: Parameters, Syntax and More." In: Entrez Programming Utilities Help. Updated 2026-03-04. Bethesda, Maryland: National Center for

Biotechnology Information, May 29, 2009. url: https://www.ncbi.nlm.nih.gov/books/NBK25499/ (visited on 09/09/2026).
SeqAn Project. Alphabets in SeqAn. Version 3.4.0. url: https://docs.seqan.de/seqan3/3.4.0/tutorial_alphabets.html (visited on 09/09/2026).
— SeqAn Manual. url: https://seqan.readthedocs.io/en/main/ (visited on 09/09/2026).
— SeqAn Tutorial: Sequences In-Depth. Version 1.4.2. String specializations, including packed and external strings. url: https://seqan.readthedocs.io/en/seqan-v1.4.2/Tutorial/SequencesInDepth.html (visited on 09/15/2026).
Shen, Wei, Botond Sipos, and Liuyang Zhao. SeqKit usage documentation: the subseq subcommand. Undated online source. Documentation for the tool described in SeqKit2, iMeta 3(3):e191, 2024. url: https://bioinf.shenwei.me/seqkit/usage/#subseq (visited on 09/15/2026).
Siragusa, Enrico, David Weese, and Knut Reinert. Fast and sensitive read mapping with approximate seeds and multiple backtracking. Version 1. Preprint. Aug. 21, 2012. doi: 10.48550/arXiv.1208.4238. arXiv: 1208.4238 [cs.DS]. url: https://arxiv.org/abs/1208.4238v1 (visited on 09/09/2026).
Steinegger, Martin and Johannes Söding. "Clustering huge protein sequence sets in linear time." In: Nature Communications 9, 2542 (June 29, 2018). doi: 10.1038/s41467-018-04964-5. url: https://doi.org/10.1038/s41467-018-04964-5.
— "MMseqs2 enables sensitive protein sequence searching for the analysis of massive data sets." In: Nature Biotechnology 35.11 (Oct. 16, 2017), pp. 1026–1028. doi: 10.1038/nbt.3988. url: https://doi.org/10.1038/nbt.3988.
— "MMseqs2: sensitive protein sequence searching for the analysis of massive data sets." Version 5. In: bioRxiv, 079681 (June 7, 2017). Preprint. doi: 10.1101/079681. url: https://www.biorxiv.org/content/early/2017/06/07/079681.
Tarjan, Robert Endre. "Depth-First Search and Linear Graph Algorithms." In: SIAM Journal on Computing 1.2 (1972), pp. 146–160. doi: 10.1137/0201010. url: https://doi.org/10.1137/0201010.
The Clang Team. C++ Support in Clang. url: https://clang.llvm.org/cxx_status.html (visited on 09/09/2026).
— Choosing the Right Interface for Your Application. url: https://clang.llvm.org/docs/Tooling.html (visited on 09/09/2026).
— LibTooling. url: https://clang.llvm.org/docs/LibTooling.html (visited on 09/09/2026).
Uhl, Tim Niklas, Matthias Schimek, and Daniel Brommer. Concepts in Practice: C++ MPI Bindings for the HPC Ecosystem. From a Standardizable Core to a Composable Interface. 2026. arXiv: 2606.09102 [cs.DC]. url: https://arxiv.org/abs/2606.09102.
UniProt Consortium. FASTA headers. June 10, 2026. url: https://www.uniprot.org/help/fasta-headers (visited on 09/09/2026).
UXL Foundation Contributors. How Task Scheduler Works. url: https://uxlfoundation.github.io/oneTBB/main/tbb_userguide/How_Task_Scheduler_Works.html (visited on 09/09/2026).
— oneTBB Benefits. url: https://uxlfoundation.github.io/oneTBB/main/intro/Benefits.html (visited on 09/09/2026).

— oneTBB Specification: task_arena. url: https://uxlfoundation.github.io/oneTBB/main/specification/source/task_scheduler/task_arena/task_arena_cls.html (visited on 09/09/2026).
UXL Foundation Contributors. When Task-Based Programming Is Inappropriate. url: https://uxlfoundation.github.io/oneTBB/main/tbb_userguide/When_Task-Based_Programming_Is_Inappropriate.html (visited on 09/09/2026).
Vardanian, Ash. StringZilla: Fast SIMD, SWAR, and GPGPU String Processing. url: https://github.com/ashvardanian/StringZilla (visited on 09/09/2026).